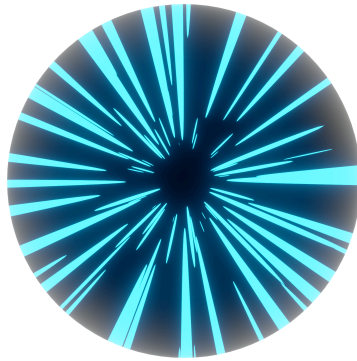




Hypertunnel

**HTTP-Based Bidirectional Tunneling without Server
Sent Events, Websockets, or Request Streaming for
Circumventing Strict TLS-Decrypting Firewalls**

OCR A Level Computer Science Project

Alex Chambers-Jones

2024-04-09

CONTENTS

1. Introduction	8
1.1. Worldwide Censorship and Circumvention	8
1.1.1. Deny-By-Default	8
1.1.2. Another Line Of Defense	9
1.1.3. Disclaimer	10
2. Analysis	11
2.1. Why Computation?	11
2.1.1. Computational Methods	11
2.1.1.1. Abstraction	11
2.1.1.2. Decomposition	11
2.1.1.3. Concurrency	11
2.2. Stakeholders	12
2.2.1. Students behind restricted networks	12
2.2.2. Dissidents and activists in oppressive regimes	13
2.2.3. Circumvention tool developers	14
2.2.4. Rust developers	15
2.3. Research	16
2.3.1. Internet in China	16
2.3.2. Socialist Market Economy	16
2.3.3. CL97	16
2.3.4. Golden Shield	16
2.3.5. Early Architecture and Circumvention	18
2.3.6. Increasing Sophistication 2006-2012	21
2.3.7. Continued Development 2012-2015	25
2.3.8. Modern Developments 2015-Present	34
2.3.9. GFWTrojan	44
2.3.10. Hysteria	44
2.3.11. What have I learnt?	46
2.3.12. Development Approach	46
2.4. Essential Features	47
2.4.1. Bidirectional HTTP Dataflow in restrictive environments	47
2.4.2. SOCKS Interface	47
2.4.3. Web-Usable Latency + Bandwidth	47
2.4.4. Secure Encryption	47
2.4.5. Secure Authentication	47
2.4.6. Passive Firewall Evasion	47
2.4.7. Active Probing Resistance	47
2.4.8. MITM Resistance	48
2.4.9. User Interface	48
2.4.10. Region Lock	48
2.5. Limitations	49
2.5.1. Security	49
2.5.2. Obscurity	49
2.5.3. Integrity	49
2.5.4. Bandwidth	49
2.5.5. Latency	49

2.5.6. User Experience and Audience	49
2.6. Requirements	50
2.6.1. Hardware	50
2.6.1.1. Client	50
2.6.1.2. Server	50
2.6.2. Software	50
2.6.2.1. Client	50
2.6.2.2. Server	50
2.6.3. Internet Connection	50
2.6.3.1. Client	50
2.6.3.2. Server	50
2.7. Success Criteria	51
3. Design	52
3.1. Decomposition	52
3.1.1. Plan	52
3.1.2. Client	52
3.1.2.1. User Interface and GUI	52
3.1.2.2. SOCKS Listener	52
3.1.2.3. SOCKS Handler	53
3.1.2.4. SOCKS Protocol Spec	53
3.1.2.5. Data Coordinator	53
3.1.2.6. Congestion Control Apparatus	53
3.1.2.7. Server Hello	53
3.1.2.8. Data Upload	53
3.1.2.9. Data Download	53
3.1.3. Server	55
3.1.3.1. Configuration Loader	55
3.1.3.2. Server Listener	55
3.1.3.3. Session Management	55
3.1.3.4. Client Hello	55
3.1.3.5. Region Lock	55
3.1.3.6. Congestion Control Apparatus	55
3.1.3.7. TCP Relay Connections	55
3.1.3.8. Data Coordinator	56
3.1.3.9. Metadata Handler	56
3.1.4. Shared	57
3.1.4.1. Cryptographic Module	57
3.1.4.2. Encoding, Decoding, and Protocol Specification	57
3.1.5. Decomposition Diagram	58
3.1.6. Code Structure	59
3.2. Definition	60
3.2.1. Core Operation	60
3.2.1.1. Diagram	60
3.2.1.2. Lifecycle of a request	61
3.2.2. Algorithmic Definitions	61
3.2.2.1. User Interface	62
3.2.2.2. SOCKS Listener	62

3.2.2.3. SOCKS Handler	62
3.2.2.4. Data Coordinator	62
3.2.2.5. Congestion Control Apparatus	64
3.2.2.6. Server Hello	64
3.2.2.7. Region Lock	67
3.2.2.8. Web Server	67
3.2.2.9. Server Configuration	67
3.2.2.10. Identification	68
3.2.3. Completionism	68
3.2.4. Tool Choice	68
3.2.4.1. Language	68
3.2.4.2. Operating System	68
3.2.4.3. Integrated Development Environment	68
3.2.4.4. Documentation and Planning	68
3.3. Usability Feature Justification	69
3.3.1. User Interface Design	69
3.3.1.1. General Layout	70
3.3.1.2. Typography	70
3.3.1.3. Accessibility	70
3.3.1.4. Client Sockets	71
3.3.1.5. Client Coordinator	71
3.3.1.6. Upload Workers	71
3.3.1.7. Client Information	71
3.3.1.8. Request Indicators	71
3.3.1.9. Server Information	71
3.3.1.10. Ingress Request Handlers	72
3.3.1.11. Server Coordinator	72
3.3.1.12. Server Sockets	72
3.3.1.13. Server Egress Request Handlers	72
3.3.1.14. Nationality Indicator	72
3.3.1.15. Modifiability Indicators	72
3.4. Code Identification	73
3.4.1. Heirarchal Breakdown	73
3.4.2. Validation	75
3.5. Test Identification	76
3.5.1. Synthetic Testing	76
3.5.2. Automated Testing	76
3.5.3. Real-World Testing	76
3.6. Further Data	77
4. Development	78
4.1. Iterative Development	78
4.1.1. SOCKS Format	78
4.1.2. Early Message Serialisation	78
4.1.3. Security Considerations	80
4.1.4. libsecrets	80
4.1.5. Client-Transit	84
4.1.6. Server-Side Configuration Management and Server Initialisation	86

4.1.7. Lock-Free Data Structures	87
4.1.8. Client Identification	88
4.1.9. Server-Side State Storage	88
4.1.10. Server-Side Connection Process	89
4.1.11. Integrated Testing	91
4.1.12. Threaded Decision Making	93
4.1.13. Client Hello Improvements	96
4.1.14. Additional Implementation	97
4.1.15. Rapid Serialisation Research	99
4.1.15.1. Log Round-Trip Data	99
4.1.15.2. Mesh Round-Trip Data	99
4.1.15.3. Minecraft World Save Round-Trip Data	100
4.1.15.4. Mk48 Round-Trip Data	100
4.1.15.5. Results	101
4.1.15.6. Serialisation Benchmarks	102
4.1.16. Transit Socket Progress	102
4.1.17. Testing	102
4.1.18. Socket Creation	102
4.1.19. TCP Implementation	103
4.1.20. Log information	103
4.1.21. Preliminary Data Transfer	106
4.1.22. Client Side Decisions	108
4.1.23. SOCKS Progress	109
4.1.24. Server Split	109
4.1.25. Passing Passers	110
4.1.26. Continued Development	110
4.1.27. Client Refactors	111
4.1.28. Rapid Development	111
4.1.29. Initial Testing	113
4.1.30. Success	118
4.1.31. Continuing Development	119
4.1.32. Iterative Testing	120
4.1.33. Messages	125
4.1.34. Architectural Changes	126
4.1.35. GUI Research	128
4.1.36. Socket Termination	128
4.1.37. Implementing the Yellow Route	130
4.1.38. Continued Termination	131
4.1.39. CLI	131
4.1.40. GUI	132
4.1.41. Refactoring	133
4.1.42. Stability	133
4.1.43. Better Logging	133
4.1.44. Metadata Handling	133
4.1.45. Improved Circumvention	133
4.1.46. False Download MIME Types and Headers	134
4.1.47. False Upload MIME Types and Headers	134

4.1.48. Client Hello False MIME Types	134
4.2. Code Appendix	136
4.2.1. 01: SOCKS Definition	136
4.2.2. 02: Multiple Messages	139
4.2.3. 03: Criterion	146
4.2.4. 04: Initial Encryption Lib	148
4.2.5. 05: Transit Socket Builder	149
4.2.6. 06: Early Server	152
4.2.7. 07: Integration Tests	156
4.2.8. 08: New Serialisation	158
4.2.9. 09: Modified Benchmark	163
4.2.10. 10: Socket Creation	165
4.2.11. 11: TCP Handler	167
4.2.12. 12: TCP Handler Rewrite	169
4.2.13. 13: New Tests	172
4.2.14. 14: Client Init	173
4.2.15. 15: Client Transit	176
4.2.16. 16: Core Client	179
4.2.17. 17: Filled Transit	183
4.2.18. 18: New Transit	190
4.2.19. 19: New TCP Handling	195
4.2.20. 20: Client Side Split TCP	198
4.2.21. 21: HypertunnelGUI	200
5. Evaluation	205
5.1. Evaluative Testing	205
5.1.1. Usability Testing and Evidence	205
5.2. Testing Evidence with respect to Success Criteria	207
5.2.1. CLI Operation	207
5.2.2. Real-World Circumvention	207
5.2.3. Meaningful Communications	207
5.2.4. Long-Term Reliability	207
5.2.5. Server Authentication	207
5.2.6. MITM Invulnerability	207
5.2.7. GUI Operation	208
5.2.8. Memory-Safe Implementation	208
5.3. Usability Features and Success	209
5.4. Maintenance	210
5.5. Limitations and Improvements	211
5.5.1. Underdeveloped GUI	211
5.5.2. Connection Reliability	211
5.5.3. Region Lock	211
5.5.4. SOCKS5 Support	211
5.5.5. BIND Support	211
5.5.6. UDP Support	211
5.5.7. Better Request Loss Handling	211
5.5.8. Dynamic Modetime	211
6. Conclusion	212

7. References	213
8. Code Appendix	217
8.1.1. client-core::transit.rs	217
8.1.2. server-monolith::new_transit.rs	228
8.1.3. server-monolith::main.rs	237
8.1.4. client-core::lib.rs	246
8.1.5. hypertunnel_gui::app.rs	253
8.1.6. libsocks::lib.rs	258
8.1.7. client-core::transit_builder.rs	262
8.1.8. benches::multiple_messages_benchmark.rs	265
8.1.9. libtransit::lib.rs	267
8.1.10. libtransit::tests.rs	270
8.1.11. libsecrets::lib.rs	272
8.1.12. client-cli::main.rs	274
8.1.13. hypertunnel_gui::main.rs	276
8.1.14. benches::decryption_benchmark.rs	277
8.1.15. server-monolith::config.rs	278
8.1.16. benches::structs_vs_strings.rs	279
8.1.17. benches::encryption_benchmark.rs	280
8.1.18. hypertunnel_gui::Cargo.toml	281
8.1.19. benches::allocate_benchmark.rs	282
8.1.20. libsecrets::Cargo.toml	283
8.1.21. server-monolith::Cargo.toml	284
8.1.22. client-core::Cargo.toml	285
8.1.23. client-cli::Cargo.toml	286
8.1.24. client-core::meta.rs	287
8.1.25. libtransit::Cargo.toml	288
8.1.26. benches::key_derivation_benchmark.rs	289
8.1.27. integration-tests::Cargo.toml	290
8.1.28. Source::Cargo.toml	291
8.1.29. libsocks::Cargo.toml	292
8.1.30. integration-tests::lib.rs	293
8.1.31. hypertunnel_gui::lib.rs	294
8.1.32. server-monolith::server.config.toml	295
8.1.33. Source::test_curl.sh	296

INTRODUCTION

Worldwide Censorship and Circumvention

The Internet allows anyone, anywhere, with a computer and a complete internet connection to talk to anyone else with those same resources. This is a powerful tool for freedom: It enables the free exchange of ideas, knowledge, and culture - as well as the exchange of less noble data.

For a variety of reasons, many internet connections are not complete. They do not allow full access - they are censored. These reasons range from the reasonable and innocuous (preventing students from accessing inappropriate content) to draconian (as an oppressive component of a totalitarian regime) [1], [2], [3].

Either way, since the very first network censors have existed, there have been people trying to circumvent them. The techniques of censorship and circumvention have rapidly advanced in a continually evolving arms race. In 2006 the most advanced censorship apparatus at the time, the Great Firewall of China (GFW), could be bypassed by simply bilaterally ignoring the injected TCP connection resets [4]. By 2011 the GFW was actively probing suspected servers [5] and by 2021 it was analyzing the entropy of traffic to catch fully-encrypted protocols [6]. And permitted protocols aren't safe either: The GFW has analysed the exterior dataflow of SSH connections using machine learning (ML) to detect and block SSH tunnels since at least 2016 [7].

The approach has changed too. Where censors were previously content with a single way of blocking each protocol, they have now moved to a more comprehensive approach based on defense in depth. The GFW implements "Triplet" DNS censoring [8], [9], multiple layers of HTTPS filtering [10], and the aforementioned entropy analysis, among other techniques. The Iranian National Information Network has imposed a similar approach of layered protocol analysis [11].

But just as the means of censorship have grown more sophisticated, so have the measures to defeat them. Early systems based on simple SOCKS proxies and primitive attempts at full-encryption have been phased out in favour of domain fronting [12] and fully encrypted protocols like Shadowsocks [13]. As censors began to block Shadowsocks through entropy analysis, circumventors introduced entropy modifications to their encryption algorithms to carry on [6]. Western research groups have pioneered the use of genetic algorithms [14] to automate the process of finding flaws in the censors [15], and there are many dozens of smaller tools that aim to avoid the attention of the counter-circumvention developers through unique approaches.

1.1.1. Deny-By-Default

All of the current generation of circumvention tools operate under a flawed assumption: They assume that censors do not decrypt TLS traffic, and that they only block that which is positively identified as a circumvention tool, rather than blocking everything that they cannot understand.

In more concrete terms, they assume that a firewall will look at a stream of random data on a random port and allow it to pass provided it cannot find anything positively identifying it as a circumvention tool.

Long-term, this is a flawed assumption. Firewalls exist - though not yet at the nationwide scale - that operate under a regime of deny-by-default and TLS decryption.

Sophos XG [16] is a real-world example of that approach. While it maintains a blacklist, not a whitelist, for websites that can be visited, it enforces this by blocking all traffic that it cannot decrypt or understand.

It achieves this through stateful analysis of data in transit. Data going over port 80 must be valid HTTP going to a website not on the blacklist - or it's blocked. Data going over port 443 must be valid, decryptable HTTPS going to a website not on the blacklist - or it's blocked. It enforces this through a mandatory client-side certificate that allows it to MITM all HTTPS traffic - if you haven't installed the certificate, and it fails to intercept and compromise your TLS handshake, the TCP connection is immediately killed through TCP RSTs and complete packet loss.

Any traffic going over a non-approved port gets blocked. This means that the traditional fully-encrypted protocols like Shadowsocks [13] no longer work: They don't look *wrong*, but they also don't look *right*, so no traffic is allowed.¹

If the GFW (or other censorship apparatus) started acting like this, all of the standard circumvention measures would be blocked.

The GFW hasn't yet taken this approach because of the amount of collateral damage it would cause. Chinese businesses rely on far more than just HTTP egress, even if the average Chinese citizen doesn't, and creating a whitelist of businesses and protocols that would work for an entire country would be extremely expensive. It would also require significantly more computational power than the current iteration of the GFW. But attitudes are changing: Entropy analysis has gone from a trigger for active probing [18] to a reason for full, prolonged blocking of a IP address [6]. The GFW is moving towards a deny-by-default approach, and it may only be a matter of time before it gets there.

1.1.2. Another Line Of Defense

So what can we do to prevent this? And if it does happen, what can we do to circumvent it?

If there were some circumvention tools capable of operating under a deny-by-default firewall, it would tip the equation against these drastic measures. If deny-by-default only reduced performance, rather than blocked entirely, there would be far less incentive to implement it. These last-ditch tools would need to operate through not just highly sophisticated firewalls, but also ones that create highly restrictive environments. Environments where, in the most extreme cases (like Sophos XG), not even Server-Sent Events [19] or Websockets [20] are allowed because of how easy it is to implement real-time bidirectionality with them.

This document details the development of such a tool. It is intended to work under highly restrictive environments, where many of the standard circumvention tools fail due to a very limited set of egress routes. If possible I intend to make it work under highly sophisticated environments, where blocking involves machine learning, active probing, and deep-packet-inspection, though that is a secondary feature. Performance is not a concern, and it is not designed to survive human analysis. I will expand on its design, features, and limitations in the following sections.

¹They do, actually, as I'm sure you know - but "magic ports" aren't guaranteed. Their lack of guaranteed availability is the reason I created the first prototype [17] of this approach in 2021.

1.1.3. Disclaimer

Due to my position as an amateur, this tool is *designed*, but not *intended*, for real-world use. This is the case throughout the document, both before and after this message, it would just get very verbose to make it clear everywhere. While I have written as if it were intended for real world use in China, I don't actually intend for it to be used in that way - it just makes for a very interesting scenario. I will design it to function in as adversarial an environment as possible, under the constraints outlined above, and I will try my best to test it in such an environment, but it should not be used in the real-world. It is a proof-of-concept school project. The only place it will ever be used in the real world is by me, at school, if the existing flaws in Sophos XG are patched.

I intend to implement a region-lock that restricts use to the west. It would be able to be removed by modifying the source code (As I may in order to test it in China).

ANALYSIS

Why Computation?

The core problem that I am trying to solve is how to get large amounts of data to and from anywhere in the world, securely, in real time, without censorship. By solving this problem, people will be able to communicate freely and learn without restriction.

This is a problem that is fundamentally suited to computers: They work well with large amounts of data and they are fast. In fact, given the “real time” requirement, it is hard to imagine a solution that does not involve computers. In order to achieve worldwide communication of large amounts of data in real time, we need a worldwide digital communications network - made up of computers.

It’s also a problem which has been solved before, through computational methods: The Internet. The Internet allows anyone, anywhere, with a computer and a complete internet connection to talk to anyone else with those same resources.

The problem comes when you don’t have a *complete* internet connection - as outlined earlier, internet censorship is commonplace. This computational problem has historically been solved by computers: The communication network and data transfer is already there, you just need a way to obfuscate and secure that data during transit to avoid censorship. I will continue to use this well-tested and effective approach of using computational methods to solve the problem of computer-enforced censorship.

2.1.1. Computational Methods

Abstraction

I intend to use abstraction to solve the problem of arbitrary data transit by exposing a generic, abstract API (conforming to the SOCKS4 spec) that allows any application to use my system without knowledge of the internal workings.

Decomposition

Hypertunnel will have a lot of shared logic between the client and server. I will decompose the problem into shared libraries for handling things like SOCKS, encryption, and protocol specification so that I can reuse components. This will allow my code to be more maintainable, easier to understand, and faster to develop.

Concurrency

Concurrency is an important part of making programs like this viable. I will use the Tokio asynchronous engine to ensure network handling occurs in a non-blocking, resource-utilising manner. I will apply selective multithreading using the Rayon library in areas where it tangibly improves performance.

Stakeholders

Hypertunnel has a variety of stakeholders, each with their own situation and needs. They are as follows:

2.2.1. Students behind restricted networks

Many educational institutions, from primary schools to elite universities, impose restrictions on their student-accessible networks. In many countries, including the UK, these restrictions are legally required for safeguarding purposes [21].

Many jurisdictions also recommend “appropriate monitoring”, in addition to “appropriate filtering” [22].

Restrictions range from those aimed at increasing productivity, such as bans on gaming sites, to essential safeguards, such as bans on politically extreme and sexual content.

Many students want to bypass these restrictions for a variety of reasons of varying legitimacy - most commonly the desire to access social media or games.

In my view, not all of these reasons are illegitimate: For example, a programmer might want a way to circumvent a firewall that blocks access to open source software, or that causes issues with a programming language’s package manager. While social media blocking is entirely reasonable, sites like Reddit [23] contain both completely inappropriate content and essential debugging information. Overblocking is both necessary and harmful - an inevitable result of a varied student population.

Because of the highly restrictive nature of school networks, many school firewalls are difficult to beat with traditional fully-encrypted evasion techniques. Typically only TCP/443 (HTTPS) and TCP/80 (HTTP) are open, which leaves only TCP/443 and TCP/80 as egress vectors. Indeed, many school networks can be circumvented with a traditional VPN or fully-encrypted protocol operating on one of those ports via TCP.

However, many schools have started to use DPI (Deep Packet Inspection) to detect protocols masquerading as HTTP(S) by simply using the correct port and layer 4 protocol. This means that many traditional techniques are no longer effective. However, tools that operate inside legitimate HTTP(S) traffic remain effective. This includes software like TrojanGFW [24] and Websocket tunnels [25], [26].

These techniques are effective because they’re indistinguishable from legitimate HTTPS traffic when viewed from the outside, with HTTPS encryption hiding the content of the request. However, this all falls apart when mandatory TLS root certificates allow a firewall to see inside the encryption: The firewall can block websockets wholesale and trivially identify TrojanGFW traffic as invalid. These firewalls can also buffer requests, preventing a thin layer of “HTTP requests” from being used to tunnel data.

My solution would be able to circumvent those TLS-decrypting firewalls by using actual valid HTTP(S), without relying on request streaming or websockets. Data masquerading as file uploads and downloads is sent in discrete chunks inside full, valid HTTP(S) requests according to a transit scheduling algorithm. This would allow students to bypass the firewall and freely access the Internet.

This makes school networks an ideal technical environment for Hypertunnel, as they are real-world examples of highly-restrictive networks that are difficult to circumvent with traditional techniques that are designed under the assumption of more permissive firewalls.

These students could make use of the software solution by deploying it themselves, or having a technical friend deploy it for them, and then using it from a laptop to access the Internet. They could also circumvent on their phones if they point the phone's proxy settings at the software solution running on the same LAN. The student demographic is quite broad, so while some students (e.g. computer science ones) may find it trivial, others may find it difficult.

There is an ethical issue with providing my tool publicly in this context. I believe that through publishing the source code, but not providing a publicly-accessible server instance, I can limit any usage to technical users who would have found a circumvention solution either way. Additionally, my software only supports laptops and desktops, which are not typically used by younger / less technical students. I believe that this is a reasonable compromise between the ethical issues of providing a circumvention tool to students and the benefits of publishing my work.

2.2.2. Dissidents and activists in oppressive regimes

See Section 1.1.3 . The following is a hypothetical scenario.

As described in the introduction, many countries around the world censor the internet for political reasons. Freedom House found that global internet freedom has declined for the last 12 consecutive years [27]. Their reports claim that only 25% of the assessed countries had a free internet, with 45% being partially free and 30% being not free.



Figure 1: Recent attacks on internet freedom. Credit: Freedom House

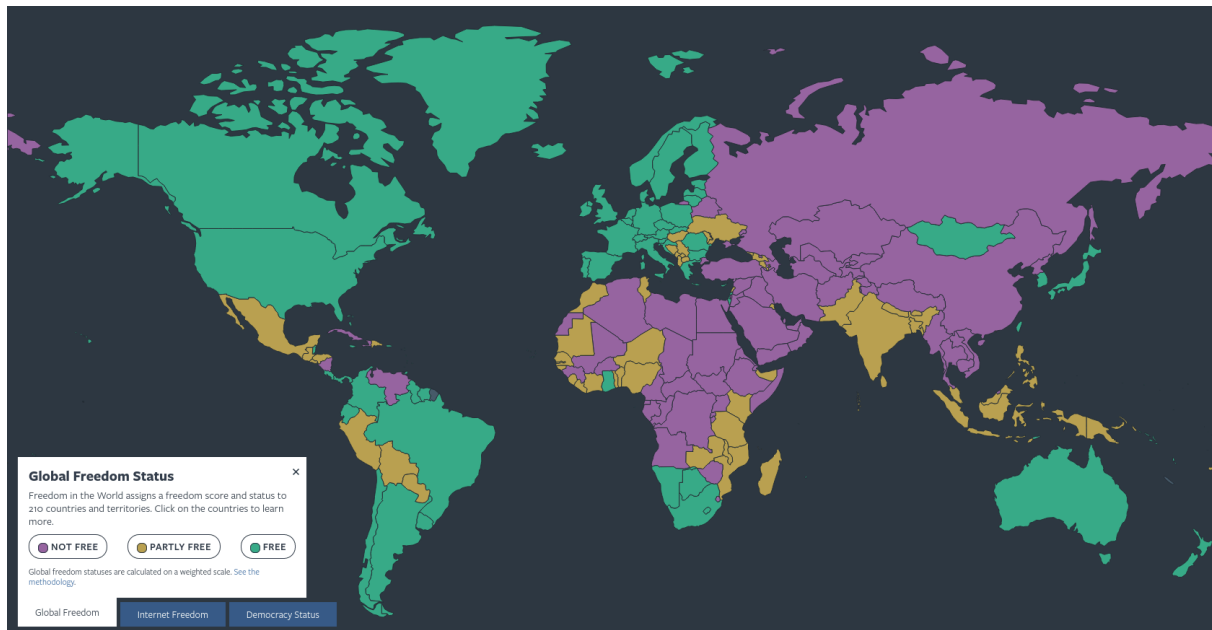


Figure 2: Internet freedom 2023 Heat Map. Credit: Freedom House

Most of these countries employ relatively primitive censorship methods - from driving tech companies out of the country to simply shutting down the internet altogether in times of unrest [28], [29]. The methods used to censor websites and services typically include DNS, hostname and IP blocking - all of which can be circumvented with a VPN.

However, countries are increasingly employing more sophisticated methods of censorship. Some, such as Iran, Egypt, and China, have established effective censorship apparatus capable of blocking non-specialised software while continuously working to beat the state-of-the-art circumvention tools. These countries also have a tendency to make their filtering more restrictive around political events, either as a show of force or to suppress protests. This is particularly true in China, where the government has previously blocked methods previously thought secure during the annual National People's Congress, the 100th anniversary of the CCP (Chinese Communist Party), and the Tiananmen Square anniversary [30], [31].

My program would provide a second line of defense, allowing dissidents, activists, and those who simply want to access the internet freely the ability to circumvent stricter censorship methods. By blending in with standard internet traffic it would place a higher bar of false positives on the censor, making it more politically costly to block, even during times of political tension. It would have significantly worse performance than standard methods due to its greater computational and network overhead, but it would be enough for those who need it.

2.2.3. Circumvention tool developers

A wide range of circumvention tools have been developed, exploiting a wide variety of methods. From the fully-encrypted protocols that “look like nothing”, to novel tunneling methods through ICMP [32] and DNS [33], the circumvention community is constantly looking for new techniques.

My system is designed to work in particularly locked-down environments, using a novel method that (as far as I can tell) has never been used before. While it has extensive limitations and shortcomings, it offers a unique set of capabilities that could be of interest to circumvention tool developers. While my tool is not designed for real-world use (See Section 1.1.3) it

proves the concept of a broader strategy that could be used by the circumvention community to develop a new class of real-world circumvention tools.

This group would typically be more interested in the theoretical workings of the system than the technical implementation. They are extremely confident with the relevant technology.

2.2.4. Rust developers

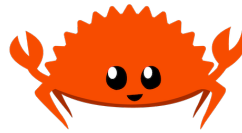


Figure 3: Ferris, Rust's mascot.

Rust is a relatively new systems programming language that is designed to have the performance of C with the safety and ergonomics of a high-level language. It achieves this through extensive borrowing of ideas from functional programming languages and academia, a strong type system, and a plethora of ownership-based memory management rules that are enforced by the compiler. This makes it perfect for my use case, where I need to write a large, high-performance, yet safe program that can be easily maintained and extended.

Its status as a new language means that it is still rapidly evolving, and as such does not have libraries for everything. My program includes large amounts of isolated library code (`libsocks` for example) that can be used by other Rust developers. Once open-sourced my program could provide reference and library code for Rust developers and the wider Rust ecosystem.

Rust developers are typically highly technically competent, and so they should not have any issues with the complexity of my solution. They would be more likely to read the program than to use it personally.

Research

In this section I will research the problem of firewall circumvention by analysing its history. I will use the history of the Great Firewall of China (henceforth “GFW”) as a case study, looking at its censorship methods and the techniques developed to circumvent them. As I go through each technique I will develop a picture of how the GFW has developed, culminating in its current architecture. Finally, I will use this knowledge to develop a circumvention technique that will penetrate the GFW and similar systems.

I will be looking at China in particular because it has one of the oldest, most advanced, and most well-known firewalls in the world. It has been the primary target of circumvention techniques for many years, and has been the primary battleground of the cat-and-mouse game between censors and circumventors. It is also a good example of a firewall that is not just a simple packet filter, but a complex system of many different components.

2.3.1. Internet in China

China’s first email was sent in 1987 - 16 years behind the United States. It contained the message that “Across the Great Wall we can reach every corner in the world”. Internet access was initially reserved for academics and public officials. It took until 1995 for it to be opened to the general public, but the CCP was enthusiastic, declaring 1996 the “Year of the Internet”. Internet users grew rapidly due to the proliferation of internet cafes.

2.3.2. Socialist Market Economy

The advent of the Internet coincided with China’s pivot to an economic doctrine of state capitalism and a “socialist market economy”. Despite this move towards a free market economy and opening of foreign investment, the CCP felt that the one-party totalitarian political apparatus had to be preserved. Deng Xiaoping, chairman at the time, described this attitude with the famous quote that “If you open the window, both fresh air and flies will be blown in” [34]. The CCP wanted a filter that could allow economic liberalisation to flow freely while blocking political liberalisation that could threaten the party’s power.

2.3.3. CL97

This attitude was formalised in the 1997 “Regulations on the Administration of Internet News Information Services” (CL97) law. The law intentionally left the implementation “flexible” so that it could be expanded in the future. In 1998, a software engineer was sent to prison for forwarding 30,000 Chinese email addresses to a US pro-democracy magazine [2]. The CCP hoped that internet users would self-censor due to high-profile arrests and awareness of constant monitoring.

2.3.4. Golden Shield

CL97 brought a political mandate for a new digital censorship and monitoring system. Fang Binxing, a Chinese computer scientist, led the technical development with extensive help from Cisco [35], [36]. This early firewall was primarily used for domestic monitoring and censorship, with little focus on international traffic. It was primarily run by civilians, with oversight from the Public Security Bureau (PSB).

The Golden Shield Project: Public Network Information Security Monitor System

Cisco.com

- **Stop the network-related crimes**
- **Guarantee the security and services of public network**
- **Combat “Falun Gong” evil religion and other hostiles**

[Note: Statement of Government goals from speech government official Li Runsen]

© 2002, Cisco Systems, Inc. All rights reserved.

Cisco Systems, Inc. Confidential

57

Figure 4: Leaked Cisco internal slide showing their involvement in the Golden Shield project [35], [36]

By 2002 the Golden Shield was blocking at least 18,000 websites [37]. The Chinese mouth-piece Xinhua stated that only “superstitious, pornographic, violence-related, gambling, and other harmful information” was blocked, yet 12 out of the top 100 websites, including gmail, were included in the list. Later that year Google was blocked entirely [38]. A censored version, `google.cn`, would be launched in 2005.

2006 brought the finalisation and deployment of the first version of the Great Firewall. This full capability brought the ability to track and observe internet usage centrally and from local bureaus, as well as the ability to block and filter traffic.

In the same year, Internet cafes were regulated by the CCP. Minors were banned from Internet cafes, and they were required to keep a log of customer accesses. This spurred the creation of underground “Dark Web Bars” for under-18s.

2.3.5. Early Architecture and Circumvention

The early GFW was sophisticated for its time, but by modern standards it was relatively simple. Websites were detected by either their destination IP address or symmetric keyword detection of then-commonplace unencrypted HTTP traffic [4]. The firewall also hijacked DNS requests to return false IP addresses if the requested domain was blocked.

One of the earliest papers on the Great Firewall, “Ignoring the Great Firewall of China” [4], from the University of Cambridge, both investigated its architecture and found an ingenious flaw in this early design.

It found that the firewall operated as a “network tap”, splitting traffic into two streams: One to the user, and one to the firewall. The network tap would then identify which TCP streams should be blocked. However, the traffic would have already passed through the other end of the network tap. Delaying traffic until it had been analysed would have caused unacceptable latency, and creating a blocking rule for every illegal connection would have also been unacceptably slow. Instead, the side-channel detection system (suspected to be based on Cisco’s intrusion detection systems) would inject TCP RSTs into the stream to end the connection.

This approach had multiple flaws. Packets would sometimes arrive before the RST, allowing data through. The authors also found that they could simply ignore the firewall by modifying their TCP stack to ignore RSTs. This was the first technique specifically created around the firewall’s design weaknesses. It was unfortunately not a general solution, as it required both the server and client to make spec-breaking modifications to their TCP stacks. Users would also still be flagged, as the firewall would still know what they’re viewing, it would just be unable to stop them.

The researchers also found that the firewall was “stateless” - it analysed each packet independently - and that through forged TCP packets with intentionally illegal content they could cause the firewall to block arbitrary IP address pairs in a denial of service attack. They calculated that a single consumer PC could block approximately 120 000 pairs of endpoints. Such an attack would also limit the firewall’s ability to censor other users.

Using this information I built a diagram of the early GFW’s architecture:

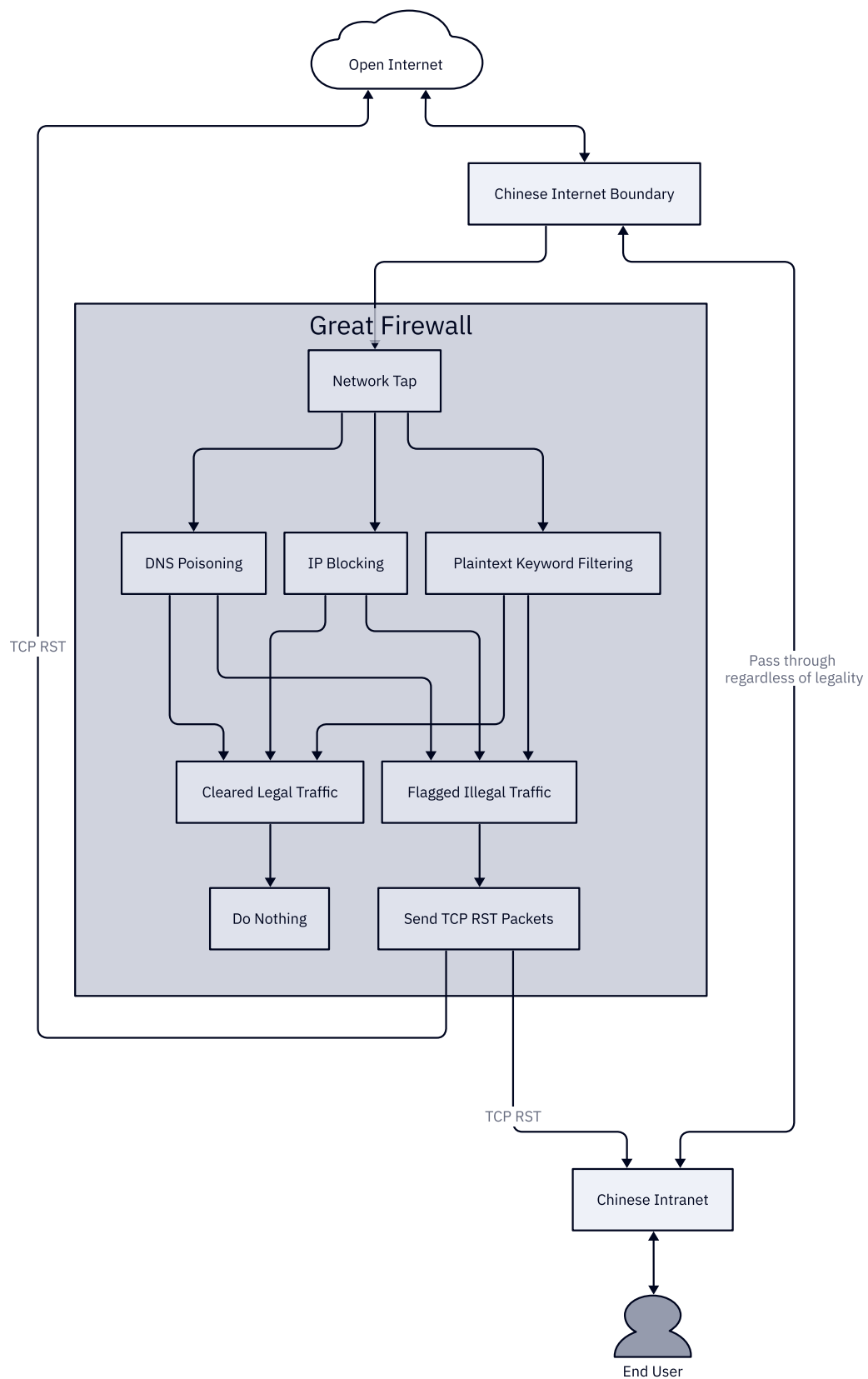


Figure 5: Approximation of the early GFW architecture

I can only learn a limited amount from this early architecture, given how far it is from the current state of the firewall. However, it informs an understanding of the underlying design. It is likely that this early parallel tapping design is still the basis of the firewall's architecture, with the addition of more advanced filtration and termination systems. IP, DNS, and keyword filtering are all still used, and the firewall will still inject RSTs, but the firewall now has backup termination methods and far more developed filtration systems.

2.3.6. Increasing Sophistication 2006-2012

The period of 2006-2012 saw the Great Firewall's architecture develop rapidly. It went beyond simply blocking websites to protecting this blocking capability through the shutdown of circumvention connections. Its sophistication increased significantly over this period.

This period saw the increased usage of HTTPS, which encrypts the vast majority of information in a HTTP connection. However, the Server Name Indicator, or SNI, remained in cleartext. The GFW was modified with the ability to block based on SNI, but the firewall did not block HTTPS entirely.

In a strange deviation from the trend of increasing sophistication, this period also saw the firewall be used to shut down all traffic in the Xinjiang province in response to the 2009 Urumqi riots [39].

This period also included Operation Aurora [40], a series of cyberattacks on Google and other US companies in order to steal intellectual property and the information of Chinese activists. Google announced that it was "no longer willing to continue censoring" content in China, and redirected all traffic to `google.cn` to `google.com.hk`, which was not censored. This was blocked by the GFW, and Google eventually shut down its Chinese operations entirely. All Google products are now blocked in China.

In late 2011, the GFW began to use active probing to identify circumvention attempts. This first operated on SSH [41], making strange requests to servers shortly after a successful connection. However, there were no observations of this being followed up by blocking. This was quickly expanded to Tor, where TLS fingerprinting [42], [18] was used to identify likely Tor connections. The GFW would then follow up with active probing, sending a request to the server to see if it was running a Tor bridge [43]. The probes were also run in near real-time. Additionally, blocked bridges would be continuously re-tested; they remained blocked as long as they were running.

Previous attempts to block Tor had worked by HTTP header filtering and simple IP blocks [44] - a straightforward and easily circumvented approach [45], [46]. This new approach was far more sophisticated.

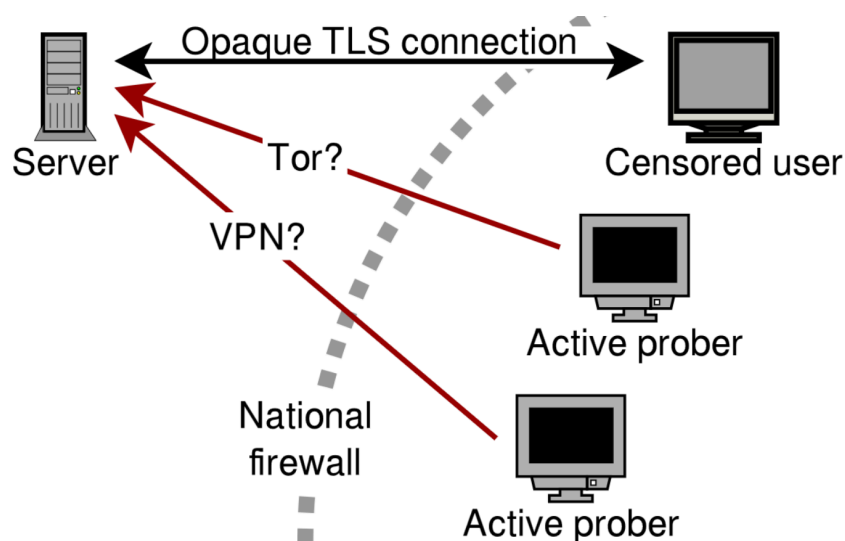


Figure 6: Active probing process [47].

Tor made several attempts to make its TLS fingerprint blend in, but quickly found that it was a losing battle. In 2012, it switched to pluggable transports, thin wrappers over normal traffic. The first of these was obfs2, designed to work around Iranian and Chinese DPI. It's a very simple protocol, which first sends a symmetric encryption key and then encrypts all later data with that key. While this worked against non-specialised censors (at the time; nowadays entropy analysis would catch it), it was trivial to detect by speculative decryption according to the protocol to see if there was TLS inside. [47]

Its censorship capabilities also increased. It began with a TCP RST, but after a short delay it would switch to a full block on all traffic.

Using this information I can build a diagram of the early GFW's architecture:

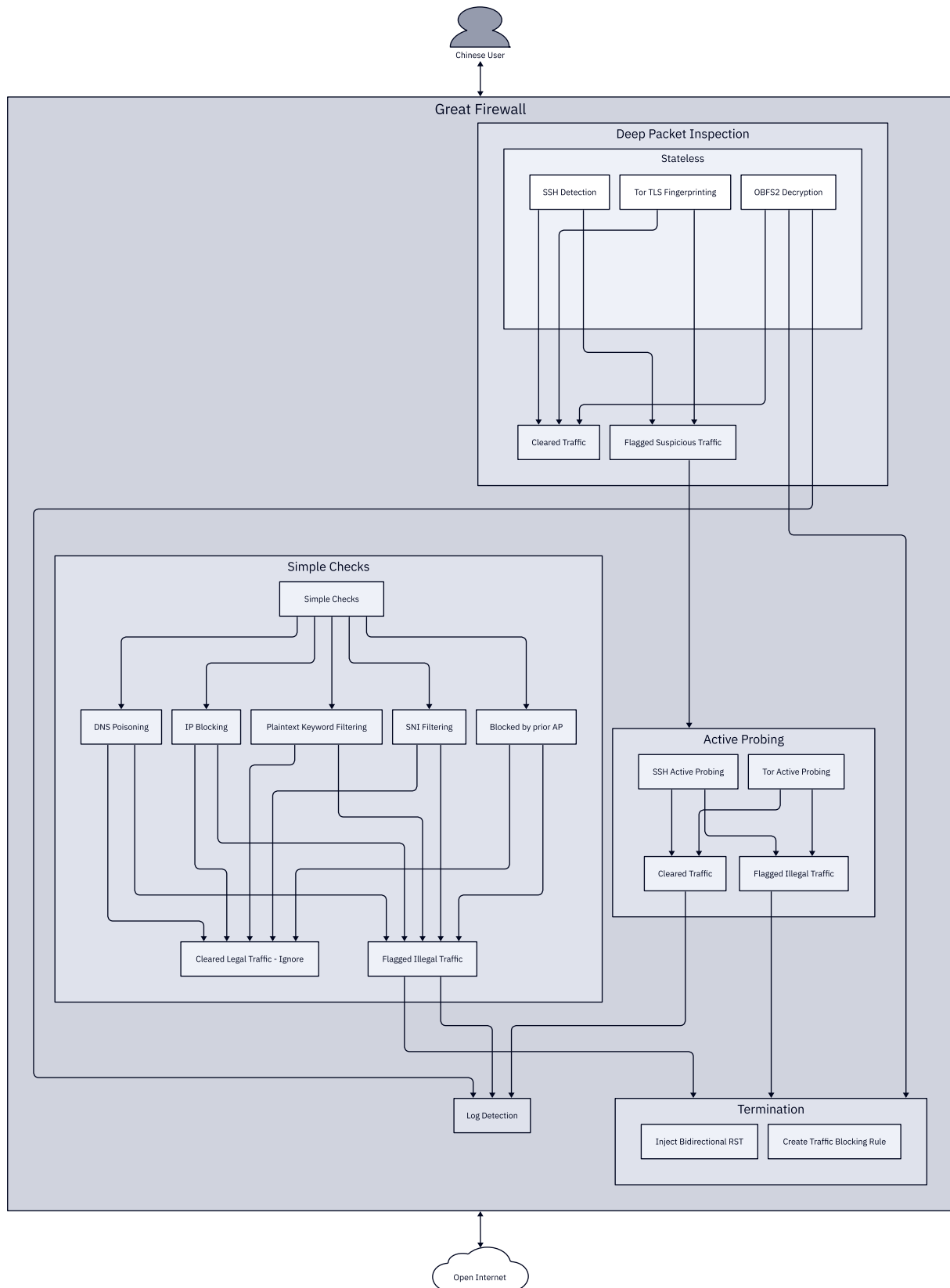


Figure 7: Approximation of its 2012 architecture. Some information about the ingress/egress network taps and periodic probing information sharing has been removed to make the text legible. I recommend zooming.

I can learn a lot from how the firewall developed in this period. It is clear that active probing is an important part of the firewall's architecture, and something to continuously defend against when developing circumvention tools. TLS fingerprinting is almost impossible to defend against, so my system must avoid having a unique signature as a matter of design, not implementation. Deep packet inspection works well at scale, and is a core part of all modern firewalls. Finally, circumvention of termination features is not a viable long-term strategy, as systems that don't rely on TCP-RSTs are now the norm.

2.3.7. Continued Development 2012-2015

In 2013, the Great Firewall began an intrusive Man-In-The-Middle (MITM) attack on Github [48], [49] via a self-signed-certificate injected into the TLS stream. This resulted in users receiving a security warning when they tried to visit the website which, if bypassed, would allow the Chinese government to view their communication with github. The suspected cause was a white house petition calling for those who built the firewall to be denied entry to the US. This linked to a Github gist [50] containing a list of Chinese individuals accused of being involved in the firewall's development.

Github had been blocked a week before, but following an unusually public protest [51] full access was restored. Github is in an unusual position for China: Blocking it fully causes significant economic issues for the tech industry - most of which knows how to circumvent anyway - but it also contains social and political content that the government would prefer to block. They can't block selectively because of its use of HTTPS, so they seem to have tried a MITM attack in order to get that additional information.

It doesn't seem to have worked, because the MITM ended just an hour later. The original list was removed, but a related one also linked in the petition [50] remains to this day.

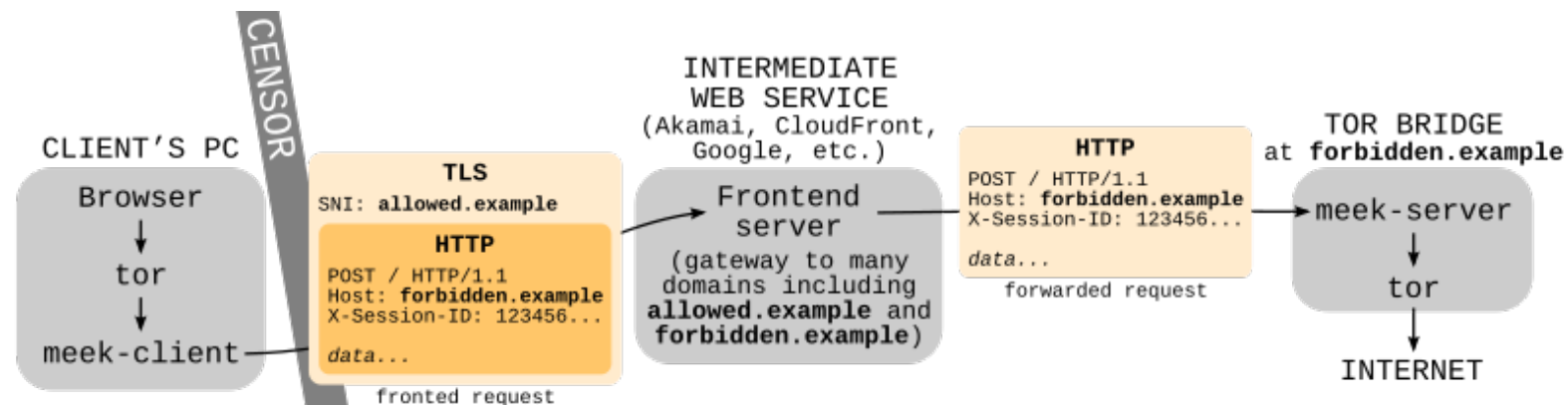


Figure 8: Meek's domain-fronting architecture [12]

In 2014, amid continuing blocks of Tor, the Tor project released the `meek` pluggable transport. Meek works through domain fronting. Domain fronting exploits multiple different aspects of the networking stack and internet infrastructure to circumvent censorship.

In a HTTPS request, the destination domain name appears in three places: The DNS query, the TLS Server Name Indication, and the HTTPS Host header. Additionally, much of the internet is run on CDNs: Content Delivery Networks. These serve many millions of different domains from a single IP address for a variety of reasons, including caching and DDOS protection. These two factors combined mean that you can send a DNS request for another (permitted) website on a CDN, then send a TLS SNI for that permitted website, then once the encrypted connection is established send a HTTPS request with the blocked website in the Host header. The CDN will then forward the request to the blocked website, and return the response to you.

There are four ways to stop this:

1. Use TLS fingerprinting to identify the connection. This is possible, but difficult, to defend against.
2. Block the entire CDN, and deal with the collateral damage.

3. Convince all CDNs you cannot feasibly block to stop allowing domain fronting (it is detectable). This can be done through threats of blocking, cyberattacks, or legal action against entities in your jurisdiction.
4. Require all HTTPS be able to be decrypted using a mandatory client certificate. This is a very invasive and difficult to implement change at the national scale, and would require immense cooperation and coercion.

China's ongoing strategy is based on 2 and 3, although many more restrictive firewalls, including Sophos XG [16], use the fourth method. Implementing the fourth method would also allow China to view all interior content, as well as the MITM attacks on Github - and later iCloud - to work without the users knowledge.

Later that year there were further MITM attempts against iCloud [52], [53], [54], [55], Google [56], and Yahoo [57]. A HTTP traceroute confirmed that these attacks occurred within GFW infrastructure.

Finally, 2014 brought additional information about the GFW's DNS censorship. An anonymous paper, "Towards a Comprehensive Picture of the Great Firewall's DNS Censorship", was published [58]. It discovered approximately 15000 blocked DNS keywords and was able to probe the composition of the GFW's censorship infrastructure. It discovered that censorship nodes only occurred on the border of Chinese infrastructure, with the majority of internal requests not passing through it, in addition to the internal architecture.

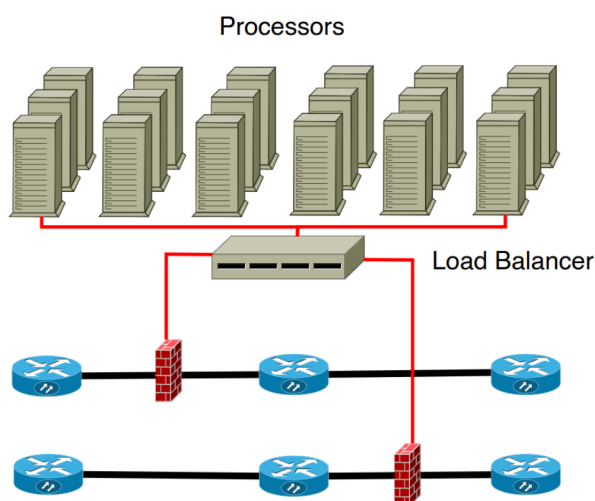


Figure 9: Internal structure of a GFW censorship node [58].

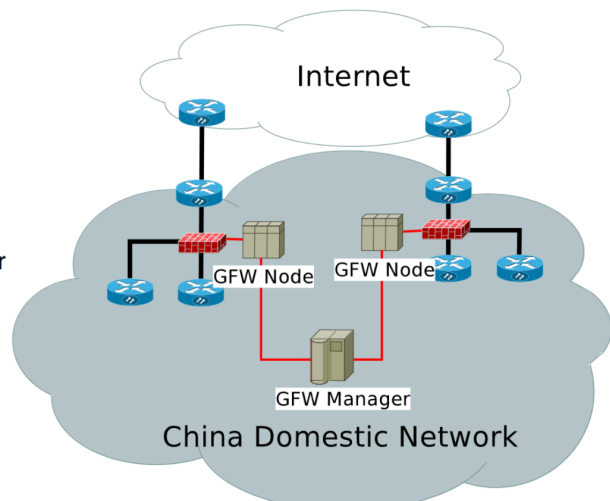


Figure 10: Overall architecture [58].

In 2015, the GFW became more than just a system for censorship: It became an offensive weapon. China wanted two anti-censorship projects removed from Github - GreatFire and cnytimes - but because of the aforementioned SSL and economic dependence on Github, they were unable to block them.

In an unprecedented move, the Chinese government launched a new kind of DDOS attack, later dubbed the "Great Cannon" [59]. It worked by injecting malicious javascript into unencrypted requests for Chinese company Baidu's analytics script in about 1.75% of requests transiting the firewall. This script would then repeatedly attack the targeted website. With enough people affected, this allowed the Chinese government to turn average users into a botnet. Additionally, because Chinese users did not transit the firewall for internal connections they would not be affected.

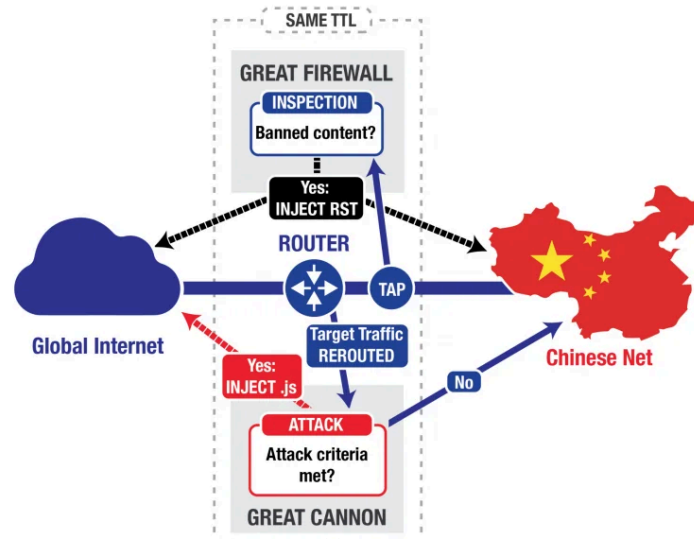


Figure 11: Topology of the Great Cannon [59].

Several aspects of the attack were completely unprecedented. First, this attack affected international users visiting websites hosted in China as well as Chinese users. This was an enormous foreign policy move, turning completely disconnected foreign citizens into unknowing pawns in a politically motivated attack. China essentially abused its status as an internet-connected country to attack other countries. Additionally, this first use was incredibly public, and it was targeted towards a US-based company. Its use caused a diplomatic incident, with the US State Department regarding it as an attack against US infrastructure [60]. There were calls to disconnect China in retaliation. This also directly damaged trust towards Baidu, a major Chinese company, despite Baidu not being complicit in the attack.

Github struggled to keep the site online, with highly variable performance [61].

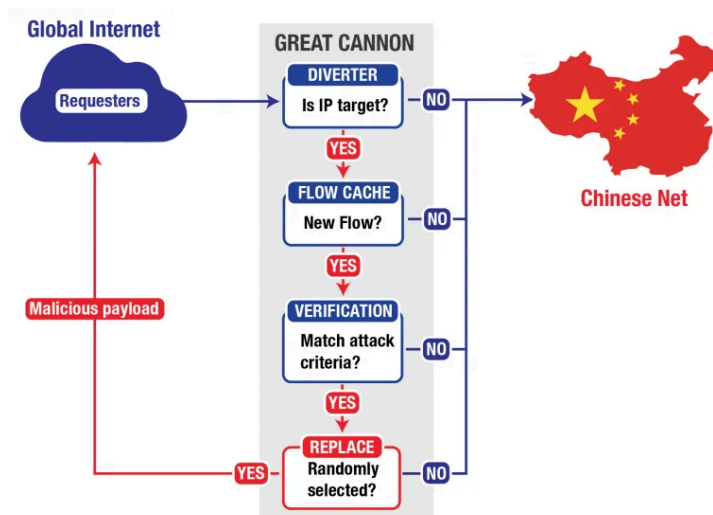


Figure 12: Probabilistic, targeted injection logic [59].

Technically, the Great Cannon is physically colocated with Great Firewall infrastructure. However, there are multiple differences in implementation. First, the GC operates in-stream and was able to *drop* requests in real time, a capability that the GFW does not have. This was observed through Citizen Lab retransmitting the exact same packet following an injection and

receiving a valid (non-injected) response from Baidu [59]. If the original packet had arrived, the second packet would have been dropped.

The Great Cannon is also stateless, and acts only on the first data packet of a connection. It does not reassemble packets, unlike the GFW.

It's also probabilistic: Most packets do not receive injections. The GFW, on the other hand, aims for 100% blocking.

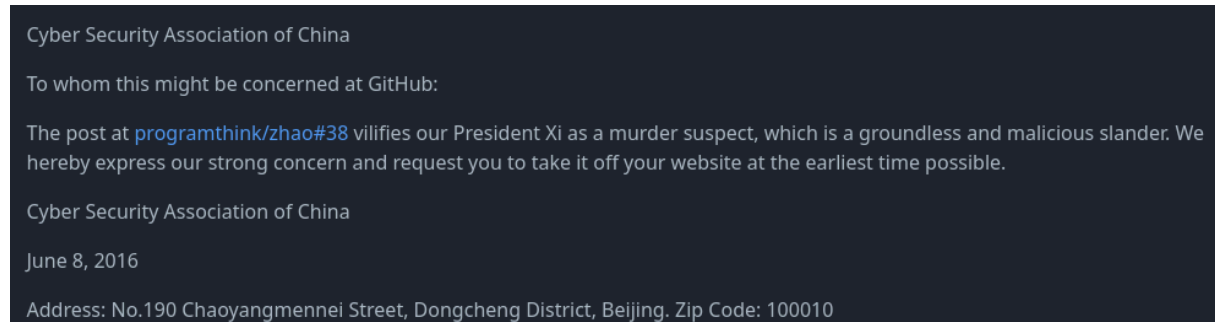


Figure 13: China asks nicely [62]

Github did not concede to the Chinese government's (implicit) demands, and the projects remained. In 2016 China tried another unprecedented strategy - *sending a strongly worded email* - and was able to have a Github gist accusing Xi Jinping of murdering his brother promptly blocked for Chinese users [62].

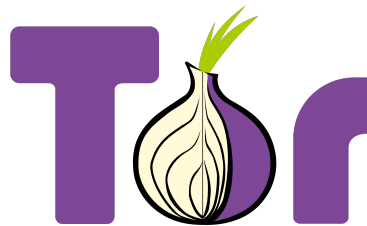


Figure 14: Tor Project logo.

A paper released in 2015 [47] detailed how the GFW was able to block Tor's obfs3 proxy, which was first defined in 2013 [63] and solved many of the issues with obfs2 through the use of diffie-hellman key exchange. The authors performed "counter-probing", discovering that the active probing infrastructure's dynamic IP space was shared with residential IP addresses. Past active probe sources were found to be running standard residential Windows installations.

The paper also found that certain periods, perhaps when filters were being updated, allowed unobfuscated Tor to work for a short period around once every 25 hours.

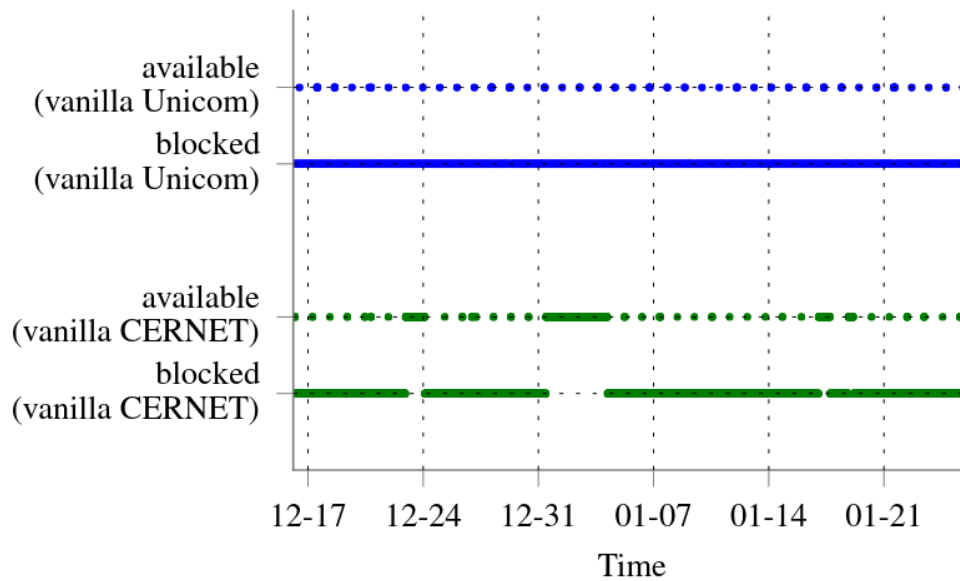


Figure 15: Graph of vanilla Tor connections from Chinese “Unicom” and “CERNET” occasionally working [47].

The authors also found that the GFW had moved from the earlier 15-minute queuing system to real-time probing. 56% of all probing connections arrived in less than a second, although four took over 800.

It also found that there were five distinct protocols being probed for, with one that they could not explain: Unexplained TLS, Tor, Obfs2, Obfs3, SoftEther, and AppSpot.

The researchers also set out to test whether the DPI sensors were stateless - do they process TCP segments in isolation, or do they consider TCP connection state? They found that the sensors were stateful - a naked packet without a prior three-way handshake would not trigger a probe. This differs from the stateless Great Cannon, likely because the Cannon operates in-stream with high latency requirements while the DPI operates on a network tap.



Figure 16: Shadowsocks logo.

But while the developers of the Great Firewall and the circumventors fought over protocol analysis and fingerprinting, the CCP was applying a less honourable approach. In 2015, the creator of Shadowsocks [13], an emerging circumvention tool based on full encryption, was identified and contacted by the police [64].

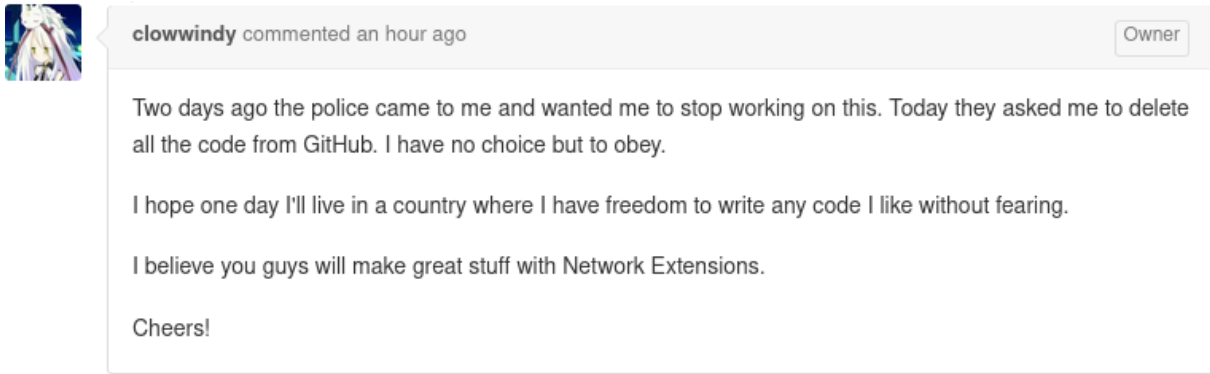


Figure 17: [64]

They deleted all of the code in the github repository and disappeared from working on circumvention tools. It appears that Clowwindy is OK, but unable to continue working on the project. Their Twitter [65] remains relatively active, with Clowwindy discussing gaming, machine learning, and other topics. The community carried on development using an offline copy of the repository.

Shadowsocks went on to become one of the most high-profile, widely-used circumvention tools. It remains at the forefront of GFW circumvention to this day, and is a key part of the circumvention ecosystem.



Figure 18: Outline VPN [66] logo.

Shadowsocks became such a major part of the circumvention ecosystem that a separate project, Outline VPN [66], was created by a Google-backed technology incubator to make it more accessible to non-technical users. It replaces the relatively confusing configuration files and command line arguments with a simple, intuitive interface.

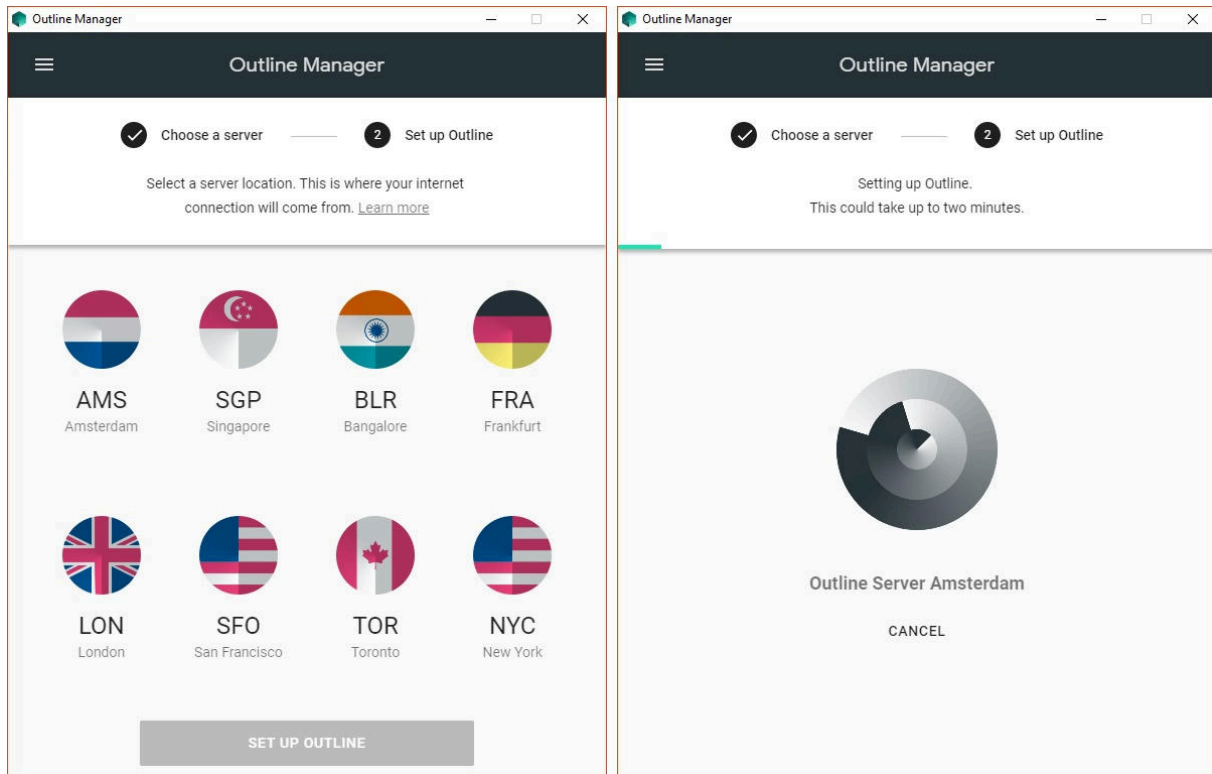


Figure 19: Outline VPN interface [67]

It has a clear, unintimidating interface that mimicks common consumer app design principles. I intend to use this to inform my UI design usability features - in particular, Outline's interface will inform the:

- Font usage
- Whitespace/Padding usage and quantity
- Colour scheme
- General look and feel of the application

An architecture diagram can be built based on what I know of the GFW in this period.

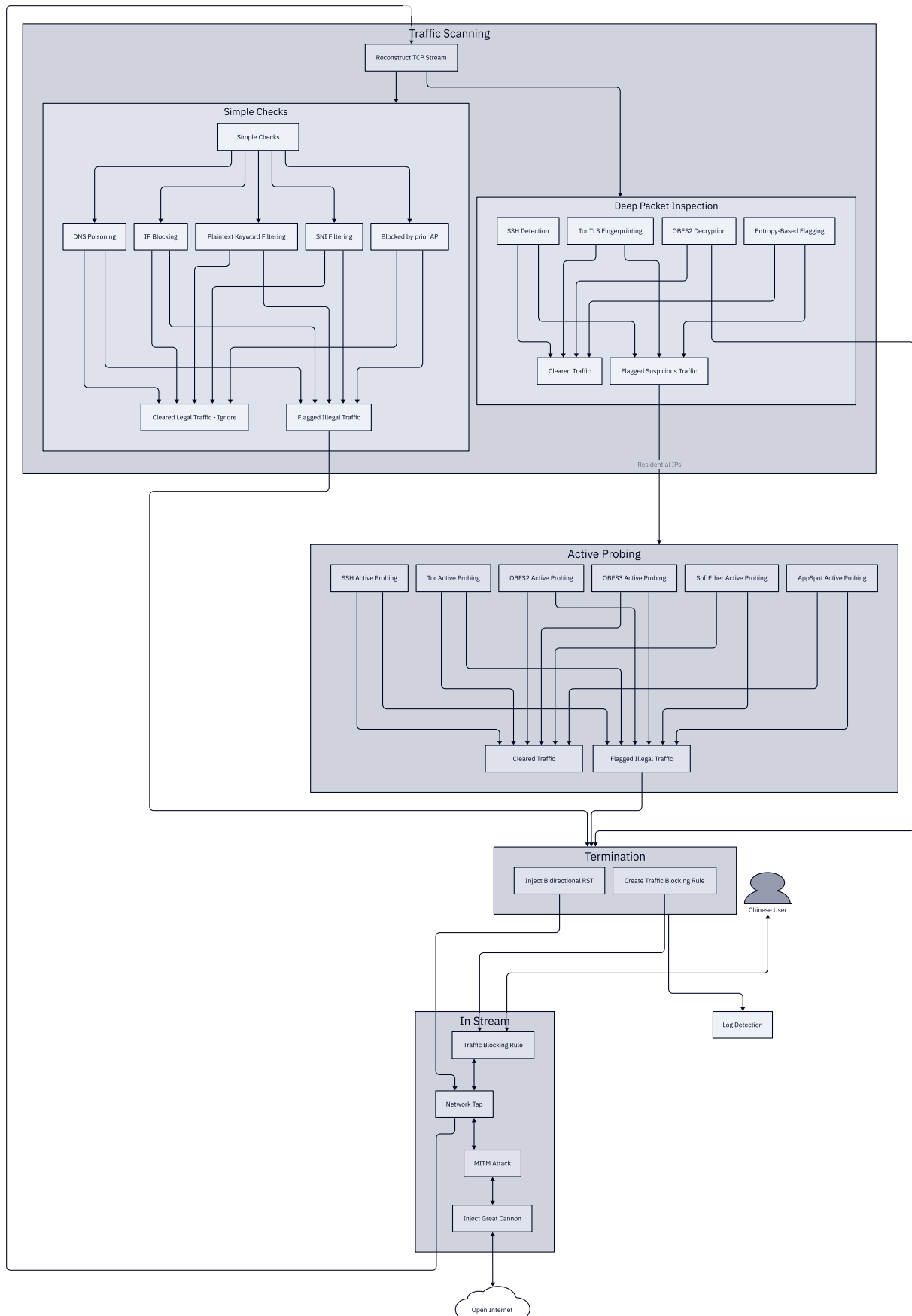


Figure 20: Approximation of its 2015 architecture. It's an SVG - you can zoom.

The lessons I learn from following the history of the GFW grow more relevant as I move closer to the current state of firewalls and circumvention techniques. I can take lots of directly relevant lessons for Hypertunnel from this period. I know that MITM attacks - something my system is directly designed to counter - are something that is within the Great Firewall's capability, and that could be deployed more widely in the future. I now know that active probing is not just a thing, but that it can come from residential IP addresses - it must be defended against through design and technical implementation, not simple IP blocks. I also know that sometimes blocking can go down, but that it's short-lived and likely due to updates. Domain fronting is an interesting strategy, but it's not perfect - and it's not to be relied on, with the quantity of attack vectors against it. Finally, reflection/amplification DDOS attacks are something actively employed by the GFW, and something that my system must be able to defend against.

2.3.8. Modern Developments 2015-Present

In 2016 a user on the Tor mailing list reported that Sophos Cyberoam firewalls had released a DPI engine update capable of blocking Tor, obfs3, obfs4, and Scramblesuit [68]. Another update about 5 weeks later added the ability to block Meek connections without even using HTTP MITM-ing. They investigated with David Fifield, a prolific anti-censorship developer, and found that it was blocking based on a combination of Meek's TLS signature and the three front domains (`www.google.com`, `a0.awsstatic.com`, `ajax.aspnetcdn.com`) [69]. This was not deployed in China at the time, but it demonstrates the ongoing issue of TLS fingerprinting. Additionally, my software will be tested against another Sophos firewall, so it is important to understand their capabilities.

Also in 2016, the great firewall's deep-packet-inspection expanded its ability to detect interior tunnels. A westerner on a business trip tried a series of unorthodox methods of escaping the firewall [7]. First he tried a standard SOCKS-over-SSH proxy, finding that whatever transited it caused severe QoS and packet loss within minutes. He then tried a web proxy exposed using SSH port redirection, but ran into the same problem. It's clear that the Great Firewall is capable of detecting bulk SSH traffic.

He then tried a secure web proxy via `stunnel` [70]. He had to use authentication, but the "407 Proxy Authentication Required" error would expose it upon active probe and TLS client authentication would likely cause a flag. To solve this he created a simple obfs-2 style system in Python based on a header-stored symmetric key; the details are irrelevant. What he found was that the GFW did not detect this system when HTTP traffic transited, but that it could when HTTPS traffic did. The reason for this can be seen through a packet capture. Here's what the firewall sees with a HTTP connection:

```
1 C: TCP SYN to proxy
2 S: TCP SYN+ACK reply from proxy
3 C: TCP ACK
4 C: ClientHello (86 bytes)
5 S: ServerHello, Certificate, ServerHelloDone (67+858+9 bytes)
6 C: ClientKeyExchange, ChangeCipherSpec, encrypted Finished (267+6+53 bytes)
7 S: NewSessionTicket, ChangeCipherSpec, encrypted Finished (207+6+53 bytes)
8 C: encrypted ApplicationData #1 (37+197 bytes)
9 S: encrypted ApplicationData #2 (37+693 bytes)
```

This looks like a normal HTTPS connection, and since he had just built a novel system, the obvious flaws of TLS fingerprinting and custom active probing were not immediately problematic.

On the other hand, here is a HTTPS connection:

```
1 C: TCP SYN to proxy
2 S: TCP SYN+ACK reply from proxy
3 C: TCP ACK
4 C: ClientHello (86 bytes)
5 S: ServerHello, Certificate, ServerHelloDone (67+858+9 bytes)
6 C: ClientKeyExchange, ChangeCipherSpec, encrypted Finished (267+6+53 bytes)
7 S: NewSessionTicket, ChangeCipherSpec, encrypted Finished (207+6+53 bytes)
8 C: encrypted ApplicationData #1 (37+197 bytes)
9 S: encrypted ApplicationData #2 (37+69 bytes)
10 C: encrypted ApplicationData #3 (37+325 bytes)
11 S: encrypted ApplicationData #4 (37+3557 bytes)
```

```

12 C: encrypted ApplicationData #5 (37+165 bytes)
13 S: encrypted ApplicationData #6 (37+85 bytes)
14 C: encrypted ApplicationData #7 (37+149 bytes)
15 S: encrypted ApplicationData #8 (37+853 bytes)

```

There's a problem here: The packet sizes and order betray an interior TLS handshake. This has already been demonstrated to be an effective attack, with the first paper on it being published that same year in 2016 [71].

C: encrypted ApplicationData 1 (37+197 bytes):

```
CONNECT www.google.com:443 HTTP/1.1\r\nHost:... \r\nUser-Agent:... \r\n\r\n
```

S: encrypted ApplicationData 2 (37+69 bytes):

```
HTTP/1.1 200 Tunnel established\r\n\r\n
```

This section is larger than expected due to record headers, AES alignment, and a MAC (Message Authentication Code).

C: encrypted ApplicationData 3 (37+325 bytes):

This contains the TCP client hello, which is typically 200-300 bytes but can be lower based on the size of the cipher suites.

S: encrypted ApplicationData 4 (37+3557 bytes):

This contains the server hello, certificates, etc.

C: encrypted ApplicationData 5 (37+165 bytes):

This contains the client key exchange, change cipher spec, and encrypted "Finished".

S: encrypted ApplicationData 6 (37+85 bytes):

ChangeCipherSpec and encrypted "Finished".

C: encrypted ApplicationData 7 (37+149 bytes):

This contains the encrypted HTTP request.

S: encrypted ApplicationData 8 (37+853 bytes):

This contains the encrypted HTTP response.

There are some unusual aspects about this TCP stream: Application data 2 is very short, application data 4 is just the right size for a certificate chain, and application data 6 is tiny - just large enough for a "proxy OK!" message.

It seems likely that machine learning is involved in order to detect these clear patterns indicating an interior TLS handshake. Further evidence of this is given by a 2008 paper demonstrating its real-time viability [72] and the fact that a simple random padding addition to each packet worked well:

```

1 if len_pkt < 1000:
2     len_pad = randint(1000 - len_pkt, 1500 - len_pkt)
3 else:
4     len_pad = randint(0, 1500 - len_pkt)

```

This change caused the system to work for about a day before being blocked. The block was likely due to the unusually high amount of data transit to a single American VPS. A change of port solved this, indicating that much of the GFW still runs on (IP, port) tuples - even as we know that much of it is moving to simply (IP).

This fascinating discovery means that my system must be prepared for machine learning analysis of packet sizes. It would also be helpful to defend against machine learning analysis of packet contents, as it would allow us to forego exterior TLS and improve performance. I believe my system, with the addition of packet padding on initial hellos, should be defended against both due to my buffering of multiple messages and use of innocent-looking HTTP headers and payloads.

Finally, there is indication that QoS is now a part of the great firewall's termination system. A throttle detection and warning system would be a good addition to my program. I could also apply [73], used in Hysteria [74] (described later) to attempt to "power through" QoS blocking.

A 2018 paper [75] revisited the GFW's active probing. It found that the scanning apparatus had adopted a uniform IP distribution in order to circumvent IP blacklists, though scanners could still be identified via protocol differences.

	Year		
	2012	2015	2018
Block Method	(IP, Port)	(IP, Port)	IP
Block Duration	12 hours	12 hours	12 hours
Scanning Queues	15 minutes	Instantly	Instantly
Scanner Distribution	Not Uniform	Uniform	Uniform
Most Common Scanner	202.108.181.70	202.108.181.70	111.202.242.93
Common Scanner ASes	4134, 4837, 17622	4134, 4837, 7497	4134, 4837, 17676

Figure 21: Temporal comparison of AP architecture [75]

It observed that the GFW blocked connections at the level of IP, not (IP, port) tuples - a different approach to the blocking of inline TLS and data-heavy SSH connections. The GFW also only unblocked a bridge if a subsequent scan at the 12-hour mark indicated no tor bridge - it would not blindly re-open then await another flagged connection to re-check. It would also not scan if a tor connection was unsuccessful, demonstrating increasing statefulness and an attempt to conserve resources.

The paper's most important contribution is that IP blocks, as alluded to before, cannot be relied upon. Additionally, the active probing infrastructure re-probes once every 12h, and you can avoid active probing by making your connection appear to fail.

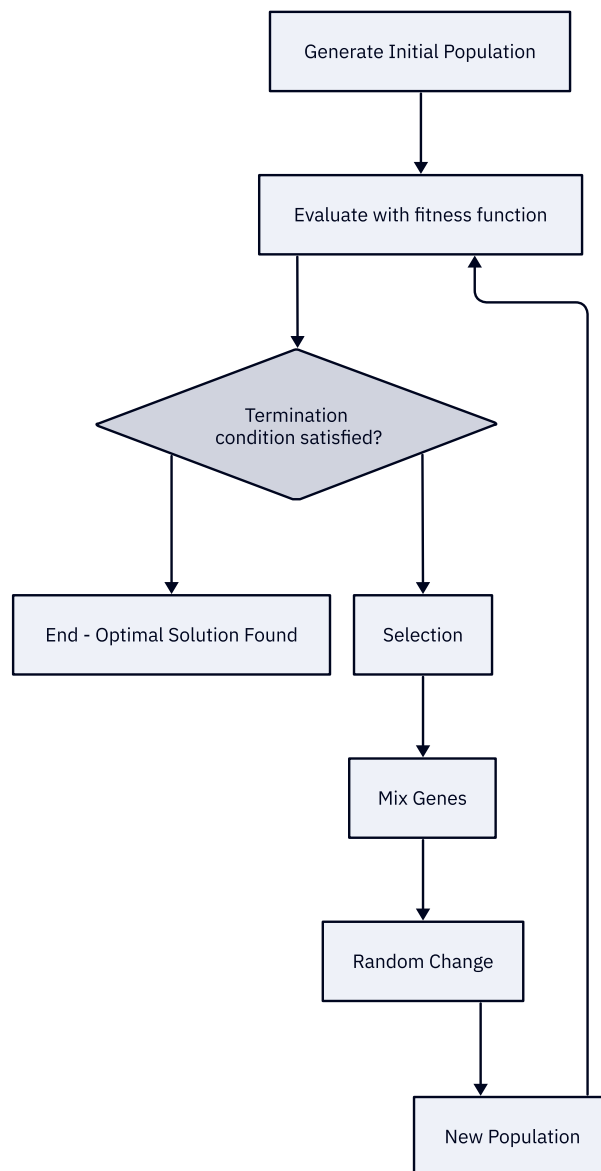


Figure 22: Genetic algorithm training process. Diagram by me, information from [76]

2019 brought an entirely new approach to the creation of circumvention systems. Where previously developers would manually attempt to find and exploit flaws in the censors, the Geneva system [15] applied genetic algorithms [14] in order to automatically find and exploit flaws. The title of the paper is “Geneva: Evolving Censorship Strategies”, and it nicely sums up how it works: It implements evolutionary algorithms to naturally select effective strategies in any environment.

It was also a new approach because it targeted a section of DPI firewalls that previously had little research: TCP flow state tracking. If packets can be crafted such that a normal (or most) TCP implementations will process them fine, but the performance-optimised censorious middleboxes process them differently, then the out-of-phase middleboxes (which operate on copies of the packets) can be tricked into ignoring the connections. Most censors “soft-fail” when hit by something unexpected, and some strategies - such as sending a TCP RST with a

low enough TTL to hit the censor but not the client - won't even be seen as anomalous. The client will ignore the RST, but the censor will delete its record of the connection.

This approach was previously not practical for real-world use because a huge amount of work is required to find a vulnerability, while it is trivial to patch. Geneva automates the process of finding these flaws, therefore making it a viable strategy.

The paper tested Geneva on various firewalls, with incredible results. Here are the novel species/subspecies/variants that were found for each firewall:

	Species	Subspecies	Novel Variants
Great Firewall	4	5	21
Kazakhstan	5	0	0
India	5	0	0

The only already-known species Geneva could not re-find (6/36) were the ones that it did not have the building blocks to express. It was able to re-derive all strategies within hours.

The use of genetic algorithms is necessary for this kind of automated search: One solution, INTANG [77], would require a Geneva tree nine-deep to represent. To enumerate all potential action trees to a depth of 9 would take (conservatively) 2^{89} evaluations. Genetic algorithms make this search depth viable.

The Geneva paper provides the knowledge that genetic algorithms are effective at finding flaws in DPI firewalls, and that TCP flow state disruptions are a viable strategy. While I likely won't need these techniques for Hypertunnel, it is good to know that they exist.

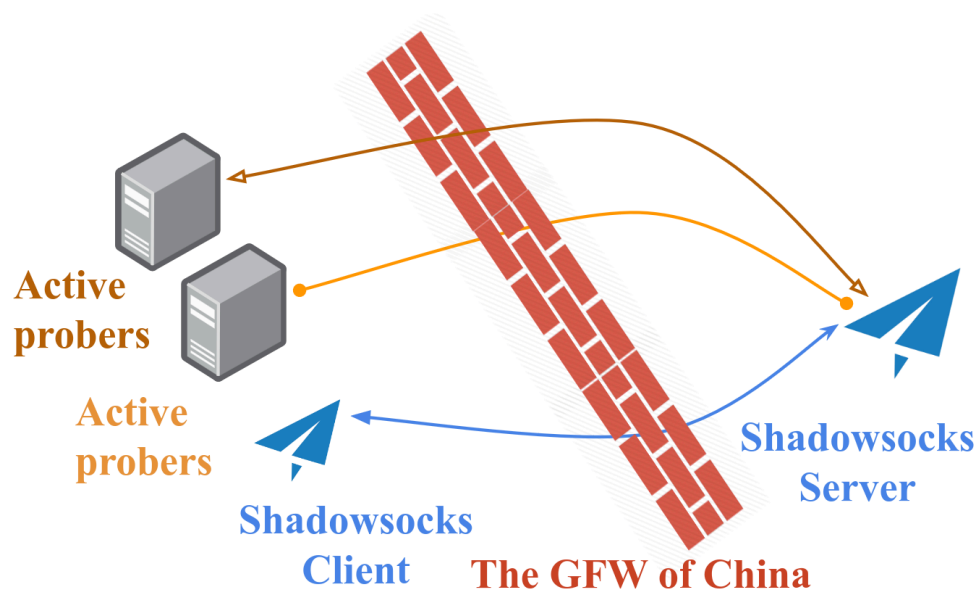


Figure 23: Shadowsocks active probing architecture circa 2020 [5] - active probing originates behind the firewall, not from it.

In 2020, a paper [5] was published that investigated the active probing of Shadowsocks. This was an unusually late development: By 2015 Shadowsocks was one of the most popular circumvention tools, with 20% of polled faculty members at Tsinghua University using it, yet active probing was deployed far later than it was for Tor. This could have been due to its fully-encrypted nature making it harder to flag for active probing, or it could be because the GFW's

development team takes a long time to react to new threats with complex solutions (like near-indistinguishable active probing).

Shadowsocks is built around the principle of full encryption and security through simplicity. The core exterior protocol is a stream of length-prefixed encrypted chunks, with even the length prefixes encrypted to avoid matchable plaintext.

The stream is preceded by a salt, which is combined with the shared secret password to produce a session key. This is similar to how my system works, with the exception that I do not use a salt, and instead have a password per-user.

Shadowsocks has had vulnerabilities in the past. An active probing vulnerability was found in 2015 that exploited its then-use of stream ciphers' lack of integrity protection to carry out a timing attack that identified shadowsocks servers. Additionally, a February 2020 vulnerability used the server as an encryption oracle to break the stream cipher encryption of recorded connections. This was fixed by the introduction of AEAD ciphers, which provide integrity protection.

Shadowsocks has been blocked before for short periods in 2017 and 2018, coinciding with political congresses. Blocking was lifted shortly after, perhaps due to the high amount of false positives caused by earlier methods. These events were not formally researched, but there is near-confirmation of this happening in 2021 [31], so the claims are plausible.

The initial flagging before an active probe appears to only use a small part of the connection, as the probes come very quickly after initialisation. 99% of the replayed packets have a payload length 160-700, so length probably plays a part, but the biggest correlation was with the entropy of the initial traffic.

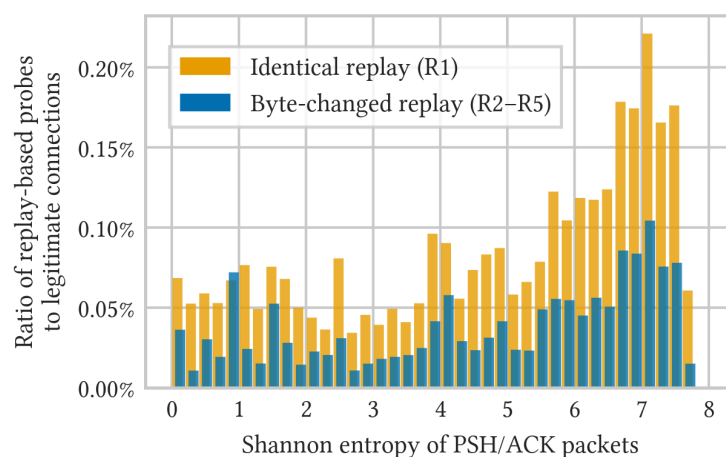


Figure 24: Entropy of the packets vs how frequently they were replayed [5]

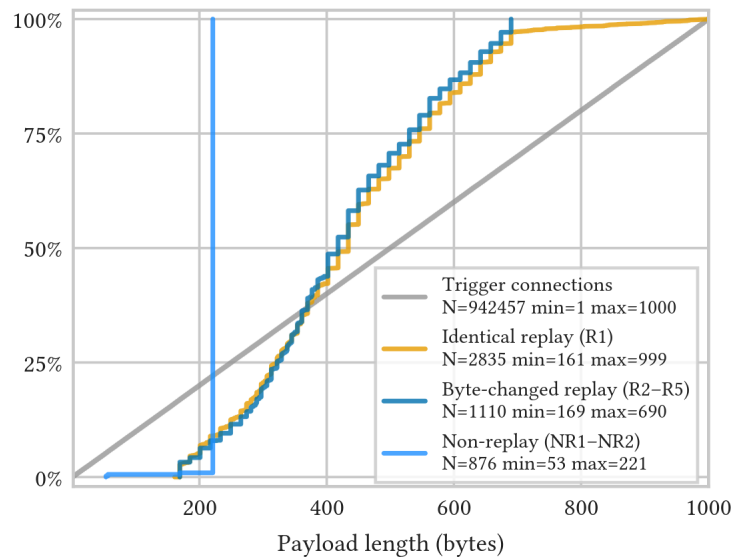


Figure 25: Length of the replayed probes and the triggering connections [5]

The probes themselves don't appear to be colocated with earlier observations, however, they are still sent from residential IPs away from firewall infrastructure.

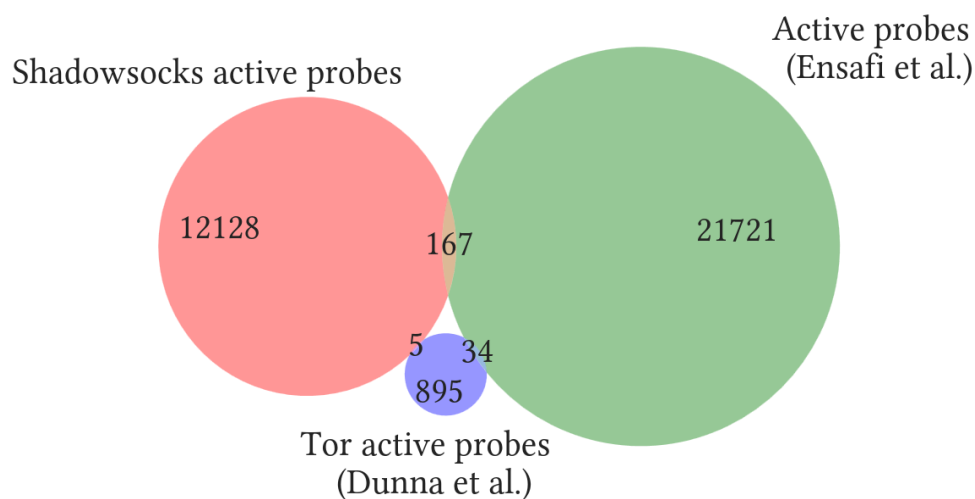


Figure 26: [5]. Ensaft: [47]. Dunna: [75]

Additionally, while previous work found that probe origin ports were evenly distributed across the entire 16-bit range, this work found that 90% of the source ports were 32768-60999, the default range for Linux. None were below 1024. They were also sent faster than previous probes.

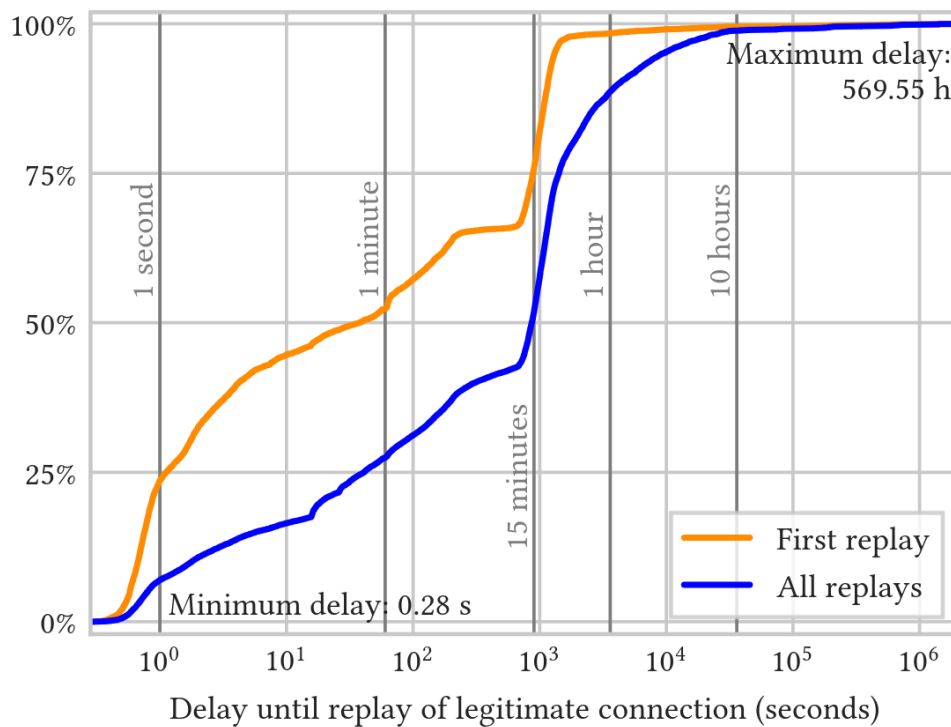


Figure 27: Time before the first probe. Note the logarithmic x axis. [5]

Termination occurred through RST injection and server → client communication blocking. This unidirectionality is similar to how Tor is blocked. However, unlike Tor, there was not a set period of time before a re-scan then unblock. The time was variable, with one node taking over a week to be unblocked, and there were no re-checks before unblocking - likely due to the higher intensity of the probes compared to Tor.

The authors made a how-to to defend against the GFW's active probing [78] which will be helpful for Hypertunnel when I develop active probing mitigation.

The standard way to block replay attacks is through nonces, but this is asymmetric: A server has to store every received nonce, while the GFW has to remember one, and can wait an arbitrarily long period. The solution was a layered system consisting of time-gated responses and short-term nonces. I can apply this approach to my system, alongside the use of AEAD ciphers and the validation that my preshared key approach is effective.

2020 and 2021 brought the interesting development of Beijing universities publishing information on obfs4 detection [79], [80]. Unfortunately I couldn't find anything useful in the papers.

More information about the Great Firewall's DNS infrastructure was published in 2020 [8]. Its findings about the architecture of the firewall's DNS infrastructure are very interesting, but not immediately relevant to Hypertunnel. The only particularly relevant finding is that the GFW still heavily relies on keyword censorship - I need to avoid any keywords that could be used to flag my system. A 2021 paper on the GFWs DNS censorship [9] had similar findings, but it also found that the GFW had a strange approach of replying with a false response but not blocking the actual response. If the client simply waits for the actual response it can bypass (that layer of) censorship. This indicates that the GFW is still quite compute-constrained when it comes to termination - good news for my system, which requires a lot of processing power to positively identify if at all.

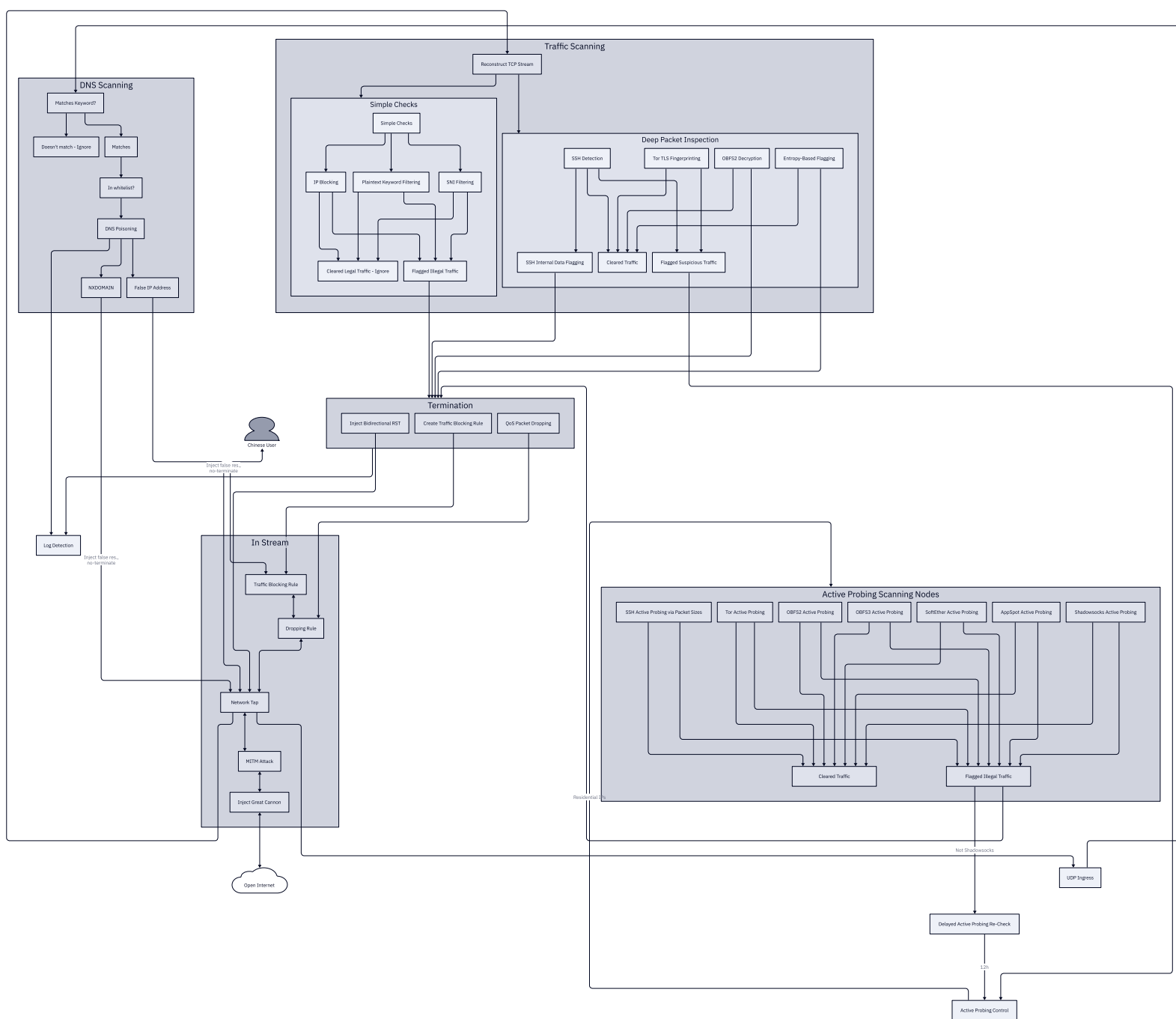
A 2021 paper [10] described how previously-discovered inconsistencies in the GFW's TCP handling was due to the GFW's HTTPS analysis actually being split into two separate middle-boxes. They were able to isolate this middlebox and probe it, finding that it operates a completely parallel stack for redundancy. This indicates that China is still interested in expanding its SNI censorship capabilities. SNI censorship should be made obsolete by encrypted SNI [81] so the continual investment means that China believes its efforts to block encrypted SNI will be successful. Encrypted SNI should not be a component of my system.

Finally, 2023 moved entropy analysis from a trigger for active probing to a wholesale blocking method [6]. Sending traffic with a high entropy caused full blocking of the same 3-tuple (client IP, server IP, server port) for 180 seconds. It has some exceptions:

1. When the entropy is low $\left(\frac{\text{popcount}(pkt)}{\text{len}(pkt)} \leq 3.4 \vee \frac{\text{popcount}(pkt)}{\text{len}(pkt)} \geq 4.6 \right)$
2. The first six or more bytes of `pkt` are `[0x20, 0x7e]`
3. More than 50% of the bytes in `pkt` are `[0x20, 0x7e]`
4. More than 20 contiguous bytes in `pkt` are `[0x20, 0x7e]`
5. It matches the protocol fingerprint for TLS or HTTP

Otherwise, it will block. The `[0x20, 0x7e]` range is the ASCII character range. This should not cause problems for my system because it goes over HTTP, and it has ASCII characters, however if those exemptions are removed it will be blocked. If this happens, I will need to implement the same mitigation Shadowsocks now uses - editing the entropy output of the encryption algorithm to be inside the allowed range, at the cost of overhead.

With this, I can create an approximate diagram of the GFW's operation.



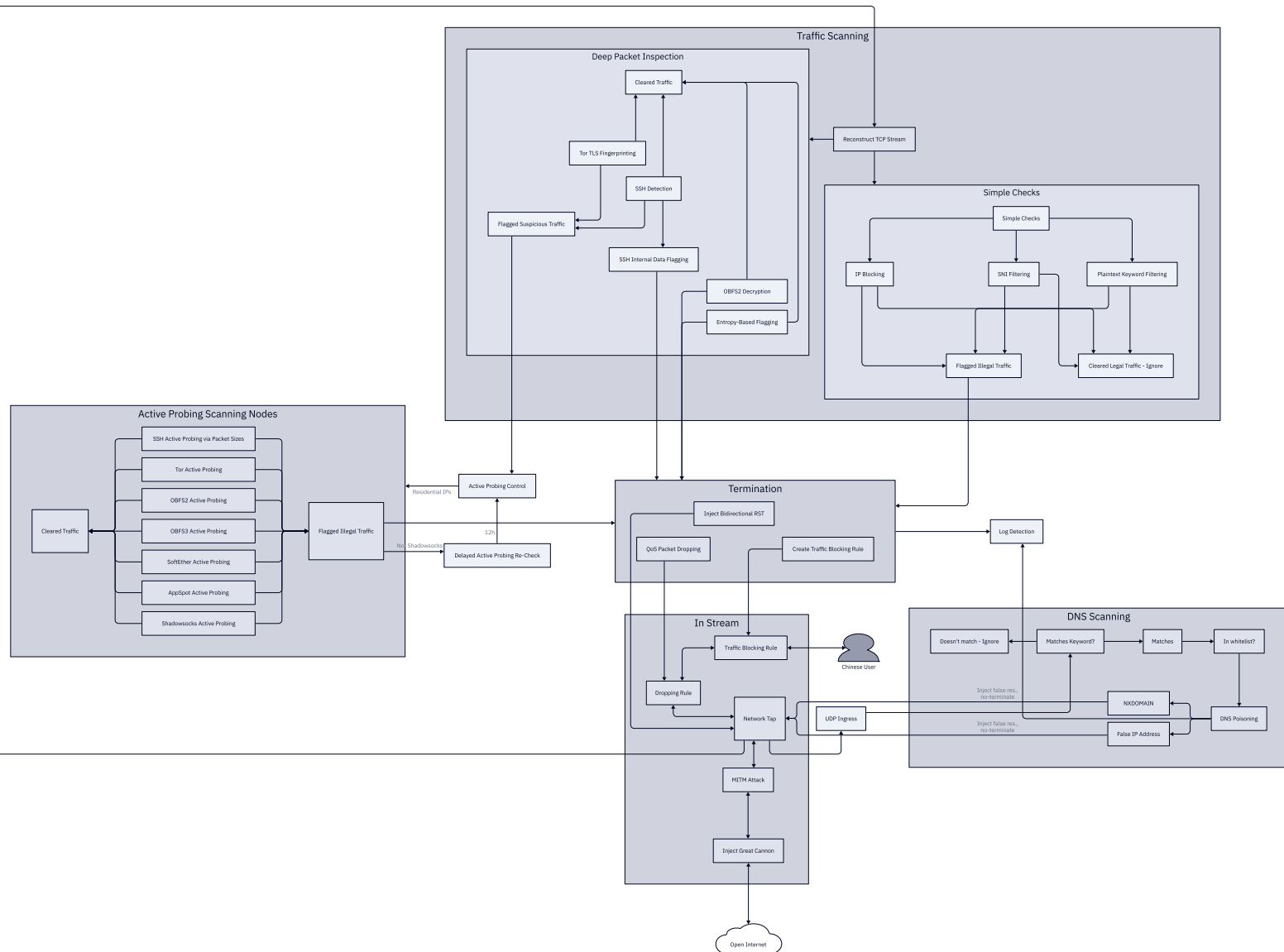


Figure 29: Approximation of its current architecture. Alternative diagram layout. You can still zoom.

2.3.9. GFWTrojan

Trojan [24] is a system that ostensibly circumvents in the same way I intend to: Hiding data in innocuous HTTP requests. However, because it relies on the secrecy of a nested TLS connection, it is vulnerable to MITM attacks and restrictions on internal bidirectionality. My system is not. It is also not actively developed and without much literature, so it is difficult to take specific lessons from it.

There are also various systems that operate over websockets [25], [26], but they are vulnerable to the same issues as Trojan.

2.3.10. Hysteria



Figure 30: Logo of the Hysteria [74] project.

Hysteria [74] is a relatively recent project that seeks to circumvent over QUIC. QUIC has the advantage of being UDP-based, and so there is no additional TCP-over-TCP overhead. However, Hysteria is vulnerable to TLS decryption and QUIC (which carries HTTP/3 [82] and is a recent development) is still widely prohibited and may never be allowed in some networks. It contains an interesting adversarial congestion control algorithm that has the user set a desired throughput before the connection is established. The program then *speeds up* when that target isn't reached in order to thwart QoS based throttling. This would be catastrophic if it were applied to the broader internet, but it works well in (selfish) moderation.

2.3.11. What have I learnt?

Throughout this research, I've learnt about the current state of censorship and circumvention using the Great Firewall of China as a case study. The key things I have learnt are:

Information	Action
Censorship is built in redundant layers	I need to consider every layer, not just the problem I am specifically targeting
Encrypted SNI is unreliable due to MITM and blocking	I should not rely on eSNI
TLS fingerprinting is very difficult to defend against	I should avoid TLS entirely if possible
Active probing is very difficult to defend against	I should consider AP throughout the design and implementation process
Entropy is pervasive as a flagging and blocking signal	I should consider entropy-modified encryption methods
MITM is relatively commonplace, even outside corporate networks	My design decision to work even when MITMed, at the cost of performance and significant complexity, is validated
DNS is heavily filtered and censors are becoming increasingly sophisticated at filtering it	I should not rely on DNS
Reflection and amplification attacks are a frequent real-world problem	I need to ensure reflection and amplification attacks can not be carried out using my system
Packet lengths are incredibly useful for detecting interior TLS	I need to ensure that my system does not leak packet lengths, particularly during the initial TLS handshake
Genetic algorithms can be employed to automatically find flaws that can be exploited	I must ensure that my system is not vulnerable to genetic algorithms, and I can incorporate them into my system to find flaws in the censors
Real-world attacks have used the server as an oracle to decrypt communications	I must ensure that my system is not vulnerable to oracle attacks
Firewalls must be extremely compute efficient	It might be enough to make it very expensive to find me, as opposed to impossible
Stream ciphers are vulnerable to a myriad of attacks that AEAD ciphers are not	I should use AEAD ciphers, not stream ciphers
Quality-of-Service is sometimes used as a blocking method	I should take steps to detect and mitigate QoS

2.3.12. Development Approach

My general development choices and their justification are encapsulated and informed by the research preceding this section. In addition to my various prior decisions regarding software architecture, I also intend to:

- Prioritise memory-safe languages (for security reasons) due to the large proportion of security bugs that are caused by memory issues.
- Develop the software using a hybrid model, building out module structure and filling in modules as appropriate, working out the exact order in which dependencies are fulfilled as I write. My research has demonstrated that firewall evasion systems are so complex that an attempt to do full top-down planning prior to full development would inevitably overlook important features and ultimately be a waste of time. This could be described as an “iterative” approach, developing a feature, reevaluating the broader program, then either moving on to another feature or re-iterating development of the current one.

Essential Features

2.4.1. Bidirectional HTTP Dataflow in restrictive environments

My system must be able to achieve bidirectional (two-way) dataflow over HTTP in environments where HTTP to uncensored domains is allowed, but HTTP(S) streaming is buffered so that only complete requests arrive, websockets and server-sent events are prohibited, and TLS is decrypted for inspection. This dataflow is a prerequisite for other features when the system is used in the restrictive MITMed environment that my system is designed for.

2.4.2. SOCKS Interface

My system must provide a SOCKS4 or SOCKS5 proxy that allows applications to access the internet in an uncensored manner by proxying their traffic through the system (over the bidirectional HTTP transit). This is the primary use case for my system.

2.4.3. Web-Usable Latency + Bandwidth

The system must provide a latency and bandwidth usable for web browsing. The Tor project [83] has extremely poor latency and bandwidth but is still usable, and indeed is widely used, for web browsing. While my system cannot - by design - provide the same level of performance as a standard internet connection or thin proxy like Shadowsocks, it must achieve the same kind of standard as Tor in providing a degraded yet usable web browsing experience in order to fulfill its primary use case.

2.4.4. Secure Encryption

The system must provide secure encryption of data in transit. This is necessary for the security of the user and for firewall evasion (unencrypted traffic provides a fingerprinting target). It must be secure against a wide range of attacks, including active oracle attacks, brute force, replay attacks, hostile peers, and data corruption.

2.4.5. Secure Authentication

The system must provide secure authentication in order to restrict server access to authorised users. This is necessary for the integrity of the server. It must be secure against a wide range of attacks, including active oracle attacks, brute force, and authentication stage replay attacks.

2.4.6. Passive Firewall Evasion

The system must be able to evade passive firewall measures based on traffic analysis, such as deep packet inspection, by disguising its traffic as HTTPS traffic. This is necessary for the system to be usable in the restrictive environment that it is designed for. Additionally, it must not rely on websockets, server-sent events, or other HTTP features that are not available - as outlined in Section 2.4.1. This is a necessity to fulfill its purpose - to provide a usable internet connection in restrictive environments.

2.4.7. Active Probing Resistance

The system must be able to resist active probing by censors in order to evade detection. This is done through “innocent” responses unless valid authentication is provided and systematic protection against replay attacks throughout the connection process. Additionally, the web server must be able to masquerade as a standard web server in order to evade detection through maximising false positives.

This is necessary to fulfill its purpose, just as passive evasion is.

2.4.8. MITM Resistance

The system must assume that any TLS traffic has been MITMed (Man-in-the-Middle) and must be able to resist this attack in order to fulfill its purpose.

2.4.9. User Interface

The client must provide an effective, intuitive interface for end users to interact with. Due to the unorthodox nature of the circumvention method and the amount of variables that affect various performance parameters such as latency, bandwidth, jitter, evasion rate, and traffic rate, the user interface may need informed, competent, and involved users to operate it. An effective user interface that provides the user with all of the data and variables necessary to maintain a high-quality connection makes this easier.

2.4.10. Region Lock

As outlined in Section 1.1.3 the system must restrict access to the server to prevent users in countries where the system is illegal from accessing it. The risk of my system getting someone arrested is incredibly small, as it is highly unlikely anyone will use it, but I still want to avoid it as much as possible.

Limitations

My software will have several limitations due to its nature as a research project and my available time as a student.

2.5.1. Security

Hypertunnel targets high security and confidentiality, however, I cannot provide any guarantees of this. I am not an expert in cryptography or cybersecurity and I cannot afford a formal audit, so the software will not be able to guarantee security. This is a consequence of its nature as a proof-of-concept.

2.5.2. Obscurity

While I will do my best to make my software able to circumvent firewalls, a skilled human or trained ML model may be able to notice patterns in activity that correspond to the tool or to generally suspicious activity. I do not have the resources to engage in this arms race in perpetuity, so I will not provide any guarantees of obscurity and circumvention. I will try my best, but it should not be used in safety-critical situations. This is reflected by the inclusion of the rejoin lock described in Section 1.1.3 .

2.5.3. Integrity

My software aims for data integrity, but in the case of unexpected errors data can be dropped. Data integrity is not necessary for its proof-of-concept use case, and indeed very few (if any) systems can guarantee this due to the fundamental “best-effort” nature of the internet.

2.5.4. Bandwidth

The nature of the obfuscation protocol is that there is additional processing at each layer and bandwidth overhead for all data transferred (both from the protocol itself and the HTTP requests they’re contained within). Additionally, downstreams must stream some useless data while waiting. This is a design limitation that cannot be avoided whilst maintaining the purpose of the system.

2.5.5. Latency

The nature of the proxy is that all data will go to another server before going on to the internet, in addition to processing (encryption, serialisation, message passing) and a delay between data transit opportunities (variable, but significant - generally 50ms at a minimum). This is a design limitation that cannot be worked around while maintaining the purpose and fundamental design of the system.

2.5.6. User Experience and Audience

Hypertunnel is a complex system with many moving parts. Many compromises must be made in various characteristics (bandwidth, latency, circumvention, overhead, CPU usage, and request frequency) based on what the user desires to optimise for. Users must choose what characteristics they care about and they must be able to understand the system in order to fully utilise it.

This requires a baseline level of technical knowledge that the vast majority of people do not possess. As such, the software will be “user-friendly” insofar as it does as the user commands, rather than making decisions for them. This limits the audience to technical users.

Requirements

2.6.1. Hardware

Client

The client must have a sufficiently powerful CPU and approximately 200MB of free ram in order to effectively run the software. It needs storage in order to store the software and its configuration. The CPU should be in the list of supported CPUs for the linux kernel and Rust compiler. A mouse, display output, and keyboard are required for the GUI client to be used, but the CLI can be used with only a keyboard and console output.

These are prerequisites for the software to run on the client well enough to meet the success criteria.

Server

The server must have a sufficiently powerful CPU, preferably multi-core, with approximately 100MB of base free RAM and approximately 50MB of additional RAM per user. In addition, it must have a functioning high-bandwidth low-latency network card. It requires storage to store the software and its configuration. The CPU should be in the list of supported CPUs for the linux kernel and Rust compiler.

These specifications are required in order to run the software with good enough performance to achieve the success criteria.

2.6.2. Software

Client

The client must run modern Linux with GUI support (Wayland or X11, any functional window manager) and a working network stack. This is a technical prerequisite in order to execute the (Linux-only) client software.

Server

The server must run modern Linux with a command line interface and a working network stack. This is a prerequisite of the Hypertunnel software.

2.6.3. Internet Connection

Client

The client requires an active, working internet connection that allows communication over HTTP or HTTPS to a remote, un-whitelisted server without abnormal rate limiting. Hypertunnel performs best with substantial bandwidth and low latency, but this is not strictly necessary.

Server

The server must be able to receive connections on either port 80/TCP or 443/TCP over HTTP or HTTPS so that it can connect to clients and function as a server. It must have unrestricted access to the open internet in order to effectively circumvent censorship. Its bandwidth requirements are dependent on user activity, but are generally high due to the additional overhead of HTTP transit and Hypertunnel's internal protocol.

Success Criteria

Hypertunnel is ultimately a research project, and so my success criteria is primarily focused around proving a minimal proof-of-concept to demonstrate that the idea is viable. My success criteria are as follows:

- The software should be able to be initiated from a command line, and should be able to be run in the background. This demonstrates that it is usable and accessible to a technical user base.
- The Hypertunnel server should refuse requests that do not present valid authentication. This demonstrates that it is secure and has basic authentication functionality.
- Hypertunnel should successfully circumvent a real-world example of a deep-packet-inspection TLS-decrypting firewall, operating behind it. This demonstrates the core functionality of the software in the extremely strict environments it targets.
- Hypertunnel should demonstrate the capability to send and receive meaningful communications through its tunnel, including the ability to send and receive files and operate Discord, a popular voice and text chat application. This demonstrates that it can be used for practical communications, demonstrating useful functionality for the stakeholders: circumvention tool developers, students, and those in restrictive regimes.
- The internal traffic should not be able to be viewed by a MITM (Man-in-the-middle) attack. This demonstrates that it is secure from eavesdropping and tampering.
- The software should be able to be initiated from a GUI, although it is not necessary for the GUI to run continuously during background operation. This demonstrates its usability to a non-technical user base.
- The software should be written in a memory safe language for security purposes.

DESIGN

Decomposition

I've broken the program down into the client and server, as these are completely separate programs deployed on different machines with very little shared code. Shared code comes under "Shared".

This is a summary of how the program operates:

3.1.1. Plan

The core plan is to build a firewall circumvention system that is able to smuggle data over HTTP without the use of websockets, request streaming, or server-sent events. This means that all data must be sent in discrete chunks. TCP uploads will need to be stored in a buffer, then, when the time is right, uploaded in a single (fully-encrypted and disguised as legitimate) HTTP request. Instead of the server having a proper way to inform the client when it has data to return, it will stream false data until it has a big enough chunk of data to return - at which time it will return it in the same stream, fully encrypted, distinguished only by a steganographic sequence. This will require a complex and intricate system, but it will also be able to operate in environments where other tools cannot.

3.1.2. Client

I have broken the client into separate constituent parts that can be effectively developed as separate modules.

User Interface and GUI

The client needs to have a graphical user interface that is intuitive, understandable, and powerful.

Users will need to have a comprehensive, intuitive view of program status on both the client and server. They also need the power to tune various areas to resolve bottlenecks and optimise for their use case.

The GUI should be handled as a separate modular component to the actual UI, communicating via shared memory and message passing.

This is suitable for a computational solution as a computer is very effective at rendering an interactive UI in real-time. It is a separately solved problem because it can be modularly removed from the core of the program without preventing the core program from functioning (provided some minimal replacement was in place to start it).

SOCKS Listener

The client needs to listen for incoming connections from SOCKS clients in order to carry on to data relay. This is a separate component because it initialises the SOCKS handlers while remaining disconnected aside from the creation of asynchronous handler tasks.

This relies on the SOCKS protocol spec to understand the metadata in the incoming request (target host, port, version number).

It makes sense to separate this component out, as it will be handled in its own asynchronous task and core loop as a separate entity in-program that can be modified independently of other components.

SOCKS Handler

Each SOCKS connection needs to be handled: Incoming data needs to be tagged and sent on to client transit, and returned data needs to be sent back to the client socket. Any errors must be propagated bidirectionally.

It makes sense to separate this component out, as it solely manages a section of data (the SOCKS connection) and carries out an independent, modular role on it (message passer negotiation).

SOCKS Protocol Spec

The program must be able to parse and handle SOCKS messages. This requires a full spec that can be used for parsing. This is suitable as an independent component because of its nature as a protocol definition - it is relatively independent, and called when data needs to be encoded and decoded without additional options. It is a “pure function” of binary to struct and struct to binary without mutable state. It could be published and used by the community as-is without any modification except for some additional documentation.

Data Coordinator

The client data coordinator accumulates ready-to-send SOCKS messages and sends them on to the uploader based on the buffer size and modetime. This is suitable as an independent component because, while it manages lots of disparate data from various places, it operates as a single hot loop in a singular function with isolated algorithms.

Congestion Control Apparatus

If congestion becomes an issue there should be a singleton congestion control management system that applies congestion control algorithms such as AIMD. This is likely to be more distributed than other components, but the algorithm itself will be a modular system (using Rust’s trait definitions) and therefore it is suitable as an independent component.

Server Hello

The client must greet the server before attempting to transfer SOCKS data. This greeting serves to verify the credentials are correct, check it is actually a hypertunnel server, and allow the server to prepare for data to come through.

This is suitable as a separate component because it is an independent block of code implementing a unique and distinct client-server communication.

Data Upload

The data upload module manages a pool of HTTP workers that upload data to the server when they receive it from the Data Coordinator. These HTTP workers also transmit and receive metadata for both sides on each request.

Each HTTP worker is an entirely isolated component - they are distributed across multiple threads and have significant internal state. They work well as separate components.

Data Download

The data upload module manages a pool of downstream HTTP workers that are constantly fed a “trickle” of fake data by the server. When the server has data to send down it covertly indicates the end of its false period, then quickly sends back the data. The downstream worker sends its data on to the sockets then immediately makes a new fake request to be ready for the next data.

Each HTTP worker is an entirely isolated component - they are distributed across multiple threads and have significant internal state. They work well as separate components.

3.1.3. Server

I have broken the server into separate constituent parts that can be effectively developed as separate modules.

Configuration Loader

The configuration module loads server configuration, including user credentials, from a configuration file. The parsed result is made available to the rest of the program such that it can be used in its operation.

This works well as an isolated component - it can be modelled as a pure function that takes a read-only reference to the file system and returns a struct. It can be implemented as a simple system that runs during program initialisation and never again.

Server Listener

The server listener awaits HTTP requests over TCP, then forms a spec-compliant HTTP/2 continuous connection. It hands off requests to the request handlers for various operations (upstreams, downstreams, client hellos, etc). This should be broadly managed by my web framework.

It is suitable as a separate function because, while it's in many ways a "junction point" for lots of state, it's implemented separately.

Session Management

The session management component keeps track of currently active sessions. This can be separated out easily as it's simply some shared mutable state behind a RwLock.

Client Hello

The client hello component manages initialisation requests from clients, verifying their encryption schemes, receiving their version, initialising their session, and validating region locks. This is a separate server route implemented as an independent function, which makes it suitable to be separated out as an independent component.

Region Lock

The region lock component evaluates whether a user is in a restricted region so that their connection request may be denied if necessary. This is isolated - it takes in a list of prohibited ASNs and an IP address and independently checks whether to allow the user to continue to connect.

Congestion Control Apparatus

If congestion becomes an issue there should be a singleton congestion control management system that applies congestion control algorithms such as AIMD while considering fairness between different users and applying backpressure via the metadata channels. This would involve a distributed data collection system, but the actual algorithm would be implemented as a pure function that is suitable for separate implementation.

TCP Relay Connections

Data must be relayed bidirectionally between remote target servers and the clients through split TCP connections. Each side (up/down) of the connection is implemented as a separate function that manage their own state, communicating with the coordinator via message passing, so it makes sense to have them as separate entities in the design phase.

Data Coordinator

The data coordinator manages the return of data in discrete chunks that reconcile TCP's nature as a stream protocol and the limited temporal opportunities for data transit while in a system without request streaming, where downstreams must be in discrete HTTP requests. It coordinates upstream data into TCP relay connections, as well as accumulating downstream data and deciding when it should be returned. It serves as an independent hub for data transit, implemented as a hot-loop in its own function. This makes it well suited to act as a separate component for design purposes.

Metadata Handler

The server needs to accumulate metadata for returning back to the client for the purpose of congestion control and performance statistics. This is centrally supported using a global singleton with atomic variable variants, and so works well as an independent component.

3.1.4. Shared

Cryptographic Module

Payloads must be securely encrypted. The cryptographic module should be an isolated, highly-rigorous system for symmetric encryption of payloads and secure key derivation. This code is shared between the client and server. The fact that it is shared demonstrates why it works well as an independent component. It could be separated into a generic public library with very few changes.

Encoding, Decoding, and Protocol Specification

Transiting data needs to be encoded into a wire format that can then be decoded on the other side. This is done through shared protocol specifications in the libtransit module. This make sense as a separate, independent component because it's simply a stateless definition that both programs utilise as a shared library.

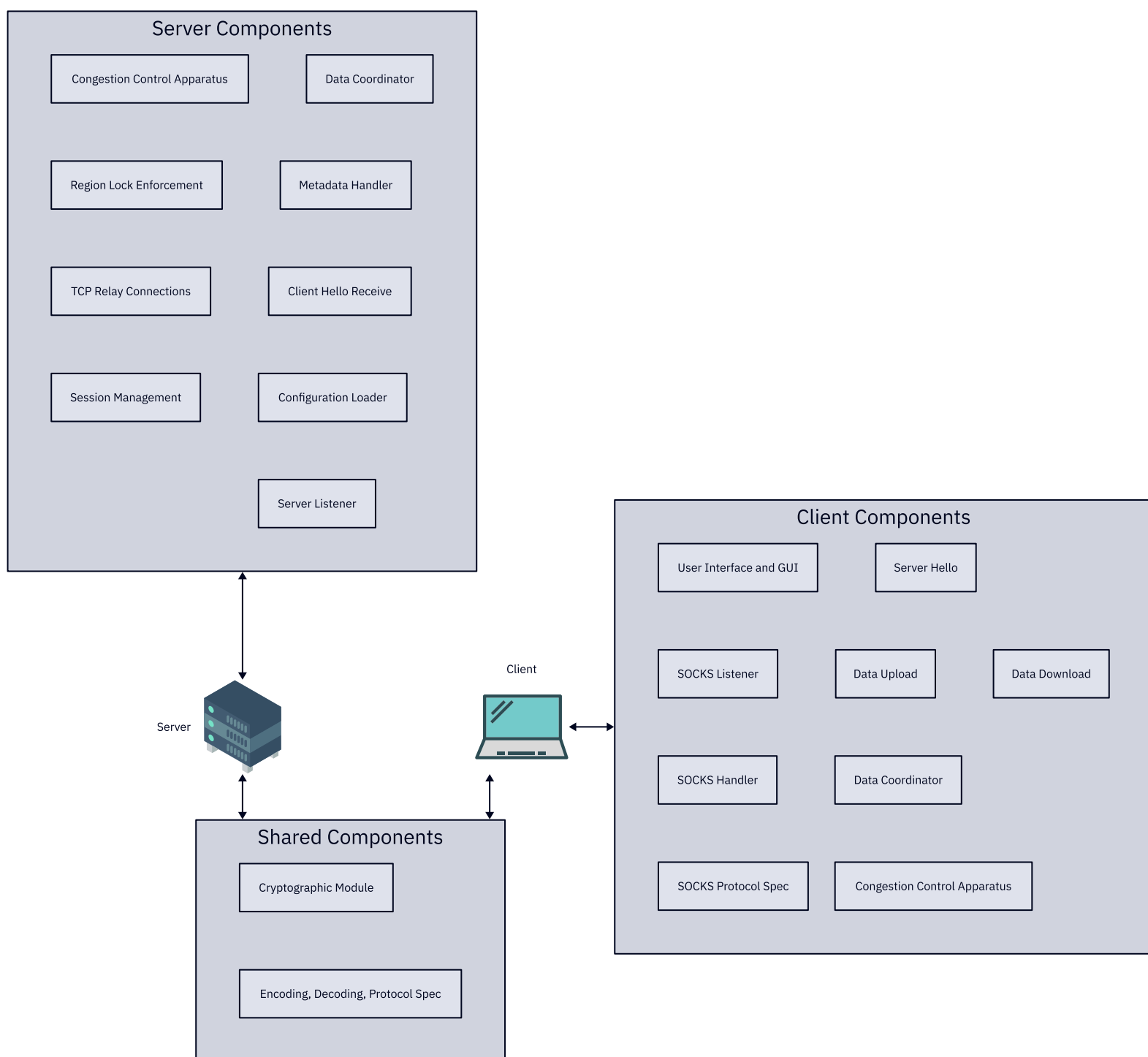
3.1.5. Decomposition Diagram

Figure 31: Simple decomposition diagram without the relationships between components.

3.1.6. Code Structure

I plan to split my code into the following discrete modules:

- client-cli (Command line interface for the client, for dev purposes).
- client-gui (GUI for the client)
- client-core (Core of the client, doing the actual circumvention)
- libsocks (SOCKS protocol definition and parsing)
- libtransit (Client-server protocol definition and parsing)
- server-monolith (Monolithic server, including HTTP receiver and upstream transit)
- libsecrets (Encryption, decryption, and authentication)
- integration-tests (Automated tests that utilise multiple modules)

Definition

3.2.1. Core Operation

Diagram

I have created a rough diagram of how the system will operate. Hypertunnel is not a combination of discrete algorithms with trivial “glue code” interconnects. Instead, it forms one large novel network-distributed algorithm that can only be adequately explained through a diagram:

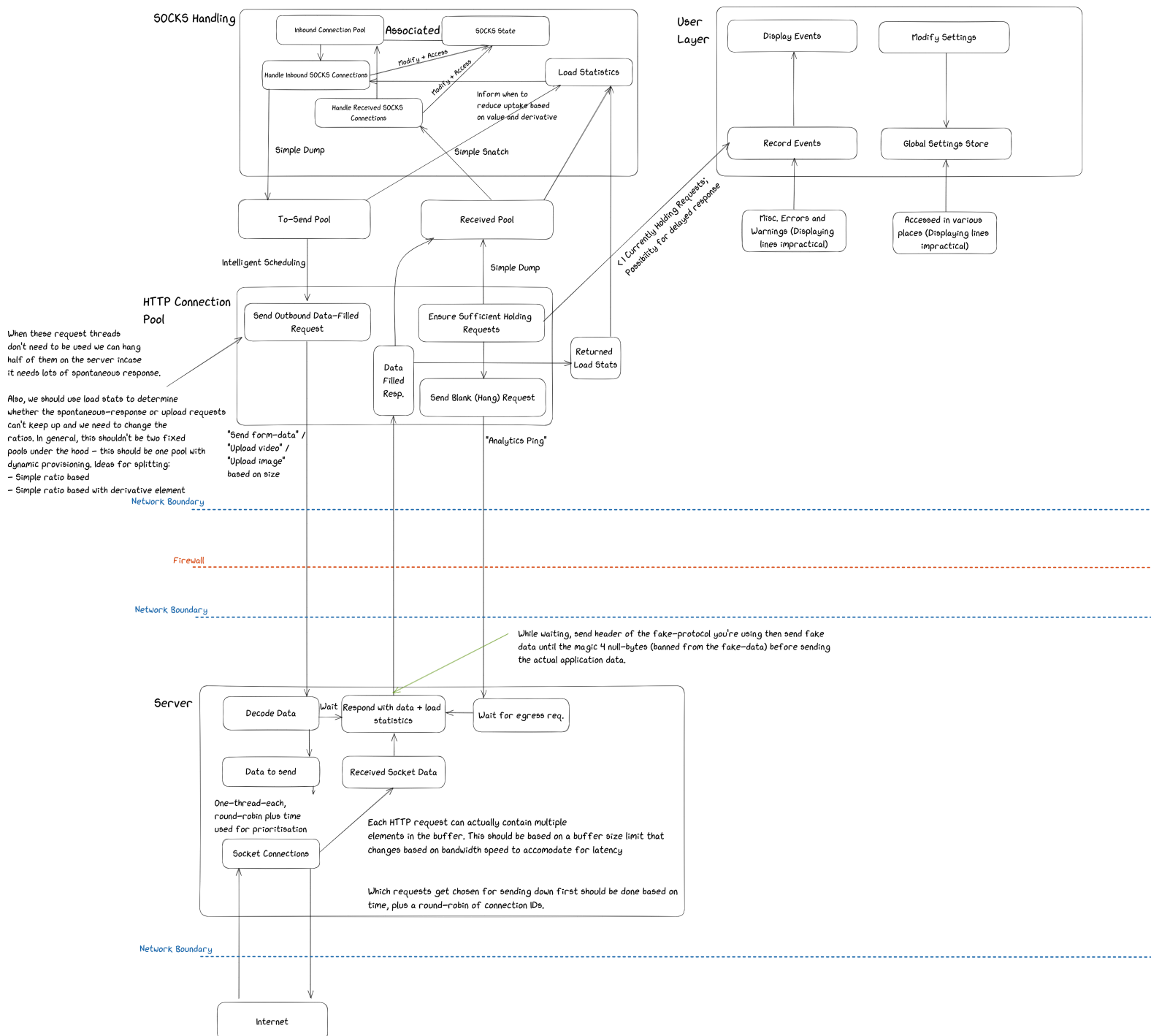


Figure 32: A rough diagram of operation. Not representative of the final product.

Lifecycle of a request

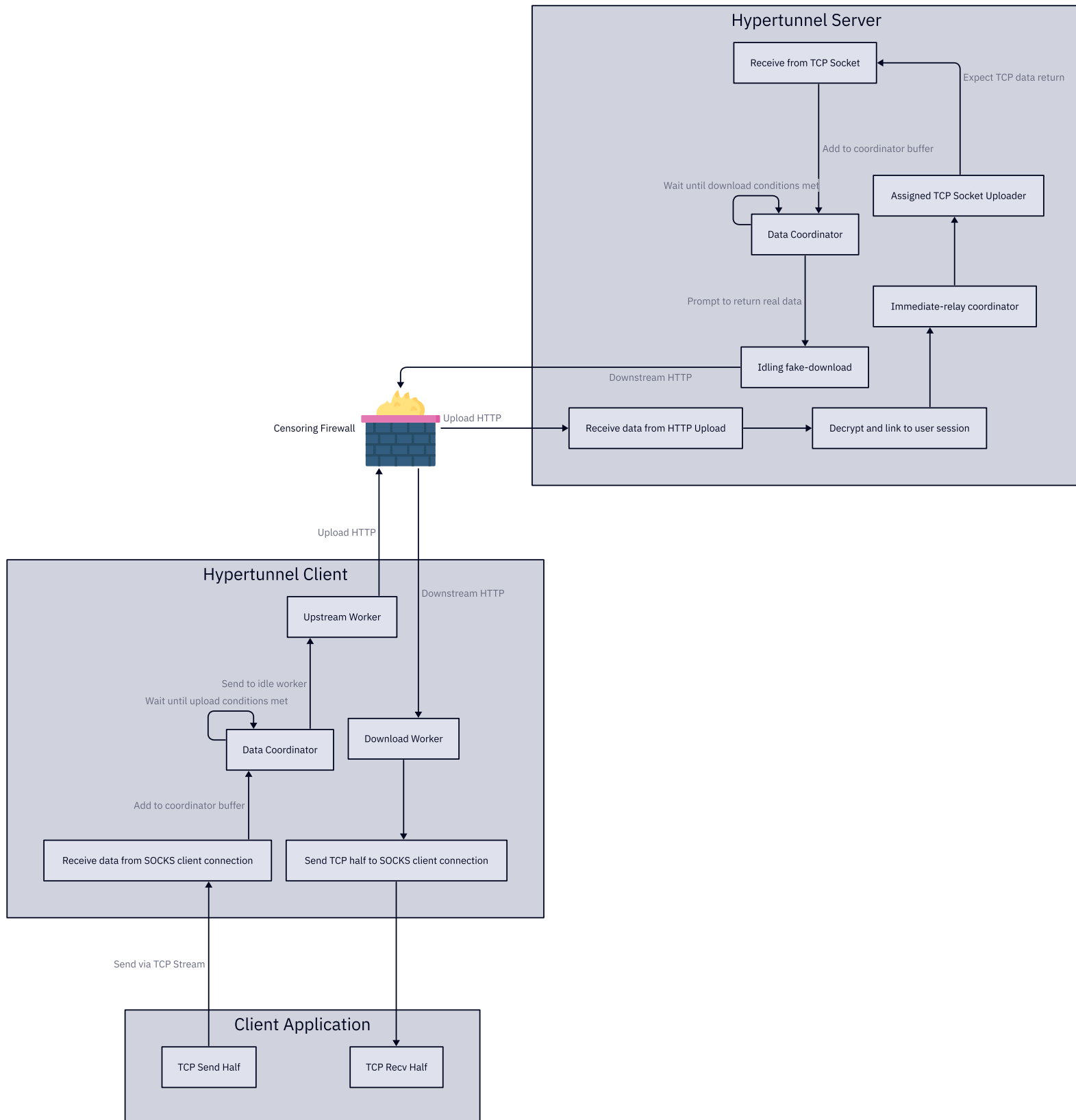


Figure 33: A rough request lifecycle when the client application has already negotiated a SOCKS connection with the listener and hypertunnel has already prepared a link on both sides. This ignores that there are many TCP streams and clients in a request and focuses on one.

3.2.2. Algorithmic Definitions

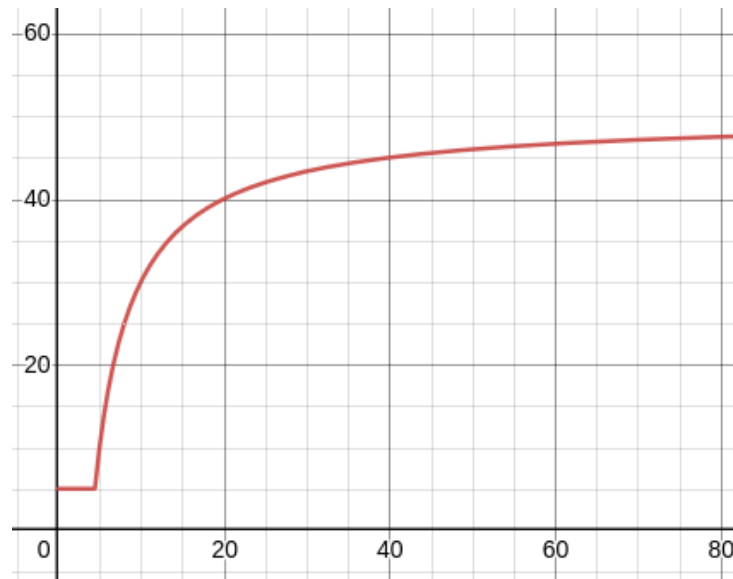


Figure 34: Graph of $y = \max(a - (bx)^{-g}, l)$ where $a = 50$, $b = 0.005$, $l = 5$, and $g = 1$.

The second, again with t as input x and modetime in milliseconds as the output y :

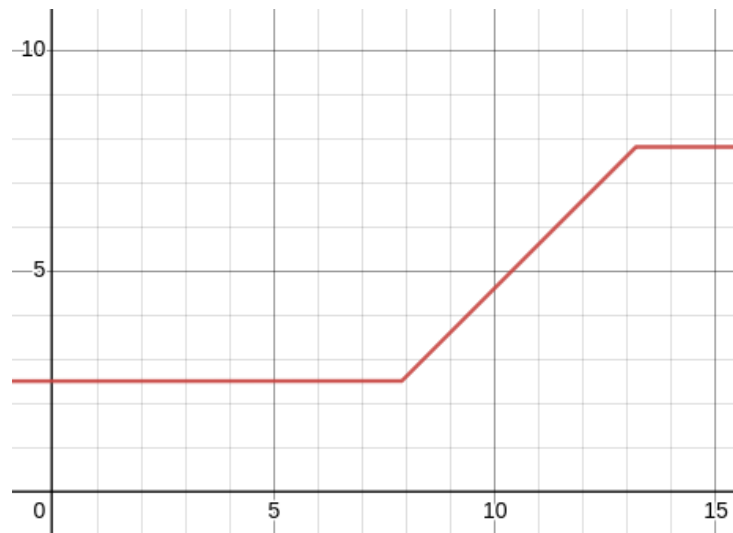


Figure 35: Graph of $y = \min(c, \max((mx - o), b))$, where $m = 1$, $o = 5.4$, $b = 2.5$, and $c = 7.8$.

The core algorithm itself can be trivially implemented in pseudocode as follows:

My algorithmic requirements are:

- Simplicity in order to lower attack surface
- Allowing any payload, no matter how small, to get through in a reasonable time without additional payload arriving after
- Minimising the integral of the bytes in the buffers by time; this core metric is essentially “buffer wastage”.
- Minimising requests in order to minimise firewall suspicion
- Maximising request size

This simple algorithm fulfills all requirements. The max buffer size limits the integral metric, while the modetime guarantees eventual delivery. Good tuning allows for extremely limited request frequency (at the cost of latency+jitter) and requests are always at an optimal size except when guaranteeing delivery.

Congestion Control Apparatus

I expect to encounter issues with congestion and bufferbloat. Should this be the case, I intend to implement a standard congestion control algorithm using an external apparatus on both the client and the server.

My algorithm of choice is the “Additive Increase; Multiplicative Decrease” algorithm. The algorithm was first described in “Congestion Avoidance and Control” [86], a 1988 paper that investigated the congestion issues in the quickly-developing internet [87], particularly over transatlantic links. It has since been formalised in multiple RFCs [88], [89]. While there are many other - more sophisticated - feedback control algorithms, AIMD is surprisingly successful for its simplicity. While it originally exhibited “feedback” through packet loss, Hypertunnel would instead reduce peak ingress bandwidth to adapt.

It can be expressed mathematically as:

Let $w(t)$ be the congestion window size for amount of data in flight during time slot t

Let a ($a > 0$) be the additive increase parameter

Let b ($0 < b < 1$) be the multiplicative decrease factor

$$w(t+1) = \begin{cases} w(t) + a, & \text{if congestion is not detected} \\ w(t) \cdot b, & \text{if congestion is detected} \end{cases}$$

Alternatively, as psuedocode:

```
1 def aimd(a, b, w, congestion_detected):
2     if congestion_detected:
3         return w * b
4
5     return w + a
```

It is able to steadily increase capacity to saturate a link while aggressively adapting to link limits or additional traffic causing congestion in order to minimise jitter. This jitter minimisation is vital. The fundamental tradeoff that my tool makes - the transfer of data in discrete chunks rather than streams - causes a small decrease in bandwidth, but a very significant increase in jitter. AIMD fits this well by minimising jitter at the cost of a little bandwidth compared to more aggressive algorithms.

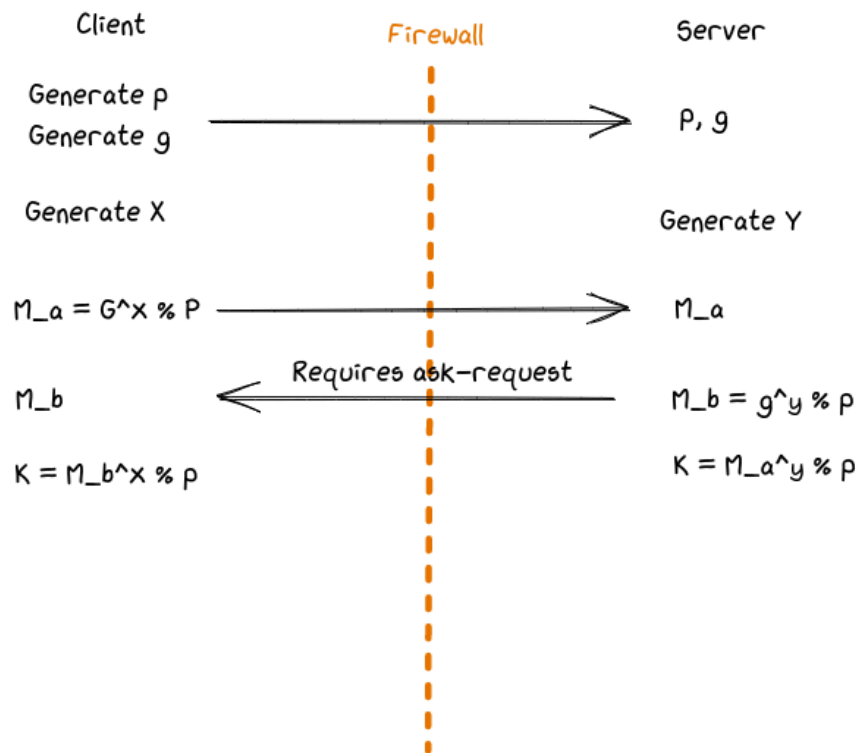
The AIMD algorithm is expected to perform all requirements, although congestion control is an area of tradeoffs and significant emergent behaviour, so this decision may not remain fixed. I have also extensively considered PID controllers (which use the first and second derivative as well as the actual value to find an equilibrium). However, AIMD appears to satisfy the requirements better due to its simplicity and the extent of its real-world use (it is widely deployed in TCP congestion control).

Server Hello

The server-hello step is required to facilitate the later circumvention. This step must remain reasonably undetectable while negotiating an encryption standard between clients.

I designed this in phases, beginning with a modified diffie-hellman key-exchange as described in the following diagram:

Diffie-Hellman Key-Exchange (Modified for Request-Required Architecture)
 Transfers an AES key
 Known-Vulnerable to fake-server attacks without additional certificate verification
 All transfers occur over HTTP



Abandoned in favour of negotiations proposal 2 due to vulnerability to MITM
 and unnecessary additional complexity

Figure 36: First draft of the encryption phase of the client hello.

I then investigated a known-password key exchange:

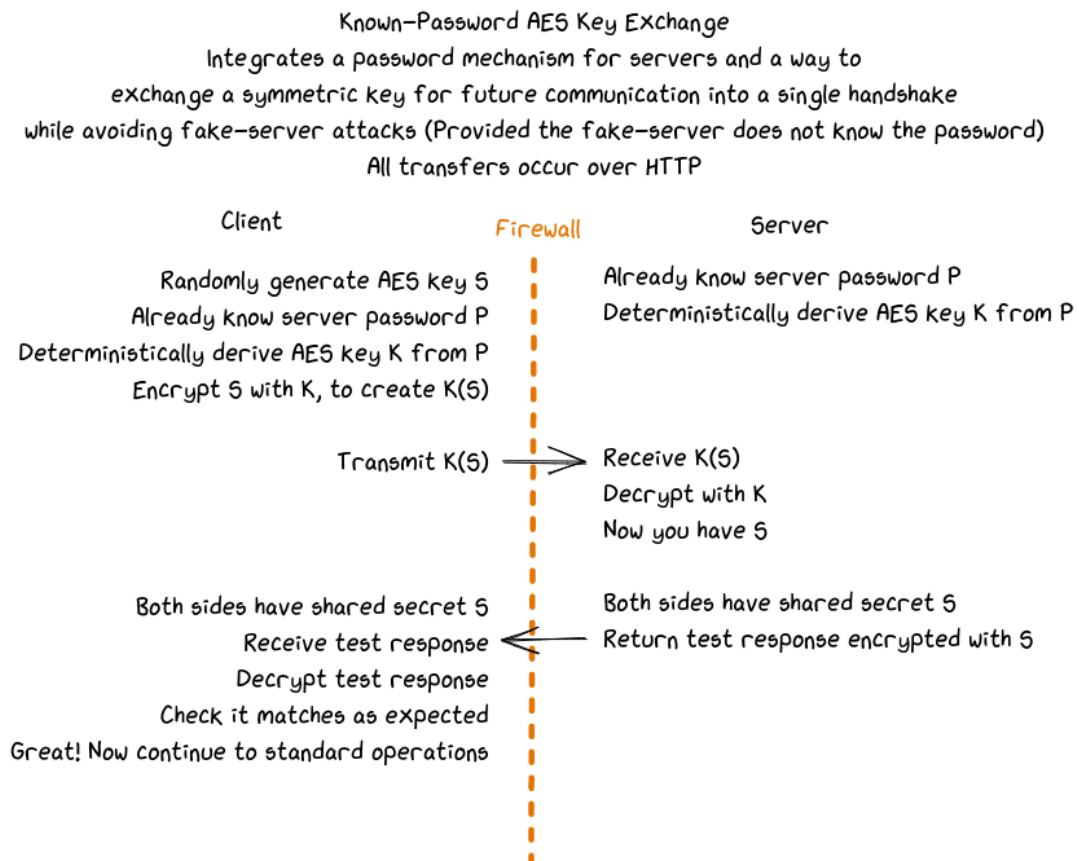


Figure 37: Second draft of the encryption phase of the client hello.

This would have worked, but remained quite complicated - and complexity is weakness. I simplified it to this:

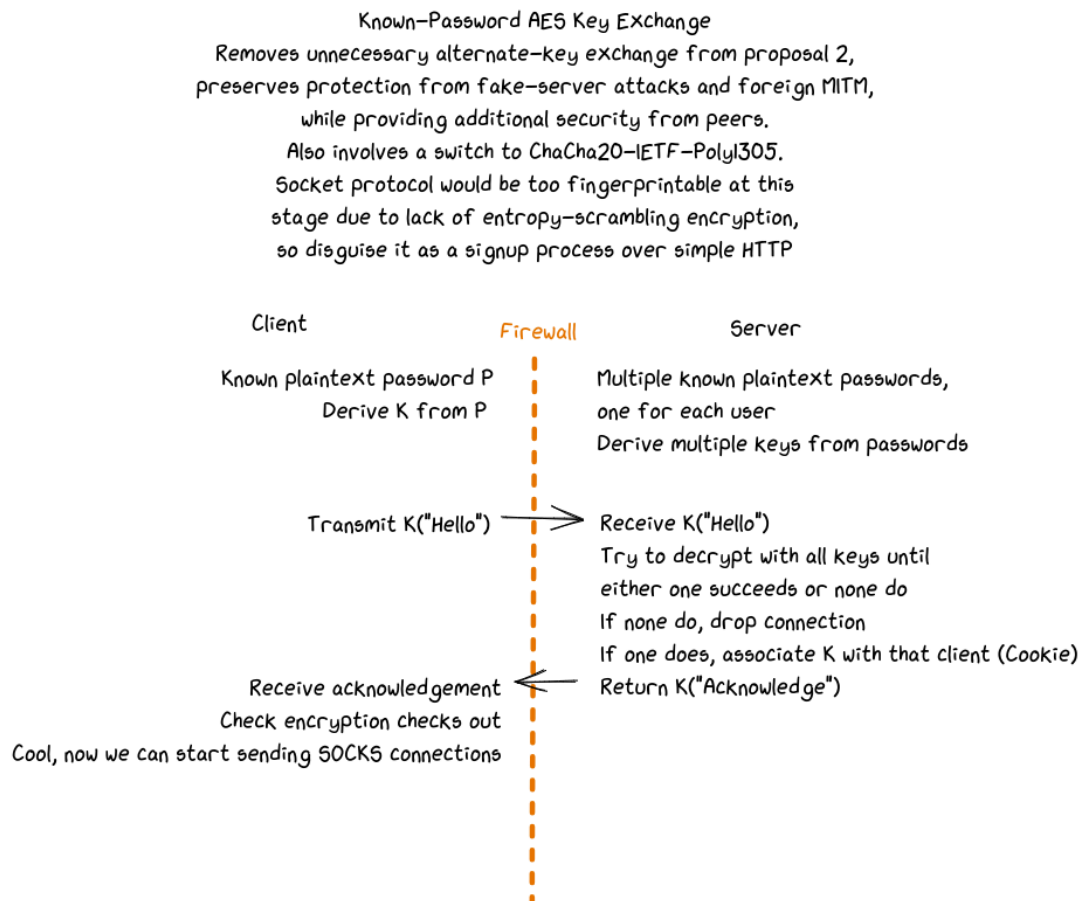


Figure 38: Implemented encryption scheme.

I have decided to use this system in the final design. “Hello” will be replaced by version and connection information, as well as a test string and dynamic padding to avoid fingerprinting.

Region Lock

As I talked about in Section 1.1.3 I intend to implement a region lock. The region lock functions as follows:

- The client sends whether it has its region lock on or off.
- The server takes its region lock state (on or off) and compares it with the client.
- If both client and server have the region lock off, the connection is permitted
- If either the client or the server have it on, the server checks (via a local ASN list) if the client is in a *whitelisted* region.
- If the client is in the whitelisted region, the connection is allowed to continue.

I intend to get the database from maxmind [90] and read it with maxminddb-rust [91].

Web Server

I intend to use a very high-performance Rust web framework to operate my server. It must be as low-overhead and high-throughput as possible in order to avoid unnecessary latency. It should expose route handling and allow for request streaming via a generator/iterator model.

Server Configuration

My server must be able to load configuration from a `config.toml` file. The configuration should include user accounts, listening host, and the listening port. I have chosen to use the `toml` format because it is well supported by the Rust programming language and very intuitive for

end users, while having a strict and well-defined specification for an intuitive descriptive configuration and serialisation language.

Identification

I need a way to differentiate between various clients. I cannot afford to attempt to decrypt with all user identities for every payload-holding request. I could try to use speculative decryption of a smaller section, but the simpler and cheaper system is just to store session identity in cookies. It's a common pattern that shouldn't be viable to block on because of collateral damage.

3.2.3. Completionism

The problem that I am solving is completely solved by the complex interconnection of the many hundreds of small, distributed algorithms that form my program. This is visible through the decomposition diagrams. Specific subsections that form more discrete and visible algorithms have been clearly described above. These discretely defined algorithms completely solve the subsections of the broader problem that they have been assigned to, and effectively combine to form a broader solution.

3.2.4. Tool Choice

Language

I chose Rust [92] to create my tool. I had some key requirements for Hypertunnel - I needed it to be safe and secure, while also being very high performance by providing me with low-level access to the computer. Rust is the perfect solution - it provides the performance of C with the ergonomics of a much higher level language, including a brilliant algebraic type system. The only other alternatives would be C++, which is memory-unsafe, and Go, which is significantly slower, requires a garbage collector, and refuses to implement many useful language features.

Operating System

I chose Linux because it has a very good networking stack and is my preferred operating system.

Integrated Development Environment

I have chosen to use Visual Studio Code [93] to write my tool. While I had previously used JetBrains tools for Rust development, I decided to switch to VSCode due to the the poor performance and buggy nature of the JetBrains Rust plugin.

Documentation and Planning

I documented my work in Obsidian [94] in Markdown and created drawings in D2 [95] and Excalidraw [96]. These are all minimal tools that allowed me to focus on the text I was writing and systems I was diagrammatically representing, rather than worrying about the tool itself.

Usability Feature Justification

3.3.1. User Interface Design

I always wanted to design something that demonstrated what was going on internally, as well as the route it took around the globe. Here's a quick mock-up of what that would look like:

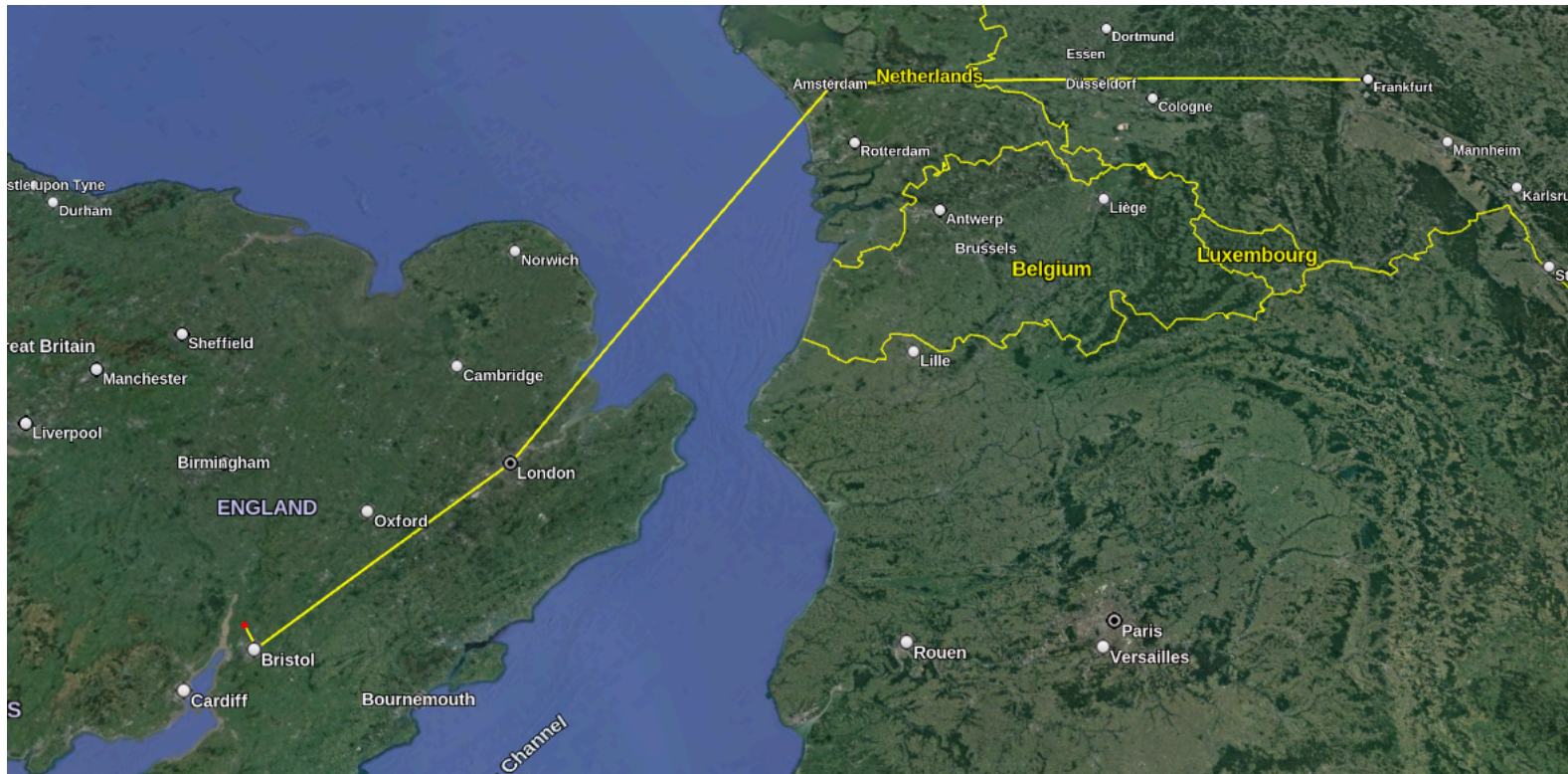


Figure 39: UI Mockup

I also wanted to demonstrate the inner workings of the tool. These two requirements were difficult to reconcile given the limited screen space. Inner workings are significantly more important for the user than an approximate geographical path, so I prioritised that in my UI mockup:

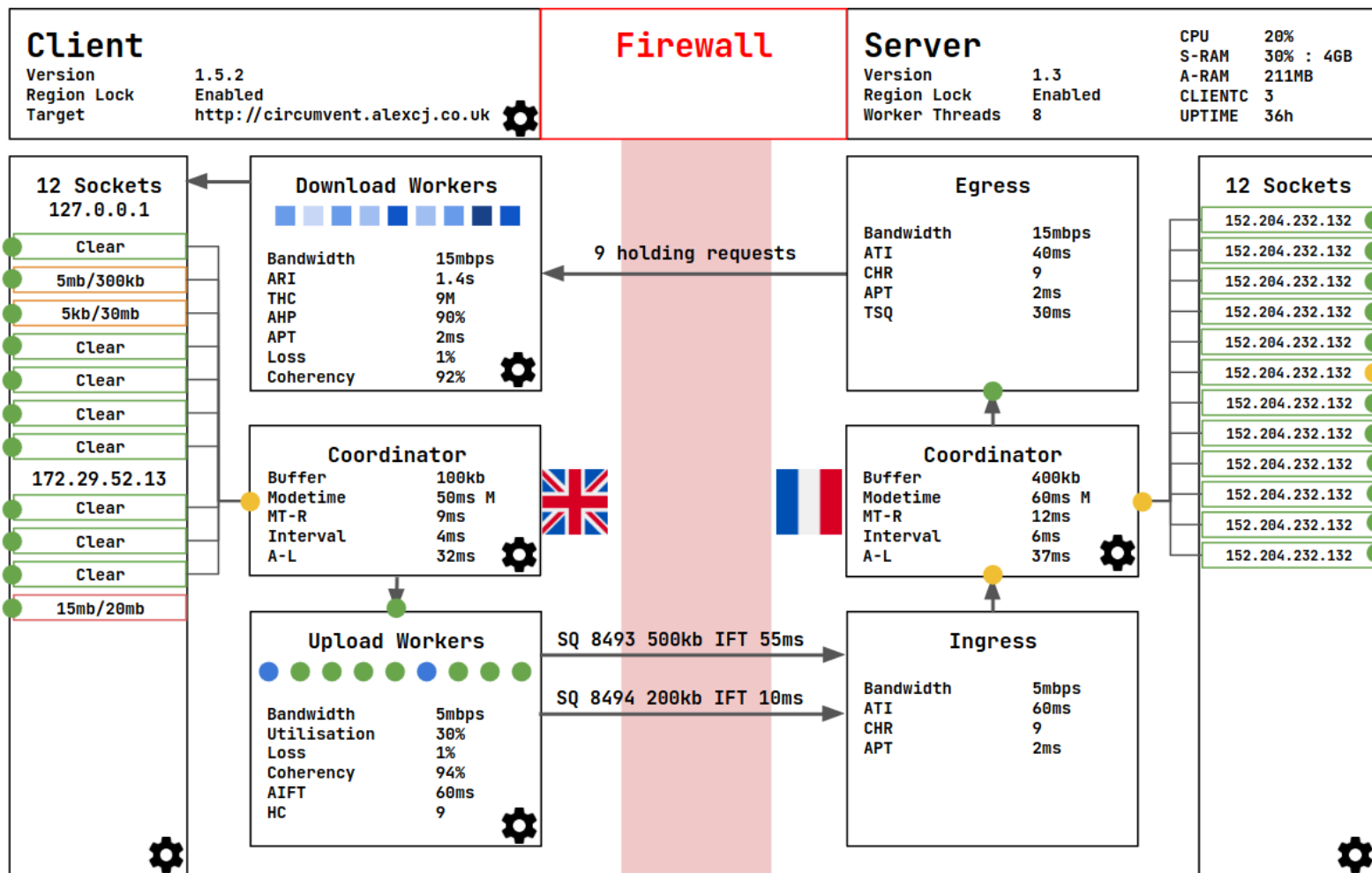


Figure 40: UI Mockup

General Layout

I chose to lay it out in this way to show a clear demarcation between client and server and to show the internal workings and dataflow. I went with a clear, bright colour scheme to avoid any negative associations with the “hacker aesthetic” and make it less conspicuous. Its layout is orderly and organised, with clear lines to designate separations between different parts. The big red bar in the center shows the hostile network barrier, and the client and server information at the top align with their components. I chose a universal indicator for modifiability - a black cog - for UI consistency.

Typography

I chose the JetBrains Mono [97] font. I wanted to use a monospace font to make UI design easier (due to guaranteed character width), and JetBrains Mono is my preferred monospace font.

Accessibility

The bright colour scheme should make the software accessible, and the large, clear text should make it easy to read. The modifiability indicators follow clear patterns, and the hierarchical, structured layout is easy to mentally parse. While the program uses colour to convey information, the information is not vital and can be ascertained through other means.

Client Sockets

The colour of the client socket's border denotes what proportion of total bandwidth is from that socket. The left dots denote the amount of buffer remaining for returning to the sockets, and the dot on the coordinator ingress indicates the size of the message passing buffer for the coordinator receiving upstream messages from the socket.

Client Coordinator

The modetime denotes the maximum time between request transfers, as described earlier. The letter next to it - "M" or "T" - denotes whether it is manual or automatically adjusted during TLS handshakes.

The MT-R is the remaining time until the modetime provision is activated.

The buffer denotes how much data is in its buffer, and the interval indicates how often it does a loop - receiving from message passers and checking for modetime conditions.

The A-L is the average amount of latency introduced to a request in this stage. Finally, the dot on egress indicates the amount of data in the message passer's buffer on its way to the upload workers.

Upload Workers

Statistic	Description
Bandwidth	Mean Bandwidth over last x seconds
Utilisation	Percentage of upload workers currently engaged
Loss	Packet loss
Coherency	Rate of out-of-order message arrival
AIFT	Average in-flight time - time from request initiation to completion
HC	Handler count - the amount of workers

The set of circles indicate the status of the various workers, with the following key:

Colour	Meaning
Green	Ready
Amber	Not Ready
Red	Retrying
Blue	In Transit

Client Information

The top left includes information about the client and target.

Request Indicators

The request indicators show currently in-progress requests. SQ is the sequence number, IFT is how long it has been in flight, and the payload size is self explanatory.

Server Information

The top right includes information about the server. It's mostly self-explanatory, with S-RAM indicating server RAM and A-RAM indicating how much has been allocated to the server instance.

Ingress Request Handlers

This displays information on the ingressing requests, representing the ingress (upload) routes.

The ATI is the average time interval between receiving data. The APT is the average time that it takes to process a request (over a recent time interval).

Server Coordinator

The coordinator box shows data for the server-side coordinator. The buffer shows the amount currently in the **return** buffer, as it handles ingress data without an intermediate buffer. The modetime shows the modetime given to the return calculation algorithm on the server side, with the M denoting manual or TLS-detecting and the MT-R denoting time remaining before it is forced to return any buffer data. The interval is how frequently it (attempts to) iterate, and the A-L is the average latency for data passing through.

The coloured dot denotes the size of the ingress->coordinator message passer.

Server Sockets

The server sockets section shows all currently in-use sockets. For display purposes it merges the upload and download halves, despite them being separately handled. The coloured dots indicate the size of the to-send buffer and the dot on the coordinator connector indicate the quantity of data that has been received and is waiting in the coordinator's message passers.

Server Egress Request Handlers

The egress request handlers indicate egress bandwidth, the average interval between egress requests (ATI), the amount of requests currently in the holding system (CHR).

Nationality Indicator

The country flags indicate the country of the client and server based on a static database of IP ASN locations. The flags should be normalised into a constant shape to improve UI consistency.

Modifiability Indicators

I wanted to make it simple to modify state. Everything with modifiable settings has a small black cog in the bottom right that can be clicked to open a menu where settings can be changed. I hope that the proximity of the modifiability indicator to the component itself will make the system more intuitive to use.

Code Identification

3.4.1. Heirarchal Breakdown

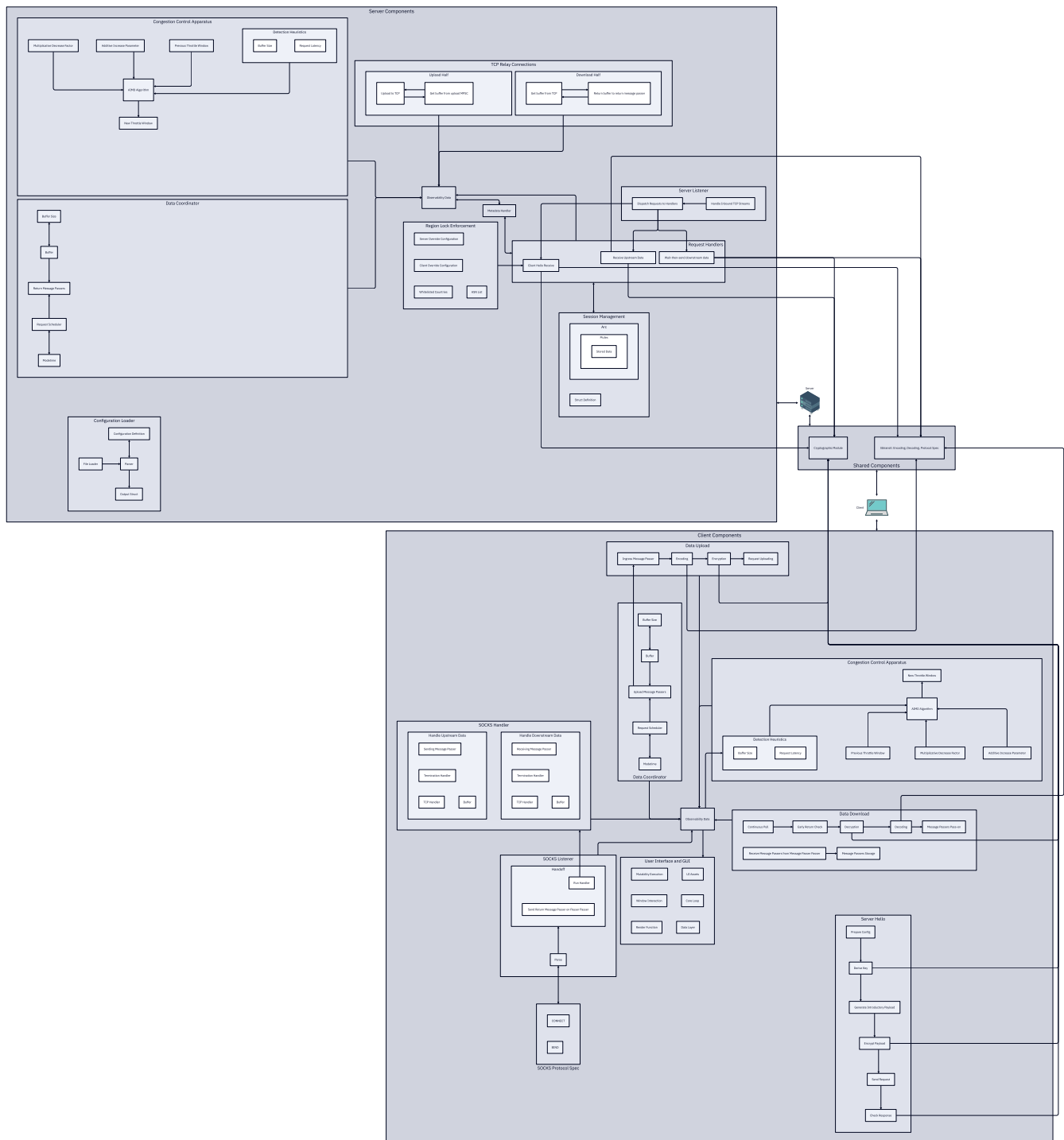


Figure 41: Heirarchal breakdown from program to struct levels, identifying components, data structures, classes, etc. Zooming is necessary to view it.

All of these components will be tested automatically using unit and integration tests written in Rust's integrated testing framework.

3.4.2. Validation

The primary attack surface is the server - a compromised server means compromised data, and therefore users, while servers are open to the internet and easily accessible for attack. In order to secure my server I have used input validation of inbound HTTP traffic to check that it is properly formed, and all internal payload data must be decrypted then decoded before it is accessed. The decryption and decoding processes refuse to accept malformed data, thus enforcing that all inbound network data abides by my internal definitions. On the client side I have similar input validation for all server responses, and the TCP streams are "dumb" and so aren't vulnerable to malicious input from that point.

Test Identification

3.5.1. Synthetic Testing

I intend to use Cloudflare's speed test [98] to test network performance in various situations. My reason for choosing this specific speed test is that it includes significantly more technical information than other tests. This technical information will help me debug and optimise my code.

3.5.2. Automated Testing

I will develop unit tests throughout the development process, measuring functionality (e.g. successful parsing of libtransit messages) and performance (e.g. time taken to parse a message or derive a key). Additionally, I will continuously develop with real-world tests, utilising the Firefox web browser and Curl to test loading real-world websites.

These tests will ensure that my code components work, even when modified.

I will use the username test and password 1234 in my test system.

3.5.3. Real-World Testing

I will continuously test my code during development, using real-world websites like google.com, ifconfig.me, and discord.com in conjunction with tools like Firefox [99] and Curl [100] to ensure that my code functions correctly. I will also continuously test circumvention effectiveness through the Sophos XG TLS-decrypting deep-packet inspection firewall [16].

Further Data

I intend to use speed tests such as <https://fast.com> and <https://speed.cloudflare.com> to test the speed of my circumvention system. I will use <https://discord.com> as a reference for a complex web application and <https://ifconfig.me> as a reference for a simple web application. <https://alexcj.co.uk> is somewhere in the middle, and I know that it's hosted as a static site at the local Cloudflare datacenter in London and so should have consistent performance for testing. I will continue to use the various tests outlined in the previous section, including circumvention tests against Sophos XG [16].

DEVELOPMENT

Iterative Development

4.1.1. SOCKS Format

I chose `binrw` [101] to implement my SOCKS protocol processing because it offered a very nice declarative API that made it very easy to implement the SOCKS4 RFC. It, in effect, created a seamless conversion between the Rust algebraic type system and the SOCKS4 protocol. I tested everything using unit tests. The code is at Section 4.2.1.

4.1.2. Early Message Serialisation

I implemented the first version of my wire message format using `binrw` [101], the same tool I used for implementing the SOCKS protocol.

Here are my initial definitions. Everything is thoroughly unit tested, and I made extensive use of sum types. The code is at Section 4.2.2. I wrote an algorithm to combine multiple message formats into a single block by storing offsets and then a binary block that is split up and individually parsed:

```

1  impl MultipleMessages {
2      pub fn from_messages(messages: Vec<Message>) -> Self {
3          if messages.len() > u8::MAX as usize {
4              panic!("Cannot encode more than 255 messages into a single
5              buffer.");
6          }
7
8          let mut sizes: Vec<u32> = Vec::with_capacity(messages.len());
9          let mut data: Vec<u8> = Vec::new();
10
11          for message in messages {
12              let message_data = match message {
13                  Message::StreamMessage(stream_message) => {
14                      let mut writer = Cursor::new(Vec::new());
15                      writer.write_be(&stream_message).unwrap();
16                      writer.into_inner()
17                  },
18                  Message::CreateSocketMessage(create_socket_message) => {
19                      let mut writer = Cursor::new(Vec::new());
20                      writer.write_be(&create_socket_message).unwrap();
21                      writer.into_inner()
22                  },
23                  Message::CloseSocketMessage(close_socket_message) => {
24                      let mut writer = Cursor::new(Vec::new());
25                      writer.write_be(&close_socket_message).unwrap();
26                      writer.into_inner()
27                  },
28              };
29
30              sizes.push(message_data.len() as u32);
31              data.extend(message_data);
32          }
33
34          Self {
35              sizes,
36              data
37          }

```

```

38
39     pub fn to_bytes(&self) -> Vec<u8> {
40         // Write the data to a multiple messages buffer
41         let buffer = Vec::with_capacity(1 + self.sizes.len() * 4 +
self.data.len());
42         let mut writer = Cursor::new(buffer);
43
44         writer.write_be(&self).unwrap();
45
46         writer.into_inner()
47     }
48
49     pub fn from_bytes(buffer: Vec<u8>) -> Self {
50         let mut reader = Cursor::new(buffer);
51         reader.read_be::<MultipleMessages>().unwrap()
52     }
53
54     pub fn pull_messages(&self) -> Vec<Message> {
55         let mut messages: Vec<Message> = Vec::with_capacity(self.sizes.len());
56
57         let mut i = 0;
58         let mut base_offset = 0;
59
60         println!("Self.data: {:?} (Length {})", self.data, self.data.len());
61
62         loop {
63             let data_start = base_offset;
64             let data_end = base_offset + self.sizes[i] as usize;
65
66             println!("Data start: {}", data_start);
67             println!("Data end: {}", data_end);
68             println!("Sizes: {:?}", self.sizes);
69             println!("Current offset: {}", i);
70
71             // Get those bytes
72             let message_data = self.data[data_start..data_end].to_vec();
73
74             // Read to a message struct
75             println!("Message data: {:?}", message_data);
76             let mut data_reader = Cursor::new(message_data);
77
78             let messagetype_reader = self.data[data_start];
79
80             let message_type = match messagetype_reader {
81                 0 => MessageType::CreateSocket,
82                 1 => MessageType::CloseSocket,
83                 2 => MessageType::SendStreamPacket,
84                 3 => MessageType::ReceiveStreamPacket,
85                 _ => panic!("Invalid message type"),
86             };
87
88             let message = match message_type {
89                 MessageType::CreateSocket => {
90                     Message::CreateSocketMessage(data_reader.read_be::<CreateSocketMessage>()
91                         .unwrap())
92                 },
93                 MessageType::CloseSocket => {
94                     Message::CloseSocketMessage(data_reader.read_be::<CloseSocketMessage>()
95                         .unwrap())
96                 },

```

```

97         MessageType::SendStreamPacket => {
98     Message::StreamMessage(data_reader.read_be::<StreamMessage>().unwrap())
99         },
100        MessageType::ReceiveStreamPacket => {
101    Message::StreamMessage(data_reader.read_be::<StreamMessage>().unwrap())
102        },
103    };
104
105    // Push to messages
106    messages.push(message);
107
108    // Check if we should stop
109    if i > self.sizes.len() - 2 {
110        break;
111    }
112
113    i = i + 1;
114    base_offset = data_end;
115    }
116
117    messages
118    }
119    }

```

The algorithm doubled as an input validation system.

I then used the Criterion benchmarking framework to get some statistics with the code at Section 4.2.3. I found the performance at the edge of acceptability - I may need to take another look later.

Messages	Payload Size	Total Data Processed	Time
50	100b	5kb	28 μ s
50	1kb	50kb	44 μ s
50	10kb	500kb	210 μ s
50	100kb	5mb	8.4ms
10	1mb	10mb	16ms
10	10mb	100mb	150ms
10	100mb	1000mb	1.5s

4.1.3. Security Considerations

I decided to develop my system without obfuscating public names (like paths), opting to hide those identification vectors later in order to improve debugging during development via clear names.

4.1.4. libsecrets

I evaluated some encryption libraries:

- <https://github.com/project-oak/tink-rust> (AEAD)
- <https://github.com/mcginty/snow> (Noise Protocol)
- <https://github.com/citahub/libsm> (Chinese)
- <https://github.com/orion-rs/orion> (AEAD + Low Level ChaCha20 Stream Cipher)
- <https://github.com/briansmith/ring> (Requires C toolchain)

- <https://github.com/RustCrypto/AEADs/tree/master/aes-gcm> (AEAD)
- <https://github.com/RustCrypto/AEADs/tree/master/chacha20poly1305> (AEAD)
- <https://github.com/RustCrypto/block-ciphers/tree/master/aes> (Low level AES, HAZMAT due to no AEAD)

The Noise Protocol is too high-level for my use case and may provide a fingerprinting vector. Cryptographic primitives originating from China are clearly undesirable due to the risk of backdoors.

I initially thought that AEAD (Authenticated Encryption with Associated Data) was undesirable because the associated data would make it easier to fingerprint. I ruled out all but <https://github.com/orion-rs/orion> and <https://github.com/RustCrypto/block-ciphers/tree/master/aes>.

Looking at the two algorithms, ChaCha20's stream nature means that it can operate on n bits, instead of $n\%$ block size. I liked the idea of not worrying about padding and being able to send arbitrary-length payloads to reduce an identification vector, so I decided to use Orion.

It's marked as "hazardous" in the docs (<https://docs.rs/orion/latest/orion/hazardous/stream/chacha20/index.html>), which include some notices:

- Don't reuse nonces.
- Don't rely on it for data integrity.
- It has no mitigations for message replay attacks.
- xchacha20 is recommended so that you can use a random nonce instead of worrying about generating safely and without collisions - the non-extended version does not have a large enough nonce to guarantee that no collisions occur.

I started implementing it in the code and noticed that it created a "poly1305 tag", which I suspected could be used for fingerprinting (and therefore blocking). A process diagram shows that it does, indeed, use additional data:

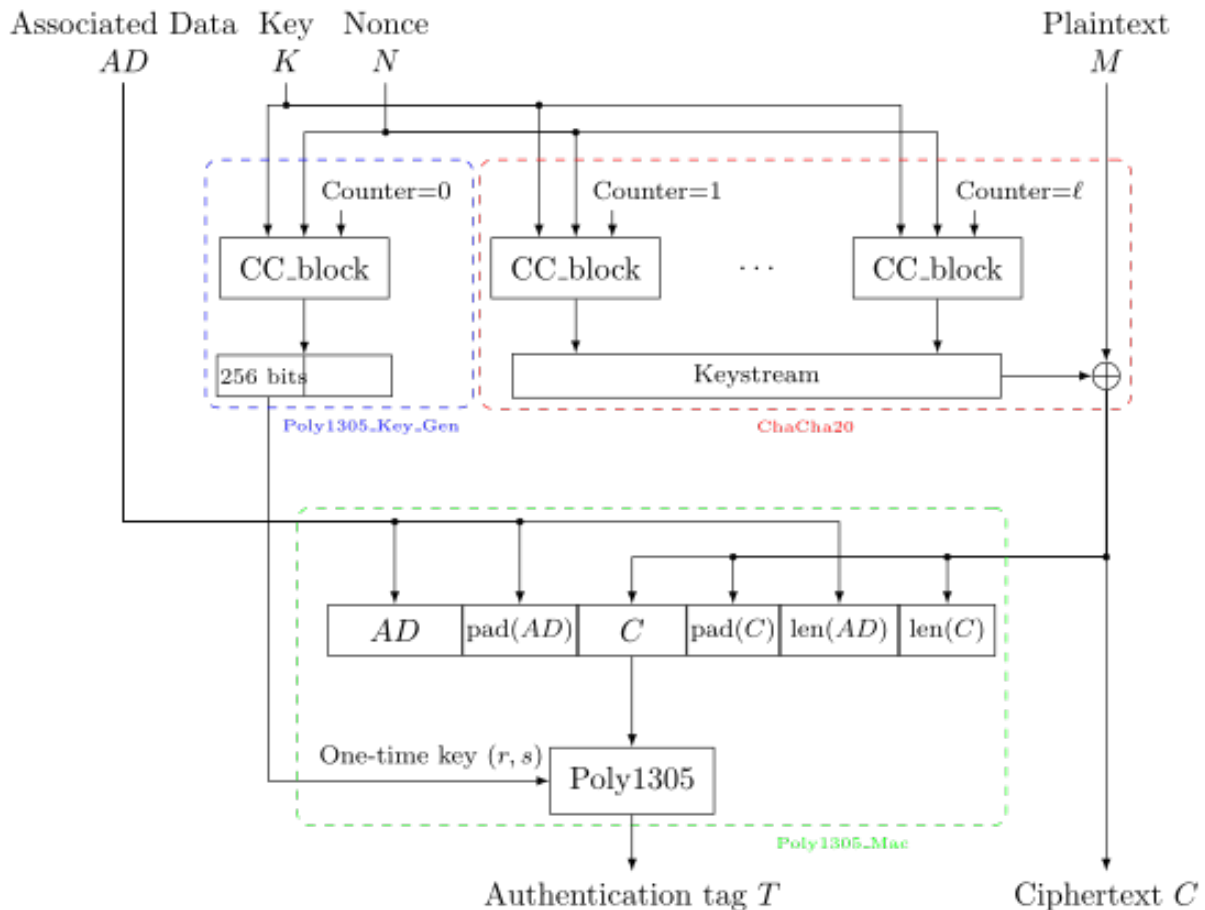


Figure 42: XChaCha20-Poly1305 diagram [102].

I decided to ask a question on the cryptography stack exchange, as I was clearly in need of some additional information. I received an excellent reply [103] that explained that the associated data is stored in an obfuscated manner, indistinguishable from the general string in a way that protects it from use as a fingerprinting attack method. He recommended a non-repeating nonce as the only major requirement.

While I could go on using `xchacha20poly1305`, I decided to reevaluate my algorithmic choices. <https://github.com/RustCrypto/AEADs> has <https://docs.rs/chacha20poly1305/latest/chacha20poly1305/>, which appears to be better built and have some level of integrity built-in.

This reduced what I needed to implement - merely a key derivation, encryption, and decryption function.

I implemented that in Section 4.2.4. Note my use of a random number generator for the nonce, eliminating patterns.

I then tested it with some unit tests, included inline:

```
1  #[test]
2  fn test_encrypt_decrypt() {
3      let key = form_key(b"01234567890123456789012345678901");
4      let data = b"Hello world!";
5      let encrypted = encrypt(data, &key).unwrap();
6      let decrypted = decrypt(&encrypted, &key).unwrap();
7      assert_eq!(data, decrypted.as_slice());
8  }
```

This encryption test worked well, but when tested on incorrect data I ran into an error when forming the (incorrect) key:

```
1 #[test]
2 fn test_decryption_wrong_key() {
3     let key = form_key(b"01234567890123456789012345678901");
4     let data = b"Hello world!";
5     let encrypted = encrypt(data, &key).unwrap();
6     let key = form_key(b"Hello World!");
7     let decrypted = decrypt(&encrypted, &key);
8     assert!(decrypted.is_err());
9 }
```

```
1 thread 'tests::test_decryption_wrong_key' panicked at 'Slice must be the same
length as the array'
```

Some additional testing found that the key needed to be 32 bytes long. I had three ways to fix this issue - I could require the user use 32-byte keys, trim/pad the key to meet the requirement, or use a key derivation function. 1 is bad for the user, 2 opens the risk of security vulnerabilities, and 3 is standard industry practice. I decided to implement a PKDF.

While ordinarily you would make a PKDF computationally intensive, my attack model does not involve offline cracking of derived keys to find the original passwords and I actually benefit from *cheap* derivation (I expect to have one per user).

I settled on SHA-2/256 as it is fast, cryptographically secure, and outputs 32 bytes (256 bits) exactly. The same organisation that made my encryption primitives also made a SHA-2 library, so I was able to make a very simple derivation function:

```
1 use sha2::{Sha256, Digest};
2
3 pub fn form_key(data: &[u8]) -> Key<ChaCha20Poly1305> {
4     let mut hasher = Sha256::new();
5     hasher.update(data);
6     hasher.finalize()
7 }
```

I then ran some benchmarking, using the same test framework as before. I found that `data.clone()` in the allocation control experiment was optimised out by the compiler, so I made it explicit with `Box::new(data.clone())` to create a unique heap-allocated smart pointer to my data. I got the following data:

Type	Small	1kb	1mb	10mb
Encryption	1.5μs	2.4μs	1.3ms	13ms
Decryption	1.0μs	2.4μs	1.3ms	13ms
Allocation (Control)	140ps	280ps	280ps	280ps

Key derivation took 65ns.

I may switch to `aes_gcm` at a later point to reduce client CPU usage, as AES has dedicated CPU instructions.

4.1.5. Client-Transit

I then created the core socket functionality in `client-transit`. My idea at the time was that I would build my own virtual sockets system, then implement circumvention on top of that. It didn't turn out that way, but much of this code ended up in the final version.

I began by implementing a new `EncryptionKey` type:

```
1 pub type EncryptionKey = Key<ChaCha20Poly1305>;
```

I then defined the transit socket struct. Rust doesn't really have classes, but it does have structs, methods on structs, and a system called "traits". You can sort of think of it as a class system that has different approaches to the problems traditionally solved by inheritance and polymorphism. This is all done to improve performance and promote composition over inheritance.

```
1 pub struct TransitSocket {
2     target: String, // Base URL, incl. protocol
3     key: EncryptionKey, // Encryption key for libsecrets
4     control_client: Client, // Does the initial encryption agreement
5     send_buffer: RwLock<Vec<libtransit::Message>>, // Messages to send
6     recv_buffer: RwLock<Vec<libtransit::Message>>, // Messages received, but not
7     yet sent to user of this socket
8     server_statistics: ServerStatistics, // Statistics about the server to
9     inform client congestion control
10    client_identifier: String, // Identifier for this client, used as a cookie
11    hybrid_client_count: usize, // Number of hybrid clients (Both push and pull)
12    pull_client_count: usize, // Number of pull clients
13    timeout_time: usize, // Time in seconds before a request is considered timed
14    out
15    headers: HeaderMap, // Headers to send with requests. Includes client
16    identifier
17 }
```

An `RwLock` is a kind of synchronisation construct - a read/write lock, which allows multiple readers but only one writer. `ServerStatistics` is another struct of mine, and `Vecs` are dynamically sized heap-allocated lists implemented as contiguous memory.

I decided to use the builder pattern for creating the transit socket, under the rationale that it makes it easier to add parameters later. In practice I only ever instantiated it once in my program so it was completely overkill, but I liked it, and it made my code cleaner if more verbose. Section 4.2.5 .

I have a function with the sole purpose of generating plausible-looking HTTP headers:

```
1 fn generate_headers(client_identifier: String, referer: String) -> HeaderMap {
2     let mut headers = HeaderMap::new();
3     headers.insert("Cookie", HeaderValue::from_str(&format!("session_token={}",
4     client_identifier)).unwrap());
5     headers.insert("Accept-Language", HeaderValue::from_str("en-
6     GB,en;q=0.5").unwrap());
7     headers.insert("Cache-Control", HeaderValue::from_str("no-cache").unwrap());
8     headers.insert("Connection", HeaderValue::from_str("keep-alive").unwrap());
9     headers.insert("Pragma", HeaderValue::from_str("no-cache").unwrap());
10    headers.insert("Referer", HeaderValue::from_str(&referer).unwrap());
```

```

9     headers.insert("Sec-Fetch-Dest",
HeaderValue::from_str("document").unwrap());
10    headers.insert("Sec-Fetch-Mode",
HeaderValue::from_str("navigate").unwrap());
11    headers.insert("Sec-Fetch-Site", HeaderValue::from_str("same-
origin").unwrap());
12    headers.insert("TE", HeaderValue::from_str("trailers").unwrap());
13    headers.insert("User-Agent", HeaderValue::from_str("Mozilla/5.0 (X11; Linux
x86_64; rv:109.0) Gecko/20100101 Firefox/115.0").unwrap());
14
15    headers
16 }

```

I then implemented the `greet_server` method, which tells the server prepare for us and checks that we agree on encryption schemes and passwords. I used variable padding to avoid making a recognisable fingerprint. This code includes a sum type for the various possible errors and a trait for auto-conversion of the `?` operator into a managed result monad.

```

1  pub enum TransitInitError {
2      EncryptionError(libsecrets::EncryptionError),
3      RequestError(request::Error),
4      ConnectionFailed,
5      ConnectionDenied,
6      LiedResponse,
7  }
8
9  impl From<libsecrets::EncryptionError> for TransitInitError {
10     fn from(error: libsecrets::EncryptionError) -> Self {
11         Self::EncryptionError(error)
12     }
13 }
14
15 impl TransitSocket {
16     fn greet_server(&self) -> Result<(), TransitInitError> {
17         let mut data = "Hello!".to_string();
18         // Pad with random amount of spaces
19         let mut rng = rand::thread_rng();
20         let amount = rng.gen_range(0..100);
21         data = data + &" ".repeat(amount);
22
23         let encrypted = libsecrets::encrypt(data.as_bytes(), &self.key)?;
24
25         let response = self.control_client.post(&format!("{}",/submit",
self.target))
26             .body(encrypted)
27             .headers(self.headers.clone())
28             .send();
29
30         match response {
31             Ok(response) => {
32                 if response.status() != 200 {
33                     return Err(TransitInitError::ConnectionDenied);
34                 }
35                 // Try to decrypt the response
36                 let encrypted = response.bytes();
37                 match encrypted {
38                     Ok(encrypted) => {
39                         let decrypted = libsecrets::decrypt(&encrypted,
&self.key)?;

```

```

40         // Convert to string
41         let decrypted = String::from_utf8(decrypted);
42         match decrypted {
43             Ok(decrypted) => {
44                 // Remove spaces
45                 let decrypted = decrypted.trim_end_matches(' ');
46                 if decrypted != "CONNECTION ACCEPTED" {
47                     return Err(TransitInitError::LiedResponse);
48                 }
49             },
50             Err(_) => {
51                 return Err(TransitInitError::LiedResponse);
52             }
53         },
54         Err(error) => {
55             return Err(TransitInitError::RequestError(error));
56         }
57     }
58 }
59
60 },
61 Err(error) => {
62     if error.is_timeout() || error.is_connect() {
63         return Err(TransitInitError::ConnectionFailed);
64     } else {
65         return Err(TransitInitError::RequestError(error));
66     }
67 }
68 }
69
70 // If we've gotten here, the connection was successful
71 // We should be ready to talk to the server now
72
73 Ok(())
74 }
75 }

```

4.1.6. Server-Side Configuration Management and Server Initialisation

I utilised the excellent Serde [104] data serialisation framework to implement configuration loading on the server. Serde integrates input validation into parsing.

```

1 use serde::{Deserialize, Serialize};
2 use std::fs;
3
4 #[derive(Serialize, Deserialize, Debug, Clone)]
5 pub struct Config {
6     pub host: String,
7     pub port: u16,
8     pub users: Vec<User>,
9 }
10
11 #[derive(Serialize, Deserialize, Debug, Clone)]
12 pub struct User {
13     pub password: String,
14 }
15
16 impl Default for Config {
17     fn default() -> Self {
18         Config {
19             host: "127.0.0.1".to_string(),

```

```

20         port: 80,
21         users: Vec::new(),
22     }
23 }
24 }
25
26 pub fn load_config() -> Config {
27     // Load the configuration file into a Config struct
28     let contents = match fs::read_to_string("server.config.toml") {
29         Ok(contents) => contents,
30         Err(_) => {
31             println!("No configuration file found, creating one...");
32             let config = Config::default();
33             let toml = toml::to_string(&config).unwrap();
34             fs::write("server.config.toml", toml).unwrap();
35             return config;
36         }
37     };
38
39     let config: Config = toml::from_str(&contents).unwrap();
40     config
41 }

```

My configuration is in the `toml` format. I chose `toml` because it's simple, intuitive, and well-supported. My code generated the following default config:

```

1 host = "127.0.0.1"
2 port = 8000
3 users = []

```

I then setup the initial web server using `actix_web`, pulling in that configuration and distributing it to all inbound requests:

```

1 #[actix_web::main]
2 async fn main() -> std::io::Result<()> {
3     println!("Loading configuration...");
4     let configuration = config::load_config();
5
6     let appstate = web::Data::new(AppState {
7         config: configuration.clone(),
8     });
9
10    println!("Starting server...");
11    HttpServer::new(move || {
12        App::new()
13            .app_data(appstate.clone())
14            .service(index)
15    })
16    .bind((configuration.host, configuration.port))?
17    .run()
18    .await
19 }

```

4.1.7. Lock-Free Data Structures

I then setup the rest of the server (See Section 4.2.6), including switching from a primitive `RwLock` of a `Vec` of user sessions to an accelerated, lock-free concurrent hashmap. I evaluated the following existing libraries:

- <https://github.com/xacrimon/dashmap>
- <https://gitlab.redox-os.org/redox-os/chashmap>
- <https://github.com/saligrama/concacha>
- <https://github.com/jonhoo/flurry>
- <https://github.com/jonhoo/evmap>
- https://docs.rs/crossbeam-skiplist/latest/crossbeam_skiplist/

I decided to use Dashmap. It's based on hashbrown, a very fast hashing algorithm used in the Rust-compiler, is widely used, has no java ancestry, and does not rely on manual write flushing to achieve eventual consistency.

4.1.8. Client Identification

I decided to change the client identifier from an arbitrary string to 16 fixed characters. I decided to avoid UTF-8 annoyances by remaining in ASCII and storing data as `u8` rather than a 4-byte `char`. I was conscious of the risk of a birthday-problem [105] collision. I read up on the birthday problem and found an excellent blog post on it [106], which provided a proof for a mathematical approximation of collision probabilities. The approximation with the least error was as follows:

$$1 - e^{-\frac{k(k-1)}{2N}}$$

Using this approximation, I found that with a “bucket count” of 255^{16} and 10,000 sessions the collision probability was 0. I kept increasing session count - we could have far more in an adversarial scenario - and it took until 1,000,000,000,000 sessions before the probability became $1.55 \cdot 10^{-15}$ - a miniscule number.

4.1.9. Server-Side State Storage

I setup my new state storage on the server, utilising the dashmap:

```
1 struct AppState {
2     config: config::Config,
3     sessions: DashMap<[u8; 16],
4     transit_socket::TransitSocket>,
5 }
```

This implied the need for a `TransitSocket`. I created a very early version, intending to expand it later:

```
1 use libsecrets::EncryptionKey;
2
3 pub struct ClientStatistics {
4     pub to_send: usize,
5     pub to_reply: usize,
6 }
7
8 pub struct TransitSocket {
9     pub key: EncryptionKey,
10    pub client_statistics: ClientStatistics,
11    // TODO information for communicating with SOCKS threads, returning
12    // requests, etc. Need to figure out how to do this.
13 }
```

I also defined a `users` section that contains already-derived keys for each user. Users have a `name`, but this is just for logging purposes for now.

```

1 struct AppState {
2     config: Config::Config,
3     sessions: DashMap<u8; 16>, transit_socket::TransitSocket>,
4     users: Vec<User>,
5 }
6
7 struct User {
8     name: String,
9     password: String,
10    key: libsecrets::EncryptionKey,
11 }

```

I modified my initialisation as appropriate - it can be viewed at Section 4.2.6.

4.1.10. Server-Side Connection Process

I then implemented the server-side connection process, commenting it extensively. This process involves a lot of client-side validation, protecting against malformed or malicious requests. Notably, it lies to the user that the page doesn't exist if the validation process fails. This is done to protect against active probing - if it told the truth it would be trivial for an active probe to identify Hypertunnel servers.

This is an example of layered input validation in order to protect security and data integrity.

```

1 // First request, which associates a client identifier with a key and proves
  // the server is legitimate
2 #[post("/submit")]
3 async fn client_greeting(app_state: web::Data<AppState>, req: HttpRequest,
4 body_bytes: Bytes) -> impl Responder {
5     dprintln!("Received request to /submit");
6     // Get cookie for client identifier
7     let token = req.cookie("token");
8
9     // Check if token exists and if so, parse it
10    let token = match token {
11        Some(token) => {
12            // Check it's 16 bytes of u8s
13            let token = token.value().to_string();
14            if token.len() != 16 {
15                // Return 404
16                return HttpResponse::NotFound().body("No page exists");
17            }
18
19            // Convert to bytes
20            let token_bytes = token.as_bytes();
21
22            // Convert to array
23            let mut token = [0u8; 16];
24            for i in 0..16 {
25                token[i] = token_bytes[i];
26            }
27
28            dprintln!("Token correct!: {:?}", token);
29
30            // Implicit return
31            token
32        }
33        None => {
34            // Return 404 - resist active probing by not telling the client
35            what went wrong
36        }
37    }
38 }

```

```

34         dprintln!("No token supplied!");
35         return HttpResponse::NotFound().body("No page exists");
36     }
37 };
38
39 // Get body
40 let body = body_bytes.to_vec();
41
42 // Iterate through users trying their keys. TODO consider Rayon here for
performance
43 let mut key: Option<EncryptionKey> = None;
44 for user in &app_state.users {
45     // Attempt to decrypt body with user's key
46     let decrypted = match libsecrets::decrypt(&body, &user.key) {
47         Ok(decrypted) => decrypted, // If it succeeds, pull out content
from Result<>
48         Err(_) => continue, // If it fails, try the next user
49     };
50
51     // Convert to string. It should always be valid ASCII, and UTF-8 is a
superset of ASCII
52     let decrypted = match String::from_utf8(decrypted) {
53         Ok(decrypted) => decrypted,
54         Err(_) => continue,
55     };
56
57     // Check if decrypted data contains "Hello!"
58     if decrypted.contains(&"Hello!".to_string()) {
59         // Set key
60         key = Some(user.key.clone()); // We've found it!
61         dprintln!("Key found! User: {}", user.name);
62         break;
63     }
64 }
65
66 // Check if key was found. If so, extract contents of Option<>
67 let key = match key {
68     Some(key) => key,
69     None => {
70         // Return 404
71         dprintln!("No key found!");
72         return HttpResponse::NotFound().body("No page exists");
73     }
74 };
75
76 // Generate response. Encrypted with their key to show we're legitimate
77 let response_text = "CONNECTION ACCEPTED";
78
79 // Pad with 0-100 spaces to avoid fingerprinting based on response length
80 let mut rng = rand::thread_rng();
81 let amount = rng.gen_range(0..100);
82 let response_text = response_text.to_string() + &" ".repeat(amount);
83
84 // Encrypt response
85 let encrypted = match libsecrets::encrypt(response_text.as_bytes(), &key)
{
86     Ok(encrypted) => encrypted,
87     Err(_) => { // This should never happen, but if it does, return 404
88         // Return 404
89         return HttpResponse::NotFound().body("No page exists");
90     }
91 };

```

```

92
93     dprintln!("Formed response: {:?}", encrypted);
94
95     // Add it to the sessions
96     let socket = TransitSocket {
97         key,
98         client_statistics: ClientStatistics {
99             to_send: 0,
100             to_reply: 0,
101         },
102     };
103
104     app_state.sessions.insert(token, socket);
105
106     dprintln!("Added session to sessions; returning response");
107
108     // Return encrypted response
109     HttpResponse::Ok().body(encrypted)
110 }

```

It runs!

```

1 [alex@alexpc server-monolith]$ cargo run
2     Finished dev [unoptimized + debuginfo] target(s) in 0.09s
3     Running `/home/alex/programming/data-smuggler/Source/target/debug/server-
4 monolith`
5 Loading configuration...
6 Hashing user keys...
7 Hashed key for user 'Test'
8 Starting server at 127.0.0.1:8000

```

4.1.11. Integrated Testing

I created a crate for integration tests that just pulls in client transit, starts up the server, and connects to it:

```

1 use client_transit::{TransitSocket, TransitSocketBuilder};
2
3 #[cfg(test)]
4 mod tests {
5     use super::*;
6
7     #[test]
8     fn check_socket_connect() {
9         let mut socket = TransitSocketBuilder::new()
10             .with_target("http://localhost:8000".to_string())
11             .with_password("12345".to_string())
12             .with_timeout_time(1)
13             .build();
14
15         socket.connect().expect("Failed to connect to server");
16     }
17 }

```

The full code is at Section 4.2.7 .

My testing found an off-by-one bug:

```

1 ---- tests::check_socket_connect stdout ----

```

```

2 thread 'tests::check_socket_connect' panicked at 'index out of bounds: the len is
  16 but the index is 16', client-transit/src/builder.rs:91:21
3 note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

```

I found the source of the issue in my client identifier formation - I forgot that Rust indexes differently to Python.

```

1 let client_identifier = match self.client_identifier {
2     Some(client_identifier) => client_identifier,
3     None => {
4         // Create a random client identifier string
5         let mut client_identifier = [0; 16];
6         for i in 0..17 {
7             client_identifier[i] = rand::random::<u8>();
8         }
9         client_identifier
10    }
11 };

```

My next connection attempt ran into the client error

```

1 Failed to connect to server: ConnectionDenied

```

and on the server:

```

1 No token (client identifier) supplied!

```

I found that I was using different names for the token on the client and server - `session_token` on the client and `token` on the server. I moved everything to `token`.

My connection was still being denied. I tracked it down, using some additional logging, to the token length being incorrect because I wasn't properly decoding from its hex form to its byte form. That was quickly fixed with a hex conversion and type cast:

```

1 let token_bytes = hex::decode(token_hex).unwrap();
2 // [...] //
3 let token: [u8; 16] = token_bytes[..16].try_into().unwrap();

```

This change fixed the code, allowing all tests to pass!

```

1 Loading configuration...
2 Hashing user keys...
3 Hashed key for user 'Test'
4 Starting server at 127.0.0.1:8000
5 Received request to /submit
6 Hex token: 3ad66d7c191a4405c84d68c338113878
7 Token correct!: [58, 214, 109, 124, 25, 26, 68, 5, 200, 77, 104, 195, 56, 17,
  56, 120]
8 Key found! User: Test
  Formed response: [120, 86, 55, 89, 177, 84, 235, 41, 80, 216, 216, 192, 86, 99,
  83, 93, 206, 147, 149, 172, 12, 171, 28, 71, 8, 64, 179, 43, 94, 177, 253, 5,
  244, 197, 130, 250, 231, 102, 18, 51, 88, 56, 10, 52, 30, 6, 78, 194, 193, 199,
9  220, 107, 207, 213, 255, 11, 71, 94, 227, 241, 107, 243, 146, 162, 148, 38, 132,
  93, 60, 193, 182, 120, 188, 23, 243, 61, 197, 79, 176, 78, 15, 183, 200, 146,

```

```

83, 26, 46, 137, 39, 79, 96, 227, 39, 243, 200, 202, 205, 182, 254, 247, 5, 153,
218, 158, 191, 118, 49, 237, 60, 241, 176, 234, 81, 39]
10 Added session to sessions; returning response

```

4.1.12. Threaded Decision Making

I went through an extended evaluation of different libraries and architectures that would meet my performance and functionality requirements. I will not go into extensive detail.

I wanted to make a mechanism for taking protocol messages and sending them onto relevant handlers through direct message passing via abstractions on `TransitSocket`. I wanted as little as possible done in the HTTP handler to minimize latency before things are sent back. We also need to actually create the SOCKS TCP thread handlers so we have something to talk to.

I thought about the following method (from my notes):

```

We'll have an initialize method that starts a hardware thread that processes the
MultipleMessages into a vector of messages, then iterates through them. Either it
1 creates a new *soft* thread (IO-bound) using Tokio, storing its message-passing
handler in a hashbrown hashmap (non-thread-shared, local to the function, to
increase L1 cache coherency) or it looks up that handler and sends on the stream
data.
2
3 The handler is a simple TCP handler that sends on SOCKS info. It's a green thread
because we'll have a lot of them and it's mostly IO bound.
4
5 There might be some funny tokio async stuff going on where the MultipleMessages
is using too much CPU time and the green threads don't get time to do their IO.
If so we'll look into detachment options.

```

Then I realised that this was false:

```

We're ignoring the above because it turns out Tokio does automatic thread
1 balancing (https://github.com/tokio-rs/tokio/discussions/3858) meaning we just
create a non-blocking Tokio task and it'll handle it all for us!
2
3 Except there's a few things I just learnt. First, actix runs in single-threaded
mode by default. This means that I don't actually need to make my actix shared
state thread-safe (and I can actually remove dashmap and replace it with
hashbrown) [TODO] but it also means that I'm not sure creating new tokio tasks
will *actually* make it run in a new thread.

```

Then I realised that actix is single threaded. I considered a new approach:

```

1 - So actix is single threaded
- Messages still cannot be unpacked inline. Even if I could take the latency it's
2 just too CPU blocking. For now I'll await a tokio task that I spawn to do it.
Still have the latency, don't have the CPU blocking.
- But how do I know that tokio's done in a proper thread?
3 `tokio::task::spawn_blocking` should guarantee that
- Once the message is unpacked we lookup the (tokio) TCP socket handlers' message
4 passing in a hashbrown map / create it. This might be sorta CPU intensive
iterating through the messages and sending them on, we'll have to see.

```

There's a problem: `actix-web` uses the `actix` ecosystem, which uses actors. I'm probably better off telling `actix` to multithread and implementing things using actors. From my notes:

```

1 1. Request comes in
2 2. Lookup relevant transit socket
3 4. Transit socket message-passes to the unpacking actor (multiple?)
4 5. Message-passing to the relevant TCP actors
5 6. TCP actors send data on
   7. What now? The transit socket isn't an actor, I can't message-pass to it. I
   need a way to either store it in a transit socket buffer (if there's nothing
6 listening for immediate data) or message pass directly to something that's
   waiting to return data. Except I also need to worry about buffer
   sizes and... I'll do this when I'm back from cambridge

```

I was doing Microsoft work experience in Cambridge at the time, so I wasn't able to test my ideas.

I then discovered that `Actix` barely uses actors anymore, and while it technically runs in single threaded mode it can spawn many workers. Each worker has its own `Tokio` task.

My plan at the time was to simply spawn a `tokio async` worker for expensive tasks using the same major `tokio` version, stick asynchronous `tokio` message passers in `dashmaps` for moving data around, and... all should be fine. It would follow the following flow:

```

1 1. Request comes in
2 2. Create new transit socket
3 3. Store that in the sessions hashmap
4 4. Data comes in again later
5 5. Access transit socket
6 6. Transit socket checks if SOCKS stream already exists
7 7. If it doesn't, create a new asynchronous TCP handler (on whatever thread was
   dispatched for the request)
8 8. Store the tokio mspc to talk to that in the future
9 10. If it does, talk to the tokio mspc to send that data on
   11. Each transit socket has a shared tokio mspc for returning data back from TCP
10 handlers - for now we won't worry about sending back larger buffers. In the
   future we can add to a middleware buffer until it's over x size then send it
   back.

```

I ran into an issue:

```

1 We have a problem here: Sometimes we want to await the mspc across tokio
   instances. I don't know if this will work or not as I don't know how the mspc is
   implemented: Is it a standard one that happens to have a nice `.await` or is it
   implemented in the runtime? If it's implemented in the runtime we may need to use
   traditional threaded mspcs with a layer I build on top that allows for unblocking
   waiting (however that's meant to work...).

```

Perhaps we'd be better off spawning a hardware thread for each `TCP` client (sorta expensive...) and using threaded message passing (sorta expensive...). Looking at `Linux` threads, the actual startup delay is fine ($10\mu s$). I'm not sure how bad the cross-thread overhead would be.

I decided to go with hardware threads for the `TCP` sockets and optimise later - "Premature optimisation is the root of all evil":

```
1 1. Request comes in
2 2. Create new transit socket
3 3. Store that in the sessions hashmap
4 4. Data comes in again later
5 5. Access transit socket
6 6. Transit socket checks if SOCKS stream already exists
7 7. If it doesn't, create a new hardware thread TCP handler and store a hardware
  mspc to talk to that in the future
8 8. If it does exist, lookup that hardware mspc and use it to send the data on
```

Each TCP thread can send to the transit socket's `wait_for_new_data` function, but there's a problem: Making that function **not** lock up the entire worker when waiting. I don't want to run `spawn_blocking`, since it's incredibly wasteful.

The `tokio` library appears to offer a message passer that can be used in both synchronous and asynchronous contexts. It would be a pain to implement (the `tokio` provided by `actix` doesn't have that module), but it should work well.

This also works cross-runtime. I considered that I could have a separate thread running a `tokio` runtime that handles **all** tcp connections.

But why have a separate thread, which guarantees cross-thread transfers? I could spawn the TCP in whatever thread the initial connection request came in on, therefore providing a roughly even spread and meaning that you sometimes have intra, rather than inter-runtime communications. This forms the following:

```
1 1. Request comes in
2 2. Create new transit socket
3 3. Store that in the sessions hashmap
4 4. Data comes in again later
5 5. Access transit socket
6 6. Transit socket checks if SOCKS stream already exists
7 7. If it doesn't, create a new hardware thread TCP handler and store a hardware
  mspc to talk to that in the future
8 8. If it does exist, lookup that hardware mspc and use it to send the data on
```

I didn't want that function to block the entire worker. While this could be done with `spawn_blocking`, it's incredibly inefficient. I found that `Tokio` offers functionality for hybrid message passers that can operate in both synchronous and asynchronous contexts. I considered having a dedicated thread with a `tokio` runtime that handles all TCP sockets - but there's actually a better way. If I spawn it on whichever socket the request came in on, it should roughly balance the sockets between workers - and, if you're lucky, `mspc`'s won't even need cross thread transit.

This produces this architecture:

```
1 1. Initiate request comes in
2 2. Create new transit socket
3 3. Store that in the sessions hashmap
4 4. Data comes in again later
5 5. Access transit socket
6 6. Transit socket checks if SOCKS stream already exists
7 7. If it does, lookup the mspc for the asynchronous TCP stream task and send
8 8. If it doesn't, create an asynchronous TCP stream task and send.
```

9. Each TCP stream task is given a reference to the return-data message passer. They can feed data in to that. Returning can call ``get_data(buffsize, modetime)`` on the transit socket. That function pulls from the data message passer. If the buffer size is filled, it returns. After ``modetime`` has elapsed it will return data regardless of if it's filled the ``buffsize``. If data appears **after** ``modetime`` where the buffer was **not filled at all** before ``modetime`` it will wait for the buffer to be filled **or** ``modetime`` to elapse again.

4.1.13. Client Hello Improvements

I modified the client hello to have two major differences: First, to declare identity by simply changing the content -

```
1 let mut data = format!("Hello. Protocol version: {}, client-transit version: {},
  client-name: {}", "0", env!("CARGO_PKG_VERSION"), self.client_name);
```

and, second, to create a new dynamic padding approach that leaks 0 bits of data with message length by creating an even distribution of lengths:

```
1 // Find minimum padding amount such that it's at least 512 bytes
2 let length = data.len();
3 let min_padding = 512 - length;
4
5 // Find maximum padding amount such that it's at most 1024 bytes
6 let max_padding = 1024 - length;
7
8 // Add random padding up to 1024 bytes
9 let mut rng = rand::thread_rng();
10 let amount = rng.gen_range(min_padding..max_padding);
11 data = data + &" ".repeat(amount);
```

I then tested this with the integration test:

```
1 Loading configuration...
2 Hashing user keys...
3 Hashed key for user 'Test'
4 Starting server at 127.0.0.1:8000
5 Received request to /submit
6 Hex token: 74b3fd57dd1ad964aab41022dbe6b0c8
7 Token correct!: [116, 179, 253, 87, 221, 26, 217, 100, 170, 180, 16, 34, 219,
  230, 176, 200]
8 Data decrypted, but does not contain client hello!
9 No key found!
```

This was solved by making the server code check for "Hello. Protocol version: ", rather than the old client hello:

```
1 Loading configuration...
2 Hashing user keys...
3 Hashed key for user 'Test'
4 Starting server at 127.0.0.1:8000
5 Received request to /submit
6 Hex token: ceb3a21cea77366bf084d37e2d72220b
7 Token correct!: [206, 179, 162, 28, 234, 119, 54, 107, 240, 132, 211, 126, 45,
  114, 34, 11]
```

```

8 Key found! User: Test
9 Decrypted data: Hello. Protocol version: 0, client-transit version: 0.1.0,
  client-name: Integration Testing
  Formed response: [234, 191, 245, 166, 203, 239, 188, 213, 9, 79, 20, 122, 165,
10 142, 102, 54, 234, 226, 200, 37, 189, 238, 16, 113, 112, 97, 138, 214, 3, 158,
  147, 126, 132, 22, 158, 183, 206, 52, 183, 158, 133, 248, 53, 180, 169, 74, 35,
  67, 14, 235, 200, 99, 189, 82, 135, 55, 175, 18, 165, 166, 107, 243, 238, 193,
  117, 94, 203, 22, 128, 131, 210, 242, 128, 148, 231, 69, 76, 210, 224, 58, 94,
  69, 63, 184, 211, 98, 135, 131, 164, 251, 131, 117, 67, 55, 59, 243, 17, 50]
11 Added session to sessions; returning response

```

4.1.14. Additional Implementation

I implemented `StreamMessage`. `sequence_number` was intended to be necessary for data reconstruction “if and when” things end up out-of-order, although I ended up guaranteeing order at a lower level instead:

```

1 #[binrw::binrw]
2 #[derive(Debug, PartialEq, Clone)]
3 #[brw(big)]
4 pub struct StreamMessage {
5     pub message_type: MessageType,
6     pub socket_id: SocketID,
7     pub sequence_number: u32,
8
9     pub send_buffer_size: u32,
10    pub receive_buffer_size: u32,
11
12    #[bw(try_calc(u32::try_from(packet_data.len())))]
13    pub packet_length: u32,
14    #[br(count = packet_length)]
15    pub packet_data: Vec<u8>,
16 }

```

I then wrote the following on the server:

```

1 pub async fn process_socks_message(&mut self, stream_message: StreamMessage) {
2     // Check if a stream already exists
3     match self.socks_streams_inbound.get(&stream_message.socket_id) {
4         Some(dashref) => {
5             // Extract the mpsc sender
6             let sender = dashref.value();
7             // Send the message
8
9             sender.send(SocksStreamInboundMessage::StreamMessage(stream_message)).await.unwrap();
10        },
11        None => {
12            // Create a new TCP handler with appropriate message passing
13            let (sender, receiver) = mpsc::channel(10000);
14            arguments = TCPHandlerArguments {
15                receiver,
16                sender: self.tcp_data_assign.clone(), // This will
17                need .resubscribe() later due to the clone
18                address: stream_message.address,
19                port: stream_message.port,
20            };
21        }
22    }
23 }

```

```
21 }
```

I realised that I was not properly handling requests to create a socks stream. There were two ways to handle this - in the first, I never actually send a request to create a new socks stream. I include the address and port in every SOCKS stream request. This has the upside of having no additional latency, and no requirement to reorder requests before they get to the TCP handler. However, there is an overhead of 48 bits (6 bytes) per request and in the strange scenario of a long wait between connection and first message there will be additional latency and it wouldn't quite match spec.

I could also send the request to create a SOCKS proxy, process that, then send back an acknowledgement. That sounds like a high-latency, high-bother task.

I went with the first message. This meant significant changes to `libtransit` to accomodate the protocol change.

This was my previous state:

```
1 use libsecrets::EncryptionKey;
2 use libtransit::{StreamMessage, CloseSocketMessage, SocketID, IPV4, Port};
3 use tokio::sync::{mpsc, broadcast};
4 use dashmap::DashMap;
5
6 pub struct ClientStatistics {
7     pub to_send: usize,
8     pub to_reply: usize,
9 }
10
11 #[derive(Debug)]
12 enum SocksStreamInboundMessage {
13     StreamMessage(StreamMessage),
14     CloseSocketMessage(CloseSocketMessage),
15 }
16
17 pub struct TransitSocket {
18     pub key: EncryptionKey,
19     pub client_statistics: ClientStatistics,
20     // TODO information for communicating with SOCKS threads, returning
21     // requests, etc. Need to figure out how to do this.
22     socks_streams: DashMap<SocketID, mpsc::Sender<SocksStreamInboundMessage>>,
23     tcp_data_return: broadcast::Receiver<StreamMessage>,
24     tcp_data_assign: broadcast::Sender<StreamMessage>,
25 }
26
27 impl TransitSocket {
28     pub fn new(key: EncryptionKey) -> Self {
29         let (tcp_data_assign, tcp_data_return) = broadcast::channel(10000);
30
31         Self {
32             key,
33             client_statistics: ClientStatistics {
34                 to_send: 0,
35                 to_reply: 0,
36             },
37             socks_streams: DashMap::new(),
38             tcp_data_return,
39             tcp_data_assign,
40         }
41     }
42 }
```

```

41     }
42
43     pub async fn process_socks_message(&mut self, message:
44 SocksStreamInboundMessage) {
45         // TODO
46     }
47 }
48 struct TCPHandlerArguments {
49     receiver: mpsc::Receiver<SocksStreamInboundMessage>,
50     sender: broadcast::Sender<StreamMessage>,
51     address: IPV4,
52     port: Port,
53 }
54
55 async fn tcp_handler_task(arguments: TCPHandlerArguments) {
56     // TODO
57 }

```

I decided that, while changing `libtransit` to remove `CreateSocketMessage` and change `StreamMessage` to include ip and port, I should also add (and run) additional benchmarks, remove the transmission of buffer sizes in each stream message (that's higher-level information), and split the client-to-server `StreamMessage` and server-to-client `StreamMessage` (because returned data does not need IP or port).

I also took this as an opportunity to switch to another data serialisation library.

4.1.15. Rapid Serialisation Research

I needed a serialisation library that was low-overhead, very fast, and able to differentiate between message types trivially (either through storing the message type in the first byte or some other method).

I found an online comparison based on real-world benchmarks [107], which compares many libraries. I entered in parameters and compared various datasets, extracting the following data:

Log Round-Trip Data

Crate	Relative Speed	Bottleneck
abomonation	100	CPU
rkyv	90	CPU
speedy	88	CPU
savefile	76	CPU
nanoserde	73	CPU
borsh	69	CPU
bitcode	68	CPU
postcard	68	CPU
bincode	66	CPU
serde_bare	65	CPU
parity-scale-codec	62	CPU

Mesh Round-Trip Data

Crate	Relative Speed	Bottleneck
postcard	100	Memory Bandwidth
speedy	99	Memory Bandwidth
nanoserde	99	Memory Bandwidth
rkyv	99	Memory Bandwidth
abomonation	99	Memory Bandwidth
savefile	99	Memory Bandwidth
parity-scale-codec	55	CPU
bincode	49	CPU
serde_bare	43	CPU
borsh	38	CPU
bitcode	36	CPU

Minecraft World Save Round-Trip Data

Crate	Relative Speed	Bottleneck
abomonation	100	CPU
rkyv	92	CPU
speedy	80	CPU
savefile	77	CPU
bitcode	71	CPU
bincode	68	CPU
borsh	68	CPU
postcard	67	CPU
nanoserde	60	CPU
parity-scale-codec	58	CPU
serde_bare	52	CPU

Mk48 Round-Trip Data

Crate	Relative Speed	Bottleneck
abomonation	100	CPU
speedy	88	CPU
rkyv	84	CPU
savefile	74	CPU
bitcode	61	CPU
postcard	53	CPU
parity-scale-codec	53	CPU
borsh	48	CPU
nanoserde	48	CPU
bincode	41	CPU
msgpacker	32	CPU

Results

I decided to further investigate abomonation, rkyv, speedy, postcard, bincode, borsh, nanoserde, savefile, and bitcode.

Abomonation [108] is incredibly unsafe and insecure, so it was quickly ruled out.

Rkyv [109] is zero-copy and according to [110] it is the fastest of the zero-copy libraries.

Speedy [111] looks to be fast, comparably so to rkyv, but it is not zero-copy. It could prove useful.

Postcard [112] is a little slower. It doesn't need the stdlib but that's useless for this project. Its heapless mode would cause stack overflow.

Bincode [113] appears to be basic but effective.

Borsh [114] looks reasonable and has an admirable focus on security.

Nanoserde [115] is just weird.

Messagepack [116] is fast but has a lot of overhead. I've used it in the past and it works reasonably well.

Savefile [117] seems inappropriate:

You may ask what savefile brings to the table that serde doesn't already do better. The answer is: Not that much! Savefile is less capable, and not as well tested. It does have versioning support built-in as a first class feature.

Savefile is not yet a very widely used project. However, although there may be bugs, the intention is that the quality should be enough for production.

Bitcode [118] does compression and has some interesting features regarding enum compression, but I didn't think it was worth it because of its version instability.

I decided to try out rkyv, quickly discovering that it was giving me weird 16-bit aligned vectors. I decided to try out one of the other promising libraries - borsh, speedy, or bitcode. Speedy isn't very widely used, so I decided to use bincode as it is far more widely used.

However, when I attempted to use it I discovered that all their docs were out of date - it had become unmaintained, with a fledgling (and poorly managed) fork. I switched to borsh and began implementing it - in Section 4.2.8.

Automated testing found an issue:

```
thread 'test_multiple_messages' panicked at 'called `Result::unwrap()` on an
1 `Err` value: Custom { kind: InvalidData, error: "Not all bytes read" }',
  libtransit/src/lib.rs:227:65
2 note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

The issue continued even with a blank payload vector. I discovered that it did not like the generic `Message` type; my solution was to implement my own header system, though later on I simply used two vectors of each type.

I then updated the serialisation benchmark (@09.ModifiedBenchmark). I tested vector arguments as both my choice (`&mut Vec<u8>`) and the correction recommended by the compiler to avoid immediately-dereferenced references (`&mut [u8]`).

Serialisation Benchmarks

Messages	Payload Size	Total Size	Original Time	Vectors Time	Slices Time
50	100b	5kb	28 μ s	5.8 μ s	6.1 μ s
50	1kb	50kb	44 μ s	11 μ s	12.1 μ s
50	10kb	500kb	210 μ s	88 μ s	88 μ s
50	100kb	5mb	8.4ms	5.8ms	6.1ms
10	1mb	10mb	16ms	10ms	11ms
10	10mb	100mb	150ms	75ms	80ms
10	100mb	1000mb	1.5s	720ms	720ms

4.1.16. Transit Socket Progress

I then returned to `transit_socket`. I went back to the plan from earlier:

1. Initiation request comes in
2. Create new transit socket
3. Store that in the sessions hashmap
4. Data comes in again later
5. Access transit socket
6. Transit socket checks if SOCKS stream already exists
7. If it does, lookup the mspc for the asynchronous TCP stream task and send
8. If it doesn't, create an asynchronous TCP stream task and send.
9. Each TCP stream task is given a reference to the return-data message passer. They can feed data in to that. Returning can call `get_data(buffsize, modetime)` on the transit socket. That function pulls from the data message passer. If the buffer size is filled, it returns. After `modetime` has elapsed it will return data regardless of if it's filled the `buffsize`. If data appears **after** `modetime` where the buffer was **not filled at all** before `modetime` it will wait for the buffer to be filled **or** `modetime` to elapse again.

4.1.17. Testing

I ran automated tests:

```

1  running 6 tests
2  test test_close_socket_message ... ok
3  test test_generate_socket_id ... ok
4  test test_downstream_message ... ok
5  test test_upstream_message ... ok
6  test test_multiple_messages_downstream ... ok
7  test test_multiple_messages_upstream ... ok

```

They all came out clear.

4.1.18. Socket Creation

I implemented backend TCP socket creation, albeit without secondary handling (Section 4.2.10). I observed the potential for lost (and intentionally malicious) packets causing issues in ordering and noted it alongside my TCP handler:

```

// Constructed for each SOCKS TCP connection. Takes in a receiver for receiving
1 upstream messages from clients, and a sender for sending downstream messages to
  clients.

```

```

2 // Remember: Incoming data may be out of order, so we need to check that it
  matches the sequence number.
3 // If it doesn't, we need to wait till it comes.
  // For now, if data is lost this is going to stay open forever. In the future we
4 should add client-side buffering of data and a "data-lost, please resend"
  message.
5 // However the lower level protocol should be reliable through request level
  retries so this should be rare-to-impossible.
6 // Though this does provide a DOS vector, so we should probably also add some
  sort of timeout. TODO
7 async fn tcp_handler_task(arguments: TCPHandlerArguments) {
8     // TODO
9 }

```

4.1.19. TCP Implementation

I planned to use Tokio's TCP stream implementation. I enabled the `net` feature as a prerequisite:

```

1 `tokio = {version = "1", features = ["sync", "net"]}`

```

My initial version (Section 4.2.11) appeared to work, but the compiler did not like its architecture - the Rust compiler sometimes sacrifices architectural flexibility for safety guarantees, even if the architecture is safe. I changed it to Section 4.2.12 .

It was made clear to me while writing the handler that I need a way to send logs back from `libtransit` to the client. I considered doing this separately to each individual message, storing it as a vector of 8-bit flags and in a hashmap of socket IDs in each transit socket to be returned on the next available message. I considered creating a kind of monolithic “shared information dump”.

4.1.20. Log information

I changed `ServerStatistics` to `ServerMetaDownstream` and moved it to `libtransit`:

Before:

```

1 pub struct ServerStatistics {
2     pub to_reply: usize, // Bytes
3     pub to_send:  usize, // Bytes
4 }

```

After:

```

1 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
2 pub struct ServerMetaDownstream {
3     pub bytes_to_reply_to_client: usize,
4     pub bytes_to_send_to_remote:  usize,
5     pub messages_to_reply_to_client: usize,
6     pub messages_to_send_to_remote:  usize,
7     pub cpu_usage: f32,
8     pub memory_usage_kb: usize,
9     pub num_open_sockets: usize,
10    pub streams: HashMap<SocketID, ServerStreamInfo>,
11 }
12

```

```

13 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
14 pub struct ServerStreamInfo {
15     pub has_terminated: bool,
16     pub errors: Vec<String>,
17     pub logs: Vec<String>,
18 }

```

I also made a distinction in direction:

```

1 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
2 pub struct ClientMetaUpstream {
3     pub bytes_to_send_to_remote: usize,
4     pub bytes_to_reply_to_client: usize,
5     pub messages_to_send_to_remote: usize,
6     pub messages_to_reply_to_client: usize,
7 }

```

I then implemented a new test (Section 4.2.13) and re-tested, with all tests passing:

```

1 running 8 tests
2 test test_close_socket_message ... ok
3 test test_client_upstream_meta ... ok
4 test test_generate_socket_id ... ok
5 test test_downstream_message ... ok
6 test test_upstream_message ... ok
7 test test_server_downstream_meta ... ok
8 test test_multiple_messages_downstream ... ok
9 test test_multiple_messages_upstream ... ok
10
11 test result: ok. 8 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
    finished in 0.02s

```

I then created two new types, impls, and tests, which passed:

```

1 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug)]
2 pub struct ServerMessageDownstream {
3     metadata: ServerMetaDownstream,
4     messages: MultipleMessagesDownstream,
5 }
6
7 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug)]
8 pub struct ClientMessageUpstream {
9     metadata: ClientMetaUpstream,
10    messages: MultipleMessagesUpstream,
11 }

```

```

1 10 tests
2 test test_client_upstream_meta ... ok
3 test test_close_socket_message ... ok
4 test test_generate_socket_id ... ok
5 test test_downstream_message ... ok
6 test test_upstream_message ... ok
7 test test_server_downstream_meta ... ok
8 test test_client_upstream_message ... ok
9 test test_multiple_messages_upstream ... ok
10 test test_server_downstream_message ... ok
11 test test_multiple_messages_downstream ... ok

```

And then a way to store it:

```
1 meta_return_data: Arc<DashMap<DeclarationToken, ServerStreamInfo>>
```

```
1 fn return_error(&self, error: String) {
2     let server_stream_info =
3     self.meta_return_data.get(&self.our_declaration_token).unwrap().value();
4     server_stream_info.errors.push(error);
5 }
6 fn return_log(&self, warning: String) {
7     let server_stream_info =
8     self.meta_return_data.get(&self.our_declaration_token).unwrap().value();
9     server_stream_info.logs.push(warning);
10 }
```

I also created a new type so that DeclarationToken was equal to [u; 16].

```
1 fn form_meta_response(app_state: web::Data<AppState>) -> ServerMetaDownstream {
2     let mut streams = Vec::new();
3     for session in &app_state.current_sessions {
4         let stream = app_state.meta_return_data.get(session).unwrap().value();
5         streams.push(stream.clone());
6     }
7
8     ServerMetaDownstream {
9         bytes_to_reply_to_client: 0, // TODO
10        bytes_to_send_to_remote: 0, // TODO
11        messages_to_reply_to_client: 0, // TODO
12        messages_to_send_to_remote: 0, // TODO
13        cpu_usage: 0.0, // TODO
14        memory_usage_kb: 0, // TODO
15        num_open_sockets: 0, // TODO
16        streams: streams,
17    }
18 }
```

I did some more protocol changes:

```
1 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug)]
2 pub struct ServerMessageDownstream {
3     metadata: ServerMetaDownstream,
4     messages: MultipleMessagesDownstream,
5 }
```

```
1 running 9 tests
2 test test_client_upstream_meta ... ok
3 test test_close_socket_message ... ok
4 test test_downstream_message ... ok
5 test test_generate_socket_id ... ok
6 test test_upstream_message ... ok
7 test test_server_downstream_meta ... ok
8 test test_client_upstream_message ... ok
9 test test_server_downstream_message ... ok
10 test test_multiple_messages_upstream ... ok
```

I changed `transit_socket` from having `process_socks_message` operating on an enum of up-stream message or close socket to two separate functions.

4.1.21. Preliminary Data Transfer

I implemented a preliminary version of data transfer. I expected it to be changed in time, but its implementation got me a lot closer to a working version:

```

1  #[post("/access")] // TODO dynamic path with multiple http methods for better
   steganography
2  async fn data_exchange(app_state: web::Data<AppState>, req: HttpRequest,
   body_bytes: Bytes) -> impl Responder {
3      dprintln!("Received request to data exchange");
4
5      // Get token
6      let token = req.cookie("token");
7
8      let token = match token {
9          Some(token) => {
10             // Get as string
11             let token_hex = token.value().to_string();
12             dprintln!("Hex token: {}", token_hex);
13
14             // Convert to bytes
15             let token_bytes = hex::decode(token_hex).unwrap();
16
17             // Check it's the correct length
18             if token_bytes.len() != 16 {
19                 // Return 404
20                 dprintln!("Token is not 16 bytes!");
21                 return HttpResponse::NotFound().body("No page exists");
22             }
23
24             // Convert to array
25             let token: DeclarationToken = token_bytes[..
16].try_into().unwrap();
26
27             dprintln!("Token correct!: {:?}", token);
28
29             // Implicit return
30             token
31         }
32         None => {
33             // Return 404 - resist active probing by not telling the client
   what went wrong
34             dprintln!("No token (client identifier) supplied!");
35             return HttpResponse::NotFound().body("No page exists");
36         }
37     };
38
39     // Get body
40     let body = body_bytes;
41
42     // Get transit socket
43     let session = match app_state.sessions.get(&token) {
44         Some(session) => session,
45         None => {
46             // Return 404 - resist active probing by not telling the client
   what went wrong
47             dprintln!("No session found for token!");
48             return HttpResponse::NotFound().body("No page exists");

```

```

49     }
50     }.value();
51
52     // Decrypt body
53     let key = session.key;
54
55     let mut decrypted = match libsecrets::decrypt(&body, &key) {
56         Ok(decrypted) => decrypted, // If it succeeds, pull out content from
Result<>
57         Err(_) => {
58             // Return 404 - resist active probing by not telling the client
what went wrong
59             dprintln!("Failed to decrypt body!");
60             return HttpResponse::NotFound().body("No page exists");
61         }
62     };
63
64     // Parse into MultipleMessagesUpstream
65     let messages: MultipleMessagesUpstream = match
libtransit::MultipleMessagesUpstream::decode_from_bytes(&mut decrypted) {
66         Ok(messages) => messages, // If it succeeds, pull out content from
Result<>
67         Err(_) => {
68             // Return 404 - resist active probing by not telling the client
what went wrong
69             dprintln!("Failed to parse decrypted body into
MultipleMessagesUpstream!");
70             return HttpResponse::NotFound().body("No page exists");
71         }
72     };
73
74     // Send on to transit socket
75     for message in messages.stream_messages {
76         session.process_upstream_message(message);
77     }
78
79     for message in messages.close_socket_messages {
80         session.process_close_socket_message(message);
81     }
82
83     // Now wait for return data. TODO replace the entire following section
with a dynamic steganographic iterator that pretends to be an image/video/etc
while pulling data...
84     // but that's for the obfuscation section. For now, just return data.
85     let downstream_messages = session.get_data(4096, 50).await;
86
87     // Get meta information
88     let meta = form_meta_response(app_state);
89
90     // Form response
91     let response = ServerMessageDownstream {
92         messages: downstream_messages,
93         metadata: meta,
94     };
95
96     // Encode response
97     let response_bytes = response.encoded().unwrap(); // TODO handle error
98
99     // Encrypt response
100    let encrypted = libsecrets::encrypt(&response_bytes, &key).unwrap(); //
TODO handle error

```

```

101
102     // Return response
103     HttpResponse::Ok().body(encrypted)
104 }

```

I tested it and ran into a series of issues regarding the Rust borrow checker. Here is one of them:

```

1  error[E0596]: cannot borrow `*session` as mutable, as it is behind a `&`
   reference
2    --> server-monolith/src/main.rs:142:9
3
4  109 |         let session = match app_state.sessions.get(&token) {
5      |             ----- consider changing this binding's type to be: `&mut
   TransitSocket`
6  ...
7  142 |             session.process_upstream_message(message);
8      |             ~~~~~~ `session` is a `&`
   reference, so the data it refers to cannot be borrowed as mutable

```

I found that the problem I previously foresaw when writing the data pull function came out to be true - it really doesn't like the multiple mutable references. I realised at that point that `DashMap` is a solution for `RwLock<HashMap<K, V>>`, rather than `HashMap<K, RwLock<V>>`. In other words, it does not provide interior mutability.

I decided to turn this to my advantage. By using an asynchronous mutex I could make each wait in grequest only access when they have exclusive data to return in a way that doesn't block Tokio. It would do request balancing as an emergent property.

I also considered that it might be better to get a parallel hashmap implementation that asynchronously locks in order to not block the event loop. At this scale, locks are cheap and contention is not.

4.1.22. Client Side Decisions

It was at this point that I decided on using hardware threads or Tokio on the client. I set out the options in this table:

Attribute	Tokio	Threads
Per connection overhead	Low	High
Message passing overhead	Low	High
Ease of use	≈ same	≈ same
Dynamic work stealing	Yes	No
Thread-Safe data structures requirement	Yes	No
Ability to set threadcount	Yes	No

I decided to use Tokio, with the following architecture:

- Start transit socket
- Create a TCP->transit socket message passer: `upstreamPasser`
- Create a TCP->transit socket message passer that sends transit socket->TCP message passers... to the transit socket... to go into a async hash map... so that things can be sent back to the TCP handlers.

- Run `handle_transit(upstreamPasser, messagePasserPasser)` in a tokio task
- Start TCP listener
- Move inbound connections to tokio tasks
- Inbound connections to the SOCKS handlers send `UpStreamMessage` to `upstreamPasser` via a tokio mpsc when they want data up, the **moment** they stop reading from that buffer
- `handle_transit()` works out when to send data up based on a buffering/optime system. Each send-up is done by spawning a task. Instead of hybrid and pull clients, we'll have push and pull clients. They actually get the messages to send by sending the messages (all in one vec) into a message passer (needs a different type that's work-stealing-y, single producer multiple consumer but consumers delete once taken) that the upstreams listen to, form via a `spawn_blocking`, then send up. Whether it's push or pull is communicated via request type: Are we POST-ing a file or GETing an image?
- When data is returned from a request (there are continuously running getters that just try to continuously resend) it's decrypted+decoded by a `spawn_blocking` then sent immediately on to the relevant TCP SOCKS handler, which is stored on `TransitSocket` (we'll probably need an mpsc that sends this over?)
- That TCP SOCKS handler then sends it back on to the client

4.1.23. SOCKS Progress

I then made a SOCKS request decoder back in `libsocks`:

```

1  pub fn decode_socks_request(data: &[u8]) -> Result<Socks4Request,
    DecodeCommandError> {
2      // Get type based on second byte
3      let command_type = match data[1] {
4          0x01 => Socks4Command::Connect,
5          0x02 => Socks4Command::Bind,
6          _ => {
7              return Err(DecodeCommandError::InvalidCommand)
8          },
9      };
10
11     match command_type {
12         Socks4Command::Connect => {
13             let mut cursor = Cursor::new(data);
14             let request: Socks4ConnectRequest = cursor.read_be().map_err(|e|
15 DecodeCommandError::BinrwError(e))?;
16             Ok(Socks4Request::Connect(request))
17         },
18         Socks4Command::Bind => {
19             let mut cursor = Cursor::new(data);
20             let request: Socks4BindRequest = cursor.read_be().map_err(|e|
21 DecodeCommandError::BinrwError(e))?;
22             Ok(Socks4Request::Bind(request))
23         },
24     }
25 }

```

4.1.24. Server Split

I briefly went back to the server to split the TCP segment into an upload and a download portion and do some code cleanup, such as separating out token parsing to make things more modular and in-keeping with my original design intentions.

```

1 fn parse_token(token_hex: String) -> Option<DeclarationToken> {
2     dprintln!("Hex token: {}", token_hex);
3
4     // Convert to bytes
5     let token_bytes = hex::decode(token_hex).unwrap();
6
7     // Check it's the correct length
8     if token_bytes.len() != 16 {
9         // Return 404
10        dprintln!("Token is not 16 bytes!");
11        return None;
12    }
13
14    // Convert to array
15    let token: DeclarationToken = token_bytes[..16].try_into().unwrap();
16
17    dprintln!("Token correct!: {:?}", token);
18
19    // Implicit return
20    Some(token)
21 }

```

4.1.25. Passing Passers

I implemented a system for getting the “downstream backpassers” (downstream messages to the correct SOCKS handler to return) to those downstream HTTP handlers by sending message passers inside other message passers:

```

1 let (upstreamPasserSend, upstreamPasserReceive): (Sender<UpStreamMessage>,
2 Receiver<UpStreamMessage>) = mpsc::new();
3 let (messagePasserPasserSend, messagePasserPasserReceive):
4 (Sender<Sender<DownstreamBackpasser>>, Receiver<Sender<DownstreamBackpasser>>) =
5 mpsc::new();
6
7 struct DownstreamBackpasser {
8     socket_id: libtransit::SocketID,
9     sender: Sender<libtransit::DownStreamMessage>,
10 }

```

I also came upon Flume [119] as a async-compatible multi-producer multi-consumer consumption-based bounded/unbounded channel with high performance.

I also ran into a problem where I couldn’t send the `transit_socket` across threads under any circumstances because the stored chacha20 encryption key is not thread-safe and tokio does automatic thread transfers. I implemented the simple solution of putting the `transit_socket` in the initiating thread.

I then wrote a lot of the shell for the new client system in Section 4.2.14 and Section 4.2.15.

4.1.26. Continued Development

I replaced the system of dynamically creating the entries in the return lookup hashmap with a simple loop because `mpsc`s cannot have multiple receivers.

```

1 // We rely on this running fast enough for it to beat the data return from a new
2 // stream; this should be OK due to the inherent latency of the connections
3 // provided this is even somewhat fast. Maybe this should be rewritten at a
4 // later date to be more robust.

```

```

3  async fn handle_new_passer_passers(&mut self, mut messagePasserPasserReceive:
Receiver<DownstreamBackpasser>) {
4      loop {
5          let data = messagePasserPasserReceive.recv().await;
6
7          if data.is_none() {
8              continue;
9          }
10
11         let data = data.unwrap();
12
13         // Add it to the hashmap of backpassers
14         self.tcp_return_passers.insert(data.socket_id, data.sender);
15     }
16 }

```

I then realised that I couldn't have self-references in tasks, so I turned it into a simple asynchronous function that ran on regular intervals.

Ultimately, though, I decided to implement that “more robust solution” I referred to. I decided to use broadcasts: I would send the `DownstreamBackpasser` to every single downstream socket. The downstream sockets would have their own hashmap, and if something wasn't in the hashmap they'd put it in there by consuming from the backpasser message passer. Flume's mechanism of each receiver handling each message would ensure there was no work-stealing.

I was going to use Tokio but discovered that the documentation was inaccurate [120]. My solution was to use the aforementioned Flume.

4.1.27. Client Refactors

I continuously ran into issues with the lifetime of the transit socket and the asynchronous tasks running on it. Considering that it would only ever be a singleton I opted to “flatten” the program, running everything through a `handle_transit` function that took a `transit_socket` and spawned various tasks that also received a `transit_socket` in its `Arc<RwLock<>>` form, acting solely as data storage.

I also implemented the pull handler, as described earlier. The pull handler does its encoding and encryption inline inside the asynchronous task, as sending it off to another thread has a greater overhead than that step. However, the push handler - also implemented at this point - does its encoding and encryption in a separate thread to minimise impact upon the rest of the program.

The `handle_transit` function doubled as a kind of client-side coordinator, handling when to upload data. It does not particularly involve itself with downstream data, which is sent directly to the relevant TCP streams using the backpasser system.

You can see the core client in Section 4.2.16 and the more developed transit in Section 4.2.17.

4.1.28. Rapid Development

As I neared a working version I rapidly developed both the client and the server. This included expanding the bidirectional TCP handler:

```

1  loop {

```

```

2     let ready = stream.ready(Interest::READABLE |
Interest::WRITABLE).await.expect("Failed to wait for socket to be ready");
3
4     if ready.is_writable() {
5         // Check if transit has sent us any data
6         match downstreamPasserReceive.try_recv() {
7             Ok(data) => {
8                 // Transit has sent us data
9                 // Send it to the client
10                let bytes = data.payload;
11
12                match stream.try_write(&bytes) {
13                    Ok(_) => {
14                        // Transit has sent us data
15                        send_seq_num += 1;
16                    },
17                    Err(error) => {
18                        // TODO handle properly
19                        eprintln!("Failed to send data to client: {:?}", error);
20                        continue
21                    }
22                }
23            },
24            Err(error) => {
25                // TODO handle properly
26                eprintln!("Failed to receive data from transit: {:?}", error);
27                continue
28            }
29        };
30    }
31
32    if ready.is_readable() {
33        let mut upstream_packet = UpStreamMessage {
34            socket_id,
35            message_sequence_number: send_seq_num,
36            dest_ip: dstip,
37            dest_port: dstport,
38            payload: Vec::with_capacity(0)
39        };
40
41        // Read into the payload buffer
42        let bytes_read = match stream.try_read_buf(&mut upstream_packet.payload)
43        {
44            Ok(bytes_read) => bytes_read,
45            Err(error) => {
46                // TODO handle properly
47                eprintln!("Failed to read from socket: {:?}", error);
48                continue
49            }
50        };
51
52        // Check if the socket was closed
53        if bytes_read == 0 {
54            // The socket was closed
55            // TODO handle properly
56        }
57
58        // Send the data to transit
59        upstreamPasserSend.send(upstream_packet).await.expect("Failed to send
data to transit");
60    }

```

This hot-loop system is easy to work with but introduces additional latency, and was later replaced with an asynchronous split system.

I then transitioned away from relying on message sequence numbers to ensure integrity, instead doing it at the higher level of ensuring that messages remained in their original vector order and (planning to) ensure HTTP messages arrived in the correct order.

4.1.29. Initial Testing

My first attempt at a full-stack test failed. I got some weird logs, with lots of blank requests being sent.

```
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Response status: 200 OK
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
Failed to receive data from transit: Empty
AC
o [alex@alexpnc client-ctrl]$ █
```

```
Hex token: 438e647e3862dcf3a8c9a5f5c33c6253
Received request to download
Token correct!: [67, 142, 100, 126, 56, 98, 220, 243, 168, 201, 165, 245, 195, 60, 98, 83]
Hex token: 438e647e3862dcf3a8c9a5f5c33c6253
Token correct!: [67, 142, 100, 126, 56, 98, 220, 243, 168, 201, 165, 245, 195, 60, 98, 83]
Received request to download
Hex token: 438e647e3862dcf3a8c9a5f5c33c6253
Token correct!: [67, 142, 100, 126, 56, 98, 220, 243, 168, 201, 165, 245, 195, 60, 98, 83]
Received request to download
Hex token: 438e647e3862dcf3a8c9a5f5c33c6253
Token correct!: [67, 142, 100, 126, 56, 98, 220, 243, 168, 201, 165, 245, 195, 60, 98, 83]
Received request to download
Hex token: 438e647e3862dcf3a8c9a5f5c33c6253
Token correct!: [67, 142, 100, 126, 56, 98, 220, 243, 168, 201, 165, 245, 195, 60, 98, 83]
Received request to download
Hex token: 438e647e3862dcf3a8c9a5f5c33c6253
Token correct!: [67, 142, 100, 126, 56, 98, 220, 243, 168, 201, 165, 245, 195, 60, 98, 83]
```

```
o ^C[alex@alexpnc server-monolith]$ ^C
o [alex@alexpnc server-monolith]$ █
```

Figure 43: Initial testing attempt. The left terminal is the client while the right is the server.

I was also seeing complaints from curl [100] about the SOCKS version being wrong. I discovered that I should reply with the version being 0, not 4. You can see the difference in how curl handled the request before hand after the change, going from a quick error to data being sent:

```
[alex@alexpc ~]$ curl -x socks4://127.0.0.1:1080 http://ifconfig.me -v
* processing: http://ifconfig.me
*   Trying 127.0.0.1:1080...
* Connected to 127.0.0.1 (127.0.0.1) port 1080
* SOCKS4 communication to ifconfig.me:80
* SOCKS4 non-blocking resolve of ifconfig.me
* SOCKS4 connect to IPv4 34.160.111.145 (locally resolved)
* SOCKS4 reply has wrong version, version should be 0.
* Closing connection
curl: (97) SOCKS4 reply has wrong version, version should be 0.
[alex@alexpc ~]$ curl -x socks4://127.0.0.1:1080 http://ifconfig.me -v
* processing: http://ifconfig.me
*   Trying 127.0.0.1:1080...
* Connected to 127.0.0.1 (127.0.0.1) port 1080
* SOCKS4 communication to ifconfig.me:80
* SOCKS4 non-blocking resolve of ifconfig.me
* SOCKS4 connect to IPv4 34.160.111.145 (locally resolved)
* SOCKS4 request granted.
* Connected to 127.0.0.1 (127.0.0.1) port 1080
> GET / HTTP/1.1
> Host: ifconfig.me
> User-Agent: curl/8.2.1
> Accept: */*
>
```

Figure 44: A programmatic test using curl [100] before and after the change to the replied SOCKS version.

I fixed the endless complaining with a simple match in the client coordinator:

```

1 Err(error) => {
2     match error {
3         TryRecvError::Empty => {
4             // No data from transit
5             // Wait a moment because async
6             tokio::time::sleep(std::time::Duration::from_millis(1)).await;
7             // DO NOT continue, we still need to read
8         },
9         _ => {
10            // TODO handle properly
11            eprintln!("Failed to receive data from transit: {:?}", error);
12            continue
13        }
14    };
15 }

```

I then investigated why the pulls were returning too early, finding that I'd managed to get the upload and download names completely inverted. Keeping track of the four different message streams as they merge and split into the two transfer directions was an ongoing issue, and one that I would later solve through a "colour" system.

Once that was resolved I got the error `Failed to read from socket: Kind(WouldBlock)`.

It seemed to originate from the following code:

```

1 if ready.is_readable() {
2     let mut upstream_packet = UpStreamMessage {
3         socket_id,
4         message_sequence_number: send_seq_num,
5         dest_ip: dstip,
6         dest_port: dstport,
7         payload: Vec::with_capacity(0)
8     };
9
10    // Read into the payload buffer
11    let bytes_read = match stream.try_read_buf(&mut upstream_packet.payload) {
12        Ok(bytes_read) => bytes_read,
13        Err(error) => {
14            // TODO handle properly
15            eprintln!("Failed to read from socket: {:?}", error);
16            continue
17        }
18    };
19
20    // Check if the socket was closed
21    if bytes_read == 0 {
22        // The socket was closed
23        // TODO handle properly
24    }
25
26    // Send the data to transit
27    upstreamPasserSend.send(upstream_packet).await.expect("Failed to send data
to transit");
28 }

```

I put in some logging and observed that it starts reading then throws an error:

```

1 Read 64 bytes from socket
2 Read 10 bytes from socket
3 Failed to read from socket: Kind(WouldBlock)

```

I replaced all the silently erroring matches with unwraps and put in print statements. This demonstrated that `handle_transit` was not properly sending data along - it wasn't receiving data sent to it.

```

1 Connected to server!
2 Push handler started
3 Push handler started
4 Push handler started
5 Push handler started
6 Push handler started
7 Push handler started
8 Said hello to client
9 Read 64 bytes from socket
10 Sent on to transit passer
    Received data to send up Some(UpStreamMessage { socket_id: 0,
    message_sequence_number: 0, dest_ip: 580939665, dest_port: 80, payload: [71, 69,
11 84, 32, 47, 32, 72, 84, 84, 80, 47, 49, 46, 49, 13, 10, 72, 111, 115, 116, 58,
    32, 105, 102, 99, 111, 110, 102, 105, 103, 46, 109, 101, 13, 10, 85, 115, 101,
    114, 45, 65, 103, 101, 110, 116, 58, 32, 99, 117, 114, 108, 47, 56, 46, 50, 46,
    49, 13, 10, 65, 99, 99, 101, 112] })
12 Read 10 bytes from socket
13 Sent on to transit passer
14 Failed to read from socket: Kind(WouldBlock)

```

I found that it was because it was waiting for close socket messages that didn't yet exist, and which would never arrive. I replaced the original asynchronous blocking code with hot-loop `try_recv()` code:

```

1 // Now do it to the close socket messages
2 let close = closePasserRcv.recv().await;
3 let close = close.unwrap(); // TODO handle this
4 // Increment buffer size
5 current_buffer_size += 2048; // Approxish I guess
6                               // Add it to the buffer
7 buffer_close.push(close);

```

```

1 let data = closePasserRcv.try_recv();
2 match data {
3     Ok(close) => {
4         // Increment buffer size
5         current_buffer_size += 2048; // Approxish I guess
6                                     // Add it to the buffer
7         buffer_close.push(close);
8     }
9     Err(error) => {
10         match error {
11             TryRecvError::Empty => {}
12             TryRecvError::Disconnected => {
13                 // TODO handle this
14                 panic!("Close socket passer disconnected");
15             }
16         }

```

```
17     }  
18 }
```

Despite these changes I was still running into the issue. I found that the `modetime` section of whether to continue was dependent on *both* closed socket messages and standard ones existing in the send buffer, rather than just one of the two existing. The fix was simple:

```
1  if last_upstream_time.elapsed().as_millis() > MODETIME  
2      && buffer_close.len() > 1  
3      && buffer_stream.len() > 1
```

```
1  if last_upstream_time.elapsed().as_millis() > MODETIME  
2      && (buffer_close.len() > 1  
3      || buffer_stream.len() > 1)
```

Once I'd fixed that I had data being sent up and upstream requests, but nothing in the request logs.

I found that the server was timing out. Some logging narrowed the issue down to the following line on the server:

```
1  let key = session.read().await.key;
```

There was a complex interplay of `RwLocks` blocking each other:

- The downloaders use the session constantly in order to wait for new data
- The uploaders need to access the session to access the key and upstream passers
- The uploaders are blocked by the downloaders
- The downloaders are blocked by the other downloaders
- The deadlock never resolves

There were two ways to go about solving this. I could add some kind of prioritisation system to the `RwLock`, or I could introduce a new system for downstreams. Downstreams would never actually own the transit socket aside from grabbing the key. Instead they would register a message passer (via... message passing) with the transit socket. The transit socket would keep an eye on when data should be returned and send it at the appropriate time using its own coordination system.

I decided to implement the latter. However, when thinking about the finer details of implementation I became overwhelmed by the spaghetti that my code had become. I decided to try a different architecture. Each client would get its own separate server instance, with a control plane spinning up a new one when needed. A load balancer would differentiate between server instances based on ID, and server instances would spin themselves down when they were no longer needed.

I also decided to remove all provisions for socket deletion at this point. I wanted to implement it later.

I'm going to skip through my investigation of the alternate approach. The summary is that I partially implemented it and decided that the complexity was inherent, then decided to go back to the original approach.

I then investigated the concept of Actors [121], which introduced me to a new way of thinking about my problem and the architecture with which I would solve it.

I created `new_transit`, which was designed to be a hybrid of the actor and traditional OOP paradigms. Sessions would be accessed just long enough to access their message passers and no longer, neatly solving the old logging problems. You can see its core architecture from a very early version here:

```

1  use dashmap::DashMap;
2  use tokio::sync::mpsc;
3  use libtransit::{
4      UpStreamMessage, DownStreamMessage, ClientMetaUpstream,
5      ServerMetaDownstream,
6      SocketID
7  };
8  use std::sync::atomic::AtomicU32;
9
10 pub struct SessionActorsStorage {
11     to_bundler_stream: mpsc::UnboundedSender<UpStreamMessage>,
12     from_bundler_stream: mpsc::UnboundedReceiver<Vec<DownStreamMessage>>,
13 }
14
15 pub async fn handle_session(mut from_http_stream:
16     mpsc::UnboundedReceiver<UpStreamMessage>,
17     to_http_stream: mpsc::UnboundedSender<Vec<DownStreamMessage>>) {
18     let stream_to_tcp_handlers: DashMap<SocketID,
19         mpsc::UnboundedSender<UpStreamMessage>> = DashMap::new();
20     let stream_from_tcp_handlers: DashMap<SocketID,
21         mpsc::UnboundedReceiver<DownStreamMessage>> = DashMap::new();
22
23     loop {
24         let mut no_received = false;
25
26         match from_http_stream.try_recv() {
27             Ok(message) => {
28                 // Get the socket ID
29                 let socket_id = message.socket_id;
30
31                 // Get the TCP handler
32                 let tcp_handler = stream_to_tcp_handlers.get(&socket_id);
33
34                 let tcp_handler = if tcp_handler.is_none() {
35                     // Create a new TCP handler
36                     // For sending upstreams to the handle_tcp
37                     let (to_tcp_handler, from_http_stream) =
38                         mpsc::unbounded_channel();
39                     // For receiving downstreams from the handle_tcp
40                     let (to_http_stream, from_tcp_handler) =
41                         mpsc::unbounded_channel();
42
43                     // Add the TCP handler to the map
44                     stream_to_tcp_handlers.insert(socket_id, to_tcp_handler);
45                     stream_from_tcp_handlers.insert(socket_id,
46                         from_tcp_handler);
47
48                     // Spawn the TCP handler
49                     tokio::spawn(handle_tcp(from_http_stream, to_http_stream));
50
51                     // Get the TCP handler

```

```

46         let tcp_handler =
47             stream_to_tcp_handlers.get(&socket_id).unwrap();
48             tcp_handler
49         } else {
50             tcp_handler.unwrap()
51         };
52         // Send the message to the TCP handler
53         tcp_handler.send(message).unwrap();
54     }
55     Err(e) => {
56         // Check if the channel is empty
57         if e == mpsc::error::TryRecvError::Empty {
58             no_received = true;
59         } else {
60             // Panic
61             panic!("Error receiving from HTTP stream: {:?}", e);
62         }
63     }
64 }
65 }
66 }
67
68 pub async fn handle_tcp(from_http_stream:
69     mpsc::UnboundedReceiver<UpStreamMessage>, to_http_stream:
70     mpsc::UnboundedSender<DownStreamMessage>) {
71     // TODO
72 }

```

This section later became Section 4.2.18, which included its own modetime system, socket opening behaviour, and complete TCP handlers.

4.1.30. Success

Two very minor bug fixes later, I had my very first working HTTPS request.

```

[alex@alexp ~]$ curl -x socks4://127.0.0.1:1080 https://ifconfig.me
86.31.175.68[alex@alexp ~]$

```

Figure 45: The first working transit. A request for my IP address from ifconfig.me completes successfully.

Figure 46: The first working transit. The left terminal is the server while the right is the client.

4.1.31. Continuing Development

Working transit didn't mean that it was a viable system in the real world. To start with, I had no provisions for out-of-order HTTP messages. I put `seq_num` in both upstream and downstream metadata, allowing me to keep track of the sequence of higher level http messages:

```
1 // libtransit
2 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
3 pub struct ClientMetaUpstream {
4     pub bytes_to_send_to_remote: u32,
5     pub bytes_to_reply_to_client: u32,
6     pub messages_to_send_to_remote: u32,
7     pub messages_to_reply_to_client: u32,
8     pub seq_num: u32,
9 }
10
11 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
12 pub struct ServerMetaDownstream {
13     pub bytes_to_reply_to_client: usize,
14     pub bytes_to_send_to_remote: usize,
15     pub messages_to_reply_to_client: usize,
16     pub messages_to_send_to_remote: usize,
17     pub cpu_usage: f32,
18     pub memory_usage_kb: usize,
19     pub num_open_sockets: usize,
20     pub seq_num: u32,
21     pub streams: Vec<ServerStreamInfo>,
22 }
```

I started storing them on the server side:

```
1 // new_transit
2 pub struct SessionActorsStorage {
3     pub to_bundler_stream: mpsc::UnboundedSender<UpStreamMessage>,
4     pub from_bundler_stream: flume::Receiver<Vec<DownStreamMessage>>,
5     pub key: EncryptionKey,
6     pub seq_num_down: u32,
7     pub next_seq_num_up: u32,
8 }
```

Then I added handling in `/upload` and `/download`:

```
1 // download
2 let meta = form_meta_response(&app_state, actor.seq_num_down.fetch_add(1,
    Ordering::SeqCst)).await;
```

```
1 // upload
2 while actor.next_seq_num_up.load(Ordering::SeqCst) != seq_num {
3     // Wait until that's the case
4     // Sleep 1ms
5     tokio::time::sleep(Duration::from_millis(1)).await;
6 }
7
8 // [...]
9
10 actor.next_seq_num_up.store(seq_num + 1, Ordering::SeqCst);
```

```

1 // client
2 static RECEIVED_SEQ_NUM: AtomicU32 = AtomicU32::new(0);
3 static SENT_SEQ_NUM: AtomicU32 = AtomicU32::new(0);

```

I considered the situation a little and discovered an issue: the client download tasks check the received seq num is the expected one, then iterates through downstream messages running `.await` each time. This provides an opportunity for multiple download tasks to be returning at once, producing out-of-order packet response and abnormal behaviour on the TCP handlers. I decided to solve this through turning the sender into an unbounded one, which doesn't use `.await`.

4.1.32. Iterative Testing

I continued to test. I opened youtube and was greeted with a cascade of senders timing out. While their timeout was artificially low for debugging purposes (1s), it was still abnormal. I tried reducing modetime, to no avail. A quick measurement of how long data parsing took showed that it was negligible at 0ms.

I was able to load alexcj.co.uk, but it was very slow. More complex websites such as github timed out quickly.

I looked at the logs and the sequence numbers seemed to be in the correct order. In lieu of an obvious culprit I turned to performance. I wanted to find where the bottleneck lied, so I needed to get metrics on what aspects of the system were taking the longest and which buffers were filling up.

I implemented a global singleton in `meta.rs` on the server to store the sizes of buffers in atomic variables:

```

1 use std::sync::atomic::{AtomicU32, Ordering};
2 use libtransit::ServerMetaDownstreamTrafficStats;
3
4 #[derive(Debug)]
5 pub struct ServerMetaDownstreamTrafficStatsSynced {
6     pub http_up_to_coordinator_bytes: AtomicU32,
7     pub coordinator_up_to_socket_bytes: AtomicU32,
8     pub socket_down_to_coordinator_bytes: AtomicU32,
9     pub coordinator_down_to_http_message_passer_bytes: AtomicU32,
10    pub coordinator_down_to_http_buffer_bytes: AtomicU32,
11    pub congestion_ctrl_intake_throttle: AtomicU32,
12 }
13
14 impl ServerMetaDownstreamTrafficStatsSynced {
15     // Convert into a ServerMetaDownstreamTrafficStats by turning AtomicU32 into
16     u32
17     pub fn into_server_meta_downstream_traffic_stats(&self) ->
18     ServerMetaDownstreamTrafficStats {
19         ServerMetaDownstreamTrafficStats {
20             http_up_to_coordinator_bytes:
21             self.http_up_to_coordinator_bytes.load(Ordering::Relaxed),
22             coordinator_up_to_socket_bytes:
23             self.coordinator_up_to_socket_bytes.load(Ordering::Relaxed),
24             socket_down_to_coordinator_bytes:
25             self.socket_down_to_coordinator_bytes.load(Ordering::Relaxed),
26             coordinator_down_to_http_message_passer_bytes:
27             self.coordinator_down_to_http_message_passer_bytes.load(Ordering::Relaxed),

```

```

22         coordinator_down_to_http_buffer_bytes:
23     self.coordinator_down_to_http_buffer_bytes.load(Ordering::Relaxed),
24         congestion_ctrl_intake_throttle:
25     self.congestion_ctrl_intake_throttle.load(Ordering::Relaxed),
26     }
27 }

```

I then incremented and decremented the variables whenever I added to or removed from a buffer. I used the `Ordering::Relaxed` as I didn't need things in perfect order.

For example:

```

1 // Modify traffic stats
2 let return_msgs_bytes = downstream_messages
3   .iter()
4   .map(|msg| msg.payload.len())
5   .sum::<usize>();
6
7 actor.traffic_stats.coordinator_down_to_http_buffer_bytes.fetch_sub(return_msgs_bytes
  as u32, Ordering::Relaxed);

```

```

1 // Send messages on
2 for message in upstream.messages.stream_messages {
3     let payload_length = message.payload.len();
4     to_bundler.send(message).unwrap();
5     actor.traffic_stats.http_up_to_coordinator_bytes.fetch_add(payload_length as
  u32, Ordering::Relaxed);
6 }
7 // Send messages on
8 for message in upstream.messages.stream_messages {
9     let payload_length = message.payload.len();
10    to_bundler.send(message).unwrap();
11    actor.traffic_stats.http_up_to_coordinator_bytes.fetch_add(payload_length as
  u32, Ordering::Relaxed);
12 }

```

Upon running and looking at the general performance statistics I noticed that the client was at 8% CPU even when idle. My CPU has 12 cores, so this was one core being used constantly. I tracked it down to a hot loop in the client coordinator (`handle_transit`). A 1ms sleep in case of an error ingesting data made idle CPU 0.1%.

Analysis of the newly-collected data indicated that server-side bufferbloat was not the issue in small requests (e.g. `ifconfig.me`), while there was a small amount of bufferbloat in larger requests (e.g. `photos on alexcj.co.uk`) in `http_up_to_coordinator_bytes`. On very large requests (e.g. `discord.com/app`) it sometimes failed entirely, but bufferbloat never seemed to become a problem. Latency was at 1.797s at this point.

I tested the effect of different threadcounts and found a 100ms delta - clearly not the cause of the issue.

I decided to implement client monitoring. This required similar changes to before, just on the client, so I will omit the code. The tests passed:

```

1  running 9 tests
2  test test_client_upstream_meta ... ok
3  test test_close_socket_message ... ok
4  test test_generate_socket_id ... ok
5  test test_downstream_message ... ok
6  test test_server_downstream_meta ... ok
7  test test_upstream_message ... ok
8  test test_multiple_messages_upstream ... ok
9  test test_client_upstream_message ... ok
10 test test_server_downstream_message ... ok
11
12 test result: ok. 9 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
    finished in 0.01s

```

I also setup a system for tracking the time that certain upstream messages got into each section. It was implemented in quite a wasteful way (u64s of milliseconds) as it was only for debugging purposes.

```

1  #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
2  pub struct UpStreamMessage {
3      pub socket_id: SocketID,
4      pub message_sequence_number: u32,
5      pub dest_ip: IPV4,
6      pub dest_port: Port,
7      pub payload: Vec<u8>,
8      pub time_ingress_client_ms: u64, // 56 bytes on latency information
9      pub time_at_coordinator_ms: u64,
10     pub time_at_client_egress_ms: u64,
11     pub time_at_server_ingress_ms: u64,
12     pub time_at_server_coordinator_ms: u64,
13     pub time_at_server_socket_ms: u64,
14     pub time_client_write_finished_ms: u64,
15 }

```

I then re-tested, getting lots of data for the time that messages took. Here's one as an example:

```

1  Message timing info:
2      ingress_to_coordinator: 2ms
3      coordinator_to_egress: 102ms
4      client_egress_server_ingress: 1ms
5      server_ingress_to_coordinator: 1ms
6      coordinator_to_socket: 0ms
7      socket to socket written: 0ms
8      Total transit time: 106ms

```

Given that a normal request to <http://ifconfig.me> took 180ms (without the proxy) and mine took about 430ms, about 40% of the additional latency was from `coordinator_to_egress`. I decided to investigate the client side coordinator.

The modetime is 10ms, so that couldn't be the issue. It only ever waited 1ms at a time if the receiver was empty.

I considered that I could be requesting a 1ms wait and getting a significantly longer one. I measured the maximum delay and it never got past 3ms except for during the startup phase.

There was clearly some inconsistency too - it sometimes worked fine:

```

1 Message timing info:
2   Sequence number: 1
3   ingress_to_coordinator: 0ms
4   coordinator_to_egress: 0ms
5   client_egress_server_ingress: 0ms
6   server_ingress_to_coordinator: 1ms
7   coordinator_to_socket: 21ms
8   socket to socket written: 0ms
9   Total transit time: 22ms for 64 bytes

```

I tested a simpler system of always ending up data if modetime had been exceeded, diverging from the algorithm I had previously designed. That didn't change average time much, but it did reduce the peaks.

I then added analytics to the downstream data in a very similar way to the upstream data. I looked at the analytics and found that the average downstream latency was about 10ms - decent, and not the source of the issue. I suspected the problem was bandwidth during intake: Because I had one thread for both upload and download, and it hot-looped, it was downloading too slowly.

I implemented my new system at Section 4.2.19 . It didn't work - no data was returned and it caused a huge and fast memory leak.

It seemed to be continuously reading empty payloads despite me using `.await`, because I wasn't handling when the `bytes_read` equalled zero.

Putting a return on `bytes_read == 0` stopped the absurd HTTP responses, but it also stopped reading data. I looked closer and found that `.read()` is a synchronous function and my `.await` ing was useless.

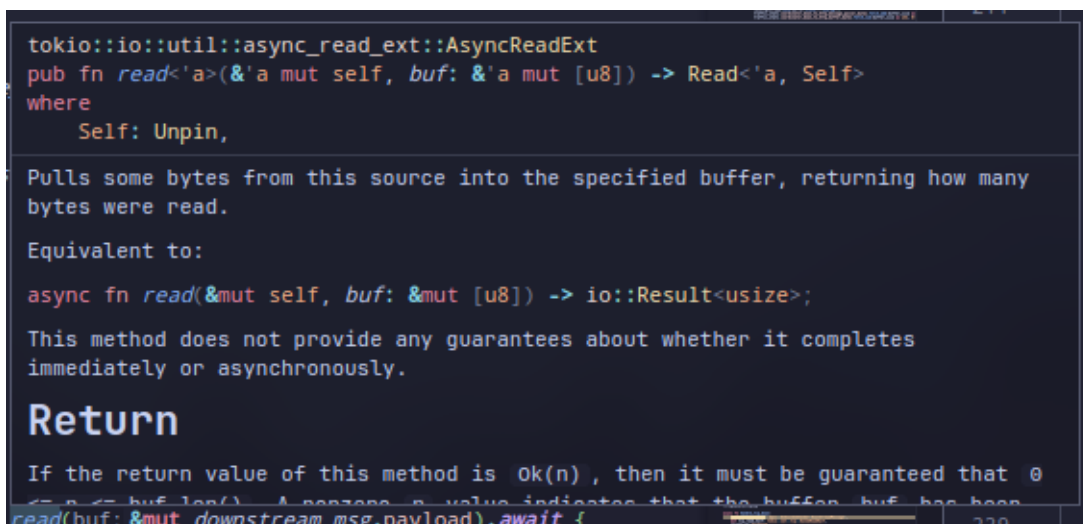


Figure 47: The hover-docs for `.read()`.

I was already using `AsyncReadExt`, but I hadn't imported `io-util` as a `tokio` feature.

I did some additional testing (panicing on more than 0 bytes to confirm nothing was being read, debug logs) and discovered that `.read()` reads into a buffer, and if that buffer has a capacity of zero (as mine did) it won't auto expand - it'll just read in zero bytes.

I instead made it read 512 bytes at a time:

```

1  let mut downstream_msg = DownStreamMessage {
2      socket_id,
3      message_sequence_number: return_sequence_number,
4      payload: vec![0; 512], // TODO make this configurable
5      has_remote_closed: false,
6      time_at_server_start_recv_ms: meta::ms_since_epoch(),
7      time_at_server_finish_recv_ms: 0,
8      time_at_server_coordinator_ms: 0,
9      time_at_server_egress_ms: 0,
10     time_at_client_ingress_ms: 0,
11     time_at_client_sendon_ms: 0,
12     time_at_client_socket_ms: 0,
13     time_at_client_socket_write_finished_ms: 0,
14 };
15
16 // Write in directly for efficiency
17 // We're using an asynchronous wait here to avoid the old hot-loop system
18 let bytes_read = match tcp_read_half.read(&mut downstream_msg.payload).await {
19     Ok(bytes_read) => bytes_read,
20     Err(e) => {
21         panic!("Error reading from TCP stream: {:?}", e);
22     }
23 };
24
25 println!("Read {} bytes", bytes_read);
26
27 if bytes_read == 0 {
28     // TODO handle socket close
29     return;
30 }
31
32 // Trim array down to size
33 downstream_msg.payload.truncate(bytes_read);
34
35 println!("Payload: {:?}", downstream_msg.payload);

```

Once this was implemented I had very promising results - latency was just 100ms above normal:

```

1  [alex@alexp ~]$ time curl -x socks4://127.0.0.1:1080 https://ifconfig.me
2  86.31.175.68
3  real  0m0.300s
4  user  0m0.047s
5  sys 0m0.007s
6  [alex@alexp ~]$ time curl -x socks4://127.0.0.1:1080 https://ifconfig.me
7  86.31.175.68
8  real  0m0.297s
9  user  0m0.032s
10 sys 0m0.008s
11 [alex@alexp ~]$ time curl -x socks4://127.0.0.1:1080 https://ifconfig.me
12 86.31.175.68
13 real  0m0.326s
14 user  0m0.048s
15 sys 0m0.014s
16 [alex@alexp ~]$ time curl -x socks4://127.0.0.1:1080 https://ifconfig.me
17 86.31.175.68
18 real  0m0.327s
19 user  0m0.059s
20 sys 0m0.007s
21 [alex@alexp ~]$ time curl https://ifconfig.me

```

```
22 86.31.175.68
23 real 0m0.200s
24 user 0m0.037s
25 sys 0m0.010s
26 [alex@alexpc ~]$ time curl https://ifconfig.me
27 86.31.175.68
28 real 0m0.180s
29 user 0m0.046s
30 sys 0m0.001s
31 [alex@alexpc ~]$ time curl https://ifconfig.me
32 86.31.175.68
33 real 0m0.191s
34 user 0m0.050s
35 sys 0m0.004s
36 [alex@alexpc ~]$ time curl https://ifconfig.me
37 86.31.175.68
38 real 0m0.208s
39 user 0m0.051s
40 sys 0m0.007s
```

I started to implement this feature on the client. The implementation is at Section 4.2.20 .

Following the introduction of both optimisations loading Discord only took 250ms, compared to 120ms without the proxy and seconds or failure pre-optimisation.

4.1.33. Messages

I had Discord running and decided to send some messages through it.

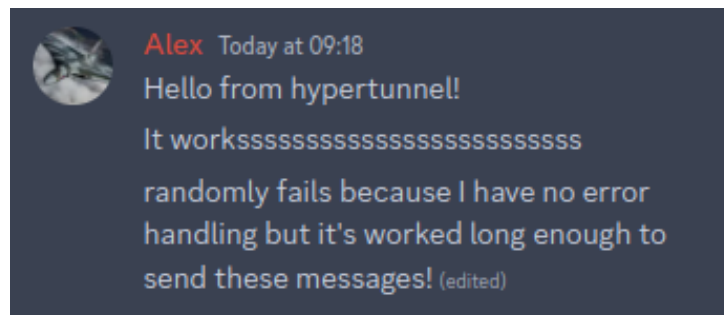


Figure 48: Discord running through the proxy.

I then ran a cloudflare speed test. It didn't completely finish, and some errors showed up, but considering I was printing all the data to the console on top this was pretty good performance:

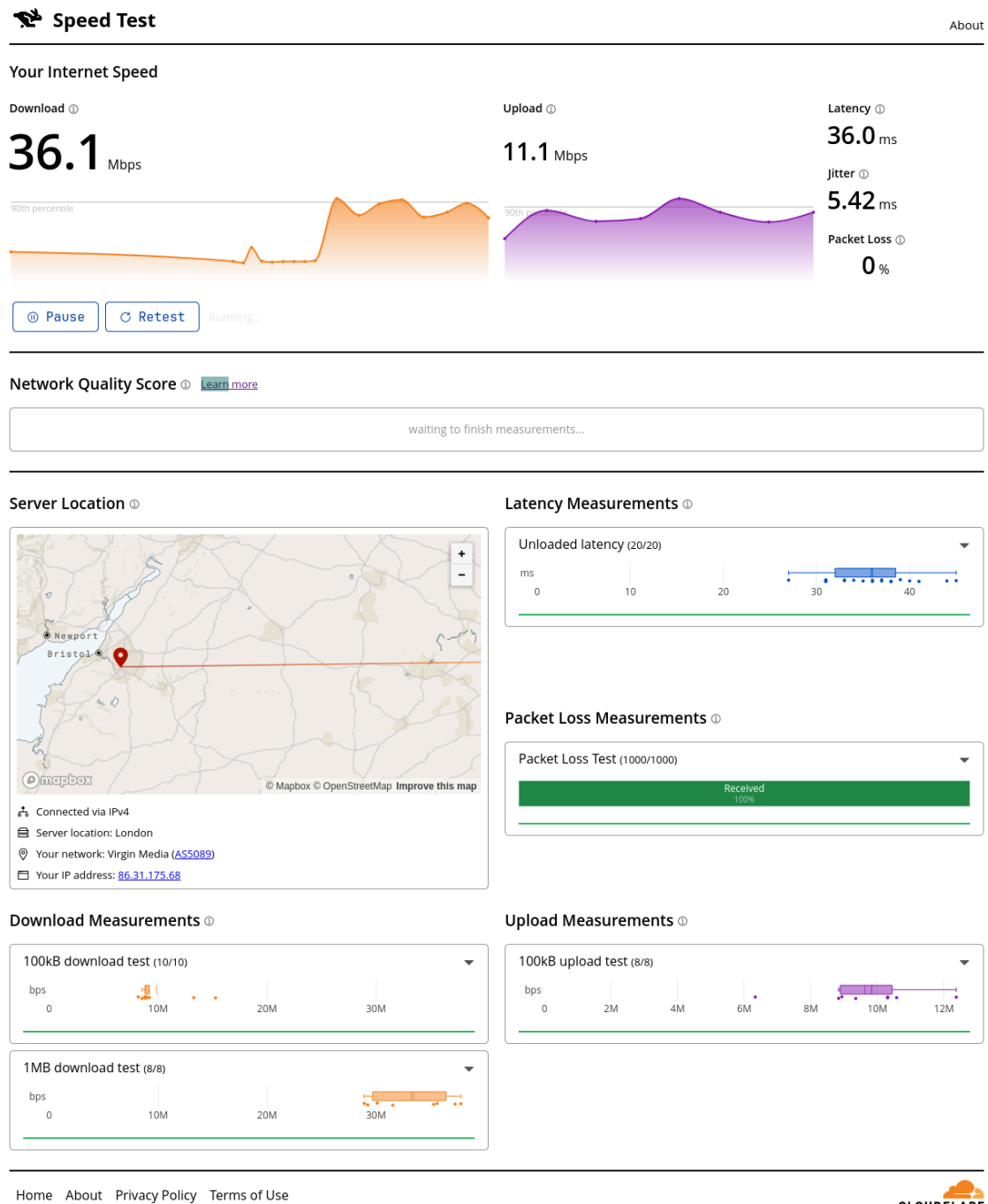


Figure 49: A cloudflare speed test running through the proxy.

4.1.34. Architectural Changes

My performance without the proxy was 75/20.9.

I decided to split the generic `Message` in `libtransit`, containing both upstream and downstream messages, into an `UpMessage` and `DownMessage` type.

```
1 #[derive(Debug, PartialEq, Clone, BorshSerialize, BorshDeserialize)]
2 pub enum UpMessage {
3     UpStream(UpStreamMessage),
4     CloseSocket(CloseSocketMessage),
5 }
```

```

6
7 #[derive(Debug, PartialEq, Clone, BorshSerialize, BorshDeserialize)]
8 pub enum DownMessage {
9     DownStream(DownStreamMessage),
10    CloseSocket(CloseSocketMessage),
11 }

```

However, I ultimately decided that I was going to remake the wire format from scratch.

This is best demonstrated through the tests that initialise a full message:

```

1 ClientMessageUpstream {
2     metadata: {
3         packet_info: {
4             unix_ms: u64
5             seq_num: u32
6         }
7         traffic_stats: {
8             socks_to_coordinator_bytes: u32,
9             coordinator_to_request_buffer_bytes: u32,
10            coordinator_to_request_channel_bytes: u32,
11            up_request_in_progress_bytes: u32,
12            response_to_socks_bytes: u32,
13        }
14        set: Option<{
15            buffer_size_to_return: u32,
16            modetime_to_return: u32,
17            intake_buffer_max: u32, // New due to new format
18        }>
19    }
20    socks_sockets: Vec<{ // Completely new system designed to minimise bandwidth
overhead
21        socket_id: SocketID,
22        dest_ip: IPV4,
23        dest_port: Port,
24        payload: Vec<u8>
25        termination_reasons: Vec<TerminationReason>, // There may be multiple
reasons
26    }>
27    payload_size: u32,
28 }

```

```

1 let msg = ServerMessageDownstream {
2     metadata: ServerMetaDownstream {
3         packet_info: UnifiedPacketInfo {
4             unix_ms: 0,
5             seq_num: 0,
6         },
7         traffic_stats: ServerMetaDownstreamTrafficStats {
8             http_up_to_coordinator_bytes: 0,
9             coordinator_up_to_socket_bytes: 0,
10            socket_down_to_coordinator_bytes: 0,
11            coordinator_down_to_http_message_passer_bytes: 0,
12            coordinator_down_to_http_buffer_bytes: 0,
13            congestion_ctrl_intake_throttle: 0,
14        },
15        server_stats: ServerMetaDownstreamServerStats {
16            cpu_usage: 0.0,
17            memory_usage_kb: 0,

```

```

18         },
19         logs: vec![ServerMetaDownstreamLog {
20             timestamp: 0,
21             severity: ServerMetaDownstreamLogSeverity::Info,
22             message: "test".to_string(),
23         }],
24     },
25     messages: vec![DownStreamMessage {
26         socket_id: 0,
27         payload: vec![0, 1, 2, 3],
28     }],
29 };

```

Tests ran fine after the changes:

```

1  running 5 tests
2  test test_client_upstream ... ok
3  test test_downstream_msg ... ok
4  test test_generate_socket_id ... ok
5  test test_server_message_downstream ... ok
6  test test_upstream_msg ... ok
7
8  test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
   finished in 0.00s

```

4.1.35. GUI Research

I had yet to decide what GUI framework to use.

Tauri [122] would allow me to use web tech, which I am already familiar with. Iced [123] would require implementing my complex diagrammatic UI as a custom widget, but everything else about it is significantly nicer.

I could also use Bevy [124], but as a full on game engine it seems both overkill and not particularly appropriate to the issue at hand.

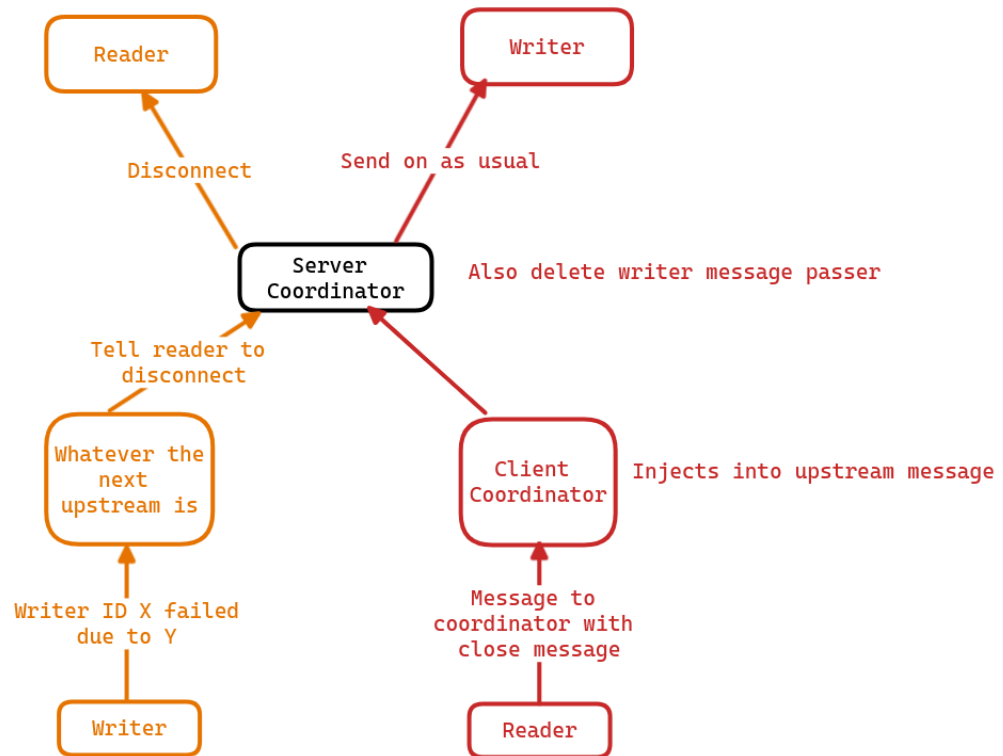
I decided on Tauri to reduce the amount of new things - I was already learning complex Rust, Tokio, Actix, and a myriad of smaller libraries.

4.1.36. Socket Termination

Socket termination isn't actually the trivial affair it seems to be at first glance. TCP sockets aren't actually necessarily bidirectional - it is possible to terminate one direction while preserving the other. That means that there are four different routes for socket termination.

In order to track these paths I used a colour coding system, linked to an extremely useful diagram:

Client → Server



Server → Client

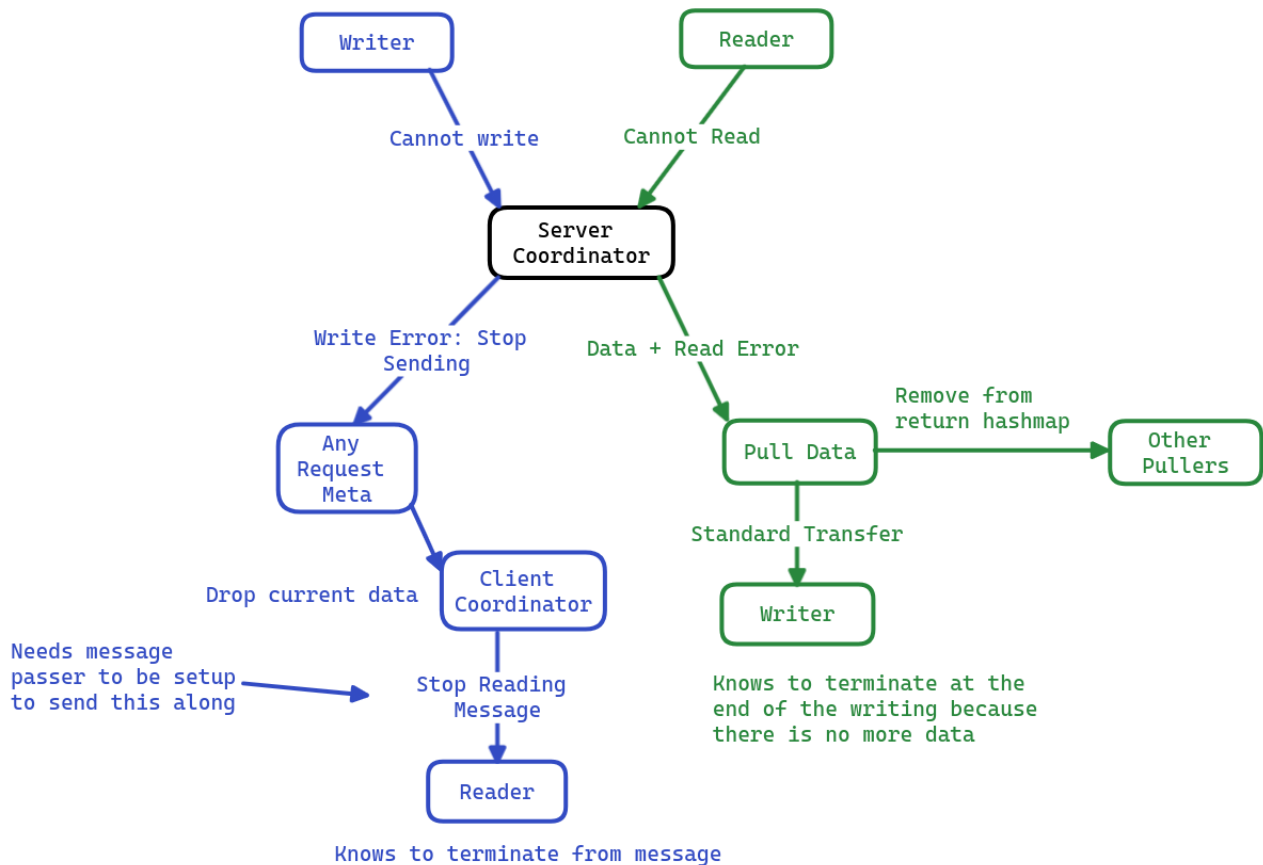


Figure 50: An extremely rough diagram of the four different socket termination paths.

While this may initially appear as a poor system for variable naming, semantic naming was extremely verbose and ended up making the code so hard to scan that a colour system and diagram produced more readable code.

4.1.37. Implementing the Yellow Route

The yellow route has some unique characteristics:

- In this route we don't care about stopping reading after we've got data because we'll have nothing to return that data to.
- We still didn't have handling for data in-transit that now has nowhere to write to. The downstream senders just `.unwrap()` if something they expect to exist doesn't.

The implementation is quite distributed, with lots of small changes. The yellow route is clearly visible in the final code in the appendix.

All tests passed:

```

1  test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
   finished in 0.00s
2
3      Running unittests src/lib.rs (target/debug/deps/integration_tests-
   clc90ea3b7962da9)
4
5  running 1 test
6  test tests::check_socket_connect ... ok
7
8  test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
   finished in 0.00s
9
10     Running unittests src/lib.rs (target/debug/deps/
   libsecrets-6bc1670bc0f993a6)
11
12  running 3 tests
13  test tests::test_form_key ... ok
14  test tests::test_encrypt_decrypt ... ok
15  test tests::test_decryption_wrong_key ... ok
16
17  test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
   finished in 0.00s
18
19     Running unittests src/lib.rs (target/debug/deps/libsocks-453ddff261080c0d)
20
21  running 7 tests
22  test tests::check_parse_reply ... ok
23  test tests::check_parse_bind ... ok
24  test tests::check_write_bind ... ok
25  test tests::check_decode_request ... ok
26  test tests::check_parse_connect ... ok
27  test tests::check_write_reply ... ok
28  test tests::check_write_connect ... ok
29
30  test result: ok. 7 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
   finished in 0.00s
31
32     Running unittests src/lib.rs (target/debug/deps/libtransit-
   b1c49fdea96f65e0)
33
34  running 5 tests
35  test test_generate_socket_id ... ok
36  test test_upstream_msg ... ok

```

```

37 test test_downstream_msg ... ok
38 test test_client_upstream ... ok
39 test test_server_message_downstream ... ok

```

4.1.38. Continued Termination

I then implemented the red route, which was - once again - pretty trivial. The green work was also pretty simple, although I accompanied it with some basic refactors focused on making the code more readable. I moved tests to a separate file to `libtransit` and decided to use a trait to define encoding/decoding:

```

1 pub trait SerialMessage: BorshSerialize + BorshDeserialize {
2     fn encoded(&self) -> Result<Vec<u8>, Box<dyn std::error::Error>> {
3         let data = self.try_to_vec()?;
4         Ok(data)
5     }
6
7     fn decode_from_bytes(data: &mut Vec<u8>) -> Result<Self, std::io::Error>
8     where
9         Self: Sized,
10    {
11        Self::try_from_slice(data)
12    }
13 }

```

I then finished the blue socket. Unfortunately, I still had some strange stability issues when I tried to run the program. I didn't seem to actually have the expected quantity of socket termination going on.

4.1.39. CLI

I utilised `clap` [125] to build a CLI for the client, rather than relying on hard-coded values. The library did the vast majority of the work for me, with me simply defining the CLI definition then passing the values on to the core client library.

```

1 use clientcore::{begin_core_client, ClientArguments};
2
3 use clap::Parser;
4
5 #[derive(Parser, Debug)]
6 #[clap(name = "Hypertunnel Client")]
7 #[command(version, about, author)]
8 struct Args {
9     #[clap(long, default_value = "127.0.0.1")]
10    listen_host: String,
11
12    #[clap(long, default_value = "1080")]
13    listen_port: u16,
14
15    #[clap(long, default_value = "http://127.0.1:8000")]
16    target_host: String,
17
18    #[clap(long, default_value = "12345")]
19    password: String,
20 }
21
22 #[tokio::main]
23 async fn main() {

```

```

24     let arguments = Args::parse();
25
26     let client_args = ClientArguments {
27         listen_address: arguments.listen_host,
28         listen_port: arguments.listen_port,
29         target_host: arguments.target_host,
30         password: arguments.password,
31     };
32
33     begin_core_client(client_args).await;
34 }

```

It works well in use:

```

1 $ ./client-cli --help
2 Usage: client-cli [OPTIONS]
3
4 Options:
5     --listen-host <LISTEN_HOST> [default: 127.0.0.1]
6     --listen-port <LISTEN_PORT> [default: 1080]
7     --target-host <TARGET_HOST> [default: http://127.0.1:8000]
8     --password <PASSWORD> [default: 12345]
9     -h, --help                Print help
10    -V, --version              Print version

```

```

1 $ ./client-cli --version
2 Hypertunnel Client 0.1.0

```

4.1.40. GUI

I decided to build a minimal initiation GUI with `egui` [126]. I setup a core desktop app using `eframe` [127]. Note that `eframe` is architected as a template which is modified; not all of the GUI code is my own. The GUI itself was very simple, consisting of some input fields and an engage/disengage button with minimal state management. It does all that it needs to do in order to fulfill its purpose.

See Section 4.2.21.

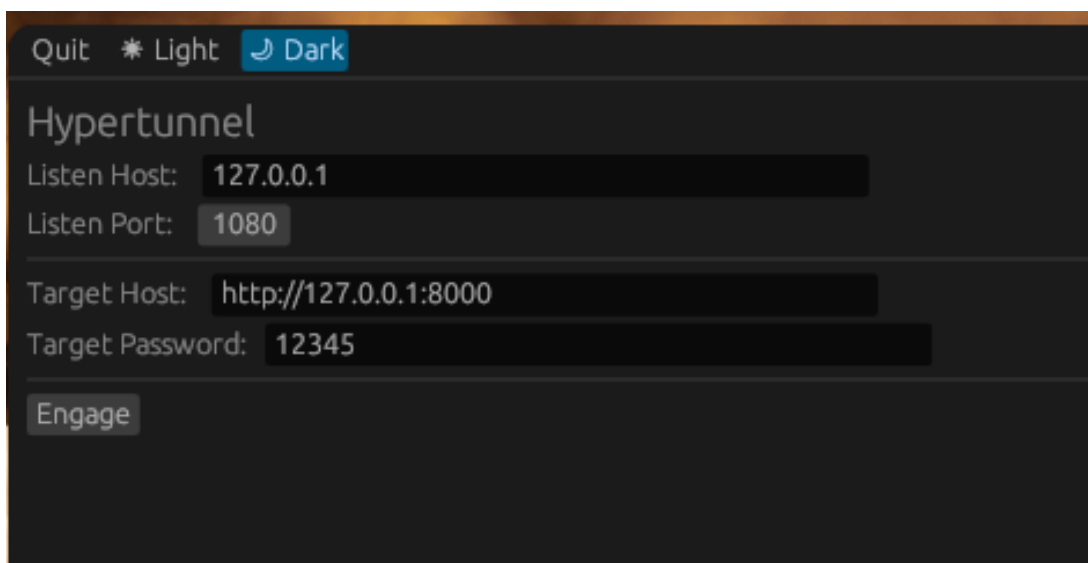


Figure 51: A screenshot of the GUI.

4.1.41. Refactoring

As I approached the end of my project I decided to update all of my dependencies. This meant introducing some strange cyclical issues, but these were solved through selective unlinking of modules. I updated tokio from 1.30.0 to 1.36.0, flume from 0.10 to 0.11, env_logger from 0.10 to 0.11, generic-array from 0.14 to 1, binrw from 0.11 to 0.13, actix-web from 4 to 4.5.1, toml from 0.7 to 0.8, flume from 0.10 to 0.11, and borsh from 0.10 to 1.

Borsh involved changing some traits and importing `derive` under features, but the `libtransit` external interfaces remained the same.

4.1.42. Stability

I found that I was panicking when I should sometimes be sending green termination. Solving this significantly improved long-term stability.

I then generally applied the “whack-a-mole” approach to bug fixing, solving panics and issues as they arose during testing. Most of this just consisted of replying explicit calls to `panic!()` and `.unwrap()` with calls to the socket termination system.

This approach worked extremely well. I achieved perfect reliability in post-debug testing, and in local tests I was able to achieve 99mbps download speed. In remote target tests I was able to browse the internet as usual without any noticeable change compared to my normal VPN. This was going to an overworked Raspberry Pi 4 with 10mbps upload speed, so I was very happy with the results.

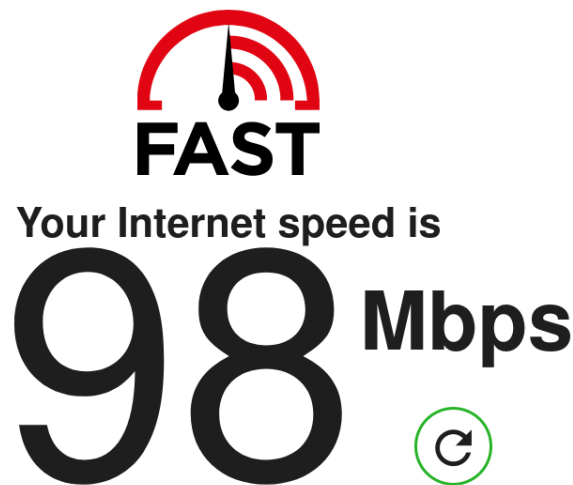


Figure 52: A speedtest demonstrating wonderful local-server performance. Tested on fast.com [128]

4.1.43. Better Logging

I switched to `log cite` and `SimpleLogger cite`. This was a pretty simple change, but it introduced greater log levels, log filtering, coloured logs, access to actix-web logs, and timestamps. I did this on both the client and the server.

4.1.44. Metadata Handling

I hadn't looked at the metadata system in a while, and I suspected that it had become redundant. Neither side actually used the exchanged data. I decided to reduce the metadata, removing logs and CPU/memory statistics.

4.1.45. Improved Circumvention

There are two areas where I would like to improve circumvention quality. First, I would like to insert fake headers, file extensions, and Content-Types in order to masquerade as simple web browsing. Second, I would like to implement dynamic padding to make it harder to detect patterns like TLS handshakes.

4.1.46. False Download MIME Types and Headers

I would like to make downloaded data look like a typical Twitch stream, because it's not something system administrators will take a close look at. Twitch exchanges binary blobs with the `application/octet-stream` format with requests of the form `/v1/segment/NONSENSE_CHARACTERS`.

I analysed its internal format with Cyberchef. It doesn't match with anything in https://en.wikipedia.org/wiki/List_of_file_signatures - which means we don't need to either!

The screenshot shows the CyberChef web application interface. On the left, the 'Recipe' panel is active, showing a recipe named 'From Base64'. The recipe steps are: 'From Base64' (selected), 'Remove non-alphabet chars' (checked), 'Strict mode' (unchecked), 'To Hexdump' (selected), 'Width 16' (selected), 'Upper case hex' (unchecked), 'Include final length' (unchecked), and 'UNIX format' (checked). The 'Input' panel on the right contains a long Base64 string. The 'Output' panel at the bottom shows the resulting hex dump, which is a series of hexadecimal values representing the raw bytes of the input data.

Figure 53: Analysis of the file data in cyberchef [129].

I decided to make the change of explicitly returning an `octet-stream` and setting the path to `/video/segment/[dynamic nonsense]`.

4.1.47. False Upload MIME Types and Headers

For the false upload I want to pretend that I'm uploading a file to a file sharing service over javascript, by sending a `application/octet-stream` with a URL of `/upload/segment/[dynamic nonsense]`.

4.1.48. Client Hello False MIME Types

The client hello is already disguised as a form submission.

Implementation was a trivial case of inserting a new header and generating the dynamic non-sense. Here is the generation function:

```
1 fn generate_nonsense_data() -> String {
2     let mut rng = rand::thread_rng();
3     (0..128).map(|_| rng.gen:::<char>()).collect()
4 }
```

I ended up changing it to only use some characters:

```
1 fn generate_nonsense_data() -> String {
2     let dataset =
3     "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
4     let count = 64;
5
6     let mut rng = rand::thread_rng();
7     let mut data = String::with_capacity(count);
8     for _ in 0..count {
9         let index = rng.gen_range(0..dataset.len());
10        data.push(dataset.chars().nth(index).unwrap());
11    }
12    data
13 }
```

I also introduced exponential backoff to the client download retrying to avoid an accidental denial-of-service when connections were dropped.

Code Appendix

This only contains code specifically referenced in the previous section. The entire final codebase is available at the end of the document.

01: SOCKS Definition

```

1  use binrw;
2  use binrw::{BinReaderExt, BinWriterExt};
3  use binrw::io::{Cursor, Seek, SeekFrom};
4
5  pub type IPV4 = u32;
6  pub type Port = u16;
7  pub type ConnID = u32;
8
9  #[binrw::binrw]
10 #[brw(repr(u8))]
11 #[derive(Debug, PartialEq)]
12 pub enum Socks4Command {
13     Connect = 1,
14     Bind = 2,
15 }
16
17 #[binrw::binrw]
18 #[brw(repr(u8))]
19 #[derive(Debug, PartialEq)]
20 pub enum Socks4Status {
21     Granted = 90,
22     Rejected = 91,
23     RejectedNoIdentd = 92,
24     RejectedIdentdMismatch = 93,
25 }
26
27 #[binrw::binrw]
28 pub struct Socks4ConnectRequest {
29     pub version: u8,
30     pub command: Socks4Command,
31     pub dstport: Port,
32     pub dstip: IPV4,
33     pub userid: binrw::NullString,
34 }
35
36 #[binrw::binrw]
37 pub struct Socks4ConnectReply {
38     pub version: u8,
39     pub status: Socks4Status,
40     pub dstport: Port,
41     pub dstip: IPV4,
42 }
43
44 #[binrw::binrw]
45 pub struct Socks4BindRequest {
46     pub version: u8,
47     pub command: Socks4Command,
48     pub dstport: Port,
49     pub dstip: IPV4,
50     pub userid: binrw::NullString,
51 }
52
53 #[binrw::binrw]
54 pub struct Socks4BindReply {
55     pub version: u8,

```

```

56     pub status: Socks4Status,
57     pub dstport: Port,
58     pub dstip: IPV4,
59 }
60
61
62 #[cfg(test)]
63 mod tests {
64     use super::*;
65
66     #[test]
67     fn check_parse_connect() {
68         let sample_data = [
69             0x04, 0x01, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01, 0x61, 0x62, 0x63, 0x64, 0x00
70         ];
71
72         let mut cursor = Cursor::new(&sample_data);
73         let request: Socks4ConnectRequest = cursor.read_be().unwrap();
74         assert_eq!(request.version, 4);
75         assert_eq!(request.command, Socks4Command::Connect);
76         assert_eq!(request.dstport, 80);
77         assert_eq!(request.dstip, 1);
78         assert_eq!(request.userid.to_string(), "abcd");
79     }
80
81     #[test]
82     fn check_write_connect() {
83         let request = Socks4ConnectRequest {
84             version: 4,
85             command: Socks4Command::Connect,
86             dstport: 80,
87             dstip: 1,
88             userid: binrw::NullString::from("abcd"),
89         };
90
91         let mut writer = Cursor::new(Vec::new());
92         writer.write_be(&request).unwrap();
93
94         let expected_data = [
95             0x04, 0x01, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01, 0x61, 0x62, 0x63, 0x64, 0x00
96         ];
97
98         assert_eq!(writer.into_inner(), expected_data);
99     }
100
101     #[test]
102     fn check_write_reply() {
103         let reply = Socks4ConnectReply {
104             version: 4,
105             status: Socks4Status::Granted,
106             dstport: 80,
107             dstip: 1,
108         };
109
110         let mut writer = Cursor::new(Vec::new());
111         writer.write_be(&reply).unwrap();
112
113         let expected_data = [
114             0x04, 0x5a, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01
115         ];
116
117         assert_eq!(writer.into_inner(), expected_data);

```

```
118     }
119
120     #[test]
121     fn check_parse_reply() {
122         let data = [
123             0x04, 0x5a, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01
124         ];
125
126         let mut cursor = Cursor::new(&data);
127         let reply: Socks4ConnectReply = cursor.read_be().unwrap();
128         assert_eq!(reply.version, 4);
129         assert_eq!(reply.status, Socks4Status::Granted);
130         assert_eq!(reply.dstport, 80);
131         assert_eq!(reply.dstip, 1);
132     }
133
134     #[test]
135     fn check_parse_bind() {
136         let data = [
137             0x04, 0x02, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01, 0x61, 0x62, 0x63, 0x64, 0x00
138         ];
139
140         let mut cursor = Cursor::new(&data);
141         let request: Socks4BindRequest = cursor.read_be().unwrap();
142         assert_eq!(request.version, 4);
143         assert_eq!(request.command, Socks4Command::Bind);
144         assert_eq!(request.dstport, 80);
145         assert_eq!(request.dstip, 1);
146         assert_eq!(request.userid.to_string(), "abcd");
147     }
148
149     #[test]
150     fn check_write_bind() {
151         let request = Socks4BindRequest {
152             version: 4,
153             command: Socks4Command::Bind,
154             dstport: 80,
155             dstip: 1,
156             userid: binrw::NullString::from("abcd"),
157         };
158
159         let mut writer = Cursor::new(Vec::new());
160         writer.write_be(&request).unwrap();
161
162         let expected_data = [
163             0x04, 0x02, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01, 0x61, 0x62, 0x63, 0x64, 0x00
164         ];
165
166         assert_eq!(writer.into_inner(), expected_data);
167     }
168 }
```

02: Multiple Messages

```

1  #[derive(Debug, PartialEq, Clone)]
2  pub enum Message {
3      StreamMessage(StreamMessage),
4      CreateSocketMessage(CreateSocketMessage),
5      CloseSocketMessage(CloseSocketMessage),
6  }
7
8  #[binrw::binrw]
9  #[brw(repr(u8))]
10 #[derive(Debug, PartialEq, Clone)]
11 pub enum MessageType {
12     CreateSocket = 0,
13     CloseSocket = 1,
14     SendStreamPacket = 2,
15     ReceiveStreamPacket = 3,
16 }
17
18 #[binrw::binrw]
19 #[derive(Debug, PartialEq, Clone)]
20 #[brw(big)]
21 pub struct StreamMessage {
22     pub message_type: MessageType,
23     pub socket_id: u32,
24
25     pub send_buffer_size: u32,
26     pub receive_buffer_size: u32,
27
28     #[bw(try_calc(u32::try_from(packet_data.len())))]
29     pub packet_length: u32,
30     #[br(count = packet_length)]
31     pub packet_data: Vec<u8>,
32 }
33
34 impl StreamMessage {
35     pub fn encoded(socket_id: u32, send_buffer_size: u32, receive_buffer_size: u32, packet_data:
36     Vec<u8>) -> Vec<u8> {
37         let data = Self {
38             message_type: MessageType::SendStreamPacket,
39             socket_id,
40             send_buffer_size,
41             receive_buffer_size,
42             packet_data,
43         };
44         let mut writer = Cursor::new(Vec::new());
45         writer.write_be(&data).unwrap();
46
47         return writer.into_inner();
48     }
49
50     pub fn from_bytes(bytes: Vec<u8>) -> Self {
51         let mut reader = Cursor::new(bytes);
52         return reader.read_be().unwrap();
53     }
54 }
55
56 #[test]
57 fn test_stream_message() {
58     let mut rng = rand::thread_rng();
59     let packet_data: Vec<u8> = (0..rng.gen_range(0..100)).map(|_| rng.gen()).collect();

```

```

60
61     let data = StreamMessage {
62         message_type: MessageType::SendStreamPacket,
63         socket_id: 0,
64         send_buffer_size: 0,
65         receive_buffer_size: 0,
66         packet_data: packet_data,
67     };
68
69     let mut writer = Cursor::new(Vec::new());
70     writer.write_be(&data).unwrap();
71     let bytes = writer.into_inner();
72
73     let mut reader = Cursor::new(bytes);
74     let read_data = reader.read_be::<StreamMessage>().unwrap();
75
76     assert_eq!(data, read_data);
77     assert_eq!(data.packet_data, read_data.packet_data);
78 }
79
80 #[binrw::binrw]
81 #[derive(Debug, PartialEq, Clone)]
82 #[brw(big)]
83 pub struct CreateSocketMessage {
84     pub message_type: MessageType,
85     pub socket_id: u32,
86     pub forwarding_address: IPV4,
87     pub forwarding_port: Port,
88 }
89
90 impl CreateSocketMessage {
91     pub fn encoded(socket_id: u32, forwarding_address: IPV4, forwarding_port: Port) -> Vec<u8> {
92         let data = Self {
93             message_type: MessageType::CreateSocket,
94             socket_id,
95             forwarding_address,
96             forwarding_port,
97         };
98
99         let mut writer = Cursor::new(Vec::new());
100         writer.write_be(&data).unwrap();
101
102         return writer.into_inner();
103     }
104
105     pub fn from_bytes(bytes: Vec<u8>) -> Self {
106         let mut reader = Cursor::new(bytes);
107         return reader.read_be().unwrap();
108     }
109 }
110
111 #[test]
112 fn test_create_socket_message() {
113     let data = CreateSocketMessage {
114         message_type: MessageType::CreateSocket,
115         socket_id: 0,
116         forwarding_address: 5,
117         forwarding_port: 0,
118     };
119
120     let mut writer = Cursor::new(Vec::new());
121     writer.write_be(&data).unwrap();

```

```

122     let bytes = writer.into_inner();
123
124     let mut reader = Cursor::new(bytes);
125     let read_data = reader.read_be::<CreateSocketMessage>().unwrap();
126
127     assert_eq!(data, read_data);
128     assert_eq!(data.forwarding_address, read_data.forwarding_address);
129     assert_eq!(data.forwarding_port, read_data.forwarding_port);
130     assert_eq!(data.socket_id, read_data.socket_id);
131     assert_eq!(data.message_type, read_data.message_type);
132     assert_eq!(read_data.message_type, MessageType::CreateSocket);
133 }
134
135 #[binrw::binrw]
136 #[derive(Debug, PartialEq, Clone)]
137 #[brw(big)]
138 pub struct CloseSocketMessage {
139     pub message_type: MessageType,
140     pub socket_id: u32,
141 }
142
143 impl CloseSocketMessage {
144     pub fn encoded(socket_id: u32) -> Vec<u8> {
145         let data = Self {
146             message_type: MessageType::CloseSocket,
147             socket_id,
148         };
149
150         let mut writer = Cursor::new(Vec::new());
151         writer.write_be(&data).unwrap();
152
153         return writer.into_inner();
154     }
155
156     pub fn from_bytes(bytes: Vec<u8>) -> Self {
157         let mut reader = Cursor::new(bytes);
158         return reader.read_be().unwrap();
159     }
160 }
161
162 #[test]
163 fn test_close_socket_message() {
164     let data = CloseSocketMessage {
165         message_type: MessageType::CloseSocket,
166         socket_id: 5,
167     };
168
169     let mut writer = Cursor::new(Vec::new());
170     writer.write_be(&data).unwrap();
171     let bytes = writer.into_inner();
172
173     let mut reader = Cursor::new(bytes);
174     let read_data = reader.read_be::<CloseSocketMessage>().unwrap();
175
176     assert_eq!(data, read_data);
177     assert_eq!(data.socket_id, read_data.socket_id);
178     assert_eq!(data.message_type, read_data.message_type);
179     assert_eq!(read_data.message_type, MessageType::CloseSocket);
180 }

```

The library didn't provide a good way to encode as a vec, so I implemented my own multiple message serialiser:

```

1 use binrw;
2 use binrw::{BinReaderExt, BinWriterExt};
3 use binrw::io::{Cursor, Seek, SeekFrom};
4 use rand::Rng;
5
6 pub type Port = u16;
7 pub type IPV4 = u32;
8
9 // We want to encode multiple messages, which could be
10 // either a stream, create, or close, into a single buffer.
11 // We need to be able to decode the buffer into the list of
12 // messages, with the correct data. We should not waste data
13 // on padding. We will begin with the length of the offsets list,
14 // and then the offsets list, and then the data for each message at
15 // the specified offsets from the end of the offsets list.
16
17 #[binrw::binrw]
18 #[derive(Debug, PartialEq)]
19 #[brw(big)]
20 pub struct MultipleMessages {
21     #[bw(try_calc(u8::try_from(sizes.len())))]
22     pub sizes_length: u8,
23     #[br(count = sizes_length)]
24     pub sizes: Vec<u32>,
25
26     #[bw(try_calc(u32::try_from(data.len())))]
27     pub data_length: u32,
28     #[br(count = data_length)]
29     pub data: Vec<u8>,
30 }
31
32 impl MultipleMessages {
33     pub fn from_messages(messages: Vec<Message>) -> Self {
34         if messages.len() > u8::MAX as usize {
35             panic!("Cannot encode more than 255 messages into a single buffer.");
36         }
37
38         let mut sizes: Vec<u32> = Vec::with_capacity(messages.len());
39         let mut data: Vec<u8> = Vec::new();
40
41         for message in messages {
42             let message_data = match message {
43                 Message::StreamMessage(stream_message) => {
44                     let mut writer = Cursor::new(Vec::new());
45                     writer.write_be(&stream_message).unwrap();
46                     writer.into_inner()
47                 },
48                 Message::CreateSocketMessage(create_socket_message) => {
49                     let mut writer = Cursor::new(Vec::new());
50                     writer.write_be(&create_socket_message).unwrap();
51                     writer.into_inner()
52                 },
53                 Message::CloseSocketMessage(close_socket_message) => {
54                     let mut writer = Cursor::new(Vec::new());
55                     writer.write_be(&close_socket_message).unwrap();
56                     writer.into_inner()
57                 },
58             };
59
60             sizes.push(message_data.len() as u32);
61             data.extend(message_data);
62         }

```

```

63
64     Self {
65         sizes,
66         data
67     }
68 }
69
70 pub fn to_bytes(&self) -> Vec<u8> {
71     // Write the data to a multiple messages buffer
72     let buffer = Vec::with_capacity(1 + self.sizes.len() * 4 + self.data.len());
73     let mut writer = Cursor::new(buffer);
74
75     writer.write_be(&self).unwrap();
76
77     writer.into_inner()
78 }
79
80 pub fn from_bytes(buffer: Vec<u8>) -> Self {
81     let mut reader = Cursor::new(buffer);
82     reader.read_be::<MultipleMessages>().unwrap()
83 }
84
85 pub fn pull_messages(&self) -> Vec<Message> {
86     let mut messages: Vec<Message> = Vec::with_capacity(self.sizes.len());
87
88     let mut i = 0;
89     let mut base_offset = 0;
90
91     println!("Self.data: {:?} (Length {})", self.data, self.data.len());
92
93     loop {
94         let data_start = base_offset;
95         let data_end = base_offset + self.sizes[i] as usize;
96
97         println!("Data start: {}", data_start);
98         println!("Data end: {}", data_end);
99         println!("Sizes: {:?}", self.sizes);
100        println!("Current offset: {}", i);
101
102        // Get those bytes
103        let message_data = self.data[data_start..data_end].to_vec();
104
105        // Read to a message struct
106        println!("Message data: {:?}", message_data);
107        let mut data_reader = Cursor::new(message_data);
108
109        let messagetype_reader = self.data[data_start];
110
111        let message_type = match messagetype_reader {
112            0 => MessageType::CreateSocket,
113            1 => MessageType::CloseSocket,
114            2 => MessageType::SendStreamPacket,
115            3 => MessageType::ReceiveStreamPacket,
116            _ => panic!("Invalid message type"),
117        };
118
119        let message = match message_type {
120            MessageType::CreateSocket => {
121                Message::CreateSocketMessage(data_reader.read_be::<CreateSocketMessage>().unwrap())
122            },
123            MessageType::CloseSocket => {

```

```

124         Message::CloseSocketMessage(data_reader.read_be::<CloseSocketMessage>().unwrap())
125     },
126     MessageType::SendStreamPacket => {
127         Message::StreamMessage(data_reader.read_be::<StreamMessage>().unwrap())
128     },
129     MessageType::ReceiveStreamPacket => {
130         Message::StreamMessage(data_reader.read_be::<StreamMessage>().unwrap())
131     },
132 };
133
134 // Push to messages
135 messages.push(message);
136
137 // Check if we should stop
138 if i > self.sizes.len() - 2 {
139     break;
140 }
141
142 i = i + 1;
143 base_offset = data_end;
144 }
145
146 messages
147 }
148 }
149
150 #[test]
151 fn test_multiple_message_writing_reading() {
152     let mut messages: Vec<Message> = Vec::new();
153
154     messages.push(Message::CreateSocketMessage(CreateSocketMessage {
155         message_type: MessageType::CreateSocket,
156         socket_id: 5,
157         forwarding_address: 43,
158         forwarding_port: 80,
159     }));
160
161     // Write message to bytes
162     let mut writer = Cursor::new(Vec::new());
163     writer.write_be(&CreateSocketMessage {
164         message_type: MessageType::CreateSocket,
165         socket_id: 5,
166         forwarding_address: 43,
167         forwarding_port: 80,
168     }).unwrap();
169     let message_bytes = writer.into_inner();
170     println!("Message bytes: {:?}", message_bytes);
171
172     messages.push(Message::StreamMessage(StreamMessage {
173         message_type: MessageType::SendStreamPacket,
174         socket_id: 5,
175         send_buffer_size: 0,
176         receive_buffer_size: 0,
177         packet_data: vec![1, 2, 3, 4, 5],
178     }));
179
180     messages.push(Message::CloseSocketMessage(CloseSocketMessage {
181         message_type: MessageType::CloseSocket,
182         socket_id: 5,
183     }));
184
185     let multiple_messages = MultipleMessages::from_messages(messages.clone());

```

```
186
187     let buffer = multiple_messages.to_bytes();
188
189     let multiple_messages = MultipleMessages::from_bytes(buffer);
190
191     let messages = multiple_messages.pull_messages();
192
193     assert_eq!(messages, messages);
194 }
```

03: Criterion

```

1 use criterion::{criterion_group, criterion_main, Criterion};
2
3 use libtransit::{Message, CreateSocketMessage, StreamMessage, CloseSocketMessage, MessageType,
4 MultipleMessages};
5
6 fn gen_data(size: usize) -> Vec<u8> {
7     Vec::from_iter((0..(size)).map(|_| rand::random:::<u8>()))
8 }
9
10 fn stress_test_multiple_messages(messagecount: u32, data: Vec<u8>) {
11     let mut messages: Vec<Message> = Vec::new();
12
13     for i in 0..messagecount {
14         messages.push(Message::CreateSocketMessage(CreateSocketMessage {
15             message_type: MessageType::CreateSocket,
16             socket_id: i,
17             forwarding_address: 43,
18             forwarding_port: 80,
19         }));
20
21         messages.push(Message::StreamMessage(StreamMessage {
22             message_type: MessageType::SendStreamPacket,
23             socket_id: i,
24             send_buffer_size: 0,
25             receive_buffer_size: 0,
26             packet_data: data.clone(),
27         }));
28
29         messages.push(Message::CloseSocketMessage(CloseSocketMessage {
30             message_type: MessageType::CloseSocket,
31             socket_id: i,
32         }));
33     }
34
35     let multiple_messages = MultipleMessages::from_messages(messages.clone());
36
37     let buffer = multiple_messages.to_bytes();
38
39     let multiple_messages = MultipleMessages::from_bytes(buffer);
40
41     let messages = multiple_messages.pull_messages();
42
43     assert_eq!(messages, messages);
44 }
45
46 fn criterion_benchmark(c: &mut Criterion) {
47     c.bench_function("stress_test_multiple_messages_50mc_100b (5kb)", |b| b.iter(||
48 stress_test_multiple_messages(50, gen_data(100))));
49     c.bench_function("stress_test_multiple_messages_50mc_1kb (50kb)", |b| b.iter(||
50 stress_test_multiple_messages(50, gen_data(1000))));
51     c.bench_function("stress_test_multiple_messages_50mc_10kb (500kb)", |b| b.iter(||
52 stress_test_multiple_messages(50, gen_data(10_000))));
53     c.bench_function("stress_test_multiple_messages_50mc_100kb (5mb)", |b| b.iter(||
54 stress_test_multiple_messages(50, gen_data(100_000))));
55
56     c.bench_function("stress_test_multiple_messages_10mc_1mb (10mb)", |b| b.iter(||
57 stress_test_multiple_messages(10, gen_data(1_000_000))));
58     c.bench_function("stress_test_multiple_messages_10mc_10mb (100mb)", |b| b.iter(||
59 stress_test_multiple_messages(10, gen_data(10_000_000))));

```

```
53     c.bench_function("stress_test_multiple_messages_10mc_100mb (1000mb)", |b| b.iter(||
54 stress_test_multiple_messages(10, gen_data(100_000_000))));
55 }
56 criterion_group!(benches, criterion_benchmark);
57 criterion_main!(benches);
```

04: Initial Encryption Lib

```
1 use chacha20poly1305::{
2     aead::{Aead, AeadCore, KeyInit, OsRng, Key},
3     ChaCha20Poly1305, Nonce, Error
4 };
5
6 use generic_array::GenericArray;
7
8 pub fn form_key(key: &[u8]) -> Key<ChaCha20Poly1305> {
9     GenericArray::clone_from_slice(key)
10 }
11
12 fn random_nonce() -> Nonce {
13     ChaCha20Poly1305::generate_nonce(&mut OsRng)
14 }
15
16 pub fn encrypt(data: &[u8], key: &Key<ChaCha20Poly1305>) -> Result<Vec<u8>, Error> {
17     let nonce = random_nonce();
18     let cipher = ChaCha20Poly1305::new(key);
19     let mut encrypted = cipher.encrypt(&nonce, data.as_ref())?;
20     encrypted.extend_from_slice(nonce.as_ref()); // Append 12-byte nonce
21     Ok(encrypted)
22 }
23
24 pub fn decrypt(data: &[u8], key: &Key<ChaCha20Poly1305>) -> Result<Vec<u8>, Error> {
25     let nonce = Nonce::from_slice(&data[data.len() - 12..]); // Extract 12-byte nonce
26     let data = &data[..data.len() - 12]; // Remove 12-byte nonce
27
28     let cipher = ChaCha20Poly1305::new(key);
29     let decrypted = cipher.decrypt(&nonce, data.as_ref())?;
30     Ok(decrypted)
31 }
```

05: Transit Socket Builder

```

1  pub struct TransitSocketBuilder {
2      target: Option<String>,
3      key: Option<EncryptionKey>,
4      password: Option<String>,
5      client_identifier: Option<String>,
6      hybrid_client_count: Option<usize>,
7      pull_client_count: Option<usize>,
8      timeout_time: Option<usize>,
9  }
10
11  impl TransitSocketBuilder {
12      pub fn new() -> Self {
13          Self {
14              target: None,
15              key: None,
16              password: None,
17              client_identifier: None,
18              hybrid_client_count: None,
19              pull_client_count: None,
20              timeout_time: None,
21          }
22      }
23
24      pub fn with_target(mut self, target: String) -> Self {
25          self.target = Some(target);
26          self
27      }
28
29      pub fn with_key(mut self, key: EncryptionKey) -> Self {
30          if self.password.is_some() {
31              panic!("Cannot set both key and password");
32          }
33          self.key = Some(key);
34          self
35      }
36
37      pub fn with_password(mut self, password: String) -> Self {
38          if self.key.is_some() {
39              panic!("Cannot set both key and password");
40          }
41          self.password = Some(password);
42          self
43      }
44
45      pub fn with_client_identifier(mut self, client_identifier: String) -> Self {
46          self.client_identifier = Some(client_identifier);
47          self
48      }
49
50      pub fn with_hybrid_client_count(mut self, hybrid_client_count: usize) -> Self {
51          self.hybrid_client_count = Some(hybrid_client_count);
52          self
53      }
54
55      pub fn with_pull_client_count(mut self, pull_client_count: usize) -> Self {
56          self.pull_client_count = Some(pull_client_count);
57          self
58      }
59
60      pub fn with_timeout_time(mut self, timeout_time: usize) -> Self {
61          self.timeout_time = Some(timeout_time);

```

```

62     self
63 }
64
65 pub fn build(self) -> TransitSocket {
66     let target = self.target.expect("Target not set");
67
68     let key = if let Some(key) = self.key {
69         key
70     } else if let Some(password) = self.password {
71         libsecrets::form_key(password.as_bytes())
72     } else {
73         panic!("Neither key nor password set");
74     };
75
76     let client_identifier = match self.client_identifier {
77         Some(client_identifier) => client_identifier,
78         None => {
79             // Create a random client identifier string
80             let mut client_identifier = String::new();
81             for _ in 0..16 {
82                 client_identifier.push(rand::random:::<u8>() as char);
83             }
84             client_identifier
85         }
86     };
87
88     let hybrid_client_count = match self.hybrid_client_count {
89         Some(hybrid_client_count) => hybrid_client_count,
90         None => 4,
91     };
92
93     let pull_client_count = match self.pull_client_count {
94         Some(pull_client_count) => pull_client_count,
95         None => 1,
96     };
97
98     let timeout_time = match self.timeout_time {
99         Some(timeout_time) => timeout_time,
100        None => 10,
101    };
102
103    let control_client = Client::new();
104    let send_buffer = RwLock::new(Vec::new());
105    let recv_buffer = RwLock::new(Vec::new());
106
107    let server_statistics = ServerStatistics {
108        to_reply: 0,
109        to_send: 0,
110    };
111
112    let headers = generate_headers(client_identifier.clone(), target.clone());
113
114    TransitSocket {
115        target,
116        key,
117        control_client,
118        send_buffer,
119        recv_buffer,
120        server_statistics,
121        client_identifier,
122        hybrid_client_count,
123        pull_client_count,

```

```
124         timeout_time,  
125         headers,  
126     }  
127 }  
128 }
```

06: Early Server

```

1 use actix_web::{get, post, web, App, HttpResponse, HttpServer, HttpRequest, Responder};
2 use actix_web::web::Bytes;
3 use libsecrets::{EncryptionKey, self};
4 use rand::Rng;
5
6 use debug_print::{
7     debug_print as dprint,
8     debug_println as dprintln,
9     debug_eprint as deprint,
10    debug_eprintln as deprintln,
11 };
12
13 use dashmap::DashMap;
14
15 mod config;
16 mod transit_socket;
17 use transit_socket::{TransitSocket, ClientStatistics};
18
19 // State passed to all request handlers
20 struct AppState {
21     config: config::Config, // Configuration
22     sessions: DashMap<u8; 16>, TransitSocket>, // Currently-in-use sessions with the
23     // client identifier as the key
24     users: Vec<User>, // Users from the configuration with the passwords preprocessed into keys
25     // for faster initial handshake when there are many users
26 }
27
28 // Simple user definition
29 struct User {
30     name: String,
31     password: String,
32     key: EncryptionKey,
33 }
34
35 // Temporary - used to check the server works
36 #[get("/")]
37 async fn index() -> impl Responder {
38     HttpResponse::Ok().body("Hello world!")
39 }
40
41 // First request, which associates a client identifier with a key and proves the server is legitimate
42 #[post("/submit")]
43 async fn client_greeting(app_state: web::Data<AppState>, req: HttpRequest, body_bytes: Bytes) -> impl
44 Responder {
45     dprintln!("Received request to /submit");
46     // Get cookie for client identifier
47     let token = req.cookie("token");
48
49     // Check if token exists and if so, parse it
50     let token = match token {
51         Some(token) => {
52             // Check it's 16 bytes of u8s
53             let token = token.value().to_string();
54             if token.len() != 16 {
55                 // Return 404
56                 return HttpResponse::NotFound().body("No page exists");
57             }
58
59             // Convert to bytes
60             let token_bytes = token.as_bytes();

```

```

61         // Convert to array
62         let mut token = [0u8; 16];
63         for i in 0..16 {
64             token[i] = token_bytes[i];
65         }
66
67         dprintln!("Token correct!: {:?}", token);
68
69         // Implicit return
70         token
71     }
72     None => {
73         // Return 404 - resist active probing by not telling the client what went wrong
74         dprintln!("No token supplied!");
75         return HttpResponse::NotFound().body("No page exists");
76     }
77 };
78
79 // Get body
80 let body = body_bytes.to_vec();
81
82 // Iterate through users trying their keys. TODO consider Rayon here for performance
83 let mut key: Option<EncryptionKey> = None;
84 for user in &app_state.users {
85     // Attempt to decrypt body with user's key
86     let decrypted = match libsecrets::decrypt(&body, &user.key) {
87         Ok(decrypted) => decrypted, // If it succeeds, pull out content from Result<>
88         Err(_) => continue, // If it fails, try the next user
89     };
90
91     // Convert to string. It should always be valid ASCII, and UTF-8 is a superset of ASCII
92     let decrypted = match String::from_utf8(decrypted) {
93         Ok(decrypted) => decrypted,
94         Err(_) => continue,
95     };
96
97     // Check if decrypted data contains "Hello!"
98     if decrypted.contains(&"Hello!".to_string()) {
99         // Set key
100         key = Some(user.key.clone()); // We've found it!
101         dprintln!("Key found! User: {}", user.name);
102         break;
103     }
104 }
105
106 // Check if key was found. If so, extract contents of Option<>
107 let key = match key {
108     Some(key) => key,
109     None => {
110         // Return 404
111         dprintln!("No key found!");
112         return HttpResponse::NotFound().body("No page exists");
113     }
114 };
115
116 // Generate response. Encrypted with their key to show we're legitimate
117 let response_text = "CONNECTION ACCEPTED";
118
119 // Pad with 0-100 spaces to avoid fingerprinting based on response length
120 let mut rng = rand::thread_rng();
121 let amount = rng.gen_range(0..100);
122 let response_text = response_text.to_string() + &" ".repeat(amount);

```

```

123
124 // Encrypt response
125 let encrypted = match libsecrets::encrypt(response_text.as_bytes(), &key) {
126     Ok(encrypted) => encrypted,
127     Err(_) => { // This should never happen, but if it does, return 404
128         // Return 404
129         return HttpResponse::NotFound().body("No page exists");
130     }
131 };
132
133 dprintln!("Formed response: {:?}", encrypted);
134
135 // Add it to the sessions
136 let socket = TransitSocket {
137     key,
138     client_statistics: ClientStatistics {
139         to_send: 0,
140         to_reply: 0,
141     },
142 };
143
144 app_state.sessions.insert(token, socket);
145
146 dprintln!("Added session to sessions; returning response");
147
148 // Return encrypted response
149 HttpResponse::Ok().body(encrypted)
150 }
151
152 #[actix_web::main]
153 async fn main() -> std::io::Result<()> {
154     println!("Loading configuration...");
155     let configuration = config::load_config();
156
157     // Create users from configuration
158     // Check if there are any users
159     if configuration.users.len() == 0 {
160         panic!("No users defined in configuration file!");
161     }
162
163     println!("Hashing user keys...");
164     let mut users = Vec::new();
165     for user in &configuration.users {
166         let key = libsecrets::form_key(user.password.as_bytes());
167
168         users.push(User {
169             name: user.name.clone(),
170             password: user.password.clone(),
171             key,
172         });
173
174         println!("Hashed key for user '{}'", user.name);
175     }
176
177     // Create appstate that will be provided to requests
178     let appstate = web::Data::new(AppState {
179         config: configuration.clone(),
180         sessions: DashMap::new(),
181         users,
182     });
183
184     println!("Starting server at {}:{}", configuration.host, configuration.port);

```

```
185     HttpServer::new(move || { // Closure - inline function. Move keyword moves ownership of
configuration into the closure
186         App::new()
187             .app_data(appstate.clone()) // Insert appdata
188             .service(index) // Insert index route
189             .service(client_greeting) // Insert client_greeting route
190         })
191     .bind((configuration.host, configuration.port))? // Bind to host and port
192     .run() // Execute
193     .await // Wait for completion with the asynchronous runtime
194 }
```

07: Integration Tests

```

1  use std::process::Command;
2  use client_transit::{TransitSocket, TransitSocketBuilder};
3  use std::thread::sleep;
4  use std::time::Duration;
5
6  static SERVER_RELIANT_TESTS: usize = 1;
7  static mut SERVER_RELIANT_TESTS_COMPLETED: usize = 0;
8
9  static mut HAS_MONOLITH_STARTED: bool = false;
10
11 #[cfg(test)]
12 mod tests {
13     use super::*;
14
15     #[test]
16     fn check_start_monolith() {
17         let mut spawned = Command::new("cargo")
18             .args(&["run"])
19             .current_dir("../server-monolith")
20             .spawn()
21             .expect("Failed to start monolith");
22
23         // Give it a second to start. If it has to compile, tests will fail and you need to rerun
24         sleep(Duration::from_secs(1));
25
26         unsafe {
27             HAS_MONOLITH_STARTED = true;
28         }
29
30         // Wait for all the tests to finish
31         // Wait for SERVER_RELIANT_TESTS to be equal to SERVER_RELIANT_TESTS_COMPLETED
32         loop {
33             unsafe {
34                 // These are atomic operations, so it *should* be safe
35                 if SERVER_RELIANT_TESTS == SERVER_RELIANT_TESTS_COMPLETED {
36                     break;
37                 }
38             }
39             sleep(Duration::from_millis(100));
40         }
41
42         // Kill the monolith
43         spawned.kill().expect("Failed to kill monolith");
44     }
45
46     #[test]
47     fn check_socket_connect() {
48         loop {
49             unsafe {
50                 if HAS_MONOLITH_STARTED {
51                     break;
52                 }
53             }
54             sleep(Duration::from_millis(10));
55         }
56
57         let mut socket = TransitSocketBuilder::new()
58             .with_target("http://localhost:8000".to_string())
59             .with_password("12345".to_string())
60             .with_timeout_time(1)
61             .build();

```

```
62
63     socket.connect().expect("Failed to connect to server");
64
65     unsafe {
66         SERVER_RELIANT_TESTS_COMPLETED += 1;
67     }
68 }
69 }
```

08: New Serialisation

```

1 use rand::Rng; // Not unused - ignore vscode
2 use borsh::{BorshSerialize, BorshDeserialize};
3
4 pub type Port = u16;
5 pub type IPV4 = u32;
6 pub type SocketID = u32;
7
8 static LAST_SOCKET_ID: std::sync::atomic::AtomicU32 = std::sync::atomic::AtomicU32::new(0);
9
10 // TODO look at using slices for performance
11
12 #[derive(Debug, PartialEq, Clone, BorshSerialize, BorshDeserialize)]
13 pub enum Message {
14     UpStreamMessage(UpStreamMessage),
15     DownStreamMessage(DownStreamMessage),
16     CloseSocketMessage(CloseSocketMessage),
17 }
18
19 #[derive(Debug, PartialEq, Clone)]
20 #[repr(u8)]
21 pub enum MessageType {
22     UpStreamMessage = 0,
23     DownStreamMessage = 1,
24     CloseSocket = 2,
25 }
26
27 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
28 pub struct CloseSocketMessage {
29     pub socket_id: SocketID,
30     pub message_sequence_number: u32,
31 }
32
33 impl CloseSocketMessage {
34     pub fn encoded_standalone(&self) -> Result<Vec<u8>, Box<dyn std::error::Error>> {
35         let mut data = self.try_to_vec()?;
36         // Insert messagetype at the _end_ of the message (to avoid reallocating)
37         data.push(MessageType::CloseSocket as u8);
38         Ok(data)
39     }
40
41     pub fn decode_from_bytes_standalone(data: &mut Vec<u8>) -> Result<Self, std::io::Error> { //
42         Borsh returns std::io::Error
43         // Remove messagetype from the _end_ of the message
44         let message_type = data.pop().unwrap(); // You should have used it already to determine the
45         message type, so it's safe to pop it here if the developer isn't utterly stupid
46         debug_assert_eq!(message_type, MessageType::CloseSocket as u8);
47         Self::try_from_slice(data)
48     }
49 }
50
51 #[test]
52 fn test_close_socket_message() {
53     let message = CloseSocketMessage {
54         socket_id: 0,
55         message_sequence_number: 0,
56     };
57
58     let mut encoded = message.encoded_standalone().unwrap();
59
60     let decoded = CloseSocketMessage::decode_from_bytes_standalone(&mut encoded).unwrap();

```

```

60     assert_eq!(message, decoded);
61 }
62
63 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
64 pub struct UpStreamMessage {
65     pub socket_id: SocketID,
66     pub message_sequence_number: u32,
67     pub dest_ip: IPV4,
68     pub dest_port: Port,
69     pub payload: Vec<u8>,
70 }
71
72 impl UpStreamMessage {
73     pub fn encoded_standalone(&self) -> Result<Vec<u8>, Box<dyn std::error::Error>> {
74         let mut data = self.try_to_vec()?;
75         // Insert messagetype at the _end_ of the message (to avoid reallocating)
76         data.push(MessageType::UpStreamMessage as u8);
77         Ok(data)
78     }
79
80     pub fn decode_from_bytes_standalone(data: &mut Vec<u8>) -> Result<Self, std::io::Error> { //
81         // Remove messagetype from the _end_ of the message
82         let message_type = data.pop().unwrap(); // You should have used it already to determine the
83         message type, so it's safe to pop it here if the developer isn't utterly stupid
84         debug_assert_eq!(message_type, MessageType::UpStreamMessage as u8);
85         Self::try_from_slice(data)
86     }
87 }
88
89 #[test]
90 fn test_upstream_message() {
91     let large_payload = vec![0; 100000];
92
93     let message = UpStreamMessage {
94         socket_id: 0,
95         message_sequence_number: 0,
96         dest_ip: 0,
97         dest_port: 0,
98         payload: large_payload,
99     };
100
101     let mut encoded = message.encoded_standalone().unwrap();
102
103     let decoded = UpStreamMessage::decode_from_bytes_standalone(&mut encoded).unwrap();
104
105     assert_eq!(message, decoded);
106 }
107
108 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
109 pub struct DownStreamMessage {
110     pub socket_id: SocketID,
111     pub message_sequence_number: u32,
112     pub payload: Vec<u8>,
113 }
114
115 impl DownStreamMessage {
116     pub fn encoded_standalone(&self) -> Result<Vec<u8>, Box<dyn std::error::Error>> {
117         let mut data = self.try_to_vec()?;
118         // Insert messagetype at the _end_ of the message (to avoid reallocating)
119         data.push(MessageType::DownStreamMessage as u8);
120         Ok(data)

```

```

120     }
121
122     pub fn decode_from_bytes_standalone(data: &mut Vec<u8>) -> Result<Self, std::io::Error> { //
Borsh returns std::io::Error
123         // Remove messagetype from the _end_ of the message
124         let message_type = data.pop().unwrap(); // You should have used it already to determine the
message type, so it's safe to pop it here if the developer isn't utterly stupid
125         debug_assert_eq!(message_type, MessageType::DownStreamMessage as u8);
126         Self::try_from_slice(data)
127     }
128 }
129
130 #[test]
131 fn test_downstream_message() {
132     let large_payload = vec![0; 100000];
133
134     let message = DownStreamMessage {
135         socket_id: 0,
136         message_sequence_number: 0,
137         payload: large_payload,
138     };
139
140     let mut encoded = message.encoded_standalone().unwrap();
141
142     let decoded = DownStreamMessage::decode_from_bytes_standalone(&mut encoded).unwrap();
143
144     assert_eq!(message, decoded);
145 }
146
147 pub fn determine_message_type(header: u8) -> Result<MessageType, &'static str> {
148     Ok(match header {
149         0 => MessageType::UpStreamMessage,
150         1 => MessageType::DownStreamMessage,
151         2 => MessageType::CloseSocket,
152         _ => return Err("Invalid message type"),
153     })
154 }
155
156 #[test]
157 fn test_determine_message_type() {
158     assert_eq!(determine_message_type(0).unwrap(), MessageType::UpStreamMessage);
159     assert_eq!(determine_message_type(1).unwrap(), MessageType::DownStreamMessage);
160     assert_eq!(determine_message_type(2).unwrap(), MessageType::CloseSocket);
161     assert_eq!(determine_message_type(3).unwrap_err(), "Invalid message type");
162 }
163
164 pub fn generate_socket_id() -> SocketID {
165     LAST_SOCKET_ID.fetch_add(1, std::sync::atomic::Ordering::SeqCst) as SocketID
166 }
167
168 #[test]
169 fn test_generate_socket_id() {
170     let socket_id = generate_socket_id();
171     assert_eq!(socket_id, 0);
172     let socket_id = generate_socket_id();
173     assert_eq!(socket_id, 1);
174 }
175
176 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
177 pub struct MultipleMessages {
178     pub messages: Vec<Message>,
179 }

```

```

180
181 impl MultipleMessages {
182     pub fn encoded(&self) -> Result<Vec<u8>, Box<dyn std::error::Error>> {
183         let mut data = self.try_to_vec()?;
184         // Insert messagetype at the _end_ of the message (to avoid reallocating)
185         data.push(MessageType::UpStreamMessage as u8);
186         Ok(data)
187     }
188
189     pub fn decode_from_bytes(data: &Vec<u8>) -> Result<Self, std::io::Error> { // Borsh returns
std::io::Error
190         Self::try_from_slice(data)
191     }
192
193     pub fn from_messages(messages: Vec<Message>) -> Self {
194         Self {
195             messages
196         }
197     }
198 }
199
200 #[test]
201 fn test_empty_multiple_messages() {
202     let message = MultipleMessages {
203         messages: vec![],
204     };
205
206     let encoded = message.encoded().unwrap();
207
208     let decoded = MultipleMessages::decode_from_bytes(&encoded).unwrap();
209
210     assert_eq!(message, decoded);
211 }
212
213 #[test]
214 fn test_multiple_messages() {
215     let large_payload = vec![0; 100000];
216
217     let message = MultipleMessages {
218         messages: vec![
219             Message::UpStreamMessage(UpStreamMessage {
220                 socket_id: 0,
221                 message_sequence_number: 0,
222                 dest_ip: 0,
223                 dest_port: 0,
224                 payload: large_payload.clone(),
225             }),
226             Message::DownStreamMessage(DownStreamMessage {
227                 socket_id: 0,
228                 message_sequence_number: 0,
229                 payload: large_payload.clone(),
230             }),
231             Message::CloseSocketMessage(CloseSocketMessage {
232                 socket_id: 0,
233                 message_sequence_number: 0,
234             }),
235         ],
236     };
237
238     let encoded = message.encoded().unwrap();
239
240     let decoded = MultipleMessages::decode_from_bytes(&encoded).unwrap();

```

```
241  
242     assert_eq!(message, decoded);  
243 }>
```

09: Modified Benchmark

```

1 use criterion::{criterion_group, criterion_main, Criterion};
2
3 use libtransit::{Message, UpStreamMessage, DownStreamMessage, MultipleMessagesUpstream,
4 MultipleMessagesDownstream, CloseSocketMessage};
5
6 fn gen_data(size: usize) -> Vec<u8> {
7     // Generate random data
8     let mut data: Vec<u8> = Vec::with_capacity(size);
9
10    for _ in 0..size {
11        data.push(rand::random::<u8>());
12    }
13
14    data
15 }
16
17 fn gen_upstream_messages(messagecount: u32, data: Vec<u8>) -> Vec<UpStreamMessage> {
18     let mut messages: Vec<UpStreamMessage> = Vec::new();
19
20     for i in 0..messagecount {
21         let message = UpStreamMessage {
22             socket_id: i,
23             message_sequence_number: i,
24             dest_ip: 0,
25             dest_port: 0,
26             payload: data.clone(),
27         };
28
29         messages.push(message);
30     }
31
32     messages
33 }
34
35 fn gen_close_sockets(messagecount: u32) -> Vec<CloseSocketMessage> {
36     let mut messages: Vec<CloseSocketMessage> = Vec::new();
37
38     for i in 0..messagecount {
39         let message = CloseSocketMessage {
40             socket_id: i,
41             message_sequence_number: i,
42         };
43
44         messages.push(message);
45     }
46
47     messages
48 }
49
50 fn stress_test_multiple_messages_upstream(upstreams: Vec<UpStreamMessage>, close_sockets:
51 Vec<CloseSocketMessage>) {
52     let multiple_messages_up = MultipleMessagesUpstream {
53         stream_messages: upstreams,
54         close_socket_messages: close_sockets,
55     };
56
57     let mut encoded = multiple_messages_up.encoded().unwrap();
58
59     let decoded = MultipleMessagesUpstream::decode_from_bytes(&mut encoded).unwrap();
60
61     assert_eq!(multiple_messages_up, decoded);

```

```

60 }
61
62 fn criterion_benchmark(c: &mut Criterion) {
63     let stream_messages = gen_upstream_messages(50, gen_data(100));
64     let close_socket_messages = gen_close_sockets(10);
65     c.bench_function("stress_test_multiple_messages_50mc_100b (5kb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
66
67     let stream_messages = gen_upstream_messages(50, gen_data(1_000));
68     let close_socket_messages = gen_close_sockets(10);
69     c.bench_function("stress_test_multiple_messages_50mc_1kb (50kb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
70
71     let stream_messages = gen_upstream_messages(50, gen_data(10_000));
72     let close_socket_messages = gen_close_sockets(10);
73     c.bench_function("stress_test_multiple_messages_50mc_10kb (500kb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
74
75     let stream_messages = gen_upstream_messages(50, gen_data(100_000));
76     let close_socket_messages = gen_close_sockets(10);
77     c.bench_function("stress_test_multiple_messages_50mc_100kb (5mb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
78
79     let stream_messages = gen_upstream_messages(10, gen_data(1_000_000));
80     let close_socket_messages = gen_close_sockets(10);
81     c.bench_function("stress_test_multiple_messages_10mc_1mb (10mb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
82
83     let stream_messages = gen_upstream_messages(10, gen_data(10_000_000));
84     let close_socket_messages = gen_close_sockets(10);
85     c.bench_function("stress_test_multiple_messages_50mc_10mb (100mb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
86
87     let stream_messages = gen_upstream_messages(10, gen_data(100_000_000));
88     let close_socket_messages = gen_close_sockets(10);
89     c.bench_function("stress_test_multiple_messages_50mc_100mb (1gb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
90 }
91
92 criterion_group!(benches, criterion_benchmark);
93 criterion_main!(benches);

```

10: Socket Creation

```

1 use libsecrets::EncryptionKey;
2 use libtransit::{CloseSocketMessage, SocketID, IPV4, Port, UpStreamMessage, DownStreamMessage};
3 use tokio::sync::{mpsc, broadcast};
4 use dashmap::DashMap;
5
6 pub struct ClientStatistics {
7     pub to_send: usize,
8     pub to_reply: usize,
9 }
10
11 #[derive(Debug)]
12 pub enum SocksStreamInboundMessage {
13     UpStreamMessage(UpStreamMessage),
14     CloseSocketMessage(CloseSocketMessage),
15 }
16
17 pub struct TransitSocket {
18     pub key: EncryptionKey,
19     pub client_statistics: ClientStatistics,
20     // TODO information for communicating with SOCKS threads, returning
21     // requests, etc. Need to figure out how to do this.
22     socks_streams: DashMap<SocketID, mpsc::Sender<SocksStreamInboundMessage>>,
23     tcp_data_return: broadcast::Receiver<DownStreamMessage>,
24     tcp_data_assign: broadcast::Sender<DownStreamMessage>,
25 }
26
27 impl TransitSocket {
28     pub fn new(key: EncryptionKey) -> Self {
29         let (tcp_data_assign, tcp_data_return) = broadcast::channel(10000);
30
31         Self {
32             key,
33             client_statistics: ClientStatistics {
34                 to_send: 0,
35                 to_reply: 0,
36             },
37             socks_streams: DashMap::new(),
38             tcp_data_return,
39             tcp_data_assign,
40         }
41     }
42
43     pub async fn process_socks_message(&mut self, message: SocksStreamInboundMessage) {
44         match message {
45             SocksStreamInboundMessage::UpStreamMessage(message) => {
46                 let socket_id = message.socket_id;
47
48                 // Check if the socket is already in the map
49                 if let Some(sender) = self.socks_streams.get(&socket_id) {
50                     // Send the message to the task
51                     sender.send(SocksStreamInboundMessage::UpStreamMessage(message)).await.unwrap();
52                 } else {
53                     // Spawn a new task to handle this socket
54                     // First create a channel to communicate with the task
55                     // The reason this section can be slightly "implicit" is because we send dest ip
56                     and dest port in
57                     let (sender, receiver) = mpsc::channel(10000);
58
59                     let address = message.dest_ip;
60                     let port = message.dest_port;

```

```

60
61         // Send the message to the task
62         sender.send(SocksStreamInboundMessage::UpStreamMessage(message)).await.unwrap();
63
64         // Insert the sender into the map
65         self.socks_streams.insert(socket_id, sender);
66
67         // Construct the arguments for the task
68         let arguments = TCPHandlerArguments {
69             receiver,
70             sender: self.tcp_data_assign.clone(),
71             address,
72             port,
73         };
74
75         // Spawn the task
76         tokio::spawn(tcp_handler_task(arguments));
77     }
78 },
79 SocksStreamInboundMessage::CloseSocketMessage(message) => {
80     let socket_id = message.socket_id;
81
82     // Re-form
83     let message = SocksStreamInboundMessage::CloseSocketMessage(message);
84
85     // Check if the socket is already in the map
86     if let Some(sender) = self.socks_streams.get(&socket_id) {
87         // Send the message to the task
88         sender.send(message).await.unwrap();
89     } else {
90         panic!("Unordered socket deletion: This should not happen because data should
91         flow before close messages. Check process_sock_message feed order.")
92     }
93 },
94 }
95 }
96
97 struct TCPHandlerArguments {
98     receiver: mpsc::Receiver<SocksStreamInboundMessage>,
99     sender: broadcast::Sender<DownStreamMessage>,
100     address: IPV4,
101     port: Port,
102 }

```

11: TCP Handler

```

1  async fn tcp_handler_task(mut arguments: TCPHandlerArguments) {
2      // Connect to the server
3      let mut stream = match TcpStream::connect(format!("{}", arguments.address,
arguments.port)).await {
4          Ok(stream) => stream,
5          Err(_) => {
6              // TODO handle this better - have an error and a way to send it back to the client
7              panic!("Could not connect to server from TCP handler task to address {}:{}",
arguments.address, arguments.port);
8          }
9      };
10
11     // Begin loop of transferring TCP data back and forth
12     let mut sequence_number = 0;
13     let mut queue: Vec<SocksStreamInboundMessage> = Vec::with_capacity(16);
14
15     loop {
16         // Get data from the receiver
17         let mut message = match arguments.receiver.recv().await {
18             Some(message) => message,
19             None => {
20                 // TODO handle this better - have an error and a way to send it back to the client
21                 panic!("TCP handler task receiver closed unexpectedly");
22             }
23         };
24
25         // Continual cycle considering both message type and sequence number handling
26         loop {
27             match message {
28                 SocksStreamInboundMessage::UpStreamMessage(interior) => {
29                     if interior.message_sequence_number != sequence_number {
30                         // Add to the queue
31                         queue.push(SocksStreamInboundMessage::UpStreamMessage(interior));
32                         // Sort queue by sequence number
33                         queue.sort_by(|a, b| {
34                             let a_seq = match a {
35                                 SocksStreamInboundMessage::UpStreamMessage(message) =>
message.message_sequence_number,
36                                 SocksStreamInboundMessage::CloseSocketMessage(message) =>
message.message_sequence_number,
37                             };
38                             let b_seq = match b {
39                                 SocksStreamInboundMessage::UpStreamMessage(message) =>
message.message_sequence_number,
40                                 SocksStreamInboundMessage::CloseSocketMessage(message) =>
message.message_sequence_number,
41                             };
42                             a_seq.cmp(&b_seq)
43                         });
44                         continue;
45                     } else {
46                         // Increment the sequence number
47                         sequence_number += 1;
48
49                         // Send the message
50                         stream.write_all(&interior.payload).await.unwrap();
51
52                         // Check if the queue has the next message
53                         match &mut queue[0] {
54

```

```

55         SocksStreamInboundMessage::UpStreamMessage(interior) => {
56             if interior.message_sequence_number == sequence_number {
57                 // Remove the message from the queue
58                 queue.remove(0);
59                 // Set the message to the next message
60                 message = SocksStreamInboundMessage::UpStreamMessage(*interior);
61                 continue;
62             } else {
63                 // Leave
64                 break;
65             }
66         },
67         SocksStreamInboundMessage::CloseSocketMessage(interior) => {
68             if interior.message_sequence_number == sequence_number {
69                 // Remove the message from the queue
70                 queue.remove(0);
71                 // Set the message to the next message
72                 message =
SocksStreamInboundMessage::CloseSocketMessage(*interior);
73                 continue;
74             } else {
75                 // Leave
76                 break;
77             }
78         }
79     }
80 }
81 },
82 SocksStreamInboundMessage::CloseSocketMessage(interior) => {
83     if interior.message_sequence_number != sequence_number {
84         // Add to the queue
85         queue.push(SocksStreamInboundMessage::CloseSocketMessage(interior));
86         continue;
87     } else {
88         // Close connection
89         stream.shutdown().await.unwrap();
90
91         // TODO have a way to remove from the dashmap
92
93         // Leave
94         return;
95     }
96 }
97 }
98 }
99 }
100 }

```

12: TCP Handler Rewrite

```

1  async fn tcp_handler_task(mut arguments: TCPHandlerArguments) {
2      // Connect to the server
3      let mut stream = match TcpStream::connect(format!("{}", arguments.address,
arguments.port)).await {
4          Ok(stream) => stream,
5          Err(_) => {
6              // TODO handle this better - have an error and a way to send it back to the client
7              panic!("Could not connect to server from TCP handler task to address {}:{}",
arguments.address, arguments.port);
8          }
9      };
10
11     // Begin loop of transferring TCP data back and forth
12     let mut sequence_number = 0;
13     let mut last_sequence_number_sanity = 0;
14     let mut return_sequence_number = 0;
15
16     let mut queue: Vec<SocksStreamInboundMessage> = Vec::with_capacity(16);
17
18     loop {
19         // Sort the queue by sequence number
20         queue.sort_by(|a, b| {
21             let seq_num_a = get_seq_num(a);
22             let seq_num_b = get_seq_num(b);
23
24             seq_num_a.cmp(&seq_num_b)
25         });
26
27         // Get data from the receiver OR the queue
28         // Check if the queue's latest message is the next one in sequence
29         let is_queue_next = if queue.len() > 0 {
30             match &queue[0] {
31                 SocksStreamInboundMessage::UpStreamMessage(interior) => {
32                     interior.message_sequence_number == sequence_number
33                 },
34                 SocksStreamInboundMessage::CloseSocketMessage(interior) => {
35                     interior.message_sequence_number == sequence_number
36                 }
37             }
38         } else {
39             false
40         };
41
42         // If the queue isn't next, get data from the receiver
43         let message = if is_queue_next {
44             // Pop the message off the queue
45             queue.remove(0)
46         } else {
47             // Get data from the receiver
48             let data = arguments.receiver.recv().await.unwrap();
49
50             // Check if the message is the next one in sequence
51             let sequence_number = match &data {
52                 SocksStreamInboundMessage::UpStreamMessage(interior) => {
53                     interior.message_sequence_number
54                 },
55                 SocksStreamInboundMessage::CloseSocketMessage(interior) => {
56                     interior.message_sequence_number
57                 }
58             };

```

```

59
60     if sequence_number == sequence_number {
61         // If it is, return it
62         data
63     } else {
64         // If it isn't, put it in the queue
65         queue.push(data);
66
67         // And loop - hopefully we'll get it later!
68         continue
69     }
70 };
71
72 // Now we have the latest message!
73 // Sanity check
74 debug_assert!(get_seq_num(&message) == last_sequence_number_sanity + 1);
75
76 // Now actually process it
77 match message {
78     SocksStreamInboundMessage::UpStreamMessage(message) => {
79         // Send the data to the server
80         stream.write_all(&message.payload).await.unwrap();
81     },
82     SocksStreamInboundMessage::CloseSocketMessage(_) => {
83         // Close the socket
84         stream.shutdown().await.unwrap();
85
86         // TODO remove the socket from the map
87
88         // Break out of the loop
89         break
90     }
91 }
92
93 // Read buffer
94 let mut buffer = vec![0; 2048]; // TODO variable buffer size
95 let bytes_read = match stream.read(&mut buffer).await {
96     Ok(bytes_read) => bytes_read,
97     Err(_) => {
98         // TODO handle this better - have an error and a way to send it back to the client
99         panic!("Could not read from server from TCP handler task to address {}:{}",
100 arguments.address, arguments.port);
101     }
102 };
103 // Sometimes less is sent than requested, so truncate the buffer. In the future we'll have a
104 way to tune the max buffer size.
105 buffer.truncate(bytes_read);
106
107 // Create a downstream message
108 let message = DownStreamMessage {
109     socket_id: arguments.socket_id,
110     message_sequence_number: return_sequence_number,
111     payload: buffer,
112     has_remote_closed: false,
113 };
114
115 // Send the message to the client
116 arguments.sender.send(message).unwrap();
117
118 // Increment the return sequence number
119 return_sequence_number += 1;
120

```

```
119         // Increment the sequence number
120         last_sequence_number_sanity = sequence_number;
121         sequence_number += 1;
122     }
123 }
```

13: New Tests

```
1  #[test]
2  fn test_server_downstream_meta() {
3      let mut streams = HashMap::new();
4
5      for i in 1..100 {
6          let stream_info = ServerStreamInfo {
7              has_terminated: i % 2 == 0,
8              errors: vec![format!("error {}", i)],
9              logs: vec![format!("log {}", i), format!("another log {}", i)],
10         };
11
12         streams.insert(i, stream_info);
13     }
14
15     let server_meta = ServerMetaDownstream {
16         bytes_to_reply_to_client: 50,
17         bytes_to_send_to_remote: 32423,
18         messages_to_reply_to_client: 4283,
19         messages_to_send_to_remote: 237482,
20         cpu_usage: 4.2,
21         memory_usage_kb: 2091,
22         num_open_sockets: 42,
23         streams,
24     };
25
26     let mut buffer = server_meta.encoded().unwrap();
27
28     let out = ServerMetaDownstream::decode_from_bytes(&mut buffer).unwrap();
29
30     assert_eq!(server_meta, out);
31 }
32
33 #[test]
34 fn test_client_upstream_meta() {
35     let client_meta = ClientMetaUpstream {
36         bytes_to_send_to_remote: 32423,
37         bytes_to_reply_to_client: 50,
38         messages_to_send_to_remote: 237482,
39         messages_to_reply_to_client: 4283,
40     };
41
42     let mut buffer = client_meta.encoded().unwrap();
43
44     let out = ClientMetaUpstream::decode_from_bytes(&mut buffer).unwrap();
45
46     assert_eq!(client_meta, out);
47 }
```

14: Client Init

```

1 use client_transit;
2 use libsocks;
3 use libtransit::UpStreamMessage;
4 use tokio::net::{TcpListener, TcpStream};
5 use tokio::sync::mpsc::{self, Receiver, Sender};
6 use std::sync::atomic::{AtomicU32, Ordering};
7 use tokio::task;
8
9 struct ClientArguments {
10     listen_address: String,
11     listen_port: u16,
12     target_host: String,
13     password: String,
14 }
15
16 // Core of the client program for now. See Obsidian
17 async fn begin_core_client(arguments: ClientArguments) {
18     let (upstreamPasserSend, upstreamPasserReceive): (
19         Sender<UpStreamMessage>,
20         Receiver<UpStreamMessage>,
21     ) = mpsc::channel(100);
22
23     let (messagePasserPasserSend, messagePasserPasserReceive): (
24         Sender<client_transit::DownstreamBackpasser>,
25         Receiver<client_transit::DownstreamBackpasser>,
26     ) = mpsc::channel(100_00);
27
28     // Cannot transfer threads
29     let mut transit_socket = client_transit::TransitSocketBuilder::new()
30         .with_target(arguments.target_host)
31         .with_password(arguments.password)
32         .with_client_name("Client-Core".to_string())
33         .build();
34
35     let status = transit_socket.connect().await;
36
37     match status {
38         Ok(_) => {
39             println!("Connected to server!");
40         }
41         Err(error) => {
42             println!("Failed to connect to server: {:?}", error);
43             return;
44         }
45     }
46
47     // Now start the TCP listener in a task
48     task::spawn(async move {
49         let listener = TcpListener::bind((arguments.listen_address,
50 arguments.listen_port)).await.expect("Failed to start TCP listener");
51
52         loop {
53             let (socket, _) = listener.accept().await.expect("Failed to accept connection");
54         }
55     });
56
57     transit_socket
58         .handle_transit(upstreamPasserReceive, messagePasserPasserReceive)
59         .await;
60 }

```

```

61  async fn tcp_listener(stream: TcpStream, upstreamPasserSend: Sender<UpStreamMessage>,
62  messagePasserPasserSend: Sender<client_transit::DownstreamBackpasser>) {
63      // Read the first packet
64      // Wait for the socket to be readable
65      let mut buf = Vec::with_capacity(4096);
66
67      stream.readable().await.expect("Failed to wait for socket to be readable");
68
69      match stream.try_read_buf(&mut buf) {
70          Ok(bytes_read) => {
71              if bytes_read == 0 {
72                  // The socket was closed
73                  return;
74              }
75          },
76          Err(error) => {
77              println!("Failed to read from socket: {:?}", error);
78              return;
79          }
80      }
81
82      // Try to parse the packet
83      let packet = libsocks::decode_socks_request(&buf);
84
85      let mut dstip: Option<libsocks::IPV4> = None;
86      let mut dstport: Option<libsocks::Port> = None;
87
88      let rejection = libsocks::Socks4ConnectReply {
89          version: 4,
90          status: libsocks::Socks4Status::Rejected,
91          dstport: 0,
92          dstip: 0,
93      };
94
95      let rejection_bytes = rejection.to_binary();
96
97      match packet {
98          Ok(packet) => {
99              // Check what kind of packet it is
100              match packet {
101                  libsocks::Socks4Request::Connect(connect_req) => {
102                      // Sanity check
103                      if connect_req.version != 4 {
104                          println!("Incorrect socks version ({} ) from program {}", connect_req.version,
105                          connect_req.userid);
106                          stream.writable().await;
107                          stream.try_write(&rejection_bytes);
108                          return;
109                      }
110
111                      dstip = Some(connect_req.dstip);
112                      dstport = Some(connect_req.dstport);
113                  },
114                  libsocks::Socks4Request::Bind(bind_req) => {
115                      // We don't support bind yet
116                      let user_id = bind_req.userid;
117                      println!("Received bind request (unsupported) from user ID {}. Killing
118                      connection", user_id);
119                      stream.writable().await;
120                      stream.try_write(&rejection_bytes);
121                      return;
122                  }
123              }
124          }
125      }

```

```

120     }
121 },
122 Err(error) => {
123     println!("Failed to parse SOCKS request: {:?}", error);
124     stream.writable().await;
125     stream.try_write(&rejection_bytes);
126     return;
127 }
128 }
129
130 // If we've gotten to this point the data's good. Let's compose a reply
131 let reply = libsocks::Socks4ConnectReply {
132     version: 4,
133     status: libsocks::Socks4Status::Granted,
134     dstport: dstport.unwrap(),
135     dstip: dstip.unwrap()
136 };
137
138 stream.writable().await.unwrap();
139
140 match stream.try_write(&reply.to_binary()) {
141     Ok(_) => {
142         println!("Sent reply to client");
143     },
144     Err(error) => {
145         println!("Failed to send reply to client: {:?}", error);
146         return;
147     }
148 }
149
150 // It seems initialisation was a success. Let's get our socket ID
151 let socket_id = allocate_socket_id();
152
153 // Now we need to let transit know how to reply to this socket. First we create a message passer
154 let (downstreamPasserSend, downstreamPasserReceive): (Sender<libtransit::DownStreamMessage>,
Receiver<libtransit::DownStreamMessage>) = mpsc::channel(100);
155
156 // Now we send the message passer to transit
157 let message = client_transit::DownstreamBackpasser {
158     socket_id,
159     sender: downstreamPasserSend
160 };
161
162 messagePasserPasserSend.send(message).await.expect("Failed to send message passer to transit");
163
164 // Now go into the constant loop of message passing
165 // TODO
166 }
167
168 static NEXT_SOCKET_ID: AtomicU32 = AtomicU32::new(0);
169
170 fn allocate_socket_id() -> libtransit::SocketID {
171     NEXT_SOCKET_ID.fetch_add(1, Ordering::SeqCst)
172 }

```

15: Client Transit

```

1 use libsecrets::{self, EncryptionKey};
2 use libtransit;
3 use rand::Rng;
4 use request::Client;
5 use std::{sync::{RwLock, Condvar}, env};
6 use libtransit::{
7     ClientMessageUpstream,
8     ServerMessageDownstream,
9     ClientMetaUpstream,
10    ServerMetaDownstream,
11    UpStreamMessage,
12    DownStreamMessage,
13    SocketID,
14    DeclarationToken
15 };
16 use tokio::sync::mpsc::{self, Sender, Receiver};
17
18 mod builder;
19 pub use self::builder::TransitSocketBuilder;
20 use request::header::{HeaderMap, HeaderValue};
21
22 use std::collections::HashMap;
23
24 pub struct TransitSocket {
25     target: String, // Base URL, incl. protocol
26     key: EncryptionKey, // Encryption key for libsecrets
27     control_client: Client, // Does the initial encryption agreement
28     server_meta: ServerMetaDownstream, // Statistics about the server to inform client congestion
29     control
30     client_identifier: DeclarationToken, // Identifier for this client, used as a cookie
31     push_client_count: usize, // Number of hybrid clients (Both push and pull)
32     pull_client_count: usize, // Number of pull clients
33     timeout_time: usize, // Time in seconds before a request is considered timed out
34     headers: HeaderMap, // Headers to send with requests. Includes client identifier
35     is_initialized: bool, // Whether the socket has been initialized by greeting the server
36     client_name: String, // Name of the client
37     tcp_return_passers: HashMap<SocketID, Sender<DownStreamMessage>>, // Passers for TCP return
38     traffic
39 }
40
41 #[derive(Debug)]
42 pub enum TransitInitError {
43     EncryptionError(libsecrets::EncryptionError),
44     RequestError(request::Error),
45     ConnectionFailed,
46     ConnectionDenied,
47     LiedResponse,
48 }
49
50 impl From<libsecrets::EncryptionError> for TransitInitError {
51     fn from(error: libsecrets::EncryptionError) -> Self {
52         Self::EncryptionError(error)
53     }
54 }
55
56 pub struct DownstreamBackpasser {
57     pub socket_id: libtransit::SocketID,
58     pub sender: Sender<DownStreamMessage>,
59 }

```

```

59 impl TransitSocket {
60     pub async fn connect(&mut self) -> Result<(), TransitInitError> {
61         self.greet_server().await?;
62         self.is_initialized = true;
63         Ok(())
64     }
65
66     // TODO this is unusual traffic, not very believable.
67     // Change this to "Get a PNG image" incl. header etc, with the
68     // encrypted data being some form of additional token in the cookies
69     async fn greet_server(&self) -> Result<(), TransitInitError> {
70         let mut data = format!("Hello. Protocol version: {}, client-transit version: {}, client-name:
71         {}, "0", env!("CARGO_PKG_VERSION"), self.client_name);
72
73         // Find minimum padding amount such that it's at least 512 bytes
74         let length = data.len();
75         let min_padding = 512 - length;
76
77         // Find maximum padding amount such that it's at most 1024 bytes
78         let max_padding = 1024 - length;
79
80         // Add random padding up to 1024 bytes
81         let mut rng = rand::thread_rng();
82         let amount = rng.gen_range(min_padding..max_padding);
83         data = data + &" ".repeat(amount);
84
85         let encrypted = libsecrets::encrypt(data.as_bytes(), &self.key)?;
86
87         let response = self.control_client.post(&format!("{}/submit", self.target))
88             .body(encrypted)
89             .headers(self.headers.clone())
90             .send()
91             .await;
92
93         match response {
94             Ok(response) => {
95                 if response.status() != 200 {
96                     return Err(TransitInitError::ConnectionDenied);
97                 }
98                 // Try to decrypt the response
99                 let encrypted = response.bytes().await;
100                 match encrypted {
101                     Ok(encrypted) => {
102                         let decrypted = libsecrets::decrypt(&encrypted, &self.key)?;
103                         // Convert to string
104                         let decrypted = String::from_utf8(decrypted);
105                         match decrypted {
106                             Ok(decrypted) => {
107                                 // Remove spaces
108                                 let decrypted = decrypted.trim_end_matches(' ');
109                                 if decrypted != "CONNECTION ACCEPTED" {
110                                     return Err(TransitInitError::LiedResponse);
111                                 }
112                             },
113                             Err(_) => {
114                                 return Err(TransitInitError::LiedResponse);
115                             }
116                         }
117                     },
118                     Err(error) => {
119                         return Err(TransitInitError::RequestError(error));
120                     }
121                 }
122             }
123         }
124     }
125 }

```

```
120         }
121     },
122     Err(error) => {
123         if error.is_timeout() || error.is_connect() {
124             return Err(TransitInitError::ConnectionFailed);
125         } else {
126             return Err(TransitInitError::RequestError(error));
127         }
128     }
129 }
130 }
131
132 // If we've gotten here, the connection was successful
133 // We should be ready to talk to the server now
134
135 Ok(())
136 }
137
138 async fn push_handler(to_send_passer: Receiver<Vec<UpStreamMessage>>) {
139 }
140
141 async fn pull_handler() {
142 }
143
144 }
145
146 pub async fn handle_transit(&mut self, upstreamPasserRcv: Receiver<UpStreamMessage>,
messagePasserPasserReceive: Receiver<DownstreamBackpasser>) {
147     // Spawn requisite push and pull handlers
148     // Create a channel for the push handler to send to so that a random not-in-use push handler
149     // can grab the data to send.
150 }
151 }
```

16: Core Client

```

1  use client_transit;
2  use libsocks;
3  use libtransit::{UpStreamMessage, CloseSocketMessage};
4  use tokio::net::{TcpListener, TcpStream};
5  use tokio::sync::mpsc::{self, Receiver, Sender};
6  use std::sync::atomic::{AtomicU32, Ordering};
7  use tokio::task;
8  use flume::{Sender as FlumeSender, Receiver as FlumeReceiver, self};
9  use tokio::sync::broadcast;
10
11 struct ClientArguments {
12     listen_address: String,
13     listen_port: u16,
14     target_host: String,
15     password: String,
16 }
17
18 // Core of the client program for now. See Obsidian
19 async fn begin_core_client(arguments: ClientArguments) {
20     let (upstreamPasserSend, upstreamPasserReceive): (
21         Sender<UpStreamMessage>,
22         Receiver<UpStreamMessage>,
23     ) = mpsc::channel(10_000);
24
25     let (closePasserSend, closePasserReceive) = mpsc::channel<CloseSocketMessage>(10_000);
26
27     let (messagePasserPasserSend, mut messagePasserPasserReceive) =
broadcast::channel<client_transit::DownstreamBackpasser>(100_000);
28     let messagePasserPasserSend = Arc::new(messagePasserPasserSend);
29
30     // Cannot transfer threads
31     let mut transit_socket = client_transit::TransitSocketBuilder::new()
32         .with_target(arguments.target_host)
33         .with_password(arguments.password)
34         .with_client_name("Client-Core".to_string())
35         .build();
36
37     let status = transit_socket.connect().await;
38
39     match status {
40         Ok(_) => {
41             println!("Connected to server!");
42         }
43         Err(error) => {
44             println!("Failed to connect to server: {:?}", error);
45             return;
46         }
47     }
48
49     // Now start the TCP listener in a task
50     task::spawn(async move {
51         let listener = TcpListener::bind((arguments.listen_address,
arguments.listen_port)).await.expect("Failed to start TCP listener");
52
53         loop {
54             let (socket, _) = listener.accept().await.expect("Failed to accept connection");
55
56             // Spawn a task to handle the connection
57             task::spawn(tcp_listener(socket, upstreamPasserSend.clone(), closePasserSend.clone(),
messagePasserPasserSend.clone()));

```

```

58     }
59 });
60
61 transit_socket
62     .handle_transit(upstreamPasserReceive, messagePasserPasserReceive)
63     .await;
64 }
65
66 async fn tcp_listener(stream: TcpStream, upstreamPasserSend: Sender<UpStreamMessage>,
67 closePasserSend: Sender<CloseSocketMessage>, messagePasserPasserSend:
68 Arc<Sender<client_transit::DownstreamBackpasser>>) {
69     // Read the first packet
70     // Wait for the socket to be readable
71     let mut buf = Vec::with_capacity(4096);
72
73     stream.readable().await.expect("Failed to wait for socket to be readable");
74
75     match stream.try_read_buf(&mut buf) {
76         Ok(bytes_read) => {
77             if bytes_read == 0 {
78                 // The socket was closed
79                 return;
80             }
81         },
82         Err(error) => {
83             println!("Failed to read from socket: {:?}", error);
84             return;
85         }
86     }
87
88     // Try to parse the packet
89     let packet = libsocks::decode_socks_request(&buf);
90
91     let mut dstip: Option<libsocks::IPV4> = None;
92     let mut dstport: Option<libsocks::Port> = None;
93
94     let rejection = libsocks::Socks4ConnectReply {
95         version: 4,
96         status: libsocks::Socks4Status::Rejected,
97         dstport: 0,
98         dstip: 0,
99     };
100
101     let rejection_bytes = rejection.to_binary();
102
103     match packet {
104         Ok(packet) => {
105             // Check what kind of packet it is
106             match packet {
107                 libsocks::Socks4Request::Connect(connect_req) => {
108                     // Sanity check
109                     if connect_req.version != 4 {
110                         println!("Incorrect socks version ({} ) from program {}", connect_req.version,
111 connect_req.userid);
112
113                         stream.writable().await;
114                         stream.try_write(&rejection_bytes);
115                         return;
116                     }
117
118                     dstip = Some(connect_req.dstip);
119                     dstport = Some(connect_req.dstport);
120                 },

```

```

117         libsocks::Socks4Request::Bind(bind_req) => {
118             // We don't support bind yet
119             let user_id = bind_req.userid;
120             println!("Received bind request (unsupported) from user ID {}. Killing
connection", user_id);
121             stream.writable().await;
122             stream.try_write(&rejection_bytes);
123             return;
124         }
125     },
126 },
127 Err(error) => {
128     println!("Failed to parse SOCKS request: {:?}", error);
129     stream.writable().await;
130     stream.try_write(&rejection_bytes);
131     return;
132 }
133 }
134
135 // If we've gotten to this point the data's good. Let's compose a reply
136 let reply = libsocks::Socks4ConnectReply {
137     version: 4,
138     status: libsocks::Socks4Status::Granted,
139     dstport: dstport.unwrap(),
140     dstip: dstip.unwrap()
141 };
142
143 stream.writable().await.unwrap();
144
145 match stream.try_write(&reply.to_binary()) {
146     Ok(_) => {
147         println!("Sent reply to client");
148     },
149     Err(error) => {
150         println!("Failed to send reply to client: {:?}", error);
151         return;
152     }
153 }
154
155 // It seems initialisation was a success. Let's get our socket ID
156 let socket_id = allocate_socket_id();
157
158 // Now we need to let transit know how to reply to this socket. First we create a message passer
159 let (downstreamPasserSend, downstreamPasserReceive): (Sender<libtransit::DownStreamMessage>,
Receiver<libtransit::DownStreamMessage>) = mpsc::channel(100);
160
161 // Now we send the message passer to transit
162 let message = client_transit::DownstreamBackpasser {
163     socket_id,
164     sender: downstreamPasserSend
165 };
166
167 messagePasserPasserSend.send(message).await.expect("Failed to send message passer to transit");
168
169 let mut send_seq_num = 0;
170
171 // Start both tasks
172 task::spawn(send_socks_to_transit(&mut stream, upstreamPasserSend, closePasserSend, socket_id,
dstip.unwrap(), dstport.unwrap()));
173 task::spawn(send_transit_to_socks(&mut stream, downstreamPasserReceive));
174 }
175

```

```

fn send_socks_to_transit(stream: &mut TcpStream, upstream_passer: Sender<UpStreamMessage>,
176 close_passer: Sender<CloseSocketMessage>, socket_id: libtransit::SocketID, ip: libsocks::IPV4, port:
libsocks::Port) {
177     let mut buf = Vec::with_capacity(4096);
178
179     let mut seq_num = 0;
180     loop {
181         let bytes_read = stream.try_read_buf(&mut buf).expect("Failed to read from socket");
182
183         // Check if the socket was closed
184         if bytes_read == 0 {
185             // The socket was closed
186             close_passer.send(CloseSocketMessage {
187                 socket_id,
188                 message_sequence_number: seq_num
189             }).expect("Failed to send close message to transit");
190             return;
191         }
192
193         // Send the data to transit
194         upstream_passer.send(UpStreamMessage {
195             socket_id,
196             message_sequence_number: seq_num,
197             dest_ip: ip,
198             dest_port: port,
199             payload: buf.clone()
200         }).expect("Failed to send data to transit");
201
202         // Blank the buffer
203         buf.clear();
204
205         seq_num += 1;
206     }
207 }
208
209 fn send_transit_to_socks(stream: &mut TcpStream, downstream_passer_rcv:
Receiver<libtransit::DownStreamMessage>) {
210     let seq_num = 0;
211     loop {
212         let message = downstream_passer_rcv.recv().expect("Failed to receive message from transit");
213
214         debug_assert!(message.message_sequence_number == seq_num, "Received message with incorrect
sequence number (expected {}, got {})", seq_num, message.message_sequence_number);
215
216         seq_num += 1;
217
218         // Send the data to the socket
219         stream.try_write(&message.payload).expect("Failed to write to socket");
220     }
221 }
222
223 static NEXT_SOCKET_ID: AtomicU32 = AtomicU32::new(0);
224
225 fn allocate_socket_id() -> libtransit::SocketID {
226     NEXT_SOCKET_ID.fetch_add(1, Ordering::SeqCst)
227 }

```

17: Filled Transit

```

1 use flume::{self, Receiver as FlumeReceiver, Sender as FlumeSender};
2 use libsecrets::{self, EncryptionKey};
3 use libtransit::{self, CloseSocketMessage, MultipleMessagesUpstream};
4 use libtransit::{
5     ClientMessageUpstream, ClientMetaUpstream, DeclarationToken, DownStreamMessage,
6     ServerMessageDownstream, ServerMetaDownstream, SocketID, UpStreamMessage,
7 };
8 use rand::Rng;
9 use request::Client;
10 use tokio::sync::mpsc::{self, Receiver, Sender};
11
12 mod builder;
13 pub use self::builder::TransitSocketBuilder;
14 use request::header::{HeaderMap, HeaderValue};
15 use std::sync::Arc;
16 use std::time::{Duration, Instant};
17 use tokio::sync::broadcast::{self, Receiver as BroadcastReceiver, Sender as BroadcastSender};
18 use tokio::sync::RwLock;
19 use tokio::task;
20
21 use std::collections::HashMap;
22
23 pub struct TransitSocket {
24     target: String, // Base URL, incl. protocol
25     key: EncryptionKey, // Encryption key for libsecrets
26     server_meta: ServerMetaDownstream, // Statistics about the server to inform client congestion
27     control
28     client_identifier: DeclarationToken, // Identifier for this client, used as a cookie
29     push_client_count: usize, // Number of hybrid clients (Both push and pull)
30     pull_client_count: usize, // Number of pull clients
31     timeout_time: usize, // Time in seconds before a request is considered timed out
32     headers: HeaderMap, // Headers to send with requests. Includes client identifier
33     is_initialized: bool, // Whether the socket has been initialized by greeting the server
34     client_name: String, // Name of the client
35 }
36
37 #[derive(Debug)]
38 pub enum TransitInitError {
39     EncryptionError(libsecrets::EncryptionError),
40     RequestError(request::Error),
41     ConnectionFailed,
42     ConnectionDenied,
43     LiedResponse,
44 }
45
46 impl From<libsecrets::EncryptionError> for TransitInitError {
47     fn from(error: libsecrets::EncryptionError) -> Self {
48         Self::EncryptionError(error)
49     }
50 }
51
52 pub struct DownstreamBackpasser {
53     pub socket_id: libtransit::SocketID,
54     pub sender: Sender<DownStreamMessage>,
55 }
56
57 pub async fn connect(transit_socket: Arc<RwLock<TransitSocket>>) -> Result<(), TransitInitError> {
58     greet_server(transit_socket.clone()).await?;
59     transit_socket.write().await.is_initialized = true;
60     Ok(())

```

```

60 }
61
62 // TODO this is unusual traffic, not very believable.
63 // Change this to "Get a PNG image" incl. header etc, with the
64 // encrypted data being some form of additional token in the cookies
65 async fn greet_server(transit_socket: Arc<RwLock<TransitSocket>>) -> Result<(), TransitInitError> {
66     let mut data = format!(
67         "Hello. Protocol version: {}, client-transit version: {}, client-name: {}",
68         "0",
69         env!("CARGO_PKG_VERSION"),
70         transit_socket.read().await.client_name
71     );
72
73     // Find minimum padding amount such that it's at least 512 bytes
74     let length = data.len();
75     let min_padding = 512 - length;
76
77     // Find maximum padding amount such that it's at most 1024 bytes
78     let max_padding = 1024 - length;
79
80     // Add random padding up to 1024 bytes
81     let mut rng = rand::thread_rng();
82     let amount = rng.gen_range(min_padding..max_padding);
83     data = data + &" ".repeat(amount);
84
85     let encrypted = libsecrets::encrypt(data.as_bytes(), &transit_socket.read().await.key)?;
86
87     let (target, headers, key, timeout_time) = {
88         let transit_socket = transit_socket.read().await;
89         (transit_socket.target.clone(), transit_socket.headers.clone(), transit_socket.key.clone(),
90 transit_socket.timeout_time)
91     };
92
93     let response = Client::new()
94         .post(&format!("{}",/submit", target))
95         .body(encrypted)
96         .headers(headers)
97         .timeout(Duration::from_secs(timeout_time as u64))
98         .send()
99         .await;
100
101     match response {
102         Ok(response) => {
103             if response.status() != 200 {
104                 return Err(TransitInitError::ConnectionDenied);
105             }
106             // Try to decrypt the response
107             let encrypted = response.bytes().await;
108             match encrypted {
109                 Ok(encrypted) => {
110                     let decrypted = libsecrets::decrypt(&encrypted, &key)?;
111                     // Convert to string
112                     let decrypted = String::from_utf8(decrypted);
113                     match decrypted {
114                         Ok(decrypted) => {
115                             // Remove spaces
116                             let decrypted = decrypted.trim_end_matches(' ');
117                             if decrypted != "CONNECTION ACCEPTED" {
118                                 return Err(TransitInitError::LiedResponse);
119                             }
120                         }
121                     }
122                 }
123                 Err(_) => {

```

```

121         return Err(TransitInitError::LiedResponse);
122     }
123 }
124 }
125 Err(error) => {
126     return Err(TransitInitError::RequestError(error));
127 }
128 }
129 }
130 Err(error) => {
131     if error.is_timeout() || error.is_connect() {
132         return Err(TransitInitError::ConnectionFailed);
133     } else {
134         return Err(TransitInitError::RequestError(error));
135     }
136 }
137 }
138
139 // If we've gotten here, the connection was successful
140 // We should be ready to talk to the server now
141
142 Ok(())
143 }
144
145 async fn push_handler(transit_socket: Arc<RwLock<TransitSocket>>, to_send_passer:
FlumeReceiver<MultipleMessagesUpstream>) {
146     // Create client
147     let client = Client::new();
148
149     let (key, headers, target, timeout_time) = {
150         let transit_socket = transit_socket.read().await;
151         (
152             transit_socket.key.clone(),
153             transit_socket.headers.clone(),
154             transit_socket.target.clone(),
155             transit_socket.timeout_time,
156         )
157     };
158
159     let timeout_duration = Duration::from_secs(timeout_time as u64);
160
161     loop {
162         // Get the data to send
163         let to_send = to_send_passer.recv_async().await;
164
165         let to_send = match to_send {
166             Ok(to_send) => to_send,
167             Err(_) => {
168                 // TODO handle this
169                 continue;
170             }
171         };
172
173         // Send the data
174         // Encode and encrypt in a thread
175         let to_send = tokio::task::spawn_blocking(move || {
176             let upstream_message = ClientMessageUpstream {
177                 messages: to_send,
178                 metadata: get_metadata(),
179             };
180
181             let data_bin = upstream_message.encoded().unwrap();

```

```

182
183         let encrypted = libsecrets::encrypt(&data_bin, &key).unwrap();
184
185         encrypted
186     });
187
188     // Send the data
189     let response = client
190         .post(&format!("{}",/submit", target))
191         .body(to_send.await.unwrap())
192         .headers(headers.clone())
193         .timeout(timeout_duration)
194         .send()
195         .await;
196
197     // TODO if things go wrong, prevent the arrival of new data until the send works. See working
198     nicely section of todo about reconstruction and error handling.
199 }
200
201 fn get_metadata() -> ClientMetaUpstream {
202     ClientMetaUpstream {
203         bytes_to_send_to_remote: 0,
204         bytes_to_reply_to_client: 0,
205         messages_to_send_to_remote: 0,
206         messages_to_reply_to_client: 0
207     }
208 }
209
210 async fn pull_handler(
211     transit_socket: Arc<RwLock<TransitSocket>>,
212     messagePasserPasser: BroadcastReceiver<DownstreamBackpasser>,
213 ) {
214     // This will hold the return lookup and be populated by the messagePasserPassers
215     let mut return_lookup: HashMap<SocketID, Sender<DownStreamMessage>> = HashMap::new();
216
217     // Create client
218     let client = Client::new();
219
220     let (key, headers, target, timeout_time) = {
221         let transit_socket = transit_socket.read().await;
222         (
223             transit_socket.key.clone(),
224             transit_socket.headers.clone(),
225             transit_socket.target.clone(),
226             transit_socket.timeout_time,
227         )
228     };
229
230     let timeout_duration = Duration::from_secs(timeout_time as u64);
231
232     loop {
233         // Could be made spawn_blocking if this turns out to take too long
234         let to_send = {
235             let upstream_message = ClientMessageUpstream {
236                 messages: MultipleMessagesUpstream { stream_messages: Vec::with_capacity(0),
237                 close_socket_messages: Vec::with_capacity(0) },
238                 metadata: get_metadata(),
239             };
240
241             let data_bin = upstream_message.encoded().unwrap();

```

```

242         let encrypted = libsecrets::encrypt(&data_bin, &key).unwrap();
243
244     encrypted
245 };
246
247 // Send the data
248 let response = client
249     .post(&format!("{}", target))
250     .body(to_send)
251     .headers(headers.clone())
252     .timeout(timeout_duration)
253     .send()
254     .await;
255
256 // Read the response
257 let response = response.unwrap();
258
259 // Get the data
260 let encrypted = response.bytes().await.unwrap();
261
262 // Decrypt the data
263 let mut decrypted = libsecrets::decrypt(&encrypted, &key).unwrap();
264
265 // Decode the data
266 let decoded = ServerMessageDownstream::decode_from_bytes(&mut decrypted).unwrap();
267
268 // TODO use the metadata to determine whether to kill the connection, as well as congestion
269 control et cetera
270 // for now just send the data to the appropriate socket
271
272 for downstream_message in decoded.messages {
273     if downstream_message.has_remote_closed {
274         // TODO handle this by removing from the lookup
275     }
276
277     let socket_id = downstream_message.socket_id;
278
279     // Get the sender
280     let sender = return_lookup.get(&socket_id);
281
282     match sender {
283         Some(sender) => {
284             // Send the message
285             sender.send(downstream_message).await.unwrap();
286         }
287         None => {
288             // Populate the sender via iteration through the messagePasserPasser
289             for messagePasser in messagePasserPasser.try_iter() {
290                 // Blindly populate
291                 return_lookup.insert(messagePasser.socket_id, messagePasser.sender);
292             }
293
294             // Try again
295             let sender = return_lookup.get(&socket_id).unwrap();
296
297             // Send the message
298             sender.send(downstream_message).await.unwrap();
299         }
300     }
301 }
302

```

```

303 pub async fn handle_transit(
304     transit_socket: Arc<RwLock<TransitSocket>>,
305     mut upstreamPasserRcv: Receiver<UpStreamMessage>,
306     mut closePasserRcv: Receiver<CloseSocketMessage>,
307     messagePasserPasserSend: Arc<BroadcastSender<DownstreamBackpasser>>,
308 ) {
309     // Spawn requisite push and pull handlers
310     // Create a channel for the push handler to send to so that a random not-in-use push handler
311     // can grab the data to send.
312
313     // Create the push handlers as flume message passers of MultipleMessagesUpstream
314     let (push_passer_send, push_passer_receive) = flume::unbounded::<MultipleMessagesUpstream>();
315
316     let (push_client_count, pull_client_count) = {
317         let transit_socket = transit_socket.read().await;
318         (transit_socket.push_client_count, transit_socket.pull_client_count)
319     };
320
321     for _ in 0..push_client_count {
322         task::spawn(push_handler(transit_socket.clone(), push_passer_receive.clone()));
323     }
324
325     // Create the pull handlers
326     for _ in 0..pull_client_count {
327         let new_receiver = messagePasserPasserSend.subscribe();
328         task::spawn(pull_handler(transit_socket.clone(), new_receiver));
329     }
330
331     // TODO make this variable
332     let BUFF_SIZE = 1024 * 256;
333     let MODETIME = 40; // In milliseconds
334
335     let mut buffer_stream: Vec<UpStreamMessage> = Vec::new(); // TODO make these (and ones in the
loop) appropriately presized
336     let mut buffer_close: Vec<CloseSocketMessage> = Vec::new();
337
338     let mut last_upstream_time = Instant::now();
339     let mut current_buffer_size = 0;
340
341     loop {
342         // Get data to send up
343         let data = upstreamPasserRcv.recv().await;
344         let data = match data {
345             Some(data) => data,
346             None => {
347                 // TODO handle this
348                 continue;
349             }
350         };
351         // Increment buffer size
352         current_buffer_size += data.payload.len();
353         // Add it to the buffer
354         buffer_stream.push(data);
355
356         // Now do it to the close socket messages
357         let close = closePasserRcv.recv().await;
358         let close = match close {
359             Some(close) => close,
360             None => {
361                 // TODO handle this
362                 continue;
363             }
364         };

```

```

364     };
365     // Increment buffer size
366     current_buffer_size += 2048; // Approxish I guess
367     // Add it to the buffer
368     buffer_close.push(close);
369
370     // If we're above buff size
371     if current_buffer_size > BUFF_SIZE {
372         // Send the buffer to the push handler
373         let multiple_messages = MultipleMessagesUpstream {
374             stream_messages: buffer_stream,
375             close_socket_messages: buffer_close,
376         };
377         push_passer_send.send_async(multiple_messages).await;
378         // Reset buffer
379         buffer_stream = Vec::new();
380         buffer_close = Vec::new();
381         // Reset buffer size
382         current_buffer_size = 0;
383     }
384
385     // If we're above mode time and there's more than one piece of data in the buffer
386     if last_upstream_time.elapsed().as_millis() > MODETIME && buffer_close.len() > 1 &&
buffer_stream.len() > 1 {
387         // Send the buffer to the push handler
388         let multiple_messages = MultipleMessagesUpstream {
389             stream_messages: buffer_stream,
390             close_socket_messages: buffer_close,
391         };
392         push_passer_send.send_async(multiple_messages).await;
393         // Reset buffer
394         buffer_close = Vec::new();
395         buffer_stream = Vec::new();
396         // Reset buffer size
397         current_buffer_size = 0;
398         // Reset last upstream time
399         last_upstream_time = Instant::now();
400     }
401
402     // If we're above mode time and this is the first piece of data, reset the last upstream time
403     if last_upstream_time.elapsed().as_millis() > MODETIME && buffer_close.len() == 1 &&
buffer_stream.len() == 1 {
404         last_upstream_time = Instant::now();
405     }
406 }
407 }

```

18: New Transit

```

1  use dashmap::DashMap;
2  use libtransit::{
3      ClientMetaUpstream, DownStreamMessage, ServerMetaDownstream, SocketID, UpStreamMessage,
4  };
5  use std::collections::HashMap;
6  use std::time::{Duration, Instant};
7  use tokio::sync::mpsc;
8  use flume;
9
10 use tokio::io::{Interest, AsyncWriteExt};
11 use tokio::net::TcpStream;
12
13 use libsecrets::EncryptionKey;
14
15 pub struct SessionActorsStorage {
16     pub to_bundler_stream: mpsc::UnboundedSender<UpStreamMessage>,
17     pub from_bundler_stream: flume::Receiver<Vec<DownStreamMessage>>,
18     pub key: EncryptionKey,
19 }
20
21 pub async fn create_actor(key: &EncryptionKey) -> SessionActorsStorage {
22     // Create the channel
23     let (to_bundler_stream, from_http_stream) = mpsc::unbounded_channel();
24     let (to_http_stream, from_bundler_stream) = flume::unbounded();
25
26     // Create the storage
27     let storage = SessionActorsStorage {
28         to_bundler_stream,
29         from_bundler_stream,
30         key: key.clone(),
31     };
32
33     // Start the handler
34     tokio::spawn(handle_session(from_http_stream, to_http_stream));
35
36     // Return the storage
37     storage
38 }
39
40 pub async fn handle_session(
41     mut from_http_stream: mpsc::UnboundedReceiver<UpStreamMessage>,
42     to_http_stream: flume::Sender<Vec<DownStreamMessage>>,
43 ) {
44     let mut stream_to_tcp_handlers: HashMap<SocketID, mpsc::UnboundedSender<UpStreamMessage>> =
45         HashMap::new();
46     let mut stream_from_tcp_handlers: HashMap<
47         SocketID,
48         mpsc::UnboundedReceiver<DownStreamMessage>,
49     > = HashMap::new();
50
51     let mut managed_sockets: Vec<SocketID> = Vec::new();
52
53     let mut last_return_time: Instant = Instant::now();
54     let mut buffer_size = 0;
55     let mut buffer: Vec<DownStreamMessage> = Vec::with_capacity(1024);
56
57     loop {
58         let mut no_received_http = true;
59         let mut no_received_tcp = false;
60
61         // TCP to HTTP messages

```

```

62     for managed_socket in &managed_sockets {
63         // Get the TCP handler
64         stream_from_tcp_handlers
65             .entry(*managed_socket)
66             .and_modify(|tcp_handler| {
67                 // Try to receive from the TCP handler
68                 match tcp_handler.try_recv() {
69                     Ok(message) => {
70                         // Add to the buffer
71                         buffer_size += message.payload.len();
72                         buffer.push(message);
73                         no_received_tcp = false;
74                     }
75                     Err(e) => {
76                         // Check if the channel is empty
77                         if e == mpsc::error::TryRecvError::Empty {
78                             return;
79                         } else {
80                             // Panic
81                             panic!("Error receiving from TCP handler: {:?}", e);
82                         }
83                     }
84                 }
85             });
86     }
87
88     // Check if we should return the buffer
89     if buffer_size > 128 * 1024 {
90         // Send the buffer
91         to_http_stream.send(buffer.clone()).unwrap();
92
93         // Reset the buffer
94         buffer.clear();
95         buffer_size = 0;
96
97         // Update the last return time
98         last_return_time = Instant::now();
99     }
100
101     // Check if it exceeds modetime, but there's nothing in the buffer
102     if last_return_time.elapsed() > Duration::from_millis(50) && buffer_size == 0 {
103         // Update the last return time
104         last_return_time = Instant::now();
105     }
106
107     // Check if last_return_time exceeds modetime
108     if last_return_time.elapsed() > Duration::from_millis(50) {
109         // Send the buffer
110         to_http_stream.send(buffer.clone()).unwrap();
111
112         // Reset the buffer
113         buffer.clear();
114         buffer_size = 0;
115
116         // Update the last return time
117         last_return_time = Instant::now();
118     }
119
120     // HTTP to TCP messages
121     match from_http_stream.try_recv() {
122         Ok(message) => {
123             // Get the socket ID

```

```

124         let socket_id = message.socket_id;
125
126         // Get the TCP handler
127         let tcp_handler = stream_to_tcp_handlers.get(&socket_id);
128
129         let tcp_handler = if tcp_handler.is_none() {
130             // Create a new TCP handler
131             // For sending upstreams to the handle_tcp
132             let (to_tcp_handler, from_http_stream) = mpsc::unbounded_channel();
133             // For receiving downstreams from the handle_tcp
134             let (to_http_stream, from_tcp_handler) = mpsc::unbounded_channel();
135
136             // Add the TCP handler to the map
137             stream_to_tcp_handlers.insert(socket_id, to_tcp_handler);
138             stream_from_tcp_handlers.insert(socket_id, from_tcp_handler);
139
140             // Add to the managed sockets
141             managed_sockets.push(socket_id);
142
143             // Spawn the TCP handler
144             tokio::spawn(handle_tcp(from_http_stream, to_http_stream, message.dest_ip,
message.dest_port, message.socket_id));
145
146             // Get the TCP handler
147             let tcp_handler = stream_to_tcp_handlers.get(&socket_id).unwrap();
148             tcp_handler
149         } else {
150             tcp_handler.unwrap()
151         };
152
153         // Send the message to the TCP handler
154         tcp_handler.send(message).unwrap();
155     }
156     Err(e) => {
157         // Check if the channel is empty
158         if e == mpsc::error::TryRecvError::Empty {
159             no_received_http = true;
160         } else {
161             // Panic
162             panic!("Error receiving from HTTP stream: {:?}", e);
163         }
164     }
165 }
166
167 if no_received_http && no_received_tcp {
168     // Sleep for 1ms
169     tokio::time::sleep(Duration::from_millis(1)).await;
170 }
171 }
172 }
173
174 pub async fn handle_tcp(
175     mut from_http_stream: mpsc::UnboundedReceiver<UpStreamMessage>,
176     to_http_stream: mpsc::UnboundedSender<DownStreamMessage>,
177     ip: u32,
178     port: u16,
179     socket_id: SocketID,
180 ) {
181     let stream = TcpStream::connect(format!("{:?}", ip, port)).await;
182     let mut stream = match stream {
183         Ok(stream) => stream,
184         Err(e) => {

```

```

185         // TODO handle this better
186         panic!("Error connecting to TCP server: {:?}", e);
187     }
188 };
189
190 let mut return_sequence_number = 0;
191 let mut last_sent_seq_number_sanity: i64 = -1;
192
193 loop {
194     let ready = stream.ready(Interest::READABLE | Interest::WRITABLE).await;
195
196     if ready.is_err() {
197         // TODO handle this better
198         panic!("Error getting ready state: {:?}", ready);
199     }
200
201     let ready = ready.unwrap();
202
203     if ready.is_readable() {
204         let mut downstream_msg = DownStreamMessage {
205             socket_id,
206             message_sequence_number: return_sequence_number,
207             payload: Vec::with_capacity(0),
208             has_remote_closed: false,
209         };
210
211         // Write in directly for efficiency
212         let bytes_read = match stream.try_read_buf(&mut downstream_msg.payload) {
213             Ok(bytes_read) => bytes_read,
214             Err(e) => {
215                 // TODO handle this better
216                 panic!("Error reading from TCP stream: {:?}", e);
217             }
218         };
219
220         if bytes_read == 0 {
221             // Remote has closed
222             // TODO handle this properly
223         }
224
225         // Send the message
226         to_http_stream.send(downstream_msg).unwrap();
227
228         // Increment the return sequence number
229         return_sequence_number += 1;
230     }
231
232     if ready.is_writable() {
233         let message = match from_http_stream.try_recv() {
234             Ok(message) => message,
235             Err(e) => {
236                 // Check if the channel is empty
237                 if e == mpsc::error::TryRecvError::Empty {
238                     continue;
239                 } else {
240                     // Panic
241                     panic!("Error receiving from HTTP stream: {:?}", e);
242                 }
243             }
244         };
245
246         let seq_num = message.message_sequence_number;

```

```
247
248     debug_assert_eq!(last_sent_seq_number_sanity + 1, seq_num as i64);
249
250     last_sent_seq_number_sanity = seq_num as i64;
251
252     // Write payload
253     stream.write_all(&message.payload).await.unwrap();
254 }
255 }
256 }
```

19: New TCP Handling

```

1  async fn handle_tcp_up(
2      mut from_http_stream: mpsc::UnboundedReceiver<UpStreamMessage>,
3      to_http_stream: mpsc::UnboundedSender<DownStreamMessage>, // For sending socket close messages
4      mut tcp_write_half: tokio::net::tcp::OwnedWriteHalf,
5      traffic_stats: Arc<meta::ServerMetaDownstreamTrafficStatsSynced>,
6      socket_id: SocketID,
7  ) {
8      let mut return_sequence_number = 0;
9      let mut last_sent_seq_number_sanity: i64 = -1;
10
11     loop {
12         let ready = tcp_write_half.ready(Interest::WRITABLE).await;
13
14         if ready.is_err() {
15             // TODO handle this better
16             panic!("Error getting ready state: {:?}", ready);
17         }
18
19         let ready = ready.unwrap();
20
21         if ready.is_writable() {
22             // We're using an asynchronous wait here to avoid the old hot-loop system
23             let mut message = match from_http_stream.recv().await {
24                 Some(message) => message,
25                 None => {
26                     // TODO handle this better
27                     panic!("Error receiving from HTTP stream: {:?}", from_http_stream);
28                 }
29             };
30
31             message.time_at_server_socket_ms = meta::ms_since_epoch();
32
33             let seq_num = message.message_sequence_number;
34
35             debug_assert_eq!(last_sent_seq_number_sanity + 1, seq_num as i64);
36
37             last_sent_seq_number_sanity = seq_num as i64;
38
39             let payload_size = message.payload.len() as u32;
40             traffic_stats.coordinator_up_to_socket_bytes.fetch_sub(payload_size, Ordering::Relaxed);
41
42             // Write payload
43             tcp_write_half.write_all(&message.payload).await.unwrap();
44
45             message.time_client_write_finished_ms = meta::ms_since_epoch();
46
47             dprintln!("Written to the stream");
48
49             println!("Message timing info: {}", message.render_latency_information());
50         }
51     }
52 }
53
54 async fn handle_tcp_down(
55     mut to_http_stream: mpsc::UnboundedSender<DownStreamMessage>,
56     mut tcp_read_half: tokio::net::tcp::OwnedReadHalf,
57     socket_id: SocketID,
58     traffic_stats: Arc<meta::ServerMetaDownstreamTrafficStatsSynced>,
59 ) {
60     let mut return_sequence_number = 0;
61

```

```

62     loop {
63         let ready = tcp_read_half.ready(Interest::READABLE).await;
64
65         if ready.is_err() {
66             // TODO handle this better
67             panic!("Error getting ready state: {:?}", ready);
68         }
69
70         let ready = ready.unwrap();
71
72         if ready.is_readable() {
73             let mut downstream_msg = DownStreamMessage {
74                 socket_id,
75                 message_sequence_number: return_sequence_number,
76                 payload: Vec::with_capacity(0),
77                 has_remote_closed: false,
78                 time_at_server_start_recv_ms: meta::ms_since_epoch(),
79                 time_at_server_finish_recv_ms: 0,
80                 time_at_server_coordinator_ms: 0,
81                 time_at_server_egress_ms: 0,
82                 time_at_client_ingress_ms: 0,
83                 time_at_client_sendon_ms: 0,
84                 time_at_client_socket_ms: 0,
85                 time_at_client_socket_write_finished_ms: 0,
86             };
87
88             // Write in directly for efficiency
89             // We're using an asynchronous wait here to avoid the old hot-loop system
90             let bytes_read = match tcp_read_half.read(&mut downstream_msg.payload).await {
91                 Ok(bytes_read) => bytes_read,
92                 Err(e) => {
93                     // TODO handle this better
94                     // Check if it's because it's blocking
95                     if e.kind() == std::io::ErrorKind::WouldBlock {
96                         // Wait a moment
97                         tokio::time::sleep(Duration::from_millis(1)).await;
98
99                         continue;
100                     } else {
101                         panic!("Error reading from TCP stream: {:?}", e);
102                     }
103                 }
104             };
105
106             if bytes_read == 0 {
107                 // Remote has closed
108                 // TODO handle this properly
109             }
110
111             // Send the message
112             dprintln!("Sending message sequence number {} to HTTP stream (socket ID {})",
113                 downstream_msg.message_sequence_number, downstream_msg.socket_id);
114             let payload_size = downstream_msg.payload.len() as u32;
115
116             // Set finish recv time
117             downstream_msg.time_at_server_finish_recv_ms = meta::ms_since_epoch();
118
119             to_http_stream.send(downstream_msg).unwrap();
120
121             traffic_stats.socket_down_to_coordinator_bytes.fetch_add(payload_size,
122                 Ordering::Relaxed);

```

```
122         // Increment the return sequence number
123         return_sequence_number += 1;
124     }
125 }
126 }
```

20: Client Side Split TCP

```

1  let (mut read_half, mut write_half) = stream.into_split();
2
3  // Spawn both tasks
4  task::spawn(tcp_handler_up(read_half, socket_id, dstip, dstport, upstream_passer_send.clone()));
5  task::spawn(tcp_handler_down(write_half, downstream_passer_receive));
6  }
7
8  async fn tcp_handler_down(write_half: tokio::net::tcp::OwnedWriteHalf,
9  mut downstream_passer_receive: UnboundedReceiver<libtransit::DownStreamMessage>,
10 ) {
11     let mut sanity_seq_num = 0;
12
13     loop {
14         let ready = write_half.ready(Interest::WRITABLE).await.expect("Failed to wait for socket to
be ready");
15
16         if ready.is_writable() {
17             match downstream_passer_receive.recv().await {
18                 Some(mut data) => {
19                     // Transit has sent us data
20                     // Send it to the client
21                     let bytes = &data.payload;
22                     let length = bytes.len();
23
24                     data.time_at_client_socket_ms = meta::ms_since_epoch();
25
26                     assert_eq!(sanity_seq_num, data.message_sequence_number, "Sequence number
mismatch");
27
28                     // Decrement counter
29                     CLIENT_META_UPSTREAM.response_to_socks_bytes.fetch_sub(length as u32,
Ordering::SeqCst);
30
31                     match write_half.try_write(bytes) {
32                         Ok(_) => {
33                             // All is fine
34                             dprintln!("Sent {} bytes to client", length);
35                         },
36                         Err(error) => {
37                             // TODO handle properly
38                             eprintln!("Failed to send data to client: {:?}", error);
39                             continue
40                         }
41                     }
42
43                     data.time_at_client_socket_write_finished_ms = meta::ms_since_epoch();
44                     println!("Downstream timing data: {}", data.display_stats());
45
46                     sanity_seq_num += 1;
47                 },
48                 None => {
49                     // Transit has disconnected
50                     // TODO handle properly
51                     eprintln!("Transit has disconnected; socket cannot be sustained");
52                     return
53                 }
54             }
55         }
56     }
57 }

```

```

58
59 async fn tcp_handler_up(mut read_half: tokio::net::tcp::OwnedReadHalf,
60     socket_id: SocketID,
61     dest_ip: libsocks::IPV4,
62     dest_port: libsocks::Port,
63     upstream_passer_send: Sender<UpStreamMessage>,
64 ) {
65     let mut msg_seq_num = 0;
66
67     loop {
68         let ready = read_half.ready(Interest::READABLE).await.expect("Failed to wait for socket to be
ready");
69
70         if ready.is_readable() {
71             let mut upstream_msg = UpStreamMessage {
72                 socket_id,
73                 message_sequence_number: msg_seq_num,
74                 dest_ip,
75                 dest_port,
76                 payload: vec![0; 512], // TODO changeable
77                 time_ingress_client_ms: 0,
78                 time_at_coordinator_ms: 0,
79                 time_at_client_egress_ms: 0,
80                 time_at_server_ingress_ms: 0,
81                 time_at_server_coordinator_ms: 0,
82                 time_at_server_socket_ms: 0,
83                 time_client_write_finished_ms: 0,
84             };
85
86             let bytes_read = match read_half.read(&mut upstream_msg.payload).await {
87                 Ok(bytes_read) => bytes_read,
88                 Err(error) => {
89                     // TODO handle properly
90                     panic!("Failed to read from socket: {:?}", error);
91                 }
92             };
93
94             if bytes_read == 0 {
95                 // The socket was closed
96                 // TODO handle properly
97                 return
98             }
99
100             // Set time
101             upstream_msg.time_ingress_client_ms = meta::ms_since_epoch();
102
103             // Trim array to actual size
104             upstream_msg.payload.truncate(bytes_read);
105
106             // Send the data to transit
107             CLIENT_META_UPSTREAM.socks_to_coordinator_bytes.fetch_add(bytes_read as u32,
Ordering::Relaxed);
108             upstream_passer_send.send(upstream_msg).await.expect("Failed to send data to transit");
109
110             msg_seq_num += 1;
111         }
112     }
113 }

```

21: HypertunnelGUI

```

1 use egui_extras;
2 use std::io::{BufRead, BufReader, Read, Write};
3 use std::process::{Stdio, Command};
4 use std::sync::RwLock;
5 use std::thread::sleep;
6 use std::io;
7 use std::sync::mpsc;
8
9 /// We derive Deserialize/Serialize so we can persist app state on shutdown.
10 #[derive(serde::Deserialize, serde::Serialize)]
11 #[serde(default)] // if we add new fields, give them default values when deserializing old state
12 pub struct HypertunnelApp {
13     // Example stuff:
14     listen_host: String,
15     listen_port: u16,
16
17     target_host: String,
18     target_password: String,
19
20     #[serde(skip)]
21     engaged: bool,
22
23     #[serde(skip)]
24     child: Option<std::process::Child>,
25
26     #[serde(skip)]
27     log_lines: Vec<String>,
28
29     #[serde(skip)]
30     abort_text: Option<String>,
31 }
32
33 impl Default for HypertunnelApp {
34     fn default() -> Self {
35         Self {
36             // Example stuff:
37             listen_host: "127.0.0.1".to_string(),
38             listen_port: 1080,
39             target_host: "https://example.com".to_string(),
40             target_password: "hunter2".to_string(),
41             engaged: false,
42             child: None,
43             log_lines: Vec::new(),
44             abort_text: None,
45         }
46     }
47 }
48
49 impl HypertunnelApp {
50     /// Called once before the first frame.
51     pub fn new(cc: &eframe::CreationContext<'_>) -> Self {
52         // This is also where you can customize the look and feel of egui using
53         // `cc.egui_ctx.set_visuals` and `cc.egui_ctx.set_fonts`.
54
55         // Load previous app state (if any).
56         // Note that you must enable the `persistence` feature for this to work.
57         if let Some(storage) = cc.storage {
58             return eframe::get_value(storage, eframe::APP_KEY).unwrap_or_default();
59         }
60
61         egui_extras::install_image_loaders(&cc.egui_ctx);

```

```

62
63     Default::default()
64 }
65 }
66
67
68 impl eframe::App for HypertunnelApp {
69     /// Called by the frame work to save state before shutdown.
70     fn save(&mut self, storage: &mut dyn eframe::Storage) {
71         eframe::set_value(storage, eframe::APP_KEY, self);
72     }
73
74     /// Called each time the UI needs repainting, which may be many times per second.
75     fn update(&mut self, ctx: &egui::Context, _frame: &mut eframe::Frame) {
76         // Put your widgets into a `SidePanel`, `TopBottomPanel`, `CentralPanel`, `Window` or `Area`.
77         // For inspiration and more examples, go to https://emilk.github.io/egui
78
79         egui::TopBottomPanel::top("top_panel").show(ctx, |ui| {
80             // The top panel is often a good place for a menu bar:
81
82             egui::menu::bar(ui, |ui| {
83                 // NOTE: no File->Quit on web pages!
84                 if ui.button("Quit").clicked() {
85                     ctx.send_viewport_cmd(egui::ViewportCommand::Close);
86                 }
87
88                 egui::widgets::global_dark_light_mode_buttons(ui);
89             });
90         });
91
92         egui::CentralPanel::default().show(ctx, |ui| {
93             // The central panel the region left after adding TopPanel's and SidePanel's
94             ui.heading("Hypertunnel");
95
96             if self.engaged {
97                 let mut child = self.child.as_mut().unwrap();
98
99                 let do_abort = match child.try_wait() {
100                     Ok(Some(status)) => {
101                         println!("Child exited with: {}", status);
102                         true
103                     }
104                     Ok(None) => false,
105                     Err(e) => {
106                         println!("Error: {}", e);
107                         true
108                     }
109                 };
110
111                 let mut child_stdout = child.stdout.as_mut();
112                 let mut child_stderr = child.stderr.as_mut();
113
114                 if ui.button("Disengage").clicked() || do_abort {
115                     // Disengage the tunnel!
116                     self.engaged = false;
117
118                     // Check if we can set an abort text
119                     if do_abort && (child_stderr.is_some() || child_stdout.is_some()) {
120                         let mut out_lines = Vec::with_capacity(16);
121
122                         for log in self.log_lines.iter().rev().take(10) {
123                             out_lines.push(log.clone());

```

```

124         }
125
126         // Reverse out_lines
127         out_lines.reverse();
128
129         if child_stdout.is_some() {
130             // Read from stdout
131             let mut reader_stdout = BufReader::new(child_stdout.unwrap());
132             for line in reader_stdout.lines() {
133                 out_lines.push(line.unwrap());
134             }
135         }
136
137         if child_stderr.is_some() {
138             // Read from stderr
139             let mut reader_stderr = BufReader::new(child_stderr.unwrap());
140             for line in reader_stderr.lines() {
141                 out_lines.push(line.unwrap());
142             }
143         }
144
145         let out_text = out_lines.join("\n");
146         self.abort_text = Some(out_text.clone());
147
148         println!("Abort text: {}", out_text);
149     }
150
151     // Kill child
152     child.kill().unwrap();
153
154     // Clear log lines
155     self.log_lines.clear();
156
157     self.child = None;
158 } else {
159     ui.label("Program executing nominally.");
160 }
161 } else {
162     ui.horizontal(|ui| {
163         ui.label("Listen Host: ");
164
165         ui.add(egui::TextEdit::singleline(&mut self.listen_host).hint_text("0.0.0.0"));
166     });
167
168     ui.horizontal(|ui| {
169         ui.label("Listen Port: ");
170         ui.add(egui::widgets::DragValue::new(&mut self.listen_port).speed(1.0));
171     });
172
173     ui.separator();
174
175     ui.horizontal(|ui| {
176         ui.label("Target Host: ");
177         ui.text_edit_singleline(&mut self.target_host);
178     });
179
180     ui.horizontal(|ui| {
181         ui.label("Target Password:");
182         ui.add(egui::widgets::TextEdit::singleline(
183             &mut self.target_password,
184         ));
185     });

```

```

186
187         ui.separator();
188
189         if ui.button("Engage").clicked() {
190             // Engage the tunnel!
191             let child = start_tunnel(
192                 &self.listen_host,
193                 self.listen_port,
194                 &self.target_host,
195                 &self.target_password,
196             );
197
198             let child = match child {
199                 Ok(child) => child,
200                 Err(e) => {
201                     ui.label(format!("Error: {}", e));
202                     return;
203                 }
204             };
205
206             self.engaged = true;
207
208             self.child = Some(child);
209
210             self.abort_text = None;
211         }
212
213         if self.abort_text.is_some() {
214             ui.label(format!("Error: {}", self.abort_text.as_ref().unwrap()));
215         }
216     }
217
218     ui.with_layout(egui::Layout::bottom_up(egui::Align::LEFT), |ui| {
219         egui::warn_if_debug_build(ui);
220     });
221 });
222 }
223 }
224
225 fn start_tunnel(
226     listen_host: &str,
227     listen_port: u16,
228     target_host: &str,
229     target_password: &str,
230 ) -> Result<std::process::Child, String> {
231     let child = Command::new("hypertunnel")
232         .arg("--listen-host")
233         .arg(listen_host)
234         .arg("--listen-port")
235         .arg(listen_port.to_string())
236         .arg("--target-host")
237         .arg(target_host)
238         .arg("--password")
239         .arg(target_password)
240         .stdout(Stdio::piped())
241         .stderr(Stdio::piped())
242         .spawn();
243
244     match child {
245         Ok(child) => {
246             return Ok(child);
247         }
248     }

```

```
248     Err(e) => {
249         return Err(e.to_string());
250     }
251 }
252 }
253
254 fn powered_by_egui_and_eframe(ui: &mut egui::Ui) {
255     ui.horizontal(|ui| {
256         ui.spacing_mut().item_spacing.x = 0.0;
257         ui.label("Powered by ");
258         ui.hyperlink_to("egui", "https://github.com/emilk/egui");
259         ui.label(" and ");
260         ui.hyperlink_to(
261             "eframe",
262             "https://github.com/emilk/egui/tree/master/crates/eframe",
263         );
264         ui.label(".");
265     });
266 }
```

EVALUATION

Evaluative Testing

5.1.1. Usability Testing and Evidence

I created a short video, accessible at compscivideos.alexcj.co.uk/cli_help.mp4, which demonstrates the CLI help command. This is a key part of usability: It helps the user learn how to use the program. It's also relevant to robustness - whenever my input validation detects an invalid argument it shows the help command rather than attempting to continue with the incorrect data.

Another video, compscivideos.alexcj.co.uk/cli_password.mp4, demonstrates how the server rejects incorrect passwords (as supplied in the CLI) but accepts correct ones. This demonstrates input validation for the purpose of security and robustness, and the client clearly conveys the error that has occurred to make it usable for the user. It can't explicitly say that the password is incorrect in order to defend against active probing attacks, but it does as much as it can.

The next video, compscivideos.alexcj.co.uk/functioning_different_ips.mp4, demonstrates the core functionality of the system. I first connect to <https://ifconfig.me>, which returns 130.185.148.162 - the public IP address of my school. This is because ifconfig.me always responds with the public IP address of the connection to it. I then activate Hypertunnel and connect to the same website via the proxy it exposes. It returns 86.31.175.68 - the IP address of my home network. My school uses the Sophos XG cite firewall, which uses strict, TLS-decrypting deep packet inspection to kill almost all circumvention methods. This demonstrates that Hypertunnel works - it uses HTTP-based bidirectional tunneling without Server Sent Events, Websockets, or request streaming to circumvent strict TLS-decrypting firewalls - exactly the title of this document. The functionality testing has succeeded.

I then demonstrated the ability to conduct meaningful communications in compscivideos.alexcj.co.uk/using_discord.mp4. I sent a series of messages and a 10MB file over Discord, a popular chat application. This demonstrates Hypertunnel's value and functionality as a communication tool.

I also looked at its traffic using Wireshark, a network protocol analyser, in compscivideos.alexcj.co.uk/wireshark.mp4. It demonstrated that the traffic is fully encrypted and secure. I put some of the intercepted data into GCHQ Cyberchef [129] to show that its entropy it has a high entropy, indicating its fully-encrypted nature. This demonstrates that it is secure.

The project was also 100% written in a memory safe language, Rust, which is a key part of robustness and one of the requirements of the project. I have not observed any memory integrity issues whatsoever during development, nor evaluation.

The project also has a minimal GUI, shown below, which I have tested for usability using feedback from beta testers. I was told that it "worked well" and they liked the "pragmatism". However, they also said that they "didn't like the colour scheme". Upon noticing the colour scheme picker ("oh, that isn't the exit button") they said that "grey looks better than the white", demonstrating that the default colour scheme was suboptimal but that the theme picker was useful and the core functionality was well-received. They also mentioned that "the port looks

like a button” as part of a general sentiment that the different types of UI elements were not sufficiently differentiated.

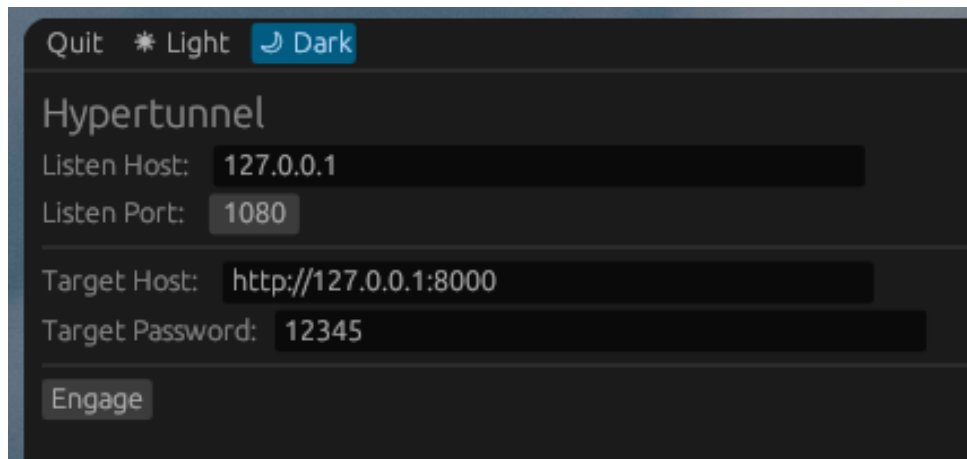


Figure 54: Dark mode GUI

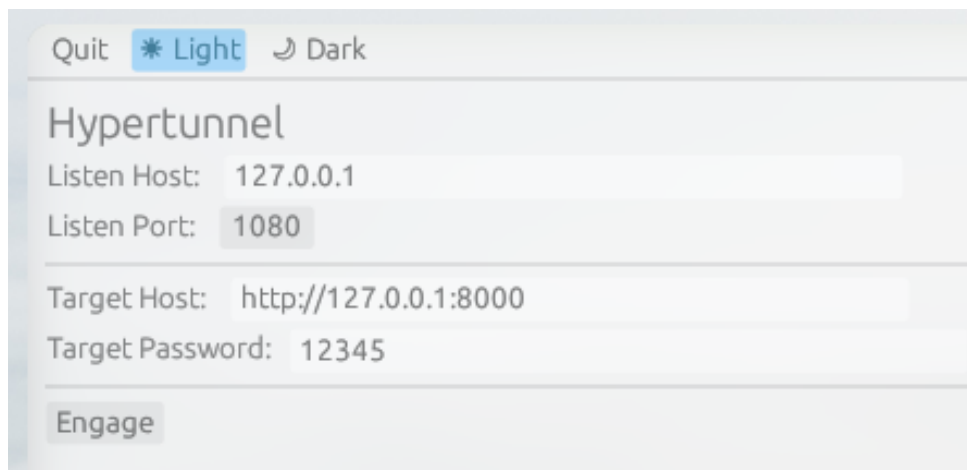


Figure 55: Light mode GUI

Testing Evidence with respect to Success Criteria

5.2.1. CLI Operation

Hypertunnel has been successfully tested and demonstrated to operate through a CLI client frontend, as demonstrated in the evaluative testing. The CLI client is easy to use and has a clear and concise interface, with all of the options exposed by the core client able to be configured. Hypertunnel fully meets the success criteria for CLI operation.

5.2.2. Real-World Circumvention

Hypertunnel has demonstrated successful real-world circumvention behind Sophos XG, and I see no reason - in theory - why the current codebase would be unable to circumvent the Iranian National Firewall, Great Firewall of China, or similar national firewalls.

This circumvention has been demonstrated both in my evaluative testing and in my real-world use - I had a 30-minute conversation over Discord while circumventing Sophos XG using Hypertunnel, and I didn't even realise that I was using Hypertunnel (rather than my usual method, which takes a different approach) until later.

Wireshark analysis shows that the packets being sent tend to be quite small, so padding small messages presents a clear area to improve. I could also increase the variation of false transfers and stream false HTTP headers - but all of this would only increase the resistance to targeted detection following widespread use, not the circumvention itself.

Hypertunnel fully meets the success criteria for real-world circumvention.

5.2.3. Meaningful Communications

Hypertunnel has demonstrated meaningful communications in both my evaluative testing and the real-world use described above. Hypertunnel is absolutely capable of meaningful communications, and it fully meets the success criteria for meaningful communications.

5.2.4. Long-Term Reliability

Hypertunnel is able to operate for reasonably long periods of time, however, when running on remote servers the connection will sometimes drop out. HTTP handlers will remain alive, yet nothing will transit through them and attempts to reconnect will fail unless the server is restarted or you wait.

I have extensively debugged the issue and I am fairly certain that it is due to internal timeout behaviour in my reverse proxy, Caddy. I have not yet been able to resolve the issue, but it can be mitigated through using it "bare" without a reverse proxy.

Hypertunnel partially meets the success criteria for long-term reliability.

5.2.5. Server Authentication

Hypertunnel successfully rejects connections from unauthenticated clients and accepts connections from authenticated clients. Furthermore, the rejection method is ambiguous and does not reveal that the server is running Hypertunnel. Hypertunnel fully meets the success criteria for server authentication.

5.2.6. MITM Invulnerability

Hypertunnel is invulnerable to MITM attacks due to its use of preshared-key encryption rather than more complex public-key authentication and key exchange. Wireshark analysis clearly demonstrates its invulnerability to MITM attacks, and it is highly unlikely to be vulnerable due

to its simple, robust, and secure design. Hypertunnel fully meets the success criteria for MITM invulnerability.

5.2.7. GUI Operation

Hypertunnel has a GUI client frontend that is able to initiate the core client with the standard set of options. It is minimal and easy to use, allowing non-technical users to operate Hypertunnel. While it does not expose all of the options, they are technical options that the average user would not need to use. Technical users would use the CLI client. Hypertunnel fully meets the success criteria for GUI operation.

5.2.8. Memory-Safe Implementation

Hypertunnel is 100% written in Rust, a memory-safe systems programming language. Rust encapsulates memory unsafe operations in safe abstractions, with all memory safety checks being performed at compile time. While you can introduce memory unsafety through incorrect usage of unsafe blocks, Hypertunnel never directly uses unsafe blocks and all of its abstractions come from widely-used and well-tested libraries. As such, Hypertunnel fully meets the success criteria for memory-safe implementation.

Usability Features and Success

Many of the previously mentioned usability features are irrelevant due to my decision to deviate from my planned UI. I was not able to show the path that the traffic took around the globe, and I wasn't able to implement the diagrammatic display of the program. I could rectify these failures with further development, switching to a server-client GUI architecture and rendering the UI through a local web server rather than a egui window.

I succeeded in avoiding the "hacker aesthetic" through the light-theme, minimal UI. I didn't use my chosen font (JetBrains Mono). However, using a more normal sans-serif font made it less conspicuous, and so I would describe typography as a partial success due to a necessary compromise.

My program is very accessible due to its integration with the system's accessibility APIs and its simplicity. I would describe this as a success.

I intended to provide easy visibility and modifiability of the following program sections through the GUI:

- Client Sockets
- Client Coordinator
- Upload Workers
- Client Information
- Request Indicators
- Server Information
- Ingress Request Handlers
- Server Coordinator
- Server Sockets
- Server Egress Request Handlers
- Nationality Indicator
- Modifiability Indicators

I was not able to provide visibility nor modifiability through the GUI. However, all of the client features can be viewed through my tiered logging system when using the CLI, and the server features can be viewed through the server's logging system. Many of the aspects that would have been modifiable through the GUI are instead able to be changed through configuration files and CLI arguments. I would describe this as a partial success. All of these features are technically possible to implement, but I was not able to do so in the time frame available. I could rectify this with further development through the construction of a server-client GUI architecture and migration of the UI to a local web server.

I feel that the program is usable, particularly to technical users, but not as usable as it could be.

Maintenance

Hypertunnel has been built in a modular, easy-to-maintain way. It is designed to be self-documenting wherever possible, and where additional documentation is needed that documentation is available in the form of this document and my notes. Maintenance and additional development should be straightforward.

Limitations and Improvements

5.5.1. Underdeveloped GUI

My GUI is underdeveloped compared to my initial designs. This has been discussed in further detail in the usability section. I could significantly improve it by implementing my initial design.

5.5.2. Connection Reliability

The connection is not always reliable, particularly behind reverse proxies. The program could be improved through further testing and bug fixing.

5.5.3. Region Lock

I didn't have time to implement the region lock. A lot of the prerequisites are already implemented, and I intend to implement it before open-sourcing the code.

5.5.4. SOCKS5 Support

My program uses SOCKS4, an older and less supported protocol than the newer SOCKS5, due to it being simpler and easier to implement. In particular, SOCKS4 does not support UDP nor more complex addressing modes. I could improve the program by implementing SOCKS5 support.

5.5.5. BIND Support

My program only supports a subset of SOCKS4. There is a lesser known argument, BIND, which some protocols (e.g. FTP) rely upon. I could improve the program by implementing BIND support.

5.5.6. UDP Support

UDP support is a part of implementing SOCKS5. It would allow for QUIC, DNS, and other UDP-based protocols to transit over the proxy. This would be quite difficult to do - it would be an entirely parallel transmission system - plus UDP is really not designed to handle the kind of traffic interruptions the discrete request segmentation system causes.

5.5.7. Better Request Loss Handling

Currently a lost request causes all transit in that direction to halt until it can be retransmitted. This is not ideal, and so I could improve the program by implementing a more robust system for handling lost requests.

5.5.8. Dynamic Modetime

Modetime is currently static. I'd like to implement something that examines the requests and network conditions to dynamically adjust the modetime.

CONCLUSION

Hypertunnel is a functional and usable system that is able to trivially circumvent a previously untargeted and theoretically impossible-to-circumvent class of firewalls.

REFERENCES

- [1] Sadia Nourin *et al.*, “Measuring and Evading Turkmenistan's Internet Censorship”. [Online]. Available: <https://geneva.cs.umd.edu/papers/2023-web-turkmenistan.pdf>
- [2] Elizabeth C Economy, “The great firewall of China: Xi Jinping's internet shutdown”. 2018. Available: <https://www.theguardian.com/news/2018/jun/29/the-great-firewall-of-china-xi-jinpings-internet-shutdown>
- [3] Freedom House, “Freedom of the Net 2018 - Iran”. [Online]. Available: <https://www.refworld.org/docid/5be16b1013.html>
- [4] Richard Clayton, Steven J. Murdoch, and Robert N. M. Watson, “Ignoring the Great Firewall of China”. [Online]. Available: <https://www.cl.cam.ac.uk/~rnc1/ignoring.pdf>
- [5] Anonymous x3, Jan Beznazwy, and Amir Houmansadr, “How China Detects and Blocks Shadowsocks”. [Online]. Available: <https://gfw.report/publications/imc20/data/paper/shadowsocks.pdf>
- [6] Mingshi Wu *et al.*, “How the Great Firewall of China Detects and Blocks Fully Encrypted Traffic”. [Online]. Available: <https://gfw.report/publications/usenixsecurity23/data/paper/paper.pdf>
- [7] Marc Bevand, “My Experience with the Great Firewall of China”. [Online]. Available: <http://blog.zorinaq.com/my-experience-with-the-great-firewall-of-china/>
- [8] Anonymous, Arian Akhavan Niaki, Nguyen Phong Hoang, Phillipa Gill, and Amir Houmansadr, “Triplet Censors: Demystifying Great Firewall's DNS Censorship Behavior”. [Online]. Available: https://gfw.report/publications/foci20_dns/dns.pdf
- [9] Nguyen Phong Hoang *et al.*, “How Great is the Great Firewall? Measuring China's DNS Censorship”. [Online]. Available: <https://www.usenix.org/system/files/sec21-hoang.pdf>
- [10] Kevin Bock, Gabriel Naval, Kyle Reese, and Dave Levin, “Even Censors Have a Backup: Examining China's Double HTTPS Censorship Middleboxes”. [Online]. Available: <https://geneva.cs.umd.edu/papers/2021-foci-censors-have-backup.pdf>
- [11] Kevin Bock, Yair Fax, Kyle Reese, Jasraj Singh, and Dave Levin, “Detecting and Evading Censorship-in-Depth: A Case Study of Iran's Protocol Filter”. [Online]. Available: <https://geneva.cs.umd.edu/papers/2020-foci-protocol-filter.pdf>
- [12] David Fifield, Chang Lan, Rod Hynes, Percy Wegmann, and Vern Paxson, “Blocking-resistant communication through domain fronting”. [Online]. Available: <https://www.bamsoftware.com/papers/fronting.pdf>
- [13] Clowwindy, “Shadowsocks”. [Online]. Available: <https://github.com/shadowsocks>
- [14] William L. Hosch and Editors of Encyclopaedia Britannica, “Genetic Algorithm”. [Online]. Available: <https://www.britannica.com/technology/genetic-algorithm>
- [15] Kevin Bock, George Hughey, Xiao Qiang, and Dave Lenin, “Geneva: Evolving Censorship Evasion Strategies”. [Online]. Available: <https://geneva.cs.umd.edu/papers/2019-ccs-geneva.pdf>
- [16] “Sophos XG Firewall”. [Online]. Available: <https://www.sophos.com/en-us/products/next-gen-firewall>
- [17] Alex Chambers-Jones, “HTTP Proxy DPI Evasion”. [Online]. Available: <https://github.com/Alex-Programs/http-proxy-dpi-evasion>
- [18] twilde, “Great Firewall Tor Probing”. [Online]. Available: <https://gist.github.com/twilde/da3c7a9af01d74cd7de7>
- [19] “Server-sent events”. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- [20] “Websockets”. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [21] Department for Education, “Meeting digital and technology standards in schools and colleges”. [Online]. Available: <https://www.gov.uk/guidance/meeting-digital-and-technology-standards-in-schools-and-colleges/broadband-internet-standards-for-schools-and-colleges>
- [22] UK Safer Internet Centre, “Appropriate filtering and monitoring: A guide for education settings and filtering providers”. [Online]. Available: <https://saferinternet.org.uk/guide-and-resource/teachers-and-school-staff/appropriate-filtering-and-monitoring>
- [23] “Reddit”. [Online]. Available: <https://reddit.com/>
- [24] Anonymous, “Trojan: An unidentifiable mechanism that helps you bypass GFW.”. [Online]. Available: <https://github.com/trojan-gfw/trojan>
- [25] erebe, “WSTunnel”. [Online]. Available: <https://github.com/erebe/wstunnel>
- [26] mhzed, “wstunnel: tunnel over websocket”. [Online]. Available: <https://github.com/mhzed/wstunnel>
- [27] Adrian Shahbaz, Allie Funk, Kian Vesteinsson, and Freedom House, “Freedom on the Net 2022”. [Online]. Available: <https://freedomhouse.org/report/freedom-net/2022/countering-authoritarian-overhaul-internet>
- [28] Al Jazeera Staff, “In 2022, the world saw 187 internet shutdowns - 84 by India alone”. [Online]. Available: <https://www.aljazeera.com/news/2023/2/28/in-2022-the-world-saw-187-internet-shutdowns-84-by-india-alone>
- [29] Rhea Mogul and CNN, “India's internet shutdowns cost the economy 2.8 billion in 2022”. [Online]. Available: <https://edition.cnn.com/2023/03/20/india/india-separatist-manhunt-internet-shutdown-intl-hnk/index.html>
- [30] Matt Haldane and Xinmei Shen, “China blocks internet anticensorship tools ahead of 20th party congress as the Great Firewall grows in sophistication”. [Online]. Available: <https://www.scmp.com/tech/policy/article/3195045/china-blocks-internet-anticensorship-tools-ahead-20th-party-congress>
- [31] “Properly configured Shadowsocks servers reportedly blocked in China?”. [Online]. Available: <https://github.com/net4people/bbs/issues/69>
- [32] “icmptunnel”. [Online]. Available: <https://github.com/DhavalKapil/icmptunnel>
- [33] “Iodine DNS Tunnel”. [Online]. Available: <https://github.com/yarrick/iodine>
- [34] “The Great Firewall of China: Background”. [Online]. Available: <https://cs.stanford.edu/people/eroberts/cs201/projects/2010-11/FreedomOfInformationChina/category/great-firewall-of-china/index.html>
- [35] Nate Cardozo and Sophia Cope, “Cisco's Latest Attempt to Dodge Responsibility for Facilitating Human Rights Abuses: Export Rules”. [Online]. Available: <https://www.eff.org/deeplinks/2016/04/ciscos-latest-attempt-dodge-responsibility-facilitating-human-rights-abuses-export>
- [36] Sarah Lai Stirland, “Cisco Leak: 'Great Firewall' of China Was a Chance to Sell More Routers”. [Online]. Available: <https://www.wired.com/2008/05/leaked-cisco-do/>
- [37] Jonathan Zittrain and Benjamin Edelman, “Empirical Analysis of Internet Filtering in China”. [Online]. Available: <https://cyber.harvard.edu/filtering/china/>

- [38] AP Wire, "China blocks Google as congress looms". [Online]. Available: <https://www.theguardian.com/technology/2002/sep/04/internetnews.china>
- [39] Ben Blanchard and Reuters, "China tightens Web screws after Xinjiang riot". [Online]. Available: <https://www.reuters.com/article/us-china-xinjiang-internet-idUSTRE5651K420090706>
- [40] Council on Foreign Relations, "Operation Aurora". [Online]. Available: <https://www.cfr.org/cyber-operations/operation-aurora>
- [41] Leif Nixon, "Some observations on the Great Firewall of China". [Online]. Available: <https://www.nsc.liu.se/~nixon/sshprobes.html>
- [42] Habdul Hazeed, "What is TLS fingerprinting?". [Online]. Available: <https://fingerprint.com/blog/what-is-tls-fingerprinting-transport-layer-security/>
- [43] twilde, "Knock Knock Knockin' on Bridges' Doors". [Online]. Available: <https://blog.torproject.org/knock-knock-knockin-bridges-doors/>
- [44] phobos, "Tor partially blocked in China". [Online]. Available: <https://blog.torproject.org/tor-partially-blocked-china/>
- [45] Philipp Winter and Stefan Lindskog, "How the Great Firewall of China Is Blocking Tor". [Online]. Available: <https://www.usenix.org/system/files/conference/foci12/foci12-final2.pdf>
- [46] Philipp Winter, "GFW actively probes obfs2 bridges". [Online]. Available: <https://gitlab.torproject.org/legacy/trac/-/issues/8591>
- [47] Roya Ensafi, David Fifield, Philipp Winter, Nick Feamster, Nicholas Weaver, and Vern Paxson, "Examining How the Great Firewall Discovers Hidden Circumvention Servers". [Online]. Available: <http://conferences.sigcomm.org/imc/2015/papers/p445.pdf>
- [48] Erik Hjelmvik, "Forensics of Chinese MITM on GitHub". [Online]. Available: <https://www.netresec.com/?page=Blog&month=2013-02&post=Forensics-of-Chinese-MITM-on-GitHub>
- [49] Martin, "China, Github, and the Man-In-The-Middle". [Online]. Available: <https://en.greatfire.org/blog/2013/jan/china-github-and-man-middle>
- [50] yefuchs, "List of Contributors to the Great Firewall of China". [Online]. Available: <https://gist.github.com/yefuchs/4648713>
- [51] Bai Tiantan and Global Times, "[Chinese Source] US tech website back online". [Online]. Available: <https://www.globaltimes.cn/content/757868.shtml>
- [52] Erik Hjelmvik, "Chinese MITM Attack on iCloud". [Online]. Available: <https://www.netresec.com/?page=Blog&month=2014-10&post=Chinese-MITM-Attack-on-iCloud>
- [53] Sean Gallagher, "Chinese government launches man-in-middle attack against iCloud". [Online]. Available: <https://arstechnica.com/information-technology/2014/10/chinese-government-launches-man-in-middle-attack-against-icloud/>
- [54] Jim Finkle, Gerry Shih, Ben Blanchard, and Reuters, "China-backed hackers targeted Apple's iCloud users". [Online]. Available: <https://www.reuters.com/article/apple-china-security-idUSL2N0SG01Q20141021>
- [55] BBC News, "China denies backing iCloud hack". [Online]. Available: <https://www.bbc.co.uk/news/technology-29704371>
- [56] Erik Hjelmvik, "Analysis of Chinese MITM on Google". [Online]. Available: <https://www.netresec.com/?page=Blog&month=2014-09&post=Analysis-of-Chinese-MITM-on-Google>
- [57] Erik Hjelmvik, "Analysis of Chinese MITM on Yahoo". [Online]. Available: <https://www.netresec.com/?page=Blog&month=2014-10&post=Verifying-Chinese-MITM-of-Yahoo>
- [58] Anonymous, "Towards a Comprehensive Picture of the Great Firewall's DNS Censorship". [Online]. Available: <https://www.usenix.org/system/files/conference/foci14/foci14-anonymous.pdf>
- [59] Bill Marczak *et al.*, "China's Great Cannon", 2015. Available: <https://citizenlab.ca/2015/04/chinas-great-cannon/>
- [60] Josh Horwitz, "China's fierce censors try a new tactic with GitHub—asking nicely". [Online]. Available: <https://qz.com/718465/chinas-fierce-censors-try-a-new-tactic-with-github-asking-nicely>
- [61] Sebastian Anthony, "GitHub battles 'largest DDoS' in site's history, targeted at anti-censorship tools". [Online]. Available: <https://arstechnica.com/information-technology/2015/03/github-battles-largest-ddos-in-sites-history-targeted-at-anti-censorship-tools/>
- [62] Cyber Security Association of China, "github: gov-takedowns/China/2016-06-08-programthink-zhao". [Online]. Available: <https://github.com/github/gov-takedowns/blob/master/China/2016/2016-06-08-programthink-zhao.md>
- [63] George Kadianakis, "obfs3 (The threebfuscator)". [Online]. Available: <https://gitweb.torproject.org/pluggable-transport/obfsproxy.git/tree/doc/obfs3/obfs3-protocol-spec.txt>
- [64] clowwindy, "Two days ago the police came to me and wanted me to stop working on this.". [Online]. Available: <https://web.archive.org/web/20150822042959/https://github.com/shadowsocks/shadowsocks-ios/issues/124#issuecomment-133630294>
- [65] clowwindy, "Clowwindy Twitter". [Online]. Available: <https://twitter.com/clowwindy?lang=en>
- [66] "Outline VPN". [Online]. Available: <https://getoutline.org/>
- [67] TechCrunch, "Alphabets Outline lets you build your own VPN". [Online]. Available: <https://techcrunch.com/2018/03/22/alphabets-outline-lets-you-build-your-own-vpn>
- [68] "Pluggable Transports and DPI". [Online]. Available: <https://lists.torproject.org/pipermail/tor-talk/2016-May/040898.html>
- [69] David Fifield, "Cyberoam firewall blocks Meek by TLS signature". [Online]. Available: <https://groups.google.com/g/traffic-obf/c/BpFSCVgi5rs>
- [70] Michal Trojnara, "Stunnel". [Online]. Available: <https://www.stunnel.org/>
- [71] Shuo Chen, Rui Wang, XiaoFeng Wang, and Kehuan Zhang, "Side-Channel Leaks in Web Applications: a Reality Today, a Challenge Tomorrow". [Online]. Available: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/WebAppSideChannel-final.pdf>
- [72] Ruixi Yuan, Zhu Li, Xiaohong Guan, and Li Xu, "An SVM-based machine learning method for accurate traffic classification". [Online]. Available: https://files.alexci.co.uk/svmpaper_mirror.pdf
- [73] apernet, "TCP Brutal". [Online]. Available: <https://github.com/apernet/tcp-brutal>
- [74] apernet, "Hysteria". [Online]. Available: <https://github.com/apernet/hysteria>
- [75] Arun Dunna, Ciaran O'Brien, and Phillipa Gill, "Analyzing China's Blocking of Unpublished Tor Bridges". [Online]. Available: <https://people.cs.umass.edu/~phillipa/papers/foci-2018.pdf>
- [76] XRPractices, "Reinforcement Learning vs Genetic Algorithm". [Online]. Available: <https://medium.com/xrpractices/reinforcement-learning-vs-genetic-algorithm-ai-for-simulations-f1f484969c56>
- [77] Tarun Kumar Yadav, Akshat Sinha, Devashish Gosain, Piyush Sharma, and Sambuddho Chakravarty, "Where the Light Gets In: Analyzing Web Censorship Mechanisms in India". [Online]. Available: <https://arxiv.org/pdf/1808.01708.pdf>

- [78] Anonymous x3, David Fifield, and Amir Houmansadr, "A practical guide to defend against the GFW's latest active probing". [Online]. Available: https://gfw.report/blog/ss_advise/en/
- [79] Yongzhong He, Liping Hu, and Rui Gao, "Detection of Tor Traffic Hiding Under Obfs4 Protocol Based on Two-Level Filtering". [Online]. Available: <https://ieeexplore.ieee.org/document/8855280>
- [80] Xuebin Wang, Zeyu Li, Wentao Huang, Meiqi Wang, Jinqiao Shi, and Yanyang Yang, "Towards Comprehensive Analysis of Tor Hidden Service Access Behavior Identification Under Obfs4 Scenario". [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3491396.3506532>
- [81] Cloudflare, "What is encrypted SNI?". [Online]. Available: <https://www.cloudflare.com/en-gb/learning/ssl/what-is-encrypted-sni/>
- [82] "HTTP/3 RFC". [Online]. Available: <https://datatracker.ietf.org/doc/rfc9114/>
- [83] Tor Project, "Tor Project". [Online]. Available: <https://www.torproject.org/>
- [84] OpenSSH, "SOCKS4 Protocol Specification". [Online]. Available: <https://www.openssh.com/txt/socks4.protocol>
- [85] Desmos, "Desmos". [Online]. Available: <https://www.desmos.com/calculator/>
- [86] Van Jacobson and Michael J. Karels, "Congestion Avoidance and Control". [Online]. Available: <https://ee.lbl.gov/papers/congavoid.pdf>
- [87] John Nagle and Network Working Group, "RFC 896". [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc896>
- [88] Network Working Group, M. Allman, V. Paxson, ICSI, and E. Blanton, "RFC 5681". [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc5681>
- [89] Internet Engineering Task Force, A. Zimmerman, W. Eddy, and L. Eggert, "RFC 7805". [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7805>
- [90] Maxmind, "MaxMind GeoLite2". [Online]. Available: <https://dev.maxmind.com/geoip/geolite2-free-geolocation-data>
- [91] oschwald, "maxminddb-rust". [Online]. Available: <https://github.com/oschwald/maxminddb-rust>
- [92] "Rust Programming Language". [Online]. Available: <https://www.rust-lang.org/>
- [93] Microsoft, "Visual Studio Code". [Online]. Available: <https://code.visualstudio.com/>
- [94] "Obsidian". [Online]. Available: <https://obsidian.md/>
- [95] Terrastruct, "D2 Language". [Online]. Available: <https://d2lang.com/>
- [96] "Excalidraw". [Online]. Available: <https://github.com/excalidraw/excalidraw>
- [97] JetBrains, "JetBrains Mono". [Online]. Available: <https://www.jetbrains.com/idea/mono/>
- [98] Cloudflare, "Cloudflare Speed Test". [Online]. Available: <https://speed.cloudflare.com/>
- [99] Mozilla, "Firefox". [Online]. Available: <https://www.mozilla.org/en-GB/firefox/new/>
- [100] Daniel Stenberg et al, "Curl". [Online]. Available: <https://curl.se/>
- [101] Jam1garner, "BinRW". [Online]. Available: <https://binrw.rs/>
- [102] Morz25, "ChaCha20-Poly1305 Encryption". [Online]. Available: https://en.wikipedia.org/wiki/ChaCha20-Poly1305#/media/File:ChaCha20-Poly1305_Encryption.svg
- [103] poncho, "Fully encrypted, non-fingerprintable symmetric encryption algorithm". [Online]. Available: <https://crypto.stackexchange.com/questions/107287/fully-encrypted-non-fingerprintable-symmetric-encryption-algorithm>
- [104] "Serde". [Online]. Available: <https://serde.rs/>
- [105] datagenetics, "Birthday Problem". [Online]. Available: <https://datagenetics.com/blog/february72019/index.html>
- [106] Kevin Galligan, "Hash Collisions Calculator". [Online]. Available: <https://kevingal.com/blog/collisions.html>
- [107] David Koloski, "Rust Serialisation Benchmark (Web)". [Online]. Available: https://david.kolo.ski/rust_serialization_benchmark/
- [108] TimelyDataflow, "Abomination". [Online]. Available: <https://github.com/TimelyDataflow/abomination>
- [109] rkyv, "rkyv". [Online]. Available: <https://crates.io/crates/rkyv/0.7.42>
- [110] David Koloski et al, "Rust Serialisation Benchmark (Source)". [Online]. Available: https://github.com/djkoloski/rust_serialization_benchmark
- [111] koute et al, "Speedy". [Online]. Available: <https://crates.io/crates/speedy/0.8.6>
- [112] James Munns et al, [Online]. Available: <https://crates.io/crates/postcard/1.0.6>
- [113] bincode-org, "Bincode". [Online]. Available: <https://crates.io/crates/bincode/1.3.3>
- [114] near et al, "Borsh". [Online]. Available: <https://crates.io/crates/borsh/0.10.3>
- [115] not-fl3 et al, "Nano Serde". [Online]. Available: <https://crates.io/crates/nanoserde/0.7.0>
- [116] 3Hren et al, "Msgpack Rust". [Online]. Available: <https://github.com/3Hren/msgpack-rust>
- [117] avl et al, "Savefile". [Online]. Available: <https://crates.io/crates/savefile/0.16.0>
- [118] SoftBearStudios, "Bitcode". [Online]. Available: <https://crates.io/crates/bitcode/0.4.0>
- [119] zesterer et al, "Flume". [Online]. Available: <https://docs.rs/flume/latest/flume/>
- [120] [Online]. Available: <https://github.com/tokio-rs/tokio/issues/2032>
- [121] lightbend, "Actors Introduction". [Online]. Available: <https://doc.akka.io/docs/akka/current/typed/guide/actors-intro.html>
- [122] "Tauri". [Online]. Available: <https://tauri.app/>
- [123] "Iced". [Online]. Available: <https://github.com/iced-rs/iced>
- [124] "Bevy". [Online]. Available: <https://bevyengine.org/>
- [125] "Clap". [Online]. Available: <https://clap.rs/>
- [126] emilk et al, "egui". [Online]. Available: <https://github.com/emilk/egui>
- [127] emilk et al, "eframe". [Online]. Available: <https://github.com/emilk/egui/tree/master/crates/eframe>
- [128] Netflix, "Fast.com Speed Test". [Online]. Available: <https://fast.com/>
- [129] GCHQ, "Cyberchef". [Online]. Available: <https://gchq.github.io/CyberChef/>
- [130] Young Xu, "Deconstructing the Great Firewall of China". 2016. Available: <https://www.thousandeyes.com/blog/deconstructing-great-firewall-china>
- [131] Roya Ensafi, Philipp Winter, Abdullah Mueen, and Jedidiah R. Crandall, "Analyzing the Great Firewall of China Over Space and Time". [Online]. Available: <https://ensa.fi/papers/Ensafi2015a.pdf>
- [132] "The Great Firewall of China". [Online]. Available: <https://thehistoryofcomputing.net/the-great-firewall-of-china>

- [133] “log”. [Online]. Available: <https://github.com/rust-lang/log>
- [134] borntyping et al, “simple_logger”. [Online]. Available: https://docs.rs/simple_logger/latest/simple_logger/
- [135] apernet, “OpenGFW”. [Online]. Available: <https://github.com/apernet/OpenGFW>

CODE APPENDIX

The following is every single `.rs` (Rust), `.sh` (Bash), and `.toml` (configuration) file in my project. The total length is 3589 loc and 116 380 characters across 33 files.

client-core::transit.rs

```

1 // transit.rs
2 use libtransit::SerialMessage;
3
4 use flume::{self, Receiver as FlumeReceiver, Sender as FlumeSender};
5 use libsecrets::{self, EncryptionKey};
6 use libtransit::{self, UnifiedPacketInfo, SocksSocketUpstream, SocksSocketDownstream};
7 use libtransit::{
8     ClientMessageUpstream, ClientMetaUpstream, DeclarationToken,
9     ServerMessageDownstream, SocketID, UpStreamMessage,
10 };
11 use rand::Rng;
12 use request::Client;
13 use tokio::sync::mpsc::{self, Receiver, Sender, UnboundedReceiver, UnboundedSender};
14 use tokio::sync::mpsc::error::TryRecvError;
15 use std::collections::HashMap;
16
17 use request::header::HeaderMap;
18 use std::sync::Arc;
19 use std::sync::atomic::AtomicU32;
20 use std::sync::atomic::Ordering;
21
22 use std::time::{Duration, Instant, SystemTime, UNIX_EPOCH};
23 use tokio::sync::broadcast::{Receiver as BroadcastReceiver, Sender as BroadcastSender};
24 use tokio::sync::RwLock;
25 use tokio::task;
26
27 static RECEIVED_SEQ_NUM: AtomicU32 = AtomicU32::new(0);
28 static SENT_SEQ_NUM: AtomicU32 = AtomicU32::new(0);
29
30 use crate::meta::YELLOW_DATA_UPSTREAM_QUEUE;
31
32 use log::{debug, error, info, warn};
33
34 pub struct TransitSocket {
35     pub target: String, // Base URL, incl. protocol
36     pub key: EncryptionKey, // Encryption key for libsecrets
37     pub client_identifier: DeclarationToken, // Identifier for this client, used as a cookie
38     pub push_client_count: usize, // Number of hybrid clients (Both push and pull)
39     pub pull_client_count: usize, // Number of pull clients
40     pub timeout_time: usize, // Time in seconds before a request is considered timed
41     out
42     pub headers: HeaderMap, // Headers to send with requests. Includes client
43     identifier
44     pub is_initialized: bool, // Whether the socket has been initialized by greeting
45     the server
46     pub client_name: String, // Name of the client
47 }
48
49 #[derive(Debug)]
50 pub enum TransitInitError {
51     EncryptionError(libsecrets::EncryptionError),
52     RequestError(request::Error),
53     ConnectionFailed,
54     ConnectionDenied,
55 }

```

```

52     LiedResponse,
53 }
54
55 impl From<libsecrets::EncryptionError> for TransitInitError {
56     fn from(error: libsecrets::EncryptionError) -> Self {
57         Self::EncryptionError(error)
58     }
59 }
60
61 #[derive(Clone, Debug)]
62 pub struct DownstreamBackpasser {
63     pub socket_id: libtransit::SocketID,
64     pub sender: UnboundedSender<SocksSocketDownstream>,
65 }
66
67 pub async fn connect(transit_socket: Arc<RwLock<TransitSocket>>) -> Result<(), TransitInitError> {
68     greet_server(transit_socket.clone()).await?;
69     transit_socket.write().await.is_initialized = true;
70     Ok(())
71 }
72
73 fn generate_nonsense_data() -> String {
74     let dataset = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
75     let count = 64;
76
77     let mut rng = rand::thread_rng();
78     let mut data = String::with_capacity(count);
79     for _ in 0..count {
80         let index = rng.gen_range(0..dataset.len());
81         data.push(dataset.chars().nth(index).unwrap());
82     }
83
84     data
85 }
86
87 async fn greet_server(transit_socket: Arc<RwLock<TransitSocket>>) -> Result<(), TransitInitError> {
88     info!("Greeting server");
89
90     let mut data = format!(
91         "Hello. Protocol version: {}, client-transit version: {}, client-name: {}",
92         "2",
93         env!("CARGO_PKG_VERSION"),
94         transit_socket.read().await.client_name
95     );
96
97     // Find minimum padding amount such that it's at least 512 bytes
98     let length = data.len();
99     let min_padding = 512 - length;
100
101     // Find maximum padding amount such that it's at most 1024 bytes
102     let max_padding = 1024 - length;
103
104     // Add random padding up to 1024 bytes
105     let mut rng = rand::thread_rng();
106     let amount = rng.gen_range(min_padding..max_padding);
107     data = data + &" ".repeat(amount);
108
109     let encrypted = libsecrets::encrypt(data.as_bytes(), &transit_socket.read().await.key)?;
110
111     let (target, headers, key, _) = {
112         let transit_socket = transit_socket.read().await;
113         (

```

```

114         transit_socket.target.clone(),
115         transit_socket.headers.clone(),
116         transit_socket.key.clone(),
117         transit_socket.timeout_time,
118     )
119 };
120
121 let response = Client::new()
122     .post(&format!("{}",/submit", target))
123     .body(encrypted)
124     .headers(headers)
125     .timeout(Duration::from_secs(2 as u64))
126     .send()
127     .await;
128
129 info!("Received response from server");
130
131 match response {
132     Ok(response) => {
133         if response.status() != 200 {
134             return Err(TransitInitError::ConnectionDenied);
135         }
136         // Try to decrypt the response
137         let encrypted = response.bytes().await;
138         match encrypted {
139             Ok(encrypted) => {
140                 let decrypted = libsecrets::decrypt(&encrypted, &key)?;
141                 // Convert to string
142                 let decrypted = String::from_utf8(decrypted);
143                 match decrypted {
144                     Ok(decrypted) => {
145                         // Remove spaces
146                         let decrypted = decrypted.trim_end_matches(' ');
147                         if decrypted != "CONNECTION ACCEPTED" {
148                             return Err(TransitInitError::LiedResponse);
149                         }
150                     }
151                     Err(_) => {
152                         return Err(TransitInitError::LiedResponse);
153                     }
154                 }
155             }
156             Err(error) => {
157                 return Err(TransitInitError::RequestError(error));
158             }
159         }
160     }
161     Err(error) => {
162         if error.is_timeout() || error.is_connect() {
163             return Err(TransitInitError::ConnectionFailed);
164         } else {
165             return Err(TransitInitError::RequestError(error));
166         }
167     }
168 }
169
170 info!("Processed response from server");
171
172 // If we've gotten here, the connection was successful
173 // We should be ready to talk to the server now
174
175 Ok(())

```

```

176 }
177
178 async fn push_handler(
179     transit_socket: Arc<RwLock<TransitSocket>>,
180     to_send_passer: FlumeReceiver<Vec<SocksSocketUpstream>>,
181 ) {
182     // Create client
183     let client = Client::new();
184
185     let (key, headers, target, timeout_time) = {
186         let transit_socket = transit_socket.read().await;
187         (
188             transit_socket.key.clone(),
189             transit_socket.headers.clone(),
190             transit_socket.target.clone(),
191             transit_socket.timeout_time,
192         )
193     };
194
195     let timeout_duration = Duration::from_secs(timeout_time as u64);
196
197     debug!("Push handler started");
198
199     loop {
200         // Get the data to send
201         let to_send = to_send_passer.recv_async().await;
202
203         let to_send = match to_send {
204             Ok(to_send) => to_send,
205             Err(_) => {
206                 // TODO handle this
207                 to_send.unwrap();
208                 continue;
209             }
210         };
211
212         let payload_size = to_send.iter().fold(0, |acc, x| acc + x.payload.len() as u32);
213
214         // Send the data
215         // Encode and encrypt in a thread
216         let upstream_message = ClientMessageUpstream {
217             socks_sockets: to_send,
218             metadata: get_metadata(SENT_SEQ_NUM.fetch_add(1, Ordering::SeqCst)).await,
219             payload_size,
220         };
221
222         let to_send = tokio::task::spawn_blocking(move || {
223             let data_bin = upstream_message.encoded().unwrap();
224
225             let encrypted = libsecrets::encrypt(&data_bin, &key).unwrap();
226
227             encrypted
228         }).await.unwrap();
229
230         let url = format!("{}", upload/segment/{}", target, generate_nonsense_data());
231
232         // Send the data
233         let response = client
234             .post(url)
235             .body(to_send)
236             .headers(headers.clone())
237             .timeout(timeout_duration) // TODO RM

```

```

238         .send()
239         .await;
240
241     let response = response.unwrap();
242
243     debug!("Sent! Response code {:?}", data: {:?}", response.status(), response.bytes().await);
244
245     // TODO if things go wrong, prevent the arrival of new data until the send works. See working
nicely section of todo about reconstruction and error handling.
246 }
247 }
248
249 async fn get_metadata(seq_num: u32) -> ClientMetaUpstream {
250     let yellow_to_stop_reading_from = {
251         let mut data = YELLOW_DATA_UPSTREAM_QUEUE.write().await;
252
253         let to_return = data.clone();
254
255         // Clear the queue
256         data.clear();
257
258         to_return
259     };
260
261     let data = ClientMetaUpstream {
262         packet_info: UnifiedPacketInfo {
263             unix_ms: SystemTime::now()
264                 .duration_since(UNIX_EPOCH)
265                 .unwrap()
266                 .as_millis() as u64,
267             seq_num,
268         },
269         set: None,
270         yellow_to_stop_reading_from,
271     };
272
273     debug!("Sending metadata: {:?}", data);
274
275     data
276 }
277
278 async fn pull_handler(
279     transit_socket: Arc<RwLock<TransitSocket>>,
280     mut message_passer_passer: BroadcastReceiver<DownstreamBackpasser>,
281     blue_termination_send: FlumeSender<SocketID>,
282 ) {
283     // This will hold the return lookup and be populated by the messagePasserPassers
284     let mut return_lookup: HashMap<SocketID, UnboundedSender<SocksSocketDownstream>> =
HashMap::new();
285
286     // Create client
287     let client = Client::new();
288
289     let (key, headers, target, timeout_time) = {
290         let transit_socket = transit_socket.read().await;
291         (
292             transit_socket.key.clone(),
293             transit_socket.headers.clone(),
294             transit_socket.target.clone(),
295             transit_socket.timeout_time,
296         )
297     };

```

```

298
299     let timeout_duration = Duration::from_secs(timeout_time as u64);
300
301     let default_timeout = 300;
302     let mut cumulative_timeout = default_timeout;
303
304     loop {
305         // Could be made spawn_blocking if this turns out to take too long
306         let upstream_message = ClientMessageUpstream {
307             socks_sockets: Vec::with_capacity(0),
308             metadata: get_metadata(0).await,
309             payload_size: 0,
310         };
311
312         let to_send = {
313             let data_bin = upstream_message.encoded().unwrap();
314
315             let encrypted = libsecrets::encrypt(&data_bin, &key).unwrap();
316
317             encrypted
318         };
319
320         let url = format!("{}/video/segment", target);
321
322         // Send the data
323         let response = client
324             .get(url)
325             .body(to_send)
326             .headers(headers.clone())
327             .timeout(timeout_duration)
328             .send()
329             .await;
330
331         let response = match response {
332             Ok(response) => {
333                 if response.status() != 200 {
334                     warn!("Request completed but with status {}", response.status());
335
336                     cumulative_timeout = default_timeout;
337
338                     continue
339                 }
340                 cumulative_timeout = default_timeout;
341
342                 response
343             }
344             Err(error) => {
345                 if error.is_timeout() {
346                     // All if fine
347                     info!("Request timeout; continue");
348                     continue
349                 }
350                 warn!("Error on request. Error: {:?}", error);
351
352                 tokio::time::sleep(Duration::from_millis(cumulative_timeout)).await;
353
354                 cumulative_timeout *= 2;
355
356                 continue
357             }
358         };
359

```

```

360 // Get the data
361 let encrypted = response.bytes().await.unwrap();
362
363 // Decrypt the data
364 let mut decrypted = libsecrets::decrypt(&encrypted, &key).unwrap();
365
366 // Decode the data
367 let decoded = ServerMessageDownstream::decode_from_bytes(&mut decrypted).unwrap();
368
369 let seq_num = decoded.metadata.packet_info.seq_num;
370
371 while seq_num != RECEIVED_SEQ_NUM.load(Ordering::SeqCst) {
372     // Wait until the seq_num is the one we're expecting
373     tokio::time::sleep(Duration::from_millis(1)).await;
374 }
375
376 // Set the seq_num to the next one
377 RECEIVED_SEQ_NUM.fetch_add(1, Ordering::SeqCst);
378
379 for socket in decoded.socks_sockets {
380     // Do we need to send on a blue termination signal?
381     if socket.do_blue_terminate {
382         // Inform both sockets and coordinator to either terminate or remove data
383         blue_termination_send.send(socket.socket_id).unwrap();
384
385         // Do not remove from lookup etc: There may still be data to send back, this is only
one side of the connection
386     }
387     let socket_id = socket.socket_id;
388     let do_term = socket.do_green_terminate;
389
390     // Get the sender
391     let sender = return_lookup.get(&socket_id);
392
393     match sender {
394         Some(sender) => {
395             // Send the message
396             match sender.send(socket) {
397                 Ok(_) => {},
398                 Err(_) => {
399                     // Remove the sender from the lookup
400                     return_lookup.remove(&socket_id);
401                 }
402             };
403         }
404         None => {
405             // Populate the sender via iteration through the messagePasserPasser
406             for _ in 0..message_passer_passer.len() {
407                 let message_passer = message_passer_passer.recv().await.unwrap();
408                 // Blindly populate
409                 return_lookup.insert(message_passer.socket_id, message_passer.sender);
410             }
411
412             // Try again
413             let sender = match return_lookup.get(&socket_id) {
414                 Some(sender) => sender,
415                 None => {
416                     // If we still don't have a sender, give up
417                     continue;
418                 }
419             };
420         }
421     };

```

```

421         // Send the message
422         match sender.send(socket) {
423             Ok(_) => {}
424             Err(_) => {
425                 // Remove the sender from the lookup
426                 return_lookup.remove(&socket_id);
427             }
428         };
429     }
430 }
431
432 // Is it set to terminate?
433 if do_term {
434     // Remove the sender from the lookup. Yes this can sometimes add then immediately
435     // remove from the hashmap if they terminate on their first transit window; I don't care, we have bigger
436     // perf issues.
437     return_lookup.remove(&socket_id);
438
439     debug!("Terminated socket from return lookup: {}", socket_id);
440 }
441 }
442 }
443
444 pub async fn handle_transit(
445     transit_socket: Arc<RwLock<TransitSocket>>,
446     mut upstream_passer_rcv: Receiver<UpStreamMessage>,
447     message_passer_passer_send: Arc<BroadcastSender<DownstreamBackpasser>>,
448     blue_termination_send: FlumeSender<SocketID>,
449     blue_termination_receive: FlumeReceiver<SocketID>
450 ) {
451     // Spawn requisite push and pull handlers
452     // Create a channel for the push handler to send to so that a random not-in-use push handler
453     // can grab the data to send.
454
455     // Create the push handlers as flume message passers of MultipleMessagesUpstream
456     let (push_passer_send, push_passer_receive) =
457     flume::unbounded:::<Vec<libtransit::SocksSocketUpstream>>();
458
459     let (push_client_count, pull_client_count) = {
460         let transit_socket = transit_socket.read().await;
461         (
462             transit_socket.push_client_count,
463             transit_socket.pull_client_count,
464         )
465     };
466
467     for _ in 0..push_client_count {
468         task::spawn(push_handler(
469             transit_socket.clone(),
470             push_passer_receive.clone(),
471         ));
472     }
473
474     // Create the pull handlers
475     for _ in 0..pull_client_count {
476         let new_receiver = message_passer_passer_send.subscribe();
477         task::spawn(pull_handler(transit_socket.clone(), new_receiver,
478             blue_termination_send.clone()));
479     }
480
481     // TODO make this variable

```

```

480     let force_send_buff_size = 1024 * 32768;
481     let mode_time = 10; // In milliseconds
482
483     let mut last_upstream_time = Instant::now();
484     let mut current_buffer_size = 0;
485     let mut last_loop_time = Instant::now();
486
487     let mut socks_sockets: Vec<libtransit::SocksSocketUpstream> = Vec::with_capacity(8);
488
489     loop {
490         let delta = Instant::now() - last_loop_time;
491         if delta > Duration::from_millis(12) {
492             warn!("Transit took an unusual amount of time ({}ms) to iterate", delta.as_millis());
493         }
494         last_loop_time = Instant::now();
495
496         // Get data to send up
497         let stream_data = upstream_passer_rcv.try_recv();
498         match stream_data {
499             Ok(mut upstream) => {
500                 // Increment buffer size
501                 let size = upstream.payload.len() as u32;
502                 current_buffer_size += size;
503
504                 debug!("Gotten data to send up: {:?}", upstream);
505
506                 // Find the relevant socks socket to insert into
507                 let mut found = false;
508                 // Iterate over socks_sockets without taking ownership
509                 for socks_socket in socks_sockets.iter_mut() {
510                     if socks_socket.socket_id == upstream.socket_id {
511                         // Is it a terminate message?
512                         if upstream.red_terminate {
513                             // Set terminate flag to true on the socks socket
514                             socks_socket.red_terminate = true;
515                         } else {
516                             // Insert into the socks socket
517                             socks_socket.payload.append(&mut upstream.payload);
518                         }
519                         found = true;
520                         break;
521                     }
522                 }
523
524                 if !found {
525                     // Create a new socks socket
526                     let socks_socket = libtransit::SocksSocketUpstream {
527                         socket_id: upstream.socket_id,
528                         dest_ip: upstream.dest_ip,
529                         dest_port: upstream.dest_port,
530                         payload: upstream.payload,
531                         red_terminate: false,
532                     };
533
534                     // Insert into the socks sockets
535                     socks_sockets.push(socks_socket);
536                 }
537             }
538             Err(error) => {
539                 match error {
540                     TryRecvError::Empty => {
541                         // Wait 1ms

```

```

542         let start_time = Instant::now();
543         tokio::time::sleep(Duration::from_millis(1)).await;
544         let end_time = Instant::now();
545         let delta = end_time - start_time;
546         let delta_ms = delta.as_millis();
547         if delta_ms > 10 {
548             warn!("Took {}ms to sleep targeted 1ms", delta_ms);
549         }
550     }
551     TryRecvError::Disconnected => {
552         // TODO handle this
553         panic!("Upstream passer disconnected");
554     }
555 }
556 }
557 }
558
559 let mut do_send = false;
560 if current_buffer_size > force_send_buff_size {
561     debug!("Sending on due to buffer size {} being greater than {}", current_buffer_size,
force_send_buff_size);
562     do_send = true;
563 }
564
565 if last_upstream_time.elapsed().as_millis() > mode_time
566    && current_buffer_size > 0
567 {
568     debug!("Sending on due to modetime");
569     do_send = true;
570 }
571
572 // If we're above buff size
573 if do_send {
574     // Wipe everything in the blue channel from the data - it can't be written so no point
sending
575     loop {
576         let socket_id = match blue_termination_receive.try_recv() {
577             Ok(socket_id) => socket_id,
578             Err(_) => break,
579         };
580
581         // Do not remove from socks sockets: We still might want to receive from it
582
583         // Instead, just remove it from the send buffer. It's ok if we remove a red terminate
- the writer's already dead
584         socks_sockets.retain(|x| x.socket_id != socket_id);
585     }
586
587     // Send the buffer to the push handler
588     push_passer_send.send_async(socks_sockets.clone()).await.unwrap();
589
590     // Eliminate socks sockets with terminate flag set
591     socks_sockets.retain(|x| !x.red_terminate); // This is *elegant*
592
593     // Reset buffer
594     for socks_socket in socks_sockets.iter_mut() {
595         socks_socket.payload.clear(); // Avoids reallocations this way, but causes
unnecessary memory usage
596     }
597
598     // Reset buffer size
599     current_buffer_size = 0;

```

```
600
601      // Reset last upstream time
602      last_upstream_time = Instant::now();
603  }
604  }
605 }
```

server-monolith::new_transit.rs

```

1 // new_transit.rs
2 use libtransit::{
3     DeclarationToken, DownStreamMessage, SocketID,
4     SocksSocketDownstream, SocksSocketUpstream,
5 };
6 use std::time::{Duration, Instant};
7 use tokio::sync::mpsc;
8
9 use tokio::io::{AsyncReadExt, AsyncWriteExt, Interest};
10 use tokio::net::TcpStream;
11
12 use libsecrets::EncryptionKey;
13
14 use log::{debug, error, info};
15
16 use std::sync::atomic::AtomicU32;
17 use std::sync::atomic::AtomicBool;
18 use std::sync::atomic::Ordering;
19 use std::sync::Arc;
20
21 use std::collections::HashMap;
22
23 type SequenceNumber = u32;
24
25 pub struct SessionActorsStorage {
26     pub to_bundler_stream: mpsc::UnboundedSender<SocksSocketUpstream>,
27     pub from_bundler_stream: flume::Receiver<(Vec<SocksSocketDownstream>, SequenceNumber)>,
28     pub key: EncryptionKey,
29     pub next_seq_num_up: AtomicU32,
30     pub declaration_token: DeclarationToken,
31     pub to_coordinator_yellow: mpsc::UnboundedSender<SocketID>,
32 }
33
34 pub async fn create_actor(key: &EncryptionKey, token: DeclarationToken) -> SessionActorsStorage {
35     // Create the channel
36     let (to_bundler_stream, from_http_stream) = mpsc::unbounded_channel();
37     let (to_http_stream, from_bundler_stream) = flume::unbounded();
38     let (to_coordinator_yellow, from_coordinator_yellow) = mpsc::unbounded_channel();
39
40     // Create the storage
41     let storage = SessionActorsStorage {
42         to_bundler_stream,
43         from_bundler_stream,
44         key: key.clone(),
45         next_seq_num_up: AtomicU32::new(0),
46         declaration_token: token,
47         to_coordinator_yellow,
48     };
49
50     // Start the handler
51     tokio::spawn(handle_session(
52         from_http_stream,
53         to_http_stream,
54         from_coordinator_yellow,
55         AtomicU32::new(0),
56     ));
57
58     // Return the storage
59     storage
60 }

```

```

61
62 pub async fn handle_session(
63     mut from_http_stream: mpsc::UnboundedReceiver<SocksSocketUpstream>,
64     to_http_stream: flume::Sender<(Vec<SocksSocketDownstream>, SequenceNumber)>,
65     mut from_coordinator_yellow: mpsc::UnboundedReceiver<SocketID>,
66     seq_num_down: AtomicU32,
67 ) {
68     let mut stream_to_tcp_handlers: HashMap<SocketID, mpsc::UnboundedSender<SocksSocketUpstream>> =
69         HashMap::new();
70     let mut stream_from_tcp_handlers: HashMap<
71         SocketID,
72         mpsc::UnboundedReceiver<DownStreamMessage>,
73     > = HashMap::new();
74
75     let (blue_termination_sender, mut blue_termination_receiver) = mpsc::unbounded_channel();
76
77     let mut yellow_informers: HashMap<SocketID, Arc<AtomicBool>> = HashMap::new();
78
79     let mut last_return_time: Instant = Instant::now();
80     let mut buffer_size = 0;
81     let mut downstream_sockets: Vec<SocksSocketDownstream> = Vec::new();
82
83     let mut last_iteration_time: Instant = Instant::now();
84
85     loop {
86         // Check how long the loop took
87         let loop_time = last_iteration_time.elapsed();
88         let loop_time_ms = loop_time.as_millis();
89         if loop_time_ms > 15 {
90             // TODO handle this better
91             info!("Coordinator took {}ms", loop_time_ms);
92         }
93         last_iteration_time = Instant::now();
94
95         let mut no_received_http = true;
96         let mut no_received_tcp = false;
97
98         // Handle inbound yellow messages
99         loop {
100             match from_coordinator_yellow.try_recv() {
101                 Ok(socket_id) => {
102                     // Remove from the map
103                     stream_to_tcp_handlers.remove(&socket_id);
104
105                     // Inform it to close ASAP
106                     let informer = yellow_informers.get(&socket_id).unwrap();
107
108                     informer.store(true, Ordering::Relaxed);
109
110                     // Remove from the map
111                     yellow_informers.remove(&socket_id);
112                 }
113                 Err(e) => {
114                     // Check if the channel is empty
115                     if e == mpsc::error::TryRecvError::Empty {
116                         break;
117                     } else {
118                         // Panic
119                         panic!("Error receiving yellow messages from HTTP stream: {:?}", e);
120                     }
121                 }
122             }

```

```

123     }
124
125     // TCP to HTTP messages
126     for downstream_socket in &mut downstream_sockets {
127         // Get the TCP handler
128         stream_from_tcp_handlers
129         .entry(downstream_socket.socket_id)
130         .and_modify(|tcp_handler| {
131             // Try to receive from the TCP handler
132             match tcp_handler.try_recv() {
133                 Ok(message) => {
134                     // Add to the buffer
135                     let payload_length = message.payload.len();
136                     buffer_size += payload_length;
137
138                     if message.do_green_terminate {
139                         downstream_socket.do_green_terminate = true; // It'll be removed
140                         later when we send the buffer back
141                     } else {
142                         downstream_socket.payload.extend(message.payload);
143                     }
144
145                     no_received_http = false;
146                 }
147                 Err(e) => {
148                     if e == tokio::sync::mpsc::error::TryRecvError::Empty {
149                         return;
150                     } else if e == tokio::sync::mpsc::error::TryRecvError::Disconnected {
151                         // Terminate the socket
152                         downstream_socket.do_green_terminate = true;
153                     }
154                 }
155             }
156         });
157     }
158
159     // Set do_blue_terminate for each socket based on ingress from the blue channel
160     loop {
161         match blue_termination_receiver.try_recv() {
162             Ok(socket_id) => {
163                 // Get the downstream socket
164                 let downstream_socket = downstream_sockets
165                     .iter_mut()
166                     .find(|socket| socket.socket_id == socket_id)
167                     .unwrap();
168
169                 downstream_socket.do_blue_terminate = true;
170
171                 // Remove from tcp_to_http handler
172                 stream_from_tcp_handlers.remove(&socket_id);
173             }
174             Err(e) => {
175                 // Check if the channel is empty
176                 if e == mpsc::error::TryRecvError::Empty {
177                     break;
178                 } else {
179                     // Panic
180                     panic!("Error receiving blue messages from TCP handler: {:?}", e);
181                 }
182             }
183         }
184     }

```

```

184
185 // Check if we should return the buffer
186 let should_return = if buffer_size > 128 * 1024 {
187     debug!("Returning; Buffer size is {}", buffer_size);
188     true
189 } else if last_return_time.elapsed() > Duration::from_millis(10) {
190     if buffer_size > 0 {
191         debug!("Returning; Time since last return is {:?}", last_return_time.elapsed());
192
193         last_return_time = Instant::now();
194
195         true
196     } else {
197         last_return_time = Instant::now();
198
199         false
200     }
201 } else {
202     false
203 };
204
205 if should_return {
206     to_http_stream.send((downstream_sockets.clone(), seq_num_down.fetch_add(1,
Ordering::SeqCst))).unwrap();
207
208     downstream_sockets.retain(|socket| !socket.do_green_terminate);
209
210     // Reset the buffer
211     for socket in &mut downstream_sockets { // NOTE: TODO: This will memory leak in a long
enough period unless we have a timeout for unused sockets
212         socket.payload.clear();
213     }
214
215     buffer_size = 0;
216
217     // Update the last return time
218     last_return_time = Instant::now();
219 }
220
221 // HTTP to TCP messages
222 match from_http_stream.try_recv() {
223     Ok(message) => {
224         // Get the socket ID
225         let socket_id = message.socket_id;
226
227         // Get the TCP handler
228         let tcp_handler = stream_to_tcp_handlers.get(&socket_id);
229
230         let tcp_handler = if tcp_handler.is_none() {
231             // Create a new TCP handler
232             // For sending upstreams to the handle_tcp
233             let (to_tcp_handler, from_http_stream) = mpsc::unbounded_channel();
234             // For receiving downstreams from the handle_tcp
235             let (to_http_stream, from_tcp_handler) = mpsc::unbounded_channel();
236
237             // Add the TCP handler to the map
238             stream_to_tcp_handlers.insert(socket_id, to_tcp_handler);
239             stream_from_tcp_handlers.insert(socket_id, from_tcp_handler);
240
241             // Add downstream socket
242             let downstream_socket = SocksSocketDownstream {
243                 socket_id,

```

```

244         dest_ip: message.dest_ip,
245         dest_port: message.dest_port,
246         payload: Vec::with_capacity(512),
247         do_green_terminate: false,
248         do_blue_terminate: false,
249     };
250
251     downstream_sockets.push(downstream_socket);
252
253     let yellow_informer = Arc::new(AtomicBool::new(false));
254
255     // Add the informer to the map
256     yellow_informers.insert(socket_id, yellow_informer.clone());
257
258     // Spawn the TCP handler
259     tokio::spawn(handle_tcp(
260         from_http_stream,
261         to_http_stream,
262         message.dest_ip,
263         message.dest_port,
264         message.socket_id,
265         yellow_informer,
266         blue_termination_sender.clone()
267     ));
268
269     // Get the TCP handler
270     let tcp_handler = stream_to_tcp_handlers.get(&socket_id).unwrap();
271     tcp_handler
272 } else {
273     tcp_handler.unwrap()
274 };
275
276 let do_red_terminate = message.red_terminate;
277
278 // Send the payload to the TCP handler
279 match tcp_handler.send(message) {
280     Ok(_) => {
281         // Just let it pass through
282     }
283     Err(e) => {
284         info!("Error sending to TCP handler: {:?}", e);
285
286         // There isn't really much we can do - the uploader is gone and should have
287         already sent down a terminate message
288
289         // Just continue
290     }
291 }
292
293 // Check if the message says to terminate
294 if do_red_terminate {
295     // Remove the HTTP-to-TCP for this socket
296     stream_to_tcp_handlers.remove(&socket_id);
297
298     // Everything else should be handled in the sent-on-message having the terminate
299     // flag in the handler
300 }
301 }
302 Err(e) => {
303     // Check if the channel is empty
304     if e == mpsc::error::TryRecvError::Empty {
305         no_received_tcp = true;

```

```

304         } else {
305             // Panic
306             panic!("Error receiving from HTTP stream: {:?}", e);
307         }
308     }
309 }
310
311 if no_received_http && no_received_tcp {
312     // Sleep for 1ms
313     tokio::time::sleep(Duration::from_millis(1)).await;
314 }
315 }
316 }
317
318 // TODO
319 fn send_down_blue_channel(blue_message_sender: mpsc::UnboundedSender<SocketID>, socket_id: SocketID)
320 {
321     debug!("Socket {} is closing on blue route", socket_id);
322
323     // Send the message
324     blue_message_sender.send(socket_id).unwrap();
325 }
326
327 async fn handle_tcp_up(
328     mut from_http_stream: mpsc::UnboundedReceiver<SocksSocketUpstream>,
329     mut tcp_write_half: tokio::net::tcp::OwnedWriteHalf,
330     socket_id: SocketID,
331     blue_message_sender: mpsc::UnboundedSender<SocketID>,
332 ) {
333     loop {
334         let ready = tcp_write_half.ready(Interest::WRITABLE).await;
335
336         if ready.is_err() {
337             panic!("Error getting ready state: {:?}", ready);
338         }
339
340         let ready = ready.unwrap();
341
342         let go_ahead = if ready.is_writable() {
343             true
344         } else if ready.is_empty() {
345             false
346         } else if ready.is_write_closed() {
347             // Send and close
348             send_down_blue_channel(blue_message_sender, socket_id);
349             return;
350         } else if ready.is_readable() {
351             panic!("Asking if something is writable returned 'readable' ready state!?!");
352         } else if ready.is_read_closed() {
353             panic!("Asking if something is writable returned 'read closed' ready state!?!");
354         } else {
355             panic!("Unknown ready state: {:?}", ready);
356         }
357     };
358
359     if go_ahead {
360         // We're using an asynchronous wait here to avoid the old hot-loop system
361         let message = match from_http_stream.recv().await {
362             Some(message) => message,
363             None => {
364                 // We need to close the socket
365                 send_down_blue_channel(blue_message_sender, socket_id);
366                 return;
367             }
368         };
369     }
370 }

```

```

365         }
366     };
367
368     // Write payload
369     tcp_write_half.write_all(&message.payload).await.unwrap();
370
371     debug!("Written to the stream");
372
373     // Check if we should terminate
374     if message.red_terminate {
375         // Simply return
376         return;
377     }
378 }
379 }
380 }
381
382 fn send_green_terminate(to_http_stream: mpsc::UnboundedSender<DownStreamMessage>, socket_id:
SocketID) {
383     debug!("Socket {} is closing on green route", socket_id);
384
385     let downstream_msg = DownStreamMessage {
386         socket_id,
387         payload: Vec::with_capacity(0),
388         do_green_terminate: true,
389     };
390
391     // Send the message
392     match to_http_stream.send(downstream_msg) {
393         Ok(_) => {}
394         Err(e) => {
395             // TODO handle this better
396             error!("Error sending green terminate: {:?}", e);
397         }
398     }
399 }
400
401 async fn handle_tcp_down(
402     to_http_stream: mpsc::UnboundedSender<DownStreamMessage>,
403     mut tcp_read_half: tokio::net::tcp::OwnedReadHalf,
404     socket_id: SocketID,
405     yellow_informer: Arc<AtomicBool>,
406 ) {
407     loop {
408         if yellow_informer.load(Ordering::Relaxed) {
409             debug!("Socket {} is yellow, closing", socket_id);
410             return;
411         }
412
413         let ready = tcp_read_half.ready(Interest::READABLE).await;
414
415         if yellow_informer.load(Ordering::Relaxed) {
416             debug!("Socket {} is yellow, closing", socket_id);
417             return;
418         }
419
420         if ready.is_err() {
421             // TODO handle this better
422             panic!("Error getting ready state: {:?}", ready);
423         }
424
425         let ready = ready.unwrap();

```

```

426
427     let go_ahead = if ready.is_readable() {
428         true
429     } else if ready.is_empty() {
430         false
431     } else if ready.is_read_closed() {
432         // Send down green channel, then return
433         send_green_terminate(to_http_stream, socket_id);
434         return;
435     } else if ready.is_writable() {
436         panic!("Asking if something is readable returned 'writable' ready state!?!?");
437     } else if ready.is_write_closed() {
438         panic!("Asking if something is readable returned 'write closed' ready state!?!?");
439     } else {
440         panic!("Unknown ready state: {:?}", ready);
441     };
442
443     if go_ahead {
444         let mut downstream_msg = DownStreamMessage {
445             socket_id,
446             payload: vec![0; 512], // TODO make this configurable
447             do_green_terminate: false,
448         };
449
450         // Write in directly for efficiency
451         // We're using an asynchronous wait here to avoid the old hot-loop system
452         let bytes_read = match tcp_read_half.read(&mut downstream_msg.payload).await {
453             Ok(bytes_read) => bytes_read,
454             Err(e) => {
455                 debug!("Sending green terminate due to error: {:?}", e);
456                 send_green_terminate(to_http_stream, socket_id);
457                 return;
458             }
459         };
460
461         if yellow_informer.load(Ordering::Relaxed) {
462             debug!("Socket {} is yellow, closing", socket_id);
463             return;
464         }
465
466         if bytes_read == 0 {
467             // Socket is closing. Send back a terminate message then close
468             send_green_terminate(to_http_stream, socket_id);
469             return;
470         }
471
472         // Trim array down to size
473         downstream_msg.payload.truncate(bytes_read);
474
475         // Send back
476         to_http_stream.send(downstream_msg).unwrap();
477     }
478 }
479 }
480
481 pub async fn handle_tcp(
482     from_http_stream: mpsc::UnboundedReceiver<SocksSocketUpstream>,
483     to_http_stream: mpsc::UnboundedSender<DownStreamMessage>,
484     ip: u32,
485     port: u16,
486     socket_id: SocketID,
487     yellow_informer: Arc<AtomicBool>,

```

```
488     handle_blue_sender: mpsc::UnboundedSender<SocketID>,
489 ) {
490     let stream = TcpStream::connect(format!("{}", ip, port)).await;
491     let stream = match stream {
492         Ok(stream) => stream,
493         Err(e) => {
494             // Just close
495             debug!("Error connecting to {}:{} (Error: {:?})", ip, port, e);
496             send_green_terminate(to_http_stream, socket_id);
497
498             return;
499         }
500     };
501
502     // Split the stream
503     let (tcp_read_half, tcp_write_half) = stream.into_split();
504
505     // Spawn the TCP handlers
506     tokio::spawn(handle_tcp_up(
507         from_http_stream,
508         tcp_write_half,
509         socket_id,
510         handle_blue_sender
511     ));
512
513     tokio::spawn(handle_tcp_down(
514         to_http_stream,
515         tcp_read_half,
516         socket_id,
517         yellow_informer,
518     ));
519 }
```

server-monolith::main.rs

```

1 // main.rs
2 use actix_web::web::Bytes;
3 use actix_web::{get, post, web, App, HttpRequest, HttpResponse, HttpServer, Responder};
4 use libsecrets::EncryptionKey;
5 use rand::Rng;
6 use libtransit::SerialMessage;
7
8 use log::{debug, info, warn};
9
10 use dashmap::DashMap;
11
12 mod config;
13 mod new_transit;
14 use new_transit::SessionActorsStorage;
15
16 use scan_fmt::scan_fmt;
17
18 use libtransit::{
19     ClientMessageUpstream, DeclarationToken, ServerMessageDownstream,
20     ServerMetaDownstream, UnifiedPacketInfo,
21 };
22
23 use std::time::{Duration, SystemTime, UNIX_EPOCH};
24
25 use std::sync::atomic::{AtomicU32, Ordering};
26
27 // State passed to all request handlers
28 struct AppState {
29     users: Vec<User>, // Users from the configuration with the passwords preprocessed into keys
30     // for faster initial handshake when there are many users,
31     actor_lookup: DashMap<DeclarationToken, SessionActorsStorage>, // Lookup table for actors
32 }
33
34 // Simple user definition
35 struct User {
36     name: String,
37     key: EncryptionKey,
38 }
39
40 async fn form_meta_response(seq_num: u32) -> ServerMetaDownstream {
41     let millis_time = SystemTime::now().duration_since(UNIX_EPOCH).expect("Time went
42     backwards").as_millis();
43
44     ServerMetaDownstream {
45         packet_info: UnifiedPacketInfo { unix_ms: millis_time as u64, seq_num },
46     }
47 }
48
49 fn parse_token(token_hex: String) -> Option<DeclarationToken> {
50     debug!("Hex token: {}", token_hex);
51
52     // Convert to bytes
53     let token_bytes = hex::decode(token_hex).unwrap();
54
55     // Check it's the correct length
56     if token_bytes.len() != 16 {
57         // Return 404
58         debug!("Token is not 16 bytes!");
59         return None;
60     }
61 }

```

```

60
61 // Convert to array
62 let token: DeclarationToken = token_bytes[..16].try_into().unwrap();
63
64 debug!("Token correct!: {:?}", token);
65
66 // Implicit return
67 Some(token)
68 }
69
70 // Temporary - used to check the server works
71 #[get("/")]
72 async fn index() -> impl Responder {
73     HttpResponse::Ok().body("Hello world!")
74 }
75
76 static DOWNSTREAM_SEQ_NUM: AtomicU32 = AtomicU32::new(0);
77
78 #[get("/video/segment")]
79 async fn downstream_data(
80     app_state: web::Data<AppState>,
81     req: HttpRequest,
82     body_bytes: Bytes,
83 ) -> impl Responder {
84     let seq_num = DOWNSTREAM_SEQ_NUM.fetch_add(1, Ordering::SeqCst);
85     info!("{}", Received request to download", seq_num);
86
87     // Get token
88     let token = req.cookie("token");
89
90     let token = match token {
91         Some(token) => {
92             match parse_token(token.value().to_string()) {
93                 Some(token) => token,
94                 None => {
95                     // Return 404 - resist active probing by not telling the client what went wrong
96                     warn!("Received token {} is invalid!", token.value());
97                     return HttpResponse::NotFound().body("No page exists");
98                 }
99             }
100         }
101         None => {
102             // Return 404 - resist active probing by not telling the client what went wrong
103             warn!("No token supplied in download req!");
104             return HttpResponse::NotFound().body("No page exists");
105         }
106     };
107
108     info!("{}", getting dashmap; may deadlock", seq_num);
109
110     // Get key
111     let (key, from_bundler_stream) = {
112         let actor = app_state.actor_lookup.get(&token);
113
114         let actor = match actor {
115             Some(actor) => actor,
116             None => {
117                 // Return 404 - resist active probing by not telling the client what went wrong
118                 warn!("No session found for token with hex representation {} in download!",
119 hex::encode(token));
120                 return HttpResponse::NotFound().body("No page exists");
121             }
122         }

```

```

121     };
122
123     (actor.key.clone(), actor.from_bundler_stream.clone())
124 };
125
126 info!("{}", data_extracted", seq_num);
127
128 // Get body
129 let body = body_bytes;
130
131 // Decrypt body
132 let mut decrypted = match libsecrets::decrypt(&body, &key) {
133     Ok(decrypted) => decrypted, // If it succeeds, pull out content from Result<>
134     Err(_) => {
135         // Return 404 - resist active probing by not telling the client what went wrong
136         warn!("Failed to decrypt download message body; returning 404");
137         return HttpResponse::NotFound().body("No page exists");
138     }
139 };
140
141 info!("{}", decrypted body", seq_num);
142
143 // Parse into ClientMessageUpstream. This is just about checking for validity really
144 let _upstream: ClientMessageUpstream =
145     match libtransit::ClientMessageUpstream::decode_from_bytes(&mut decrypted) {
146         Ok(upstream) => upstream, // If it succeeds, pull out content from Result<>
147         Err(_) => {
148             // Return 404 - resist active probing by not telling the client what went wrong
149             warn!("Failed to parse decrypted download body into ClientMessageUpstream; returning
150 404");
151             return HttpResponse::NotFound().body("No page exists");
152         }
153     };
154
155 info!("{}", parsed into ClientMessageUpstream", seq_num);
156
157 // Don't send on - just get
158 let (return_messages, seq_num) = {
159     let responded_data = from_bundler_stream.recv_async().await;
160
161     match responded_data {
162         Ok(responded_data) => responded_data,
163         Err(_) => {
164             // Return 404 - resist active probing by not telling the client what went wrong
165             warn!("Failed to receive messages from actor in download; returning 404");
166             return HttpResponse::NotFound().body("No page exists");
167         }
168     }
169 };
170
171 // Get meta information
172 let meta = form_meta_response(seq_num).await;
173
174 info!("{}", Formed meta: {:?}:", seq_num, meta);
175
176 // Form response
177 let response = ServerMessageDownstream {
178     metadata: meta,
179     socks_sockets: return_messages,
180     payload_size: 0,
181 };

```

```

182     info!("{}", Formed response", seq_num);
183
184     // Encode response
185     let response_bytes = response.encoded().unwrap(); // TODO handle error
186
187     info!("{}", Encoded response", seq_num);
188
189     // Encrypt response
190     let encrypted = libsecrets::encrypt(&response_bytes, &key).unwrap(); // TODO handle error
191
192     info!("{}", Encrypted response", seq_num);
193
194     // Return response
195     HttpResponse::Ok().body(encrypted)
196 }
197
198 #[post("/upload/segment/{tail:.*}")]
199 async fn upstream_data(
200     app_state: web::Data<AppState>,
201     req: HttpRequest,
202     body_bytes: Bytes,
203     _segment_name: web::Path<String>,
204 ) -> impl Responder {
205     debug!("Received request to upload");
206
207     // Get token
208     let token = req.cookie("token");
209
210     let token = match token {
211         Some(token) => {
212             match parse_token(token.value().to_string()) {
213                 Some(token) => token,
214                 None => {
215                     // Return 404 - resist active probing by not telling the client what went wrong
216                     warn!("Received token {} is invalid!", token.value());
217                     return HttpResponse::NotFound().body("No page exists");
218                 }
219             }
220         }
221         None => {
222             // Return 404 - resist active probing by not telling the client what went wrong
223             warn!("No token supplied in upload req!");
224             return HttpResponse::NotFound().body("No page exists");
225         }
226     };
227
228     // Get actor
229     let actor = app_state.actor_lookup.get(&token);
230     if actor.is_none() {
231         // Return 404 - resist active probing by not telling the client what went wrong
232         warn!("No session found for token with hex representation {} in upload!",
hex::encode(token));
233         return HttpResponse::NotFound().body("No page exists");
234     }
235
236     let actor = actor.unwrap();
237
238     // Get body
239     let body = body_bytes;
240
241     // Decrypt body
242     let key = actor.key;

```

```

243
244 let mut decrypted = match libsecrets::decrypt(&body, &key) {
245     Ok(decrypted) => decrypted, // If it succeeds, pull out content from Result<>
246     Err(_) => {
247         // Return 404 - resist active probing by not telling the client what went wrong
248         warn!("Failed to decrypt upload message body; returning 404");
249         return HttpResponse::NotFound().body("No page exists");
250     }
251 };
252
253 // Parse into ClientMessageUpstream
254 let upstream: ClientMessageUpstream =
255     match libtransit::ClientMessageUpstream::decode_from_bytes(&mut decrypted) {
256         Ok(upstream) => upstream, // If it succeeds, pull out content from Result<>
257         Err(_) => {
258             // Return 404 - resist active probing by not telling the client what went wrong
259             warn!("Failed to parse decrypted upload body into ClientMessageUpstream; returning
404");
260             return HttpResponse::NotFound().body("No page exists");
261         }
262     };
263
264 // Get sequence number
265 let seq_num = upstream.metadata.packet_info.seq_num;
266
267 // Get yellow sockets to close
268 let yellow_sockets = upstream.metadata.yellow_to_stop_reading_from;
269
270 while actor.next_seq_num_up.load(Ordering::SeqCst) != seq_num {
271     // Wait until that's the case
272     // Sleep 1ms
273     tokio::time::sleep(Duration::from_millis(1)).await;
274 }
275
276 // Send on to actor
277 debug!("Sending on messages to actor...");
278 {
279     let to_bundler = &actor.to_bundler_stream;
280
281     for socket in upstream.socks_sockets {
282         to_bundler.send(socket).unwrap();
283     }
284
285     // Send yellow messages to mspc
286     for yellow_socket in yellow_sockets {
287         let _ = &actor.to_coordinator_yellow.send(yellow_socket).unwrap();
288     }
289
290     actor.next_seq_num_up.store(seq_num + 1, Ordering::SeqCst);
291 }
292
293 debug!("Sent on messages to actor");
294
295 // Get meta information
296 let meta = form_meta_response(0).await;
297
298 debug!("Formed meta: {:?}" , meta);
299
300 // Form response
301 let response = ServerMessageDownstream {
302     socks_sockets: Vec::with_capacity(0),
303     metadata: meta,

```

```

304     payload_size: 0,
305 };
306
307 debug!("Formed response: {:?}", response);
308
309 // Encode response
310 let response_bytes = response.encoded().unwrap(); // TODO handle error
311 debug!("Encoded response");
312
313 // Encrypt response
314 let encrypted = libsecrets::encrypt(&response_bytes, &key).unwrap(); // TODO handle error
315
316 debug!("Encrypted response");
317
318 // Return response
319 HttpResponse::Ok().body(encrypted)
320 }
321
322 // First request, which associates a client identifier with a key and proves the server is legitimate
323 #[post("/submit")]
324 async fn client_greeting(
325     app_state: web::Data<AppState>,
326     req: HttpRequest,
327     body_bytes: Bytes,
328 ) -> impl Responder {
329     info!("Received request to /submit (client hello)");
330
331     // Get token
332     let token = req.cookie("token");
333
334     let token = match token {
335         Some(token) => {
336             match parse_token(token.value().to_string()) {
337                 Some(token) => token,
338                 None => {
339                     // Return 404 - resist active probing by not telling the client what went wrong
340                     warn!("Received token {} is invalid for client hello!", token.value());
341                     return HttpResponse::NotFound().body("No page exists");
342                 }
343             }
344         }
345         None => {
346             // Return 404 - resist active probing by not telling the client what went wrong
347             warn!("No token supplied in client hello req!");
348             return HttpResponse::NotFound().body("No page exists");
349         }
350     };
351
352     // Get body
353     let body = body_bytes.to_vec();
354
355     // Iterate through users trying their keys. TODO consider Rayon here for performance
356     let mut key: Option<EncryptionKey> = None;
357     for user in &app_state.users {
358         // Attempt to decrypt body with user's key
359         let decrypted = match libsecrets::decrypt(&body, &user.key) {
360             Ok(decrypted) => decrypted, // If it succeeds, pull out content from Result<>
361             Err(_) => continue,         // If it fails, try the next user
362         };
363
364         // Convert to string. It should always be valid ASCII, and UTF-8 is a superset of ASCII
365         let decrypted = match String::from_utf8(decrypted) {

```

```

366         Ok(decrypted) => decrypted,
367         Err(_) => continue,
368     };
369
370     // Check if decrypted data contains "Hello!"
371     if decrypted.contains(&"Hello. Protocol version: 2".to_string()) {
372         // Set key
373         key = Some(user.key.clone()); // We've found it!
374         info!("Key found! User: {}", user.name);
375         info!("Decrypted data: {}", decrypted);
376
377         // Parse out the properties
378         let (protocol_version, client_transit_version, client_name) = match scan_fmt!(
379             &decrypted,
380             "Hello. Protocol version: {}, client-transit version: {}, client-name: {}",
381             String,
382             String,
383             String
384         ) {
385             Ok((protocol_version, client_transit_version, client_name)) => {
386                 (protocol_version, client_transit_version, client_name)
387             }
388             Err(e) => {
389                 // Return 404
390                 warn!("Failed to parse client hello! Error {:?}", e);
391                 return HttpResponse::NotFound().body("No page exists");
392             }
393         };
394
395         println!("Protocol version: {}", protocol_version);
396         println!("Client transit version: {}", client_transit_version);
397         println!("Client name: {}", client_name);
398
399         break;
400     } else {
401         warn!("Data decrypted, but does not contain client hello!");
402     }
403 }
404
405 // Check if key was found. If so, extract contents of Option<>
406 let key = match key {
407     Some(key) => key,
408     None => {
409         // Return 404
410         warn!("No key found in client hello!");
411         return HttpResponse::NotFound().body("No page exists");
412     }
413 };
414
415 // Generate response. Encrypted with their key to show we're legitimate
416 let response_text = "CONNECTION ACCEPTED";
417
418 // Pad with 0-100 spaces to avoid fingerprinting based on response length
419 let mut rng = rand::thread_rng();
420 let amount = rng.gen_range(0..100);
421 let response_text = response_text.to_string() + &" ".repeat(amount);
422
423 // Encrypt response
424 let encrypted = match libsecrets::encrypt(response_text.as_bytes(), &key) {
425     Ok(encrypted) => encrypted,
426     Err(_) => {
427         // This should never happen, but if it does, return 404

```

```

428         // Return 404
429         return HttpResponse::NotFound().body("No page exists");
430     }
431 };
432
433 debug!("Formed response: {:?}", encrypted);
434
435 // Create actor
436 let actor_storage = new_transit::create_actor(&key, token).await;
437 // Add to sessions
438 app_state.actor_lookup.insert(token, actor_storage);
439
440 debug!("Added session to sessions; returning response");
441
442 info!("Client hello successful!");
443
444 // Return encrypted response
445 HttpResponse::Ok().body(encrypted)
446 }
447
448 #[actix_web::main]
449 async fn main() -> std::io::Result<> {
450     simple_logger::set_up_color_terminal();
451     simple_logger::init_with_env().unwrap();
452
453     info!("Loading configuration...");
454     let configuration = config::load_config();
455
456     // Create users from configuration
457     // Check if there are any users
458     if configuration.users.is_empty() {
459         panic!("No users defined in configuration file!");
460     }
461
462     info!("Hashing user keys...");
463     let mut users = Vec::new();
464     for user in &configuration.users {
465         // Check for malformed config
466         let mut duplicate_user_before = false;
467         let mut duplicate_password_before = false;
468
469         for other_user in &configuration.users {
470             if user.name == other_user.name {
471                 if duplicate_user_before {
472                     panic!("Duplicate user '{}'", user.name);
473                 }
474                 duplicate_user_before = true;
475             }
476             if user.password == other_user.password && user.name != other_user.name {
477                 if duplicate_password_before {
478                     panic!("Duplicate password '{}'", user.password);
479                 }
480                 duplicate_password_before = true;
481             }
482         }
483
484         let key = libsecrets::form_key(user.password.as_bytes());
485
486         users.push(User {
487             name: user.name.clone(),
488             key,
489         });

```

```
490
491     info!("Derived key for user '{}'", user.name);
492 }
493
494 let appstate = web::Data::new(AppState {
495     users,
496     actor_lookup: DashMap::new()
497 });
498
499 info!(
500     "Starting server at {}:{} with {} workers",
501     configuration.host, configuration.port, configuration.workers
502 );
503
504 HttpServer::new(move || {
505     // Closure - inline function. Move keyword moves ownership of configuration into the closure
506     App::new()
507         .app_data(appstate.clone()) // Insert appdata
508         .service(index) // Insert index route
509         .service(client_greeting) // Insert client_greeting route
510         .service(upstream_data)
511         .service(downstream_data)
512 })
513 .workers(configuration.workers) // Set number of workers
514 .bind((configuration.host, configuration.port))? // Bind to host and port
515 .run() // Execute
516 .await // Wait for completion with the asynchronous runtime
```

client-core::lib.rs

```

1 // lib.rs
2 use libsocks;
3 use libtransit::{UpStreamMessage, SocketID, SocksSocketDownstream};
4 use tokio::net::{TcpListener, TcpStream};
5 #[allow(unused)] // Not unused - the warning is wrong
6 use tokio::sync::mpsc::{self, Receiver, Sender, UnboundedReceiver, UnboundedSender};
7 use tokio::sync::mpsc::error::TryRecvError;
8 use std::sync::atomic::{AtomicU32, Ordering};
9 use tokio::task;
10 use tokio::sync::broadcast::{self, Sender as BroadcastSender, Receiver as BroadcastReceiver};
11 use std::sync::Arc;
12 use tokio::io::{Interest, AsyncWriteExt, AsyncReadExt};
13 use tokio::sync::RwLock;
14
15 mod transit_builder;
16 use transit_builder::TransitSocketBuilder;
17 mod transit;
18 mod meta;
19 use meta::YELLOW_DATA_UPSTREAM_QUEUE;
20
21 #[allow(unused)]
22 use log::{debug, error, info, warn};
23
24 pub struct ClientArguments {
25     pub listen_address: String,
26     pub listen_port: u16,
27     pub target_host: String,
28     pub password: String,
29     pub push_client_count: usize,
30     pub pull_client_count: usize,
31     pub timeout_time_s: usize,
32     pub client_name: String,
33 }
34
35 // Core of the client program for now. See Obsidian
36 pub async fn begin_core_client(arguments: ClientArguments) {
37     let (upstream_passer_send, upstream_passer_receive): (
38         Sender<UpStreamMessage>,
39         Receiver<UpStreamMessage>,
40     ) = mpsc::channel(10_000);
41
42     let (message_passer_passer_send, _): (BroadcastSender<transit::DownstreamBackpasser>,
43 BroadcastReceiver<transit::DownstreamBackpasser>) = broadcast::channel(100_000);
44
45     let message_passer_passer_send = Arc::new(message_passer_passer_send);
46
47     let (blue_terminate_send, blue_terminate_receive): (flume::Sender<SocketID>,
48 flume::Receiver<SocketID>) = flume::unbounded();
49
50     // Cannot transfer threads
51     let transit_socket = TransitSocketBuilder::new()
52         .with_target(arguments.target_host)
53         .with_password(arguments.password)
54         .with_client_name(arguments.client_name)
55         .with_timeout_time(arguments.timeout_time_s)
56         .with_pull_client_count(arguments.pull_client_count)
57         .with_push_client_count(arguments.push_client_count)
58         .build();
59
60     let transit_socket = Arc::new(RwLock::new(transit_socket));

```

```

59
60     let status = transit::connect(transit_socket.clone()).await;
61
62     match status {
63         Ok(_) => {
64             info!("Connected to server!");
65         }
66         Err(error) => {
67             error!("Failed to connect to server: {:?}", error);
68             return;
69         }
70     }
71
72     // Now start the TCP listener in a task
73     let moved_passer_passer = message_passer_passer_send.clone();
74
75     let to_other_task = blue_terminate_receive.clone();
76     task::spawn(async move {
77         let listener = TcpListener::bind((arguments.listen_address,
arguments.listen_port)).await.expect("Failed to start TCP listener");
78
79         loop {
80             let (socket, _) = listener.accept().await.expect("Failed to accept connection");
81
82             // Spawn a task to handle the connection
83             task::spawn(tcp_listener(socket, upstream_passer_send.clone(),
moved_passer_passer.clone(), to_other_task.clone()));
84         }
85     });
86
87     transit::handle_transit(transit_socket, upstream_passer_receive,
message_passer_passer_send.clone(), blue_terminate_send, blue_terminate_receive).await;
88 }
89
90 #[allow(unused)]
91 async fn tcp_listener(stream: TcpStream, upstream_passer_send: Sender<UpStreamMessage>,
message_passer_passer_send: Arc<BroadcastSender<transit::DownstreamBackpasser>>,
blue_terminate_receive: flume::Receiver<SocketID>) {
92     // Read the first packet
93     // Wait for the socket to be readable
94     let mut buf = Vec::with_capacity(4096);
95
96     stream.readable().await.expect("Failed to wait for socket to be readable");
97
98     match stream.try_read_buf(&mut buf) {
99         Ok(bytes_read) => {
100             if bytes_read == 0 {
101                 // The socket was closed
102                 return;
103             }
104         },
105         Err(error) => {
106             info!("Failed to read from socket in initial SOCKS discussion: {:?}", error);
107             return;
108         }
109     }
110
111     // Try to parse the packet
112     let packet = libsocks::decode_socks_request(&buf);
113
114     let mut dstip: Option<libsocks::IPV4> = None;
115     let mut dstport: Option<libsocks::Port> = None;

```

```

116
117     let rejection = libsocks::Socks4ConnectReply {
118         version: 0, // yes, this is correct
119         status: libsocks::Socks4Status::Rejected,
120         dstport: 0,
121         dstip: 0,
122     };
123
124     let rejection_bytes = rejection.to_binary();
125
126     match packet {
127         Ok(packet) => {
128             // Check what kind of packet it is
129             match packet {
130                 libsocks::Socks4Request::Connect(connect_req) => {
131                     // Sanity check
132                     if connect_req.version != 4 {
133                         warn!("Incorrect socks version ({{}}) from program {{}}", connect_req.version,
connect_req.userid);
134
135                         stream.writable().await.unwrap();
136                         stream.try_write(&rejection_bytes).unwrap();
137                         return;
138                     }
139
140                     dstip = Some(connect_req.dstip);
141                     dstport = Some(connect_req.dstport);
142                 },
143                 libsocks::Socks4Request::Bind(bind_req) => {
144                     // We don't support bind yet
145                     let user_id = bind_req.userid;
146                     error!("Received bind request (unsupported) from user ID {{}}. Killing connection",
user_id);
147
148                     stream.writable().await.unwrap();
149                     stream.try_write(&rejection_bytes).unwrap();
150                     return;
151                 }
152             },
153             Err(error) => {
154                 error!("Failed to parse SOCKS request: {:?}", error);
155                 match stream.writable().await {
156                     Ok(_) => {
157                         let res = stream.try_write(&rejection_bytes);
158                         if res.is_err() {
159                             error!("Failed to send rejection to client: {:?}", res.err().unwrap());
160                         }
161                     },
162                     Err(error) => {
163                         error!("Failed to send rejection to client: {:?}", error);
164                     }
165                 };
166                 match stream.try_write(&rejection_bytes) {
167                     Ok(_) => {
168                         info!("Sent rejection to client");
169                     },
170                     Err(error) => {
171                         error!("Failed to send rejection to client: {:?}", error);
172                     }
173                 }
174             }
175         }
176     }
177     return;
178 }

```

```

176     }
177
178     let (dstip, dstport) = (dstip.unwrap(), dstport.unwrap());
179
180     // If we've gotten to this point the data's good. Let's compose a reply
181     let reply = libsocks::Socks4ConnectReply {
182         version: 0,
183         status: libsocks::Socks4Status::Granted,
184         dstport,
185         dstip,
186     };
187
188     stream.writable().await.unwrap();
189
190     match stream.try_write(&reply.to_binary()) {
191         Ok(_) => {
192             info!("Said hello to client on recv");
193         },
194         Err(error) => {
195             error!("Failed to send reply to client: {:?}", error);
196             return;
197         }
198     }
199
200     // It seems initialisation was a success. Let's get our socket ID
201     let socket_id = allocate_socket_id();
202
203     // Now we need to let transit know how to reply to this socket. First we create a message passer
204     let (downstream_passer_send, mut downstream_passer_receive):
205     (UnboundedSender<SocksSocketDownstream>, UnboundedReceiver<SocksSocketDownstream>) =
206     mpsc::unbounded_channel();
207
208     // Now we send the message passer to transit
209     let message = transit::DownstreamBackpasser {
210         socket_id,
211         sender: downstream_passer_send
212     };
213
214     message_passer_passer_send.send(message).expect("Failed to send message passer to transit");
215
216     let (mut read_half, mut write_half) = stream.into_split();
217
218     // Spawn both tasks
219     task::spawn(tcp_handler_up(read_half, socket_id, dstip, dstport, upstream_passer_send.clone(),
220     blue_terminate_receive.clone()));
221     task::spawn(tcp_handler_down(write_half, downstream_passer_receive, socket_id));
222 }
223
224 async fn tcp_handler_down(mut write_half: tokio::net::tcp::OwnedWriteHalf,
225     mut downstream_passer_receive: UnboundedReceiver<SocksSocketDownstream>,
226     socket_id: SocketID,
227 ) {
228     loop {
229         let ready = write_half.ready(Interest::WRITABLE).await.expect("Failed to wait for socket to
230         be ready");
231
232         if ready.is_writable() {
233             match downstream_passer_receive.recv().await {
234                 Some(data) => {
235                     // Transit has sent us data
236                     // Send it to the client
237                     let bytes = &data.payload;

```

```

234         let length = bytes.len();
235
236         if length == 0 {
237             // No point writing. Check that we're not meant to close, though.
238             if data.do_green_terminate {
239                 // We've been told to close
240                 debug!("Closing writer for id {} due to green terminate at point of zero
length", socket_id);
241                 return
242             }
243             continue
244         }
245
246         match write_half.try_write(bytes) {
247             Ok(_) => {
248                 // All is fine
249                 debug!("Sent {} bytes to client", length);
250             },
251             Err(error) => {
252                 // We can't write anymore. This should be propagated
253                 // up the yellow route, unless we've already been told to close.
254                 if data.do_green_terminate {
255                     // We've already been told to close
256                     debug!("Closing writer for id {} due to simultaneous error and green
terminate", socket_id);
257                     return
258                 } else {
259                     yellow_route_record_error(socket_id, error.to_string()).await;
260                     return;
261                 }
262             }
263         }
264
265         // Check if we've been told to close
266         if data.do_green_terminate {
267             // We've been told to close
268             debug!("Closing writer for id {} due to green terminate", socket_id);
269             return
270         }
271     },
272     None => {
273         // Message passer has disconnected due to green route closing
274         // Simply close this half of the socket
275         debug!("Closing writer for id {} due to no data on passer due to (likely) green
terminate on passer", socket_id);
276         write_half.shutdown().await.expect("Failed to shutdown socket at green route
passer close");
277
278         // Now leave
279         return
280     }
281 }
282 }
283 }
284 }
285
286 async fn yellow_route_record_error(socket_id: SocketID, reason: String) {
287     debug!("Yellow route error on socket id {} : {}", socket_id, reason);
288
289     YELLOW_DATA_UPSTREAM_QUEUE.write().await.push(socket_id);
290 }
291

```

```

292 async fn tcp_handler_up(mut read_half: tokio::net::tcp::OwnedReadHalf,
293     socket_id: SocketID,
294     dest_ip: libsocks::IPV4,
295     dest_port: libsocks::Port,
296     upstream_passer_send: Sender<UpStreamMessage>,
297     blue_terminate_receive: flume::Receiver<SocketID>
298 ) {
299     // Send the first msg in order to get the socket open
300     let upstream_msg = UpStreamMessage {
301         socket_id,
302         dest_ip,
303         dest_port,
304         payload: Vec::with_capacity(0),
305         red_terminate: false,
306     };
307
308     upstream_passer_send.send(upstream_msg).await.expect("Failed to send initialising socket open
data to transit");
309
310     loop {
311         let ready = read_half.ready(Interest::READABLE).await.expect("Failed to wait for socket to be
ready");
312
313         while let Ok(term_socket_id) = blue_terminate_receive.try_recv() {
314             if term_socket_id == socket_id {
315                 // We've been told to close
316                 debug!("Closing reader for id {} due to blue terminate", socket_id);
317                 return
318             }
319         }
320
321         if ready.is_readable() {
322             let mut upstream_msg = UpStreamMessage {
323                 socket_id,
324                 dest_ip,
325                 dest_port,
326                 payload: vec![0; 512], // TODO changeable, try to reuse buffers
327                 red_terminate: false,
328             };
329
330             let bytes_read = match read_half.read(&mut upstream_msg.payload).await {
331                 Ok(bytes_read) => bytes_read,
332                 Err(error) => {
333                     upstream_msg.red_terminate = true;
334                     upstream_msg.payload = Vec::with_capacity(0);
335                     upstream_passer_send.send(upstream_msg).await.expect("Failed to send data to
transit");
336                     debug!("Failed to read from socket: {:?}", error);
337                     return;
338                 }
339             };
340
341             if bytes_read == 0 {
342                 // The socket was closed
343                 upstream_msg.red_terminate = true;
344                 upstream_msg.payload = Vec::with_capacity(0);
345                 upstream_passer_send.send(upstream_msg).await.expect("Failed to send data to
transit");
346                 return
347             }
348
349             // Trim array to actual size

```

```
350         upstream_msg.payload.truncate(bytes_read);
351
352         upstream_passer_send.send(upstream_msg).await.expect("Failed to send data to transit");
353     }
354 }
355 }
356
357 static NEXT_SOCKET_ID: AtomicU32 = AtomicU32::new(0);
358
359 fn allocate_socket_id() -> libtransit::SocketID {
360     NEXT_SOCKET_ID.fetch_add(1, Ordering::SeqCst)
361 }
```

hypertunnel_gui::app.rs

```

1 // app.rs
2 use egui_extras;
3 use std::io::{BufRead, BufReader, Read, Write};
4 use std::process::{Stdio, Command};
5 use std::sync::RwLock;
6 use std::thread::sleep;
7 use std::io;
8 use std::sync::mpsc;
9
10 /// We derive Deserialize/Serialize so we can persist app state on shutdown.
11 #[derive(serde::Deserialize, serde::Serialize)]
12 #[serde(default)] // if we add new fields, give them default values when deserializing old state
13 pub struct HypertunnelApp {
14     // Example stuff:
15     listen_host: String,
16     listen_port: u16,
17
18     target_host: String,
19     target_password: String,
20
21     #[serde(skip)]
22     engaged: bool,
23
24     #[serde(skip)]
25     child: Option<std::process::Child>,
26
27     #[serde(skip)]
28     log_lines: Vec<String>,
29
30     #[serde(skip)]
31     abort_text: Option<String>,
32 }
33
34 impl Default for HypertunnelApp {
35     fn default() -> Self {
36         Self {
37             // Example stuff:
38             listen_host: "127.0.0.1".to_string(),
39             listen_port: 1080,
40             target_host: "https://example.com".to_string(),
41             target_password: "hunter2".to_string(),
42             engaged: false,
43             child: None,
44             log_lines: Vec::new(),
45             abort_text: None,
46         }
47     }
48 }
49
50 impl HypertunnelApp {
51     /// Called once before the first frame.
52     pub fn new(cc: &eframe::CreationContext<'>) -> Self {
53         // This is also where you can customize the look and feel of egui using
54         // `cc.egui_ctx.set_visuals` and `cc.egui_ctx.set_fonts`.
55
56         // Load previous app state (if any).
57         // Note that you must enable the `persistence` feature for this to work.
58         if let Some(storage) = cc.storage {
59             return eframe::get_value(storage, eframe::APP_KEY).unwrap_or_default();
60         }

```

```

61
62     egui_extras::install_image_loaders(&cc.egui_ctx);
63
64     Default::default()
65 }
66 }
67
68
69 impl eframe::App for HypertunnelApp {
70     /// Called by the frame work to save state before shutdown.
71     fn save(&mut self, storage: &mut dyn eframe::Storage) {
72         eframe::set_value(storage, eframe::APP_KEY, self);
73     }
74
75     /// Called each time the UI needs repainting, which may be many times per second.
76     fn update(&mut self, ctx: &egui::Context, _frame: &mut eframe::Frame) {
77         // Put your widgets into a `SidePanel`, `TopBottomPanel`, `CentralPanel`, `Window` or `Area`.
78         // For inspiration and more examples, go to https://emilk.github.io/egui
79
80         egui::TopBottomPanel::top("top_panel").show(ctx, |ui| {
81             // The top panel is often a good place for a menu bar:
82
83             egui::menu::bar(ui, |ui| {
84                 // NOTE: no File->Quit on web pages!
85                 if ui.button("Quit").clicked() {
86                     ctx.send_viewport_cmd(egui::ViewportCommand::Close);
87                 }
88
89                 egui::widgets::global_dark_light_mode_buttons(ui);
90             });
91         });
92
93         egui::CentralPanel::default().show(ctx, |ui| {
94             // The central panel the region left after adding TopPanel's and SidePanel's
95             ui.heading("Hypertunnel");
96
97             if self.engaged {
98                 let mut child = self.child.as_mut().unwrap();
99
100                 let do_abort = match child.try_wait() {
101                     Ok(Some(status)) => {
102                         println!("Child exited with: {}", status);
103                         true
104                     }
105                     Ok(None) => false,
106                     Err(e) => {
107                         println!("Error: {}", e);
108                         true
109                     }
110                 };
111
112                 let mut child_stdout = child.stdout.as_mut();
113                 let mut child_stderr = child.stderr.as_mut();
114
115                 if ui.button("Disengage").clicked() || do_abort {
116                     // Disengage the tunnel!
117                     self.engaged = false;
118
119                     // Check if we can set an abort text
120                     if do_abort && (child_stderr.is_some() || child_stdout.is_some()) {
121                         let mut out_lines = Vec::with_capacity(16);
122

```

```

123         for log in self.log_lines.iter().rev().take(10) {
124             out_lines.push(log.clone());
125         }
126
127         // Reverse out_lines
128         out_lines.reverse();
129
130         if child_stdout.is_some() {
131             // Read from stdout
132             let mut reader_stdout = BufReader::new(child_stdout.unwrap());
133             for line in reader_stdout.lines() {
134                 out_lines.push(line.unwrap());
135             }
136         }
137
138         if child_stderr.is_some() {
139             // Read from stderr
140             let mut reader_stderr = BufReader::new(child_stderr.unwrap());
141             for line in reader_stderr.lines() {
142                 out_lines.push(line.unwrap());
143             }
144         }
145
146         let out_text = out_lines.join("\n");
147         self.abort_text = Some(out_text.clone());
148
149         println!("Abort text: {}", out_text);
150     }
151
152     // Kill child
153     child.kill().unwrap();
154
155     // Clear log lines
156     self.log_lines.clear();
157
158     self.child = None;
159 } else {
160     ui.label("Program executing nominally.");
161 }
162 } else {
163     ui.horizontal(|ui| {
164         ui.label("Listen Host: ");
165
166         ui.add(egui::TextEdit::singleline(&mut self.listen_host).hint_text("0.0.0.0"));
167     });
168
169     ui.horizontal(|ui| {
170         ui.label("Listen Port: ");
171         ui.add(egui::widgets::DragValue::new(&mut self.listen_port).speed(1.0));
172     });
173
174     ui.separator();
175
176     ui.horizontal(|ui| {
177         ui.label("Target Host: ");
178         ui.text_edit_singleline(&mut self.target_host);
179     });
180
181     ui.horizontal(|ui| {
182         ui.label("Target Password:");
183         ui.add(egui::widgets::TextEdit::singleline(
184             &mut self.target_password,

```

```

185         ));
186     });
187
188     ui.separator();
189
190     if ui.button("Engage").clicked() {
191         // Engage the tunnel!
192         let child = start_tunnel(
193             &self.listen_host,
194             self.listen_port,
195             &self.target_host,
196             &self.target_password,
197         );
198
199         let child = match child {
200             Ok(child) => child,
201             Err(e) => {
202                 ui.label(format!("Error: {}", e));
203                 return;
204             }
205         };
206
207         self.engaged = true;
208
209         self.child = Some(child);
210
211         self.abort_text = None;
212     }
213
214     if self.abort_text.is_some() {
215         ui.label(format!("Error: {}", self.abort_text.as_ref().unwrap()));
216     }
217 }
218
219 ui.with_layout(egui::Layout::bottom_up(egui::Align::LEFT), |ui| {
220     egui::warn_if_debug_build(ui);
221 });
222 });
223 }
224 }
225
226 fn start_tunnel(
227     listen_host: &str,
228     listen_port: u16,
229     target_host: &str,
230     target_password: &str,
231 ) -> Result<std::process::Child, String> {
232     let child = Command::new("hypertunnel")
233         .arg("--listen-host")
234         .arg(listen_host)
235         .arg("--listen-port")
236         .arg(listen_port.to_string())
237         .arg("--target-host")
238         .arg(target_host)
239         .arg("--password")
240         .arg(target_password)
241         .stdout(Stdio::piped())
242         .stderr(Stdio::piped())
243         .spawn();
244
245     match child {
246         Ok(child) => {

```

```
247         return Ok(child);
248     }
249     Err(e) => {
250         return Err(e.to_string());
251     }
252 }
253 }
254
255 fn powered_by_egui_and_eframe(ui: &mut egui::Ui) {
256     ui.horizontal(|ui| {
257         ui.spacing_mut().item_spacing.x = 0.0;
258         ui.label("Powered by ");
259         ui.hyperlink_to("egui", "https://github.com/emilk/egui");
260         ui.label(" and ");
261         ui.hyperlink_to(
262             "eframe",
263             "https://github.com/emilk/egui/tree/master/crates/eframe",
264         );
265         ui.label(".");
266     });
267 }
```

libsocks::lib.rs

```

1 // lib.rs
2 use binrw;
3 use binrw::{BinReaderExt, BinWriterExt};
4 use binrw::io::{Cursor, Seek, SeekFrom};
5
6 pub type IPV4 = u32;
7 pub type Port = u16;
8 pub type ConnID = u32;
9
10 #[binrw::binrw]
11 #[brw(repr(u8))]
12 #[derive(Debug, PartialEq)]
13 pub enum Socks4Command {
14     Connect = 1,
15     Bind = 2,
16 }
17
18 #[derive(Debug, PartialEq)]
19 pub enum Socks4Request {
20     Connect(Socks4ConnectRequest),
21     Bind(Socks4BindRequest),
22 }
23
24 #[binrw::binrw]
25 #[brw(repr(u8))]
26 #[derive(Debug, PartialEq)]
27 pub enum Socks4Status {
28     Granted = 90,
29     Rejected = 91,
30     RejectedNoIdentd = 92,
31     RejectedIdentdMismatch = 93,
32 }
33
34 #[binrw::binrw]
35 #[derive(Debug, PartialEq)]
36 pub struct Socks4ConnectRequest {
37     pub version: u8,
38     pub command: Socks4Command,
39     pub dstport: Port,
40     pub dstip: IPV4,
41     pub userid: binrw::NullString,
42 }
43
44 #[binrw::binrw]
45 #[derive(Debug, PartialEq)]
46 pub struct Socks4ConnectReply {
47     pub version: u8,
48     pub status: Socks4Status,
49     pub dstport: Port,
50     pub dstip: IPV4,
51 }
52
53 impl Socks4ConnectReply {
54     pub fn to_binary(&self) -> Vec<u8> {
55         let mut writer = Cursor::new(Vec::new());
56         writer.write_be(self).unwrap();
57         writer.into_inner()
58     }
59 }
60

```

```

61 #[binrw::binrw]
62 #[derive(Debug, PartialEq)]
63 pub struct Socks4BindRequest {
64     pub version: u8,
65     pub command: Socks4Command,
66     pub dstport: Port,
67     pub dstip: IPV4,
68     pub userid: binrw::NullString,
69 }
70
71 #[binrw::binrw]
72 #[derive(Debug, PartialEq)]
73 pub struct Socks4BindReply {
74     pub version: u8,
75     pub status: Socks4Status,
76     pub dstport: Port,
77     pub dstip: IPV4,
78 }
79
80 #[derive(Debug)]
81 pub enum DecodeCommandError {
82     InvalidCommand,
83     BinrwError(binrw::Error),
84 }
85
86 pub fn decode_socks_request(data: &[u8]) -> Result<Socks4Request, DecodeCommandError> {
87     // Get type based on second byte
88     let command_type = match data[1] {
89         0x01 => Socks4Command::Connect,
90         0x02 => Socks4Command::Bind,
91         _ => {
92             return Err(DecodeCommandError::InvalidCommand)
93         },
94     };
95
96     match command_type {
97         Socks4Command::Connect => {
98             let mut cursor = Cursor::new(data);
99             let request: Socks4ConnectRequest = cursor.read_be().map_err(|e|
100 DecodeCommandError::BinrwError(e))?;
101             Ok(Socks4Request::Connect(request))
102         },
103         Socks4Command::Bind => {
104             let mut cursor = Cursor::new(data);
105             let request: Socks4BindRequest = cursor.read_be().map_err(|e|
106 DecodeCommandError::BinrwError(e))?;
107             Ok(Socks4Request::Bind(request))
108         },
109     }
110 }
111
112 #[cfg(test)]
113 mod tests {
114     use super::*;
115
116     #[test]
117     fn check_decode_request() {
118         let sample_data = [
119             0x04, 0x01, 0x00, 0x50, 0x00, 0x00, 0x00, 0x00, 0x01, 0x61, 0x62, 0x63, 0x64, 0x00
120         ];
121
122         let request = decode_socks_request(&sample_data).unwrap();

```

```

121     assert_eq!(request, Socks4Request::Connect(Socks4ConnectRequest {
122         version: 4,
123         command: Socks4Command::Connect,
124         dstport: 80,
125         dstip: 1,
126         userid: binrw::NullString::from("abcd"),
127     }));
128 }
129
130 #[test]
131 fn check_parse_connect() {
132     let sample_data = [
133         0x04, 0x01, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01, 0x61, 0x62, 0x63, 0x64, 0x00
134     ];
135
136     let mut cursor = Cursor::new(&sample_data);
137     let request: Socks4ConnectRequest = cursor.read_be().unwrap();
138     assert_eq!(request.version, 4);
139     assert_eq!(request.command, Socks4Command::Connect);
140     assert_eq!(request.dstport, 80);
141     assert_eq!(request.dstip, 1);
142     assert_eq!(request.userid.to_string(), "abcd");
143 }
144
145 #[test]
146 fn check_write_connect() {
147     let request = Socks4ConnectRequest {
148         version: 4,
149         command: Socks4Command::Connect,
150         dstport: 80,
151         dstip: 1,
152         userid: binrw::NullString::from("abcd"),
153     };
154
155     let mut writer = Cursor::new(Vec::new());
156     writer.write_be(&request).unwrap();
157
158     let expected_data = [
159         0x04, 0x01, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01, 0x61, 0x62, 0x63, 0x64, 0x00
160     ];
161
162     assert_eq!(writer.into_inner(), expected_data);
163 }
164
165 #[test]
166 fn check_write_reply() {
167     let reply = Socks4ConnectReply {
168         version: 4,
169         status: Socks4Status::Granted,
170         dstport: 80,
171         dstip: 1,
172     };
173
174     let mut writer = Cursor::new(Vec::new());
175     writer.write_be(&reply).unwrap();
176
177     let expected_data = [
178         0x04, 0x5a, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01
179     ];
180
181     assert_eq!(writer.into_inner(), expected_data);
182 }

```

```
183
184     #[test]
185     fn check_parse_reply() {
186         let data = [
187             0x04, 0x5a, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01
188         ];
189
190         let mut cursor = Cursor::new(&data);
191         let reply: Socks4ConnectReply = cursor.read_be().unwrap();
192         assert_eq!(reply.version, 4);
193         assert_eq!(reply.status, Socks4Status::Granted);
194         assert_eq!(reply.dstport, 80);
195         assert_eq!(reply.dstip, 1);
196     }
197
198     #[test]
199     fn check_parse_bind() {
200         let data = [
201             0x04, 0x02, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01, 0x61, 0x62, 0x63, 0x64, 0x00
202         ];
203
204         let mut cursor = Cursor::new(&data);
205         let request: Socks4BindRequest = cursor.read_be().unwrap();
206         assert_eq!(request.version, 4);
207         assert_eq!(request.command, Socks4Command::Bind);
208         assert_eq!(request.dstport, 80);
209         assert_eq!(request.dstip, 1);
210         assert_eq!(request.userid.to_string(), "abcd");
211     }
212
213     #[test]
214     fn check_write_bind() {
215         let request = Socks4BindRequest {
216             version: 4,
217             command: Socks4Command::Bind,
218             dstport: 80,
219             dstip: 1,
220             userid: binrw::NullString::from("abcd"),
221         };
222
223         let mut writer = Cursor::new(Vec::new());
224         writer.write_be(&request).unwrap();
225
226         let expected_data = [
227             0x04, 0x02, 0x00, 0x50, 0x00, 0x00, 0x00, 0x01, 0x61, 0x62, 0x63, 0x64, 0x00
228         ];
229
230         assert_eq!(writer.into_inner(), expected_data);
231     }
232 }
```

client-core::transit_builder.rs

```
1 // transit_builder.rs
2 use libsecrets::{self, EncryptionKey};
3 use request::Client;
4 use crate::transit::TransitSocket;
5 use libtransit::ServerMetaDownstream;
6 use rand;
7 use request::header::{HeaderMap, HeaderValue};
8
9 pub struct TransitSocketBuilder {
10     target: Option<String>,
11     key: Option<EncryptionKey>,
12     password: Option<String>,
13     client_identifier: Option<[u8; 16]>,
14     push_client_count: Option<usize>,
15     pull_client_count: Option<usize>,
16     timeout_time: Option<usize>,
17     client_name: Option<String>,
18 }
19
20 impl TransitSocketBuilder {
21     pub fn new() -> Self {
22         Self {
23             target: None,
24             key: None,
25             password: None,
26             client_identifier: None,
27             push_client_count: None,
28             pull_client_count: None,
29             timeout_time: None,
30             client_name: None,
31         }
32     }
33
34     pub fn with_target(mut self, target: String) -> Self {
35         self.target = Some(target);
36         self
37     }
38
39     #[allow(dead_code)]
40     pub fn with_key(mut self, key: EncryptionKey) -> Self {
41         if self.password.is_some() {
42             panic!("Cannot set both key and password");
43         }
44         self.key = Some(key);
45         self
46     }
47
48     pub fn with_password(mut self, password: String) -> Self {
49         if self.key.is_some() {
50             panic!("Cannot set both key and password");
51         }
52         self.password = Some(password);
53         self
54     }
55
56     #[allow(dead_code)]
57     pub fn with_client_identifier(mut self, client_identifier: [u8; 16]) -> Self {
58         self.client_identifier = Some(client_identifier);
59         self
60     }
61 }
```

```

61
62     pub fn with_push_client_count(mut self, push_client_count: usize) -> Self {
63         self.push_client_count = Some(push_client_count);
64         self
65     }
66
67     pub fn with_pull_client_count(mut self, pull_client_count: usize) -> Self {
68         self.pull_client_count = Some(pull_client_count);
69         self
70     }
71
72     pub fn with_timeout_time(mut self, timeout_time: usize) -> Self {
73         self.timeout_time = Some(timeout_time);
74         self
75     }
76
77     pub fn with_client_name(mut self, client_name: String) -> Self {
78         self.client_name = Some(client_name);
79         self
80     }
81
82     pub fn build(self) -> TransitSocket {
83         let target = self.target.expect("Target not set");
84
85         let key = if let Some(key) = self.key {
86             key
87         } else if let Some(password) = self.password {
88             libsecrets::form_key(password.as_bytes())
89         } else {
90             panic!("Neither key nor password set");
91         };
92
93         let client_identifier = match self.client_identifier {
94             Some(client_identifier) => client_identifier,
95             None => {
96                 // Create a random client identifier string
97                 let mut client_identifier = [0; 16];
98                 for i in 0..16 {
99                     client_identifier[i] = rand::random:::<u8>();
100                 }
101                 client_identifier
102             }
103         };
104
105         let push_client_count = match self.push_client_count {
106             Some(push_client_count) => push_client_count,
107             None => 6,
108         };
109
110         let pull_client_count = match self.pull_client_count {
111             Some(pull_client_count) => pull_client_count,
112             None => 6,
113         };
114
115         let timeout_time = match self.timeout_time {
116             Some(timeout_time) => timeout_time,
117             None => 10,
118         };
119
120         let client_id_string = hex::encode(client_identifier);
121         let headers = generate_headers(client_id_string, target.clone());
122

```

```
123     let client_name = match self.client_name {
124         Some(client_name) => client_name,
125         None => panic!("Client name not set"),
126     };
127
128     TransitSocket {
129         target,
130         key,
131         client_identifier,
132         push_client_count,
133         pull_client_count,
134         timeout_time,
135         headers,
136         is_initialized: false,
137         client_name,
138     }
139 }
140 }
141
142 fn generate_headers(client_identifier: String, referer: String) -> HeaderMap {
143     let mut headers = HeaderMap::new();
144     headers.insert("Cookie", HeaderValue::from_str(&format!("token={}",
client_identifier)).unwrap());
145     headers.insert("Accept-Language", HeaderValue::from_str("en-GB,en;q=0.5").unwrap());
146     headers.insert("Cache-Control", HeaderValue::from_str("no-cache").unwrap());
147     headers.insert("Connection", HeaderValue::from_str("keep-alive").unwrap());
148     headers.insert("Pragma", HeaderValue::from_str("no-cache").unwrap());
149     headers.insert("Referer", HeaderValue::from_str(&referer).unwrap());
150     headers.insert("Sec-Fetch-Dest", HeaderValue::from_str("document").unwrap());
151     headers.insert("Sec-Fetch-Mode", HeaderValue::from_str("navigate").unwrap());
152     headers.insert("Sec-Fetch-Site", HeaderValue::from_str("same-origin").unwrap());
153     headers.insert("TE", HeaderValue::from_str("trailers").unwrap());
154     headers.insert("User-Agent", HeaderValue::from_str("Mozilla/5.0 (X11; Linux x86_64; rv:109.0)
Gecko/20100101 Firefox/115.0").unwrap());
155     headers.insert("Content-Type", HeaderValue::from_str("application/octet-stream").unwrap());
156
157     headers
158 }
```

benches::multiple_messages_benchmark.rs

```
1 // multiple_messages_benchmark.rs
2 use criterion::{criterion_group, criterion_main, Criterion};
3
4 use libtransit::{Message, UpStreamMessage, DownStreamMessage, MultipleMessagesUpstream,
5 CloseSocketMessage};
6
7 fn gen_data(size: usize) -> Vec<u8> {
8     // Generate random data
9     let mut data: Vec<u8> = Vec::with_capacity(size);
10
11     for _ in 0..size {
12         data.push(rand::random::<u8>());
13     }
14
15     data
16 }
17
18 fn gen_upstream_messages(messagecount: u32, data: Vec<u8>) -> Vec<UpStreamMessage> {
19     let mut messages: Vec<UpStreamMessage> = Vec::new();
20
21     for i in 0..messagecount {
22         let message = UpStreamMessage {
23             socket_id: i,
24             message_sequence_number: i,
25             dest_ip: 0,
26             dest_port: 0,
27             payload: data.clone(),
28         };
29
30         messages.push(message);
31     }
32
33     messages
34 }
35
36 fn gen_close_sockets(messagecount: u32) -> Vec<CloseSocketMessage> {
37     let mut messages: Vec<CloseSocketMessage> = Vec::new();
38
39     for i in 0..messagecount {
40         let message = CloseSocketMessage {
41             socket_id: i,
42             message_sequence_number: i,
43         };
44
45         messages.push(message);
46     }
47
48     messages
49 }
50
51 fn stress_test_multiple_messages_upstream(upstreams: Vec<UpStreamMessage>, close_sockets:
52 Vec<CloseSocketMessage>) {
53     let multiple_messages_up = MultipleMessagesUpstream {
54         stream_messages: upstreams,
55         close_socket_messages: close_sockets,
56     };
57
58     let mut encoded = multiple_messages_up.encoded().unwrap();
59
60     let decoded = MultipleMessagesUpstream::decode_from_bytes(&mut encoded).unwrap();
```

```

59
60     assert_eq!(multiple_messages_up, decoded);
61 }
62
63 fn criterion_benchmark(c: &mut Criterion) {
64     let stream_messages = gen_upstream_messages(50, gen_data(100));
65     let close_socket_messages = gen_close_sockets(10);
66     c.bench_function("stress_test_multiple_messages_50mc_100b    (5kb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
67
68     let stream_messages = gen_upstream_messages(50, gen_data(1_000));
69     let close_socket_messages = gen_close_sockets(10);
70     c.bench_function("stress_test_multiple_messages_50mc_1kb    (50kb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
71
72     let stream_messages = gen_upstream_messages(50, gen_data(10_000));
73     let close_socket_messages = gen_close_sockets(10);
74     c.bench_function("stress_test_multiple_messages_50mc_10kb   (500kb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
75
76     let stream_messages = gen_upstream_messages(50, gen_data(100_000));
77     let close_socket_messages = gen_close_sockets(10);
78     c.bench_function("stress_test_multiple_messages_50mc_100kb   (5mb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
79
80     let stream_messages = gen_upstream_messages(10, gen_data(1_000_000));
81     let close_socket_messages = gen_close_sockets(10);
82     c.bench_function("stress_test_multiple_messages_10mc_1mb    (10mb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
83
84     let stream_messages = gen_upstream_messages(10, gen_data(10_000_000));
85     let close_socket_messages = gen_close_sockets(10);
86     c.bench_function("stress_test_multiple_messages_50mc_10mb   (100mb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
87
88     let stream_messages = gen_upstream_messages(10, gen_data(100_000_000));
89     let close_socket_messages = gen_close_sockets(10);
90     c.bench_function("stress_test_multiple_messages_50mc_100mb   (1gb)", |b| b.iter(||
stress_test_multiple_messages_upstream(stream_messages.clone(), close_socket_messages.clone())));
91 }
92
93 criterion_group!(benches, criterion_benchmark);
94 criterion_main!(benches);

```

libtransit::lib.rs

```

1 // lib.rs
2 #![allow(unused_imports)]
3 use rand::Rng; // Not unused - the warning
4
5 use borsh::{BorshSerialize, BorshDeserialize, to_vec};
6
7 #[cfg(test)]
8 mod tests;
9
10 pub type Port = u16;
11 pub type IPV4 = u32;
12 pub type SocketID = u32;
13 pub type DeclarationToken = [u8; 16];
14
15 static LAST_SOCKET_ID: std::sync::atomic::AtomicU32 = std::sync::atomic::AtomicU32::new(0);
16
17 pub trait SerialMessage: BorshSerialize + BorshDeserialize {
18     fn encoded(&self) -> Result<Vec<u8>, Box<dyn std::error::Error>> {
19         let data = to_vec(self)?;
20         Ok(data)
21     }
22
23     fn decode_from_bytes(data: &mut Vec<u8>) -> Result<Self, std::io::Error>
24     where
25         Self: Sized,
26     {
27         Self::try_from_slice(data)
28     }
29 }
30
31 pub fn generate_socket_id() -> SocketID {
32     LAST_SOCKET_ID.fetch_add(1, std::sync::atomic::Ordering::SeqCst) as SocketID
33 }
34
35 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
36 pub struct UpStreamMessage {
37     pub socket_id: SocketID,
38     pub dest_ip: IPV4,
39     pub dest_port: Port,
40     pub payload: Vec<u8>,
41     pub red_terminate: bool,
42 }
43
44 impl SerialMessage for UpStreamMessage {}
45
46 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
47 pub struct DownStreamMessage {
48     pub socket_id: SocketID,
49     pub payload: Vec<u8>,
50     pub do_green_terminate: bool,
51 }
52
53 impl SerialMessage for DownStreamMessage {}
54
55 // ===== //
56
57 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug)]
58 pub struct ClientMessageUpstream {
59     pub metadata: ClientMetaUpstream,
60     pub socks_sockets: Vec<SocksSocketUpstream>,
61     pub payload_size: u32,

```

```

62 }
63
64 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
65 pub struct ClientMetaUpstream {
66     pub packet_info: UnifiedPacketInfo,
67     pub set: Option<ClientMetaUpstreamSet>,
68     pub yellow_to_stop_reading_from: Vec<SocketID>,
69 }
70
71 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
72 pub struct UnifiedPacketInfo {
73     pub unix_ms: u64, // As u32 it would only last 49 days
74     pub seq_num: u32,
75 }
76
77 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
78 pub struct ClientMetaUpstreamSet {
79     pub buffer_size_to_return: u32,
80     pub modetime_to_return: u32,
81 }
82
83 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
84 pub struct SocksSocketUpstream {
85     pub socket_id: SocketID,
86     pub dest_ip: IPV4,
87     pub dest_port: Port,
88     pub payload: Vec<u8>,
89     pub red_terminate: bool,
90 }
91
92 impl SerialMessage for ClientMessageUpstream {}
93
94 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug)]
95 pub struct ServerMessageDownstream {
96     pub metadata: ServerMetaDownstream,
97     pub socks_sockets: Vec<SocksSocketDownstream>,
98     pub payload_size: u32,
99 }
100
101 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
102 pub struct ServerMetaDownstream {
103     pub packet_info: UnifiedPacketInfo,
104 }
105
106
107 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
108 pub struct SocksSocketDownstream {
109     pub socket_id: SocketID,
110     pub dest_ip: IPV4,
111     pub dest_port: Port,
112     pub payload: Vec<u8>,
113     pub do_green_terminate: bool,
114     pub do_blue_terminate: bool,
115 }
116
117 #[repr(u8)]
118 #[derive(BorshSerialize, BorshDeserialize, PartialEq, Debug, Clone)]
119 pub enum ServerMetaDownstreamLogSeverity {
120     Info,
121     Warning,
122     Error,
123 }

```

```
124  
125  impl SerialMessage for ServerMessageDownstream {}
```

libtransit::tests.rs

```

1 // tests.rs
2 use crate::*;
3
4 #[test]
5 fn test_generate_socket_id() {
6     let socket_id = generate_socket_id();
7     assert_eq!(socket_id, 0);
8     let socket_id = generate_socket_id();
9     assert_eq!(socket_id, 1);
10 }
11
12 #[test]
13 fn test_client_upstream() {
14     let msg = ClientMessageUpstream {
15         metadata: ClientMetaUpstream {
16             packet_info: UnifiedPacketInfo {
17                 unix_ms: 0,
18                 seq_num: 0,
19             },
20             set: None,
21             yellow_to_stop_reading_from: vec![0],
22         },
23         socks_sockets: vec![SocksSocketUpstream {
24             socket_id: 0,
25             dest_ip: 0,
26             dest_port: 0,
27             payload: vec![0, 1, 2, 3],
28             red_terminate: false,
29         }],
30         payload_size: 4,
31     };
32     let encoded = msg.encoded().unwrap();
33     let decoded = ClientMessageUpstream::decode_from_bytes(&mut encoded.clone()).unwrap();
34     assert_eq!(msg, decoded);
35 }
36
37 #[test]
38 fn test_upstream_msg() {
39     let msg = UpStreamMessage {
40         socket_id: 0,
41         dest_ip: 0,
42         dest_port: 0,
43         payload: vec![0, 1, 2, 3],
44         red_terminate: false,
45     };
46     let encoded = msg.encoded().unwrap();
47     let decoded = UpStreamMessage::decode_from_bytes(&mut encoded.clone()).unwrap();
48     assert_eq!(msg, decoded);
49 }
50
51 #[test]
52 fn test_downstream_msg() {
53     let msg = DownStreamMessage {
54         socket_id: 0,
55         payload: vec![0, 1, 2, 3],
56         do_green_terminate: false,
57     };
58     let encoded = msg.encoded().unwrap();
59     let decoded = DownStreamMessage::decode_from_bytes(&mut encoded.clone()).unwrap();
60     assert_eq!(msg, decoded);

```

```
61 }
62
63 #[test]
64 fn test_server_message_downstream() {
65     let msg = ServerMessageDownstream {
66         metadata: ServerMetaDownstream {
67             packet_info: UnifiedPacketInfo {
68                 unix_ms: 0,
69                 seq_num: 0,
70             },
71         },
72         socks_sockets: vec![SocksSocketDownstream {
73             socket_id: 0,
74             dest_ip: 0,
75             dest_port: 0,
76             payload: vec![0, 1, 2, 3],
77             do_green_terminate: false,
78             do_blue_terminate: false,
79         }],
80         payload_size: 4,
81     };
82     let encoded = msg.encoded().unwrap();
83     let decoded = ServerMessageDownstream::decode_from_bytes(&mut encoded.clone()).unwrap();
84     assert_eq!(msg, decoded);
85 }
```

libsecrets::lib.rs

```

1 // lib.rs
2 use chacha20poly1305::{
3     aead::{Aead, AeadCore, KeyInit, OsRng, Key},
4     ChaCha20Poly1305, Nonce
5 };
6
7 pub use chacha20poly1305::Error;
8 pub type EncryptionError = Error;
9
10 use sha2::{Sha256, Digest};
11
12 pub type EncryptionKey = Key<ChaCha20Poly1305>;
13
14 pub fn form_key(data: &[u8]) -> EncryptionKey {
15     let mut hasher = Sha256::new();
16     hasher.update(data);
17     hasher.finalize()
18 }
19
20 fn random_nonce() -> Nonce {
21     ChaCha20Poly1305::generate_nonce(&mut OsRng)
22 }
23
24 pub fn encrypt(data: &[u8], key: &EncryptionKey) -> Result<Vec<u8>, EncryptionError> {
25     let nonce = random_nonce();
26     let cipher = ChaCha20Poly1305::new(key);
27     let mut encrypted = cipher.encrypt(&nonce, data.as_ref())?;
28     encrypted.extend_from_slice(nonce.as_ref()); // Append 12-byte nonce
29     Ok(encrypted)
30 }
31
32 pub fn decrypt(data: &[u8], key: &EncryptionKey) -> Result<Vec<u8>, EncryptionError> {
33     let nonce = Nonce::from_slice(&data[data.len() - 12..]); // Extract 12-byte nonce
34     let data = &data[..data.len() - 12]; // Remove 12-byte nonce
35
36     let cipher = ChaCha20Poly1305::new(key);
37     let decrypted = cipher.decrypt(&nonce, data.as_ref())?;
38     Ok(decrypted)
39 }
40
41 #[cfg(test)]
42 mod tests {
43     use super::*;
44
45     #[test]
46     fn test_encrypt_decrypt() {
47         let key = form_key(b"01234567890123456789012345678901");
48         let data = b"Hello world!";
49         let encrypted = encrypt(data, &key).unwrap();
50         let decrypted = decrypt(&encrypted, &key).unwrap();
51         assert_eq!(data, decrypted.as_slice());
52     }
53
54     #[test]
55     fn test_decryption_wrong_key() {
56         let key = form_key(b"01234567890123456789012345678901");
57         let data = b"Hello world!";
58         let encrypted = encrypt(data, &key).unwrap();
59         let key = form_key(b"Hello World!");
60         let decrypted = decrypt(&encrypted, &key);

```

```
61         assert!(decrypted.is_err());
62     }
63
64     #[test]
65     fn test_form_key() {
66         let key = form_key(b"Hello, world!");
67         assert_eq!(key.len(), 32);
68     }
69 }
```

client-cli::main.rs

```

1 // main.rs
2 use clientcore::{begin_core_client, ClientArguments};
3 use log::{info, Level};
4 use clap::Parser;
5
6 #[derive(Parser, Debug)]
7 #[clap(name = "Hypertunnel Client")]
8 #[command(version, about, author)]
9 struct Args {
10     #[clap(long, default_value = "127.0.0.1")]
11     listen_host: String,
12
13     #[clap(long, default_value = "1080")]
14     listen_port: u16,
15
16     #[clap(long, default_value = "http://127.0.1:8000")]
17     target_host: String,
18
19     #[clap(long, default_value = "12345")]
20     password: String,
21
22     #[clap(long, default_value = "8")]
23     push_client_count: usize,
24
25     #[clap(long, default_value = "8")]
26     pull_client_count: usize,
27
28     #[clap(long, default_value = "128")]
29     timeout_time_s: usize,
30
31     #[clap(long, default_value = "info")]
32     log_level: String,
33 }
34
35 #[tokio::main]
36 async fn main() {
37     let arguments = Args::parse();
38
39     let level: Level = match arguments.log_level.to_lowercase().as_str() {
40         "trace" => Level::Trace,
41         "debug" => Level::Debug,
42         "info" => Level::Info,
43         "warn" => Level::Warn,
44         "error" => Level::Error,
45         _ => Level::Info,
46     };
47
48     simple_logger::set_up_color_terminal();
49     simple_logger::init_with_level(level).unwrap();
50
51     info!("Received arguments: {:?}", arguments);
52
53     let client_args = ClientArguments {
54         listen_address: arguments.listen_host,
55         listen_port: arguments.listen_port,
56         target_host: arguments.target_host,
57         password: arguments.password,
58         push_client_count: arguments.push_client_count,
59         pull_client_count: arguments.pull_client_count,
60         timeout_time_s: arguments.timeout_time_s,
61         client_name: "Client CLI via Client Core".to_string(),

```

```
62     };  
63  
64     begin_core_client(client_args).await;  
65 }
```

hypertunnel_gui::main.rs

```

1 // main.rs
2 #![warn(clippy::all, rust_2018_idioms)]
3 #![cfg_attr(not(debug_assertions), windows_subsystem = "windows")] // hide console window on Windows
4 in release
5
6 // When compiling natively:
7 #[cfg(not(target_arch = "wasm32"))]
8 fn main() -> eframe::Result<()> {
9     env_logger::init(); // Log to stderr (if you run with `RUST_LOG=debug`).
10
11     let native_options = eframe::NativeOptions {
12         viewport: egui::ViewportBuilder::default()
13             .with_inner_size([400.0, 300.0])
14             .with_min_inner_size([300.0, 220.0])
15             .with_icon(
16                 // NOE: Adding an icon is optional
17                 eframe::icon_data::from_png_bytes(&include_bytes!("../assets/icon-256.png")[..])
18                     .unwrap(),
19             ),
20         ..Default::default()
21     };
22     eframe::run_native(
23         "eframe template",
24         native_options,
25         Box::new(|cc| Box::new(hypertunnel_gui::HypertunnelApp::new(cc))),
26     )
27 }
28
29 // When compiling to web using trunk:
30 #[cfg(target_arch = "wasm32")]
31 fn main() {
32     // Redirect `log` message to `console.log` and friends:
33     eframe::WebLogger::init(log::LevelFilter::Debug).ok();
34
35     let web_options = eframe::WebOptions::default();
36
37     wasm_bindgen_futures::spawn_local(async {
38         eframe::WebRunner::new()
39             .start(
40                 "the_canvas_id", // hardcode it
41                 web_options,
42                 Box::new(|cc| Box::new(hypertunnel_gui::HypertunnelApp::new(cc))),
43             )
44             .await
45             .expect("failed to start eframe");
46     });
47 }

```

benches::decryption_benchmark.rs

```
1 // decryption_benchmark.rs
2 use criterion::{criterion_group, criterion_main, Criterion};
3 use libsecrets::{encrypt, form_key, decrypt};
4
5 fn criterion_benchmark(c: &mut Criterion) {
6     let key = form_key(b"01234567890123456789012345678901");
7     let data = b"Hello world!";
8     let encrypted = encrypt(data, &key).unwrap();
9
10    c.bench_function("decrypt_small", |b| b.iter(|| {
11        let decrypted = decrypt(&encrypted, &key);
12    }));
13
14    let data_1kb = b"12345678".repeat(128); // 8 * 128 = 1024
15    let encrypted = encrypt(data_1kb.as_slice(), &key).unwrap();
16
17    c.bench_function("decrypt_1kb", |b| b.iter(|| {
18        let decrypted = decrypt(&encrypted, &key);
19    }));
20
21    let data_1mb = b"12345678".repeat(128 * 1024); // 8 * 128 * 1024 = 1048576
22    let encrypted = encrypt(data_1mb.as_slice(), &key).unwrap();
23
24    c.bench_function("decrypt_1mb", |b| b.iter(|| {
25        let decrypted = decrypt(&encrypted, &key);
26    }));
27
28    let data_10mb = b"12345678".repeat(128 * 1024 * 10); // 8 * 128 * 1024 * 10 = 10485760
29    let encrypted = encrypt(data_10mb.as_slice(), &key).unwrap();
30
31    c.bench_function("decrypt_10mb", |b| b.iter(|| {
32        let decrypted = decrypt(&encrypted, &key);
33    }));
34 }
35
36 criterion_group!(benches, criterion_benchmark);
37 criterion_main!(benches);
```

server-monolith::config.rs

```

1 // config.rs
2 use serde::{Deserialize, Serialize};
3 use std::fs;
4 use log::{debug, warn};
5
6
7 #[derive(Serialize, Deserialize, Debug, Clone)]
8 pub struct Config {
9     pub host: String,
10    pub port: u16,
11    pub users: Vec<User>,
12    pub workers: usize,
13 }
14
15 #[derive(Serialize, Deserialize, Debug, Clone)]
16 pub struct User {
17     pub name: String,
18     pub password: String,
19 }
20
21 impl Default for Config {
22     fn default() -> Self {
23         Config {
24             host: "127.0.0.1".to_string(),
25             port: 80,
26             users: Vec::new(),
27             workers: get_thread_count() as usize,
28         }
29     }
30 }
31
32 fn get_thread_count() -> usize {
33     num_cpus::get()
34 }
35
36 pub fn load_config() -> Config {
37     // Load the configuration file into a Config struct
38     let contents = match fs::read_to_string("server.config.toml") {
39         Ok(contents) => contents,
40         Err(_) => {
41             warn!("No configuration file found, creating one...");
42             let config = Config::default();
43             let toml = toml::to_string(&config).unwrap();
44             fs::write("server.config.toml", toml).unwrap();
45             return config;
46         }
47     };
48
49     let config: Config = toml::from_str(&contents).unwrap();
50
51     debug!("Loaded configuration: {:?}", config);
52
53     config
54 }

```

benches::structs_vs_strings.rs

```

1 // structs_vs_strings.rs
2 use criterion::{criterion_group, criterion_main, Criterion};
3
4 struct ErrorStruct {
5     error_type: u32,
6     error_value: u32,
7     idk_what_goes_here: u32,
8 }
9
10 struct OtherErrorType {
11     thing: u64,
12 }
13
14 enum ErrorEnum {
15     ErrorStruct(ErrorStruct),
16     OtherErrorType(OtherErrorType),
17 }
18
19 fn create_structs() {
20     let mut fill = Vec::with_capacity(1_000_000);
21     for i in 0..1_000_000 {
22         let error = ErrorEnum::ErrorStruct(ErrorStruct {
23             error_type: 1,
24             error_value: 2,
25             idk_what_goes_here: 3,
26         });
27         fill.push(error);
28     }
29 }
30
31 fn create_strings() {
32     let mut fill = Vec::with_capacity(1_000_000);
33     for i in 0..1_000_000 {
34         let error = String::from("Hello world! I'm doing a string memory allocation here! :D");
35         fill.push(error);
36     }
37 }
38
39 fn criterion_benchmark(c: &mut Criterion) {
40     c.bench_function("fill_with_structs", |b| {
41         b.iter(|| {
42             create_structs();
43         })
44     });
45     c.bench_function("fill_with_strings", |b| {
46         b.iter(|| {
47             create_strings();
48         })
49     });
50 }
51
52
53 criterion_group!(benches, criterion_benchmark);
54 criterion_main!(benches);

```

benches::encryption_benchmark.rs

```
1 // encryption_benchmark.rs
2 use criterion::{criterion_group, criterion_main, Criterion};
3 use libsecrets::{encrypt, form_key};
4
5 fn criterion_benchmark(c: &mut Criterion) {
6     let key = form_key(b"01234567890123456789012345678901");
7
8     c.bench_function("encrypt_small", |b| b.iter(|| {
9         let data = b"Hello world!";
10        let encrypted = encrypt(data, &key).unwrap();
11    }));
12
13    let data_1kb = b"12345678".repeat(128); // 8 * 128 = 1024
14
15    c.bench_function("encrypt_1kb", |b| b.iter(|| {
16        let data = data_1kb.as_slice();
17        let encrypted = encrypt(data, &key).unwrap();
18    }));
19
20    let data_1mb = b"12345678".repeat(128 * 1024); // 8 * 128 * 1024 = 1048576
21
22    c.bench_function("encrypt_1mb", |b| b.iter(|| {
23        let data = data_1mb.as_slice();
24        let encrypted = encrypt(data, &key).unwrap();
25    }));
26
27    let data_10mb = b"12345678".repeat(128 * 1024 * 10); // 8 * 128 * 1024 * 10 = 10485760
28
29    c.bench_function("encrypt_10mb", |b| b.iter(|| {
30        let data = data_10mb.as_slice();
31        let encrypted = encrypt(data, &key).unwrap();
32    }));
33 }
34
35 criterion_group!(benches, criterion_benchmark);
36 criterion_main!(benches);
```

hypertunnel_gui::Cargo.toml

```
1 // Cargo.toml
2 [package]
3 name = "hypertunnel_gui"
4 version = "0.1.0"
5 authors = ["Alex <alexcj2802@gmail.com>"]
6 edition = "2021"
7
8 [dependencies]
9 egui = "0.26.0"
10 eframe = { version = "0.26.0", default-features = false, features = [
11     "accesskit",      # Make egui compatible with screen readers. NOTE: adds a lot of dependencies.
12     "default_fonts",  # Embed the default egui fonts.
13     "glow",           # Use the glow rendering backend. Alternative: "wgpu".
14     "persistence",    # Enable restoring app state when restarting the app.
15 ] }
16 egui_extras = {version = "0.26.2", features = ["all_loaders", "image", "file"]}
17 log = "0.4"
18
19 # You only need serde if you want app persistence:
20 serde = { version = "1", features = ["derive"] }
21
22 # native:
23 [target.'cfg(not(target_arch = "wasm32"))'.dependencies]
24 env_logger = "0.11"
25
26 [profile.release]
27 opt-level = 2 # fast and small wasm
28
29 # Optimize all dependencies even in debug builds:
30 [profile.dev.package."*"]
31 opt-level = 2
```

benches::allocate_benchmark.rs

```
1 // allocate_benchmark.rs
2 use criterion::{criterion_group, criterion_main, Criterion};
3
4 fn criterion_benchmark(c: &mut Criterion) {
5     let data = b"Hello world!";
6     c.bench_function("alloc_small", |b| b.iter(|| {
7         let other = Box::new(data.clone());
8     }));
9
10    let data_1kb = b"12345678".repeat(128); // 8 * 128 = 1024
11
12    c.bench_function("alloc_1kb", |b| b.iter(|| {
13        let other = Box::new(data_1kb.clone());
14    }));
15
16    let data_1mb = b"12345678".repeat(128 * 1024); // 8 * 128 * 1024 = 1048576
17
18    c.bench_function("alloc_1mb", |b| b.iter(|| {
19        let other = Box::new(data_1mb.clone());
20    }));
21
22    let data_10mb = b"12345678".repeat(128 * 1024 * 10); // 8 * 128 * 1024 * 10 = 10485760
23
24    c.bench_function("alloc_10mb", |b| b.iter(|| {
25        let other = Box::new(data_10mb.clone());
26    }));
27 }
28
29 criterion_group!(benches, criterion_benchmark);
30 criterion_main!(benches);
```

libsecrets::Cargo.toml

```
1 // Cargo.toml
2 [package]
3 name = "libsecrets"
4 version = "0.1.0"
5 edition = "2021"
6
7 # See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
8
9 [dependencies]
10 chacha20poly1305 = "0.10.1"
11 generic-array = "1"
12 sha2 = "0.10.7"
13
14 [dev-dependencies]
15 criterion = { version = "0.5", features = ["html_reports"] }
16
17 [[bench]]
18 name = "encryption_benchmark"
19 harness = false
20
21 [[bench]]
22 name = "decryption_benchmark"
23 harness = false
24
25 [[bench]]
26 name = "key_derivation_benchmark"
27 harness = false
28
29 [[bench]]
30 name = "allocate_benchmark"
31 harness = false
32
33 [[bench]]
34 name = "structs_vs_strings"
35 harness = false
```

server-monolith::Cargo.toml

```
1 // Cargo.toml
2 [package]
3 name = "server-monolith"
4 version = "0.1.0"
5 edition = "2021"
6
7 # See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
8
9 [dependencies]
10 actix-web = "4.5.1"
11
12 libtransit = { path = "../libtransit" }
13 libsecrets = { path = "../libsecrets" }
14
15 dashmap = "5"
16
17 once_cell = "1.18.0"
18 serde = { version = "1.0", features = ["derive"] }
19
20 rand = "0.8"
21 debug_print = "1"
22
23 hex = "0.4"
24
25 num_cpus = "1.16"
26
27 tokio = {version = "1", features = ["sync", "net", "io-util"]}
28
29 flume = "0.11"
30
31 toml = "0.8"
32
33 log = "0.4"
34 simple_logger = "4.3"
35
36 scan_fmt = "0.2.6"
```

client-core::Cargo.toml

```
1 // Cargo.toml
2 [package]
3 name = "clientcore"
4 version = "0.2.0"
5 edition = "2021"
6
7 # See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
8
9 [dependencies]
10 libsocks = {path = "../libsocks"}
11 libtransit = {path = "../libtransit"}
12 libsecrets = { path = "../libsecrets" }
13
14 tokio = { version = "1.36.0", features = ["net"] }
15
16 request = { version = "0.11", features = ["json"] }
17
18 rand = "0.8"
19 hex = "0.4"
20
21 flume = "0.11"
22 debug_print = "1"
23
24 once_cell = "1.18.0"
25
26 log = "0.4"
27
28 scan_fmt = "0.2.6"
```

client-cli::Cargo.toml

```
1 // Cargo.toml
2 [package]
3 name = "client-cli"
4 version = "0.1.0"
5 authors = ["Alex <alexcj2802@gmail.com>"]
6 edition = "2021"
7
8 # See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
9
10 [dependencies]
11 clap = { version = "4.5.0", features = ["derive"] }
12 clientcore = { path = "../client-core" }
13 tokio = { version = "1.36.0", features = ["full"] }
14 simple_logger = "4.3"
15 log = "0.4"
16
17 [profile.release]
18 debug = true
```

client-core::meta.rs

```
1 // meta.rs
2 use libtransit::SocketID;
3 use tokio::sync::RwLock;
4 use once_cell::sync::Lazy;
5
6 // Simple format: List of socket IDs to terminate reading from ASAP. Order doesn't really matter,
7 // the info should be sent as soon as possible. See diagram.
8 pub type YellowDataUpstreamQueue = Lazy<RwLock<Vec<SocketID>>>;
9
10 pub static YELLOW_DATA_UPSTREAM_QUEUE: YellowDataUpstreamQueue = Lazy::new(|| {
11     RwLock::new(Vec::new())
12 });
```

libtransit::Cargo.toml

```
1 // Cargo.toml
2 [package]
3 name = "libtransit"
4 version = "0.1.0"
5 edition = "2021"
6
7 # See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
8
9 [dependencies]
10 borsh = {version = "1", features = ["derive"]}
11 rand = "0.8"
12
13 [dev-dependencies]
14 criterion = { version = "0.5", features = ["html_reports"] }
15
16 [[bench]]
17 name = "multiple_messages_benchmark"
18 harness = false
```

benches::key_derivation_benchmark.rs

```
1 // key_derivation_benchmark.rs
2 use criterion::{criterion_group, criterion_main, Criterion};
3 use libsecrets::{encrypt, form_key};
4
5 fn criterion_benchmark(c: &mut Criterion) {
6     c.bench_function("key_derivation", |b| b.iter(|| {
7         let key = form_key(b"01234567890123456789012345678901");
8     }));
9
10 }
11
12 criterion_group!(benches, criterion_benchmark);
13 criterion_main!(benches);
```

integration-tests::Cargo.toml

```
1 // Cargo.toml
2 [package]
3 name = "integration-tests"
4 version = "0.1.0"
5 edition = "2021"
6
7 # See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
8
9 [dependencies]
10 clientcore = { path = "../client-core" }
11 once_cell = "1"
```

Source::Cargo.toml

```
1 // Cargo.toml
2 [workspace]
3
4 members = [
5     "client-cli",
6     "client-core",
7     "libsocks",
8     "libtransit",
9     "server-monolith",
10    "libsecrets",
11    "integration-tests",
12    "hypertunnel_gui"
13 ]
14
15 workspace.resolver = "2"
```

libsocks::Cargo.toml

```
1 // Cargo.toml
2 [package]
3 name = "libsocks"
4 version = "0.1.0"
5 edition = "2021"
6
7 # See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
8
9 [dependencies]
10 binrw = "0.13"
```

integration-tests::lib.rs

```
1 // lib.rs
2 #[cfg(test)]
3 mod tests {
4     use super::*;
5
6     // TODO rewrite for new system
7     #[test]
8     fn check_socket_connect() {
9         assert!(true);
10    }
11 }
```

hypertunnel_gui::lib.rs

```
1 // lib.rs
2 #![warn(clippy::all, rust_2018_idioms)]
3 #![feature(buf_read_has_data_left)]
4
5 mod app;
6 pub use app::HypertunnelApp;
```

server-monolith::server.config.toml

```
1 // server.config.toml
2 host = "127.0.0.1"
3 port = 8000
4 workers = 12
5
6 users = [
7     {"name" = "Test", "password" = "12345"}
8 ]
```

Source::test_curl.sh

```
1 // test_curl.sh
2 curl -x socks4://127.0.0.1:1080 https://ifconfig.me
```